Diffusion, Flows and Beyond:

Fundamentals, Adaptation, Post-Training, Distillation and Inference-Time Scaling of Generative Models

Niklas Schweiger

October 18, 2026

Abstract

Diffusion and flow models are trained with a simple regression loss and now generate images, video, audio, language, robot actions and molecular structures. The central questions in the field have moved from how to train these models to what happens after pretraining. This survey organizes that work as a lifecycle with five stages. Pretraining learns a score or velocity field. Adaptation specializes the model to new data, subjects and forms of control. Alignment tilts the model towards rewards and human preferences. Distillation compresses its sampler to one or a few steps. Search spends additional compute at inference time. We develop the theory in one notation, from energy-based models and score matching through stochastic and ordinary differential equations, flow matching and stochastic optimal control, with derivations at the level of a first course and complete proofs in an appendix. We then show that most adaptation, alignment and search methods target the same reward-tilted distribution and differ in whether they reach it by changing weights, the sampling dynamics or the choice of noise, which makes their costs and failure modes directly comparable. The survey covers more than 450 works with an emphasis on 2024 to 2026, including discrete diffusion for language, hybrids of autoregression and diffusion, causal video generation and world models, and applications in robotics and science. Conceptual figures are computed exactly on two-dimensional Gaussian mixtures and are accompanied by animations in a web companion. We close with open problems that span the lifecycle: the relation between autoregression and diffusion, evaluation that tracks human judgement, reward over-optimization, data provenance and the allocation of compute between training, post-training and inference.

Contents

1 Introduction **4**

2 Fundamentals **7**

2.1 Energies, scores and Langevin dynamics 8
2.2 Learning the score: from Hyvärinen to Tweedie 9
2.3 Diffusion as an SDE: Fokker–Planck, time reversal and the probability-flow ODE . . 12
2.4 Flows: the continuity equation and flow matching 14
2.5 The likelihood view: ELBO, VDM and loss weighting 15
2.6 Couplings, optimal transport, Schrödinger bridges and control 18
2.7 Why diffusion models generalize 20
2.8 Representations and architectures 22
2.9 Discrete and language diffusion 23
2.10 Autoregressive and hybrid visual generation 28
2.11 Conditioning and guidance 29

3 Fine-Tuning and Adaptation **32**
3.1 Parameter-efficient fine-tuning . . . 32
3.2 Personalization . . . 33
3.3 Spatial and structural control . . . 34
3.4 Instruction-based editing . . . 35
3.5 Video adaptation . . . 35
3.6 Risks of cheap adaptation . . . 36

4 Post-Training: Preference and Reward Alignment **36**
4.1 Reward models . . . 37
4.2 Policy gradients on the denoising chain . . . 38
4.3 Differentiable rewards . . . 41
4.4 Preference optimization . . . 41
4.5 A stochastic optimal control view . . . 42
4.6 Reward hacking and regularization . . . 44
4.7 Reinforcement learning for discrete diffusion . . . 46

5 Distillation and Few-Step Generation **47**
5.1 Trajectory distillation . . . 47
5.2 Distribution matching . . . 50
5.3 Adversarial distillation . . . 52
5.4 Flow maps: a unifying lens . . . 53
5.5 Causal and real-time video . . . 56
5.6 Systems-level acceleration . . . 57

6 Inference-Time Scaling **59**
6.1 Solvers and step schedules . . . 60
6.2 Search over noise . . . 61
6.3 Sequential Monte Carlo and twisted sampling . . . 63
6.4 Verifiers, reward guidance and reflection . . . 65
6.5 Inverse problems . . . 67
6.6 Test-time compute for diffusion language models . . . 68

7 Applications **70**
7.1 Image generation and editing . . . 70
7.2 Video . . . 73
7.3 World models and interactive simulation . . . 74
7.4 3D, audio and motion . . . 77
7.5 Robotics . . . 78
7.6 Science: proteins, molecules, materials, medicine and weather . . . 79

8 Open Problems and Outlook **82**
8.1 Autoregression and diffusion as one family . . . 83
8.2 Evaluation that tracks human judgement . . . 84
8.3 Reward over-optimization . . . 84
8.4 Data provenance and memorization . . . 85
8.5 Compute-optimal trade-offs across the lifecycle . . . 86
8.6 Beyond iterative sampling . . . 87

A Derivations for the fundamentals **88**
A.1 Implicit score matching . . . 88
A.2 Regression onto a noisy target and denoising score matching . . . 88
A.3 The Fokker–Planck equation . . . 88
A.4 The probability-flow ODE and the reverse-time SDE . . . 89
A.5 The marginal velocity . . . 89
A.6 The velocity–score identity . . . 89
A.7 The ELBO as a weighted denoising loss . . . 90
A.8 Closed-form fields for Gaussian mixtures . . . 90

1 Introduction

Diffusion models generate data by learning to reverse a process that gradually turns data into noise [123, 316, 320, 322]. Flow matching and related methods learn a velocity field that transports noise to data along prescribed paths [4, 5, 202, 214]. Both families are trained with a simple regression loss, they scale reliably with data and compute, and they now underlie most systems for generating images [84, 287], video [273, 340], audio [85], robot actions [28, 60], protein structures [1, 376] and weather forecasts [278]. Diffusion has also become a serious alternative to autoregression for language [252, 295].

From training to the lifecycle of a model. The first wave of research asked how to train such models: which noising process to use, how to parametrize the network, which loss weighting to choose and how to sample from the result [123, 152, 202, 322]. These questions are now largely settled, and the questions that occupy researchers and practitioners in 2026 come after pretraining. How can a large pretrained model be adapted to a new domain, subject or form of control with little data and compute? How can it be aligned with human preferences or with a reward that measures a property such as docking score or text rendering [27, 208, 356]? How can its sampler, which needs tens of network evaluations, be compressed to one or a few steps [297, 325, 417]? And how should additional compute be spent at inference time, by searching over noises and sampling paths against a verifier [232]? This survey is organized around these questions. We call the sequence of stages, pretraining, adaptation, alignment, compression and search, the lifecycle of a generative model, and we treat each stage with the same notation and, where possible, the same mathematical objects.

One target, several routes. A single distribution recurs throughout the survey and explains why the stages belong together. Given a pretrained model p, a reward r and a regularization strength $\beta > 0$, the distribution that maximizes the expected reward minus β times the KL divergence from p is the tilted distribution

$$p^\star(\boldsymbol{x}) = \frac{1}{Z}\, p(\boldsymbol{x})\, e^{r(\boldsymbol{x})/\beta}, \qquad Z = \mathbb{E}_p\big[e^{r(\boldsymbol{x})/\beta}\big],$$

derived in Section 4 as Eq. (60). Reward fine-tuning (Section 4) changes the weights of the model so that it samples from $p^\star$. Guidance and stochastic optimal control change the drift of the sampler instead, and the exact change is given by a Doob h-transform (Sections 2 and 4.5). Sequential Monte Carlo reaches $p^\star$ by reweighting and resampling particles during sampling, and best-of-N reaches a closely related tilt by selection (Section 6). Conditioning fits the same form: with $\beta = 1$ and $r(\boldsymbol{x}) = \log p(\boldsymbol{y} \mid \boldsymbol{x})$, the tilt is Bayes' rule and $p^\star$ is the posterior $p(\boldsymbol{x} \mid \boldsymbol{y})$. The routes differ in when the cost is paid, during training or for every sample, and in what they require: a differentiable reward, a stochastic sampler or only the ability to score finished samples. Table 1 summarizes these differences. The common target also explains a common failure. A learned reward is only a proxy, and every route that moves far towards $p^\star$ can exploit its errors, which is the reward hacking discussed in Sections 4.6 and 8.3.

Taxonomy. Figure 1 shows the lifecycle on a two-dimensional toy problem for which every quantity is exact. A flow is pretrained on six Gaussians arranged on a ring, fine-tuned to data with an additional mode in the centre, aligned with a reward that prefers the upper half of the plane, distilled into a one-step map and finally improved at test time by best-of-8 selection. The same 400 noise draws pass through all five stages, so the effect of each stage on individual samples can

Table 1: **The stages of the lifecycle.** For each stage, what is changed, when its compute is spent and what it needs. All stages except distillation can be read as moving a pretrained model towards a target distribution, in the alignment and search stages the tilted target Eq. (60). Distillation keeps the distribution and changes the cost of sampling from it.

Stage	What changes	Compute paid	Typical requirement	Section
Pretrain	all weights	once, very large	large dataset	2
Adapt	few weights or adapters	once, small	small dataset or condition	3
Align	weights	once, moderate	reward or preference pairs	4
Compress	sampler (new network)	once, moderate	teacher model	5
Search	noise, path or drift	per sample	verifier	6

be followed. The chapters of the survey follow the same order. Applications (Section 7) and open problems (Section 8) come at the end, because every application combines several stages.

Scope and related surveys. Earlier surveys of diffusion models [39, 70, 405] were written when the field was organized around improvements to the original denoising model: faster samplers, better likelihoods and new data modalities. They predate flow matching as a standard training method, latent transformers at scale and most of the work on alignment and distillation. The flow matching guide of Lipman et al. [203] gives a thorough and self-contained treatment of flow matching with code, but it focuses on training. The tutorial of Uehara et al. [351] reviews reinforcement learning for fine-tuning diffusion models towards rewards, with an emphasis on biological sequence and structure design. Surveys of single topics, such as efficient inference for masked diffusion language models [112], cover one stage in depth. This survey differs in three ways. It treats diffusion and flow models in one notation and does not separate them by family. It covers the whole lifecycle after pretraining and emphasizes the connections between the stages. And it concentrates on the literature of 2024 to 2026, in which most of the post-training, distillation and inference-scaling methods appeared. We cover continuous and discrete state spaces, images, video, language and scientific data. We do not cover generative adversarial networks, variational autoencoders or normalizing flows except where they are used as components, and we discuss applications only to the extent that they raise new methodological questions.

Notation and conventions. Throughout, $t = 0$ denotes noise and $t = 1$ denotes data, and the noisy state is

$$\boldsymbol{x}_t = \alpha_t \, \boldsymbol{x}_1 + \sigma_t \, \boldsymbol{x}_0, \qquad \boldsymbol{x}_0 \sim \mathcal{N}(0, I), \quad \boldsymbol{x}_1 \sim p_{\text{data}},$$

with $\alpha_0 = 0$, $\sigma_0 = 1$, $\alpha_1 = 1$ and $\sigma_1 = 0$. This is the direction used by flow matching and stochastic interpolants. Score-based diffusion and DDPM use the opposite direction, and many rectified-flow image models write $\boldsymbol{x}_t = (1 - t)\boldsymbol{x}_1 + t\boldsymbol{x}_0$ with $t = 0$ at the data. Table 2 translates between these conventions. We write $\boldsymbol{s}_t$ for the score, $\boldsymbol{u}_t$ for the velocity, $\hat{\boldsymbol{x}}_1(\boldsymbol{x}_t, t)$ for the denoiser, that is, the posterior mean of the data given the noisy state, r for a reward and β for the strength of a KL regularizer.

How to read this survey. The survey is written for two audiences. Readers new to the field can read Section 2 from the beginning, since it builds the theory from energy-based models through score matching, stochastic and ordinary differential equations, flow matching and optimal control in one chain of derivations, summarized in the dependency map of Fig. 2. Readers who know the basics can start with any later chapter, each of which refers back to the specific results it uses. The

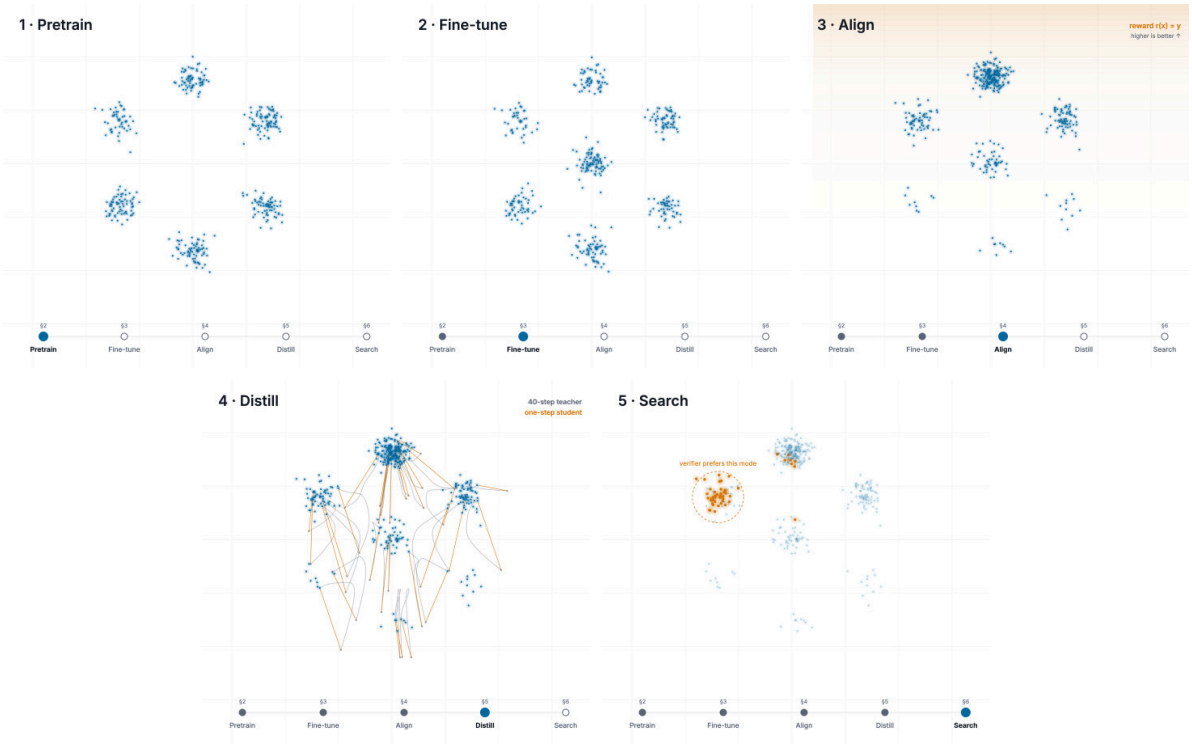


Figure 1: **The lifecycle covered by this survey** (Animation A0). One 2D toy and the same 400 noise draws pass through five stages. Pretraining learns a flow from noise to six Gaussians on a ring (Section 2). Fine-tuning adapts it to data with an added centre mode (Section 3). Alignment tilts the model by a reward $r(x) = y$, which for a KL penalty gives $p^*(x) \propto p(x)\exp(r(x)/\beta)$ (Section 4). Distillation replaces the 40-step ODE with a one-step map to the same endpoints (Section 5). Search spends test-time compute, here best-of-8 against a verifier (Section 6). Every stage is an exact Gaussian mixture.

text has three layers. The main text gives the idea, the key equation and a short derivation. Boxes titled *Derivation sketch* give the steps of a proof, and full proofs of the results of Section 2 are collected in Appendix A. Paragraphs marked *Advanced* discuss technical conditions, convergence theory and recent extensions and can be skipped on a first reading. Most conceptual figures are computed on two-dimensional Gaussian mixtures, for which scores, velocities and posteriors are available in closed form, so the figures show exact quantities rather than the outputs of a trained network. Each of these figures is a still from an animation in the web companion of this survey, where the dynamics can be watched over time.

Contributions. The survey makes the following contributions.

- A unified derivation of diffusion and flow models in a single notation, from energies and scores to flows, couplings and stochastic control, with the identities that connect them and complete proofs in the appendix (Section 2).
- An organization of the work after pretraining as a lifecycle of adaptation, alignment, compression and search, with the tilted distribution Eq. (60) as the common target, which makes the trade-offs between training-time and inference-time methods explicit (Sections 3 to 6).

- Coverage of more than 450 works, with an emphasis on 2024 to 2026, including discrete diffusion for language, autoregressive and diffusion hybrids, and applications to video, world models, robotics and science (Section 7).
- Short derivations of results that are often stated without proof, such as the best-of-N divergence bound Eq. (91), and exact two-dimensional illustrations with accompanying animations.
- A discussion of open problems that span several stages, including the relation between autoregression and diffusion, evaluation, reward over-optimization, data provenance and the allocation of compute across the lifecycle (Section 8).

Organization. Section 2 develops the theory and covers architectures, discrete diffusion, hybrid models and guidance. Section 3 covers parameter-efficient fine-tuning, personalization, spatial control, editing and the adaptation of image models to video. Section 4 covers reward models and alignment by policy gradients, reward backpropagation, direct preference optimization and stochastic optimal control. Section 5 covers trajectory distillation, distribution matching, adversarial distillation, flow maps and causal distillation for video. Section 6 covers solvers, noise search, sequential Monte Carlo, reward guidance, inverse problems and test-time scaling for diffusion language models. Section 7 reviews applications, and Section 8 discusses open problems.

2 Fundamentals

This chapter builds the mathematical foundation for the rest of the survey. It is organized as one chain of derivations rather than as a list of model families. We start from energy-based models and the observation that the score of a density does not depend on its normalizing constant (Section 2.1). Score matching and Tweedie's formula show how to learn the score from samples (Section 2.2). Adding noise continuously gives a stochastic differential equation, and the Fokker–Planck equation gives both its reversal in time and a deterministic flow with the same marginals (Section 2.3). Reading that flow through the continuity equation leads to flow matching, and to the fact that scores and velocities are linear transforms of each other (Section 2.4). The later sections treat the likelihood view, couplings and control, generalization, architectures, discrete data and guidance. Figure 2 shows how the pieces connect and where each one is used later in the survey.

The chapter is written for three kinds of reader. The main text states each idea, its key equation and a short derivation. Complete proofs are collected in Appendix A, so that every identity in the main text can be checked line by line. Paragraphs marked *Advanced* contain measure-theoretic detail, convergence theory and recent extensions, and can be skipped on a first reading.

Conventions. Throughout the survey, time runs from noise to data. A sample from the model at time $t \in [0, 1]$ is

$$\boldsymbol{x}_t = \alpha_t \boldsymbol{x}_1 + \sigma_t \boldsymbol{x}_0, \qquad \boldsymbol{x}_0 \sim \mathcal{N}(0, I), \quad \boldsymbol{x}_1 \sim p_{\text{data}}, \tag{1}$$

with $\boldsymbol{x}_0$ and $\boldsymbol{x}_1$ independent unless stated otherwise. The schedule satisfies $\alpha_0 = 0$, $\sigma_0 = 1$, $\alpha_1 = 1$ and $\sigma_1 = 0$, and we write p_t for the law of $\boldsymbol{x}_t$, so that $p_0 = \mathcal{N}(0, I)$ and $p_1 = p_{\text{data}}$. The log signal-to-noise ratio $\lambda_t = \log(\alpha_t^2/\sigma_t^2)$ increases from $-\infty$ to $+\infty$. Dots denote time derivatives. Many papers, including the original score-based SDE papers and most rectified-flow image models, run time in the opposite direction. Table 2 translates between the conventions.

The last row deserves emphasis. Rectified-flow image models such as SD3 call the *data* $\boldsymbol{x}_0$ and run time from data to noise, so their velocity has the opposite sign to ours. This is the most common source of sign errors when moving between papers.

Table 2: **Translating time conventions.** Our time t runs from noise ($t = 0$) to data ($t = 1$). The last column gives the substitution that maps each convention to ours.

Framework	Corrupted sample	Data at	Map to (α_t, σ_t)
This survey	$\boldsymbol{x}_t = \alpha_t \boldsymbol{x}_1 + \sigma_t \boldsymbol{x}_0$	$t = 1$	identity
DDPM [123]	$\sqrt{\bar{\alpha}_k}\,\boldsymbol{x} + \sqrt{1-\bar{\alpha}_k}\,\boldsymbol{\epsilon}$, step k	$k = 0$	$\alpha_t = \sqrt{\bar{\alpha}_k}$, $t = 1 - k/K$
Score SDE [322]	$\boldsymbol{y}_\tau$, $\tau \in [0, T]$	$\tau = 0$	$t = 1 - \tau/T$
EDM [152]	$\boldsymbol{y} + \sigma \boldsymbol{n}$, indexed by σ	$\sigma = 0$	$\sigma = \sigma_t/\alpha_t$, $\boldsymbol{x} = \alpha_t(\boldsymbol{y} + \sigma \boldsymbol{n})$
Flow matching [202]	$t\boldsymbol{x}_1 + (1-t)\boldsymbol{x}_0$	$t = 1$	$\alpha_t = t$, $\sigma_t = 1 - t$
Rectified flow [214]	$tX_1 + (1-t)X_0$	$t = 1$	$\alpha_t = t$, $\sigma_t = 1 - t$
Interpolants [5]	$\alpha_t \boldsymbol{x}_0 + \beta_t \boldsymbol{x}_1$ $(+\gamma_t \boldsymbol{z})$	$t = 1$	$\alpha_t \to \sigma_t$, $\beta_t \to \alpha_t$
SD3 and successors [84]	$(1-t)\boldsymbol{x}_0 + t\boldsymbol{\epsilon}$, $\boldsymbol{x}_0$ data	$t = 0$	$t \to 1 - t$

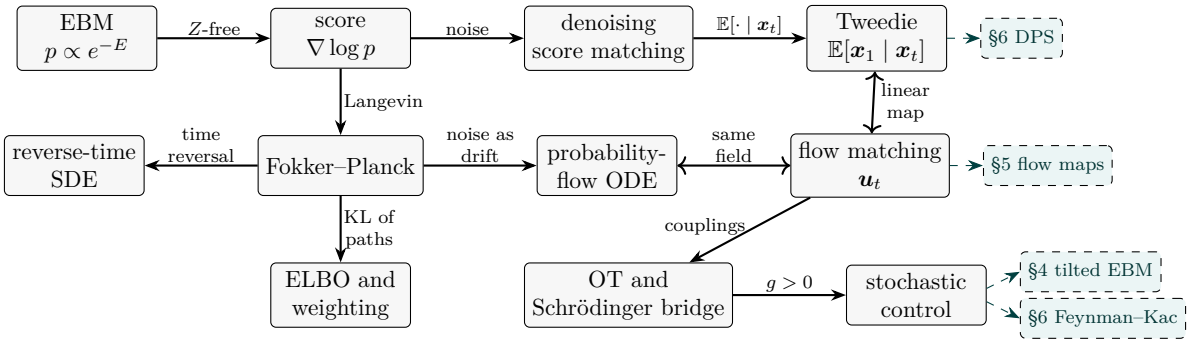


Figure 2: **Dependency map of the fundamentals.** Each arrow is an identity derived in this chapter, labelled by the step it uses. Dashed boxes show where a result is used in a later chapter.

Running example. For a Gaussian mixture $p_{\text{data}} = \sum_k \pi_k \mathcal{N}(\boldsymbol{\mu}_k, \Sigma_k)$, every quantity in this chapter is available in closed form, because p_t is again a Gaussian mixture with means $\alpha_t \boldsymbol{\mu}_k$ and covariances $\alpha_t^2 \Sigma_k + \sigma_t^2 I$ (Appendix A.8). All animations of the web companion that show 2D fields use these exact expressions instead of trained networks, so the figures illustrate the mathematics rather than the quirks of a particular training run.

2.1 Energies, scores and Langevin dynamics

Energy-based models. Any positive density can be written as

$$p(\boldsymbol{x}) = \frac{e^{-E(\boldsymbol{x})}}{Z}, \qquad Z = \int e^{-E(\boldsymbol{x})}\, \mathrm{d}\boldsymbol{x}, \tag{2}$$

where E is the *energy*. Parametrizing E_θ by a neural network is the most flexible way to define a density, but both maximum likelihood and sampling run into the normalizer Z_θ, an integral over the whole space that has no closed form [321].

The score does not depend on Z. The gradient of the log-density,

$$\boldsymbol{s}(\boldsymbol{x}) = \nabla_{\boldsymbol{x}} \log p(\boldsymbol{x}) = -\nabla_{\boldsymbol{x}} E(\boldsymbol{x}), \tag{3}$$

is called the *score*.[1] Because Z does not depend on $\boldsymbol{x}$, it disappears from the score. This observation is the starting point of the whole chapter. If a model can learn $\boldsymbol{s}$ instead of p, the normalization

[1] In statistics the score is the gradient with respect to parameters. In generative modelling it is the gradient with respect to the data, and we follow that usage.

problem never arises. The rest of this section shows that the score is enough to sample, and Section 2.2 shows that it can be learned from samples.

Langevin dynamics. The overdamped Langevin equation

$$\mathrm{d}\boldsymbol{x}_\tau = \nabla \log p(\boldsymbol{x}_\tau)\,\mathrm{d}\tau + \sqrt{2}\,\mathrm{d}\boldsymbol{W}_\tau \tag{4}$$

drifts uphill along the score and is kicked by Brownian noise $\boldsymbol{W}_\tau$, where τ is an auxiliary time unrelated to t. Its density ρ_τ satisfies the Fokker–Planck equation $\partial_\tau \rho = -\nabla\cdot(\rho \nabla \log p) + \Delta\rho$, which we derive in Section 2.3. Rewriting the right-hand side as

$$\partial_\tau \rho_\tau = \nabla\cdot\Big(\rho_\tau\, \nabla \log \frac{\rho_\tau}{p}\Big) \tag{5}$$

shows that $\rho = p$ is stationary. A deeper statement, due to Jordan et al. [148], is that Eq. (5) is the gradient flow of $\mathrm{KL}(\rho\,\|\,p)$ in the space of probability measures with the Wasserstein-2 metric. Langevin dynamics is therefore steepest descent on the KL divergence to the target, and it needs only the score. The Euler–Maruyama discretization with step η,

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \eta\, \boldsymbol{s}(\boldsymbol{x}_k) + \sqrt{2\eta}\,\boldsymbol{\xi}_k, \qquad \boldsymbol{\xi}_k \sim \mathcal{N}(0, I), \tag{6}$$

is the unadjusted Langevin algorithm.

Why Langevin alone is not enough. Two problems appear as soon as the score is learned rather than known [320]. First, a score estimated from data is accurate only where data is dense, and the initial particles start far from the data. Second, for a multimodal target the time needed to cross a low-density barrier grows exponentially with its height, so the relative mass of separated modes is not recovered in practice. Both problems are addressed by the same device. Smoothing the target with Gaussian noise of scale σ fills in the low-density regions and lowers the barriers. Running Langevin dynamics at a decreasing sequence of noise levels then moves particles at a coarse scale first and refines them afterwards. Figure 3 shows both effects on the running example. Making the sequence of noise levels continuous turns annealed Langevin dynamics into the diffusion models of Section 2.3.

Advanced. Energy parametrizations remain useful even though most modern models learn scores or velocities directly. An energy gives a scalar that can be compared across samples, composed by addition, and used for Metropolis corrections. Equilibrium Matching [365] learns a time-independent implicit energy whose gradient field generates samples, and the reward-tilted targets of Section 4 are energy-based models by construction (Section 2.6).

2.2 Learning the score: from Hyvärinen to Tweedie

Explicit and implicit score matching. The natural objective for a score model $\boldsymbol{s}_\theta$ is the Fisher divergence

$$J_{\mathrm{ESM}}(\theta) = \tfrac{1}{2}\,\mathbb{E}_p \big\|\boldsymbol{s}_\theta(\boldsymbol{x}) - \nabla \log p(\boldsymbol{x})\big\|^2, \tag{7}$$

but it contains the unknown $\nabla \log p$. Hyvärinen [136] showed that integration by parts removes it. Expanding the square, the cross term is $\mathbb{E}_p[\boldsymbol{s}_\theta \cdot \nabla \log p] = \int \boldsymbol{s}_\theta \cdot \nabla p\,\mathrm{d}\boldsymbol{x} = -\mathbb{E}_p[\nabla\cdot \boldsymbol{s}_\theta]$, provided $p\,\boldsymbol{s}_\theta$ vanishes at infinity. Hence

$$J_{\mathrm{ESM}}(\theta) = \mathbb{E}_p\Big[\tfrac{1}{2}\|\boldsymbol{s}_\theta(\boldsymbol{x})\|^2 + \nabla\cdot \boldsymbol{s}_\theta(\boldsymbol{x})\Big] + C, \tag{8}$$

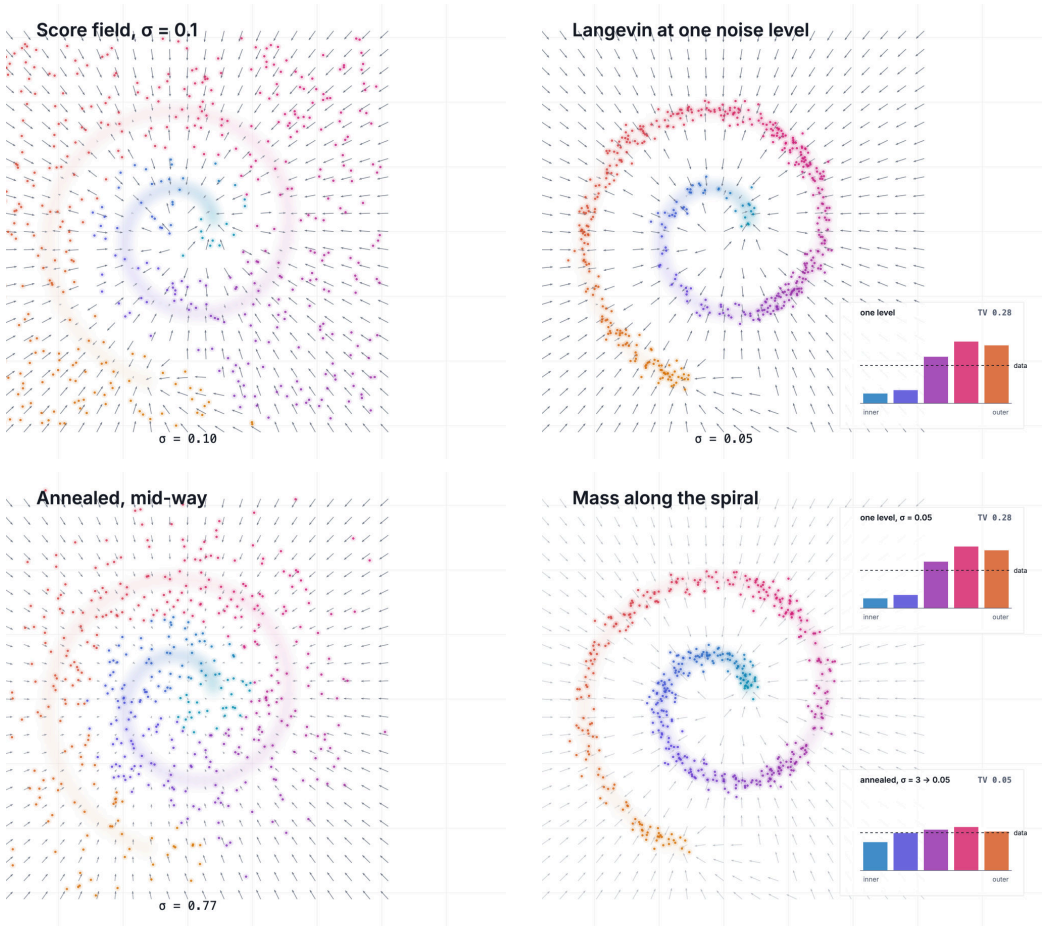


Figure 3: **Scores and annealed Langevin dynamics** (Animation A2). The score of the noise-smoothed spiral mixture is exact at every noise level σ (top left). Langevin dynamics at a single small σ leaves each particle on the arm nearest to its uniform start, so the inner fifth of the spiral receives 6% of the mass instead of 20% (top right). Annealing σ from 3 to 0.05 over ten levels [320] first spreads particles at a coarse scale and then refines them (bottom left), which recovers the correct mass along the spiral (bottom right).

where C does not depend on θ (Appendix A.1). The objective now involves only samples from p. Its weakness is the divergence term, the trace of a $d \times d$ Jacobian, which costs d backward passes per sample for images with $d \approx 10^5$ or more.

Denoising score matching. Vincent [355] removed the Jacobian by learning the score of a *noised* distribution. With our interpolant, the conditional law of $\boldsymbol{x}_t$ given the data point $\boldsymbol{x}_1$ is $p_t(\boldsymbol{x} \mid \boldsymbol{x}_1) = \mathcal{N}(\boldsymbol{x}; \alpha_t \boldsymbol{x}_1, \sigma_t^2 I)$, whose score is known:

$$\nabla \log p_t(\boldsymbol{x}_t \mid \boldsymbol{x}_1) = -\frac{\boldsymbol{x}_t - \alpha_t \boldsymbol{x}_1}{\sigma_t^2} = -\frac{\boldsymbol{x}_0}{\sigma_t}. \tag{9}$$

The denoising score matching objective regresses onto this conditional score,

$$J_{\mathrm{DSM}}(\theta) = \tfrac{1}{2}\, \mathbb{E}_{\boldsymbol{x}_1, \boldsymbol{x}_0} \left\| \boldsymbol{s}_\theta(\boldsymbol{x}_t, t) + \frac{\boldsymbol{x}_0}{\sigma_t} \right\|^2. \tag{10}$$

It differs from the explicit objective for p_t only by a constant. The reason is a lemma that we use repeatedly in this chapter.

Lemma 2.1 (Regression onto a noisy target)**.** *Let (X, Y) be random variables with $\mathbb{E}\|Y\|^2 < \infty$. For every function f,*

$$\mathbb{E}\|f(X) - Y\|^2 = \mathbb{E}\|f(X) - \mathbb{E}[Y \mid X]\|^2 + \mathbb{E}\|Y - \mathbb{E}[Y \mid X]\|^2.$$

In particular, the minimizer is $f^\star(\boldsymbol{x}) = \mathbb{E}[Y \mid X = \boldsymbol{x}]$, and the second term does not depend on f.

Applied with $X = \boldsymbol{x}_t$ and $Y = -\boldsymbol{x}_0/\sigma_t$, the lemma says that the minimizer of Eq. (10) is $\mathbb{E}[-\boldsymbol{x}_0/\sigma_t \mid \boldsymbol{x}_t]$. Differentiating $p_t(\boldsymbol{x}) = \int p_t(\boldsymbol{x} \mid \boldsymbol{x}_1)\, p_{\text{data}}(\boldsymbol{x}_1)\, \mathrm{d}\boldsymbol{x}_1$ under the integral shows that this conditional expectation *is* the marginal score (Appendix A.2):

$$\nabla \log p_t(\boldsymbol{x}) = \mathbb{E}\big[\nabla \log p_t(\boldsymbol{x}_t \mid \boldsymbol{x}_1) \,\big|\, \boldsymbol{x}_t = \boldsymbol{x}\big] = -\frac{1}{\sigma_t}\, \mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{x}_t = \boldsymbol{x}]. \tag{11}$$

The pattern of regressing on a tractable conditional target and obtaining an intractable marginal quantity as the minimizer will reappear in flow matching (Section 2.4), in distillation and in reward fine-tuning.

Tweedie's formula. Combining Eq. (11) with $\boldsymbol{x}_t = \alpha_t \mathbb{E}[\boldsymbol{x}_1 \mid \boldsymbol{x}_t] + \sigma_t \mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{x}_t]$, which holds because $\boldsymbol{x}_t$ is known given itself, gives

$$\mathbb{E}[\boldsymbol{x}_1 \mid \boldsymbol{x}_t = \boldsymbol{x}] = \frac{\boldsymbol{x} + \sigma_t^2 \nabla \log p_t(\boldsymbol{x})}{\alpha_t}. \tag{12}$$

This is Tweedie's formula [82]. It says that the posterior-mean denoiser and the score of the noised distribution carry exactly the same information. A network trained to denoise is a score model, and a score model is a denoiser. The identity is used throughout the survey: posterior sampling for inverse problems plugs the denoiser into a likelihood (Section 6), distribution-matching distillation compares two denoisers (Section 5), and representation alignment shapes the internal features of the denoiser (Section 2.8).

Prediction targets are interchangeable. Write $\hat{\boldsymbol{x}}_1(\boldsymbol{x}, t) = \mathbb{E}[\boldsymbol{x}_1 \mid \boldsymbol{x}_t = \boldsymbol{x}]$ and $\hat{\boldsymbol{x}}_0(\boldsymbol{x}, t) = \mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{x}_t = \boldsymbol{x}]$. Because $\boldsymbol{x} = \alpha_t \hat{\boldsymbol{x}}_1 + \sigma_t \hat{\boldsymbol{x}}_0$, either one determines the other, and both determine the score and the velocity of Section 2.4:

$$\hat{\boldsymbol{x}}_0 = -\sigma_t\, \boldsymbol{s}_t, \qquad \hat{\boldsymbol{x}}_1 = \frac{\boldsymbol{x} + \sigma_t^2\, \boldsymbol{s}_t}{\alpha_t}, \qquad \hat{\boldsymbol{u}}_t = \dot{\alpha}_t\, \hat{\boldsymbol{x}}_1 + \dot{\sigma}_t\, \hat{\boldsymbol{x}}_0. \tag{13}$$

Noise prediction ("ϵ-prediction") learns $\hat{\boldsymbol{x}}_0$, data prediction learns $\hat{\boldsymbol{x}}_1$, and v-prediction [297] and velocity prediction learn fixed linear combinations of the two. At the optimum all parametrizations give the same model. They differ in two practical respects. The first is the implied weighting over time. Since the errors satisfy $\alpha_t(\hat{\boldsymbol{x}}_1 - \boldsymbol{x}_1) + \sigma_t(\hat{\boldsymbol{x}}_0 - \boldsymbol{x}_0) = 0$ pointwise, the squared errors are related by

$$\|\hat{\boldsymbol{x}}_0 - \boldsymbol{x}_0\|^2 = \frac{\alpha_t^2}{\sigma_t^2}\|\hat{\boldsymbol{x}}_1 - \boldsymbol{x}_1\|^2, \qquad \|\hat{\boldsymbol{u}}_t - \boldsymbol{u}_t\|^2 = \Big(\dot{\alpha}_t - \dot{\sigma}_t \frac{\alpha_t}{\sigma_t}\Big)^2 \|\hat{\boldsymbol{x}}_1 - \boldsymbol{x}_1\|^2. \tag{14}$$

For the linear path $\alpha_t = t$, $\sigma_t = 1 - t$ the velocity loss equals the data-prediction loss multiplied by $(1 - t)^{-2}$. The second is numerical conditioning. Near $t = 0$ the input is almost pure noise, so

Table 3: **Prediction targets.** Each row expresses one target in terms of the data prediction $\hat{\boldsymbol{x}}_1$ and the input $\boldsymbol{x} = \boldsymbol{x}_t$. The weight is the factor that turns the squared error of the target into the squared error of $\hat{\boldsymbol{x}}_1$, as in Eq. (14).

Target	Regression label	In terms of $\hat{\boldsymbol{x}}_1$	Weight on $\|\hat{\boldsymbol{x}}_1 - \boldsymbol{x}_1\|^2$
data ($\hat{\boldsymbol{x}}_1$)	$\boldsymbol{x}_1$	$\hat{\boldsymbol{x}}_1$	1
noise ($\hat{\boldsymbol{x}}_0$)	$\boldsymbol{x}_0$	$(\boldsymbol{x} - \alpha_t \hat{\boldsymbol{x}}_1)/\sigma_t$	$\alpha_t^2/\sigma_t^2 = e^{\lambda_t}$
score ($\boldsymbol{s}_t$)	$-\boldsymbol{x}_0/\sigma_t$	$(\alpha_t \hat{\boldsymbol{x}}_1 - \boldsymbol{x})/\sigma_t^2$	α_t^2/σ_t^4
velocity ($\hat{\boldsymbol{u}}_t$)	$\dot{\alpha}_t \boldsymbol{x}_1 + \dot{\sigma}_t \boldsymbol{x}_0$	$\dot{\alpha}_t \hat{\boldsymbol{x}}_1 + \dot{\sigma}_t (\boldsymbol{x} - \alpha_t \hat{\boldsymbol{x}}_1)/\sigma_t$	$(\dot{\alpha}_t - \dot{\sigma}_t \alpha_t/\sigma_t)^2$

predicting $\hat{\boldsymbol{x}}_0$ is trivial and $\hat{\boldsymbol{x}}_1$ is hard, while near $t = 1$ the opposite holds. Table 3 collects the conversions, and Section 2.5 returns to the choice of weighting.

Advanced. Sliced score matching replaces the trace in Eq. (8) by random projections and avoids noise altogether, but denoising score matching has displaced it because the noise is needed anyway for sampling. Data prediction has regained popularity in high-dimensional latent spaces. Singh et al. [312] argue that representation alignment is effectively data prediction in the latent space of a representation autoencoder, and report strong ImageNet results from this view.

2.3 Diffusion as an SDE: Fokker–Planck, time reversal and the probability-flow ODE

Annealed Langevin dynamics uses a finite ladder of noise levels. Letting the number of levels go to infinity turns the corruption into a stochastic differential equation [322]. This section derives the three equations behind every continuous-time diffusion model. In this section only, we write the corruption process in its natural direction, from data to noise, with time $\tau = 1 - t$, and translate back to our convention at the end.

The forward SDE. Consider the linear SDE

$$\mathrm{d}\boldsymbol{y}_\tau = f_\tau \, \boldsymbol{y}_\tau \, \mathrm{d}\tau + g_\tau \, \mathrm{d}\boldsymbol{W}_\tau, \qquad \boldsymbol{y}_0 \sim p_{\mathrm{data}}. \tag{15}$$

Given $\boldsymbol{y}_0$, its solution is Gaussian with mean $a_\tau \boldsymbol{y}_0$ and covariance $b_\tau^2 I$, where $\dot{a}_\tau = f_\tau a_\tau$ and $\frac{\mathrm{d}}{\mathrm{d}\tau} b_\tau^2 = 2 f_\tau b_\tau^2 + g_\tau^2$. Matching $a_\tau = \alpha_{1-\tau}$ and $b_\tau = \sigma_{1-\tau}$ to our interpolant Eq. (1) gives, after a short calculation,

$$f = -\frac{\dot{\alpha}_t}{\alpha_t}, \qquad g^2 = 2\sigma_t^2 \Big(\frac{\dot{\alpha}_t}{\alpha_t} - \frac{\dot{\sigma}_t}{\sigma_t} \Big) = \sigma_t^2 \, \dot{\lambda}_t, \tag{16}$$

where the right-hand sides are evaluated at $t = 1 - \tau$. Every Gaussian interpolant with increasing log-SNR is therefore the marginal law of some linear SDE, and the diffusion coefficient g^2 is the rate at which the log-SNR changes, scaled by the noise variance. The variance-preserving process of DDPM [123, 316] has $\alpha_t^2 + \sigma_t^2 = 1$, and the variance-exploding process of score-based models [320] has $\alpha_t \equiv 1$ up to rescaling.[2]

The Fokker–Planck equation. For a general SDE $\mathrm{d}\boldsymbol{y} = \boldsymbol{b}(\boldsymbol{y}, \tau) \, \mathrm{d}\tau + g_\tau \, \mathrm{d}\boldsymbol{W}$, the density ρ_τ of $\boldsymbol{y}_\tau$ evolves by

$$\partial_\tau \rho_\tau = -\nabla \cdot (\boldsymbol{b} \, \rho_\tau) + \tfrac{1}{2} g_\tau^2 \, \Delta \rho_\tau. \tag{17}$$

[2] Strictly, $\alpha_0 = 0$ requires $f \to -\infty$ at the noise end. Practical schedules stop at a small α or a large σ, and the mismatch between p_0 and $\mathcal{N}(0, I)$ enters the error bounds below.

The derivation takes three lines (Appendix A.3). Itô's formula gives $\mathrm{d}\varphi(\boldsymbol{y}) = (\boldsymbol{b}\cdot\nabla\varphi + \frac{1}{2}g^2\Delta\varphi)\,\mathrm{d}\tau +$ martingale for any smooth test function φ. Taking expectations gives $\frac{\mathrm{d}}{\mathrm{d}\tau}\mathbb{E}\varphi(\boldsymbol{y}_\tau) = \mathbb{E}[\boldsymbol{b}\cdot\nabla\varphi + \frac{1}{2}g^2\Delta\varphi]$. Writing both sides as integrals against ρ_τ and integrating by parts moves the derivatives from φ to ρ_τ. The first term on the right of Eq. (17) transports mass along the drift, and the second spreads it out. The Langevin equation Eq. (4) is the special case $\boldsymbol{b} = \nabla\log p$, $g = \sqrt{2}$.

One equation, two readings. The key step is to write the diffusion term as a transport term. Since $\Delta\rho = \nabla\cdot(\rho\nabla\log\rho)$,

$$\partial_\tau\rho_\tau = -\nabla\cdot\Big((\boldsymbol{b} - \tfrac{1}{2}g_\tau^2\nabla\log\rho_\tau)\rho_\tau\Big). \tag{18}$$

This is a continuity equation with velocity $\boldsymbol{b} - \frac{1}{2}g^2\nabla\log\rho_\tau$. Two consequences follow, one from reading Eq. (18) forwards and one from reading it backwards.

Probability-flow ODE. The deterministic dynamics $\dot{\boldsymbol{y}} = \boldsymbol{b} - \frac{1}{2}g^2\nabla\log\rho_\tau$ has exactly the same marginals ρ_τ as the SDE [322]. The randomness of the SDE can be traded for a dependence of the velocity on the current density, through its score.

Reverse-time SDE. Reversing time, the density $p_t = \rho_{1-t}$ satisfies

$$\partial_t p_t = -\nabla\cdot\left((-\boldsymbol{b} + \tfrac{1}{2}g^2\nabla\log p_t)\,p_t\right).$$

Adding and subtracting $\frac{1}{2}g^2\Delta p_t = \nabla\cdot(\frac{1}{2}g^2 p_t\nabla\log p_t)$ turns this into a Fokker–Planck equation with drift $-\boldsymbol{b} + g^2\nabla\log p_t$ and the same diffusion coefficient. The SDE

$$\mathrm{d}\boldsymbol{x}_t = \big(-\boldsymbol{b} + g^2\,\nabla\log p_t(\boldsymbol{x}_t)\big)\,\mathrm{d}t + g\,\mathrm{d}\boldsymbol{W}_t \tag{19}$$

therefore runs from noise to data with marginals p_t. Anderson [9] proved the stronger statement that the reversed process has this law as a whole path, not only at each time. The derivation via Eq. (18) gives the marginal statement, which is all that sampling needs (Appendix A.4).

Back to our convention. Substituting Eq. (16) with $\boldsymbol{b} = f\boldsymbol{y} = -(\dot\alpha_t/\alpha_t)\boldsymbol{x}$ and using $\mathrm{d}\tau = -\mathrm{d}t$, the probability-flow ODE and the reverse SDE become

$$\dot{\boldsymbol{x}}_t = \frac{\dot\alpha_t}{\alpha_t}\,\boldsymbol{x}_t + \tfrac{1}{2}\sigma_t^2\dot\lambda_t\,\nabla\log p_t(\boldsymbol{x}_t), \tag{20}$$

$$\mathrm{d}\boldsymbol{x}_t = \Big(\frac{\dot\alpha_t}{\alpha_t}\,\boldsymbol{x}_t + \sigma_t^2\dot\lambda_t\,\nabla\log p_t(\boldsymbol{x}_t)\Big)\mathrm{d}t + \sigma_t\sqrt{\dot\lambda_t}\,\mathrm{d}\boldsymbol{W}_t. \tag{21}$$

A learned score $\boldsymbol{s}_\theta \approx \nabla\log p_t$ turns either equation into a sampler. DDPM ancestral sampling is a discretization of Eq. (21), and DDIM is a discretization of Eq. (20) [318].

Schedules are reparametrizations. Dividing Eq. (1) by α_t gives $\boldsymbol{x}_t/\alpha_t = \boldsymbol{x}_1 + (\sigma_t/\alpha_t)\boldsymbol{x}_0$. Every Gaussian path is therefore the variance-exploding path $\boldsymbol{y} + \sigma\boldsymbol{n}$ in rescaled coordinates, indexed by the noise-to-signal ratio $\sigma = \sigma_t/\alpha_t = e^{-\lambda_t/2}$. EDM [152] makes this explicit and writes the probability-flow ODE as $\mathrm{d}\boldsymbol{y}/\mathrm{d}\sigma = -\sigma\nabla_{\boldsymbol{y}}\log p(\boldsymbol{y};\sigma)$. Two schedules with the same range of λ define the same family of distributions. What changes is how the integration steps are spaced and how training effort is distributed over noise levels, which Section 2.5 discusses. EDM2 [153] builds on this view to analyze training dynamics.

Advanced. How accurate does the score need to be? Under mild assumptions on p_{data}, the KL divergence between the data and the distribution sampled by a discretized reverse SDE is bounded by three terms: the L^2 score error accumulated along the path, the discretization error, and the error from starting at $\mathcal{N}(0, I)$

instead of the exact p_0 [56]. No log-concavity or smoothness of p_{data} is required, so "sampling is as easy as learning the score". Benton et al. [19] sharpen the dependence on dimension and show that the number of steps needed for a fixed accuracy grows nearly linearly in d. Similar bounds hold for the probability-flow ODE but require control of the score's Jacobian, because a deterministic sampler cannot wash out errors through noise.

2.4 Flows: the continuity equation and flow matching

The probability-flow ODE shows that a deterministic velocity field can carry noise to data. Flow matching [202], rectified flow [214] and stochastic interpolants [4, 5] start from this end. They specify the path p_t directly through Eq. (1) and learn the velocity that generates it, without passing through SDEs.

The continuity equation. A time-dependent velocity field $\boldsymbol{u}_t$ defines an ODE $\dot{\boldsymbol{x}}_t = \boldsymbol{u}_t(\boldsymbol{x}_t)$. If $\boldsymbol{x}_0 \sim p_0$, the density p_t of $\boldsymbol{x}_t$ satisfies the continuity equation

$$\partial_t p_t + \nabla \cdot (p_t \, \boldsymbol{u}_t) = 0, \tag{22}$$

which is Eq. (17) without diffusion. We say that $\boldsymbol{u}_t$ *generates* the path p_t. Many velocity fields generate the same path, since any divergence-free addition to $p_t \boldsymbol{u}_t$ leaves Eq. (22) unchanged, so we need a canonical choice.

The marginal velocity. Differentiating the interpolant along a single pair $(\boldsymbol{x}_0, \boldsymbol{x}_1)$ gives the conditional velocity $\dot{\boldsymbol{x}}_t = \dot{\alpha}_t \boldsymbol{x}_1 + \dot{\sigma}_t \boldsymbol{x}_0$. Each pair moves along its own path, and many paths pass through any given point. The velocity that generates the marginal path averages over them:

$$\boldsymbol{u}_t(\boldsymbol{x}) = \mathbb{E}\big[\dot{\alpha}_t \boldsymbol{x}_1 + \dot{\sigma}_t \boldsymbol{x}_0 \,\big|\, \boldsymbol{x}_t = \boldsymbol{x}\big]. \tag{23}$$

Proposition 2.2 (Marginalization)**.** *The field Eq. (23) satisfies the continuity equation Eq. (22) for the path p_t defined by Eq. (1).*

Derivation sketch. For a test function φ, the chain rule gives $\frac{\mathrm{d}}{\mathrm{d}t}\mathbb{E}\varphi(\boldsymbol{x}_t) = \mathbb{E}[\nabla\varphi(\boldsymbol{x}_t) \cdot \dot{\boldsymbol{x}}_t]$. Conditioning on $\boldsymbol{x}_t$ replaces $\dot{\boldsymbol{x}}_t$ by $\boldsymbol{u}_t(\boldsymbol{x}_t)$, so $\frac{\mathrm{d}}{\mathrm{d}t}\int \varphi \, p_t = \int \nabla\varphi \cdot \boldsymbol{u}_t \, p_t$. Integrating by parts gives the weak form of Eq. (22). Details are in Appendix A.5.

Flow matching. The marginal velocity is a conditional expectation, so Lemma 2.1 applies again. The flow matching loss

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t,\boldsymbol{x}_0,\boldsymbol{x}_1} \big\| \boldsymbol{u}_\theta(\boldsymbol{x}_t, t) - (\dot{\alpha}_t \boldsymbol{x}_1 + \dot{\sigma}_t \boldsymbol{x}_0) \big\|^2 \tag{24}$$

regresses onto the tractable conditional velocity and is minimized by the marginal velocity Eq. (23). For the linear path $\alpha_t = t$, $\sigma_t = 1 - t$ of rectified flow the label is simply $\boldsymbol{x}_1 - \boldsymbol{x}_0$. Training is simulation-free: it needs one noise sample, one data sample and one time per step, and never integrates the ODE. Sampling integrates $\dot{\boldsymbol{x}} = \boldsymbol{u}_\theta(\boldsymbol{x}, t)$ from $t = 0$ to $t = 1$.

Velocity and score are the same object. By Eq. (13), the marginal velocity is a linear combination of $\hat{\boldsymbol{x}}_1$ and $\hat{\boldsymbol{x}}_0$, and Tweedie's formula expresses both through the score. Substituting gives the velocity–score identity (Appendix A.6)

$$\boldsymbol{u}_t(\boldsymbol{x}) = \frac{\dot{\alpha}_t}{\alpha_t}\boldsymbol{x} + \tfrac{1}{2}\sigma_t^2 \dot{\lambda}_t \nabla \log p_t(\boldsymbol{x}) = \frac{\dot{\alpha}_t}{\alpha_t}\boldsymbol{x} + \Big(\sigma_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \sigma_t \dot{\sigma}_t\Big) \nabla \log p_t(\boldsymbol{x}). \tag{25}$$

This is exactly the probability-flow ODE Eq. (20). For Gaussian paths, flow matching and score-based diffusion learn the same field up to a known affine transformation, and their differences reduce to the choice of schedule, the parametrization of the network output and the weighting over time (Table 3). The perceived gap between "diffusion" and "flow" models is therefore mostly a gap in conventions.

One marginal path, a family of samplers. The velocity–score identity also shows how to add noise to a flow without changing its marginals.

Proposition 2.3 (SDEs with prescribed marginals)**.** *For any $\varepsilon_t \geq 0$, the SDE*

$$\mathrm{d}\boldsymbol{x}_t = \Big(\boldsymbol{u}_t(\boldsymbol{x}_t) + \tfrac{1}{2}\varepsilon_t^2 \nabla \log p_t(\boldsymbol{x}_t)\Big)\mathrm{d}t + \varepsilon_t \,\mathrm{d}\boldsymbol{W}_t, \qquad \boldsymbol{x}_0 \sim p_0, \tag{26}$$

has marginals p_t for all $t \in [0,1]$.

Derivation sketch. By Eq. (17), the extra drift contributes $-\nabla\cdot(\frac{1}{2}\varepsilon_t^2 p_t \nabla \log p_t) = -\frac{1}{2}\varepsilon_t^2 \Delta p_t$, which cancels the diffusion term $+\frac{1}{2}\varepsilon_t^2\Delta p_t$. What remains is the continuity equation Eq. (22).

The choice $\varepsilon \equiv 0$ is the probability-flow ODE. The choice $\varepsilon_t^2 = \sigma_t^2 \dot{\lambda}_t$ recovers the reverse-time SDE Eq. (21). This particular schedule has a property that matters for reward fine-tuning: the noise injected along the path is just enough to make the final sample independent of the initial noise. It is the "memoryless" schedule on which Adjoint Matching is built [79], and we return to it in Sections 2.6 and 4. Intermediate choices interpolate between the two. Larger ε corrects errors in the learned field by contracting towards high-density regions, at the cost of needing more steps, while $\varepsilon = 0$ allows high-order ODE solvers and the few-step methods of Section 5. A single trained model therefore supports a whole family of samplers.

Straight conditional paths, curved marginal paths. For the linear path each conditional trajectory is a straight line from $\boldsymbol{x}_0$ to $\boldsymbol{x}_1$, but different lines cross. Trajectories of an ODE cannot cross, so the marginal velocity averages the directions of all lines through a point, and the resulting paths curve (Fig. 4). Curvature is what makes few-step sampling inaccurate, and reducing it is the purpose of the couplings in Section 2.6 and of reflow in Section 5.

Advanced. Several recent works extend this picture. Stochastic interpolants allow arbitrary couplings of $\boldsymbol{x}_0$ and $\boldsymbol{x}_1$ and non-Gaussian sources [5]. Flux Matching [264] learns vector fields that need not be gradients of a log-density and asks only that their flux matches that of the data, which generalizes score matching. Shlenskii et al. [310] relate conditional flow matching to interaction field matching, a descendant of electrostatic and Poisson-flow generative models. Drift models aim at one-step generation, and Ma et al. [230] combine them with flow matching to recover test-time scaling. Ranganath and Singhal [283] give a measure-theoretic treatment that unifies diffusion, score-based and flow matching models, and Lipman et al. [203] provide a detailed tutorial on flow matching.

2.5 The likelihood view: ELBO, VDM and loss weighting

The previous sections justified denoising losses through regression: their minimizers are the right conditional expectations. They did not say how to weight the noise levels against each other, and every weighting has the same minimizer when the network is flexible enough. In practice the network is not, and the weighting decides where its capacity goes. This section derives the one weighting that has a probabilistic meaning, the evidence lower bound, and then shows that the weightings used in practice are close relatives of it.

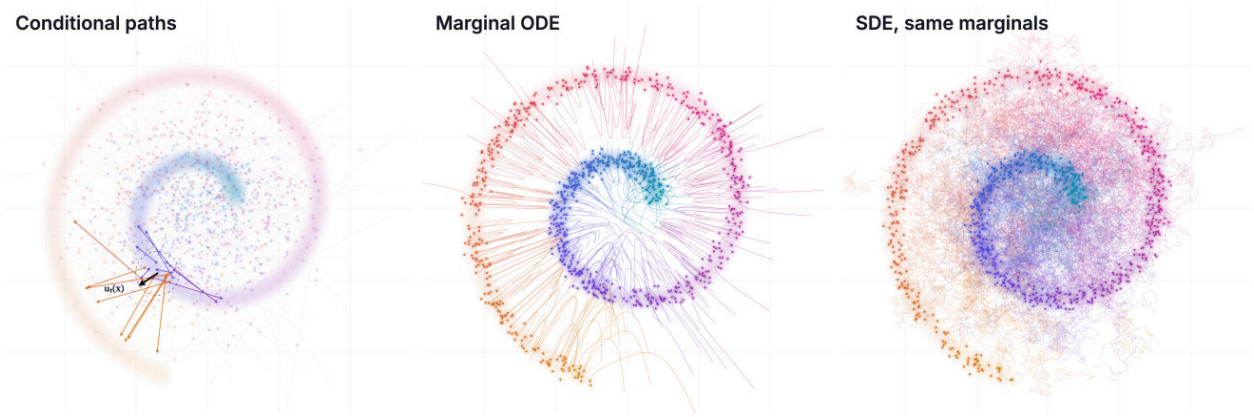


Figure 4: **Flow matching on an exact 2D mixture** (Animation A1). Left: conditional paths pair noise with data samples; they are straight but cross. Middle: the marginal velocity field averages them, giving curved paths that never cross. Right: an SDE with the same marginals yields the same distribution along stochastic paths. All fields are computed in closed form.

Diffusion as a hierarchical VAE. Fix a grid $0 = t_0 < t_1 < \cdots < t_N = 1$ and treat $\boldsymbol{x}_{t_0}, \ldots, \boldsymbol{x}_{t_{N-1}}$ as latent variables. The *encoder* is the corruption process: it is fixed, has no parameters, and maps a data point to progressively noisier versions with $q(\boldsymbol{x}_t \mid \boldsymbol{x}_1) = \mathcal{N}(\alpha_t \boldsymbol{x}_1, \sigma_t^2 I)$. The *decoder* is the generative chain $p_\theta(\boldsymbol{x}_{t_i} \mid \boldsymbol{x}_{t_{i-1}})$ that removes noise one step at a time, starting from $p(\boldsymbol{x}_{t_0}) = \mathcal{N}(0, I)$. The standard variational argument [123, 316] bounds the negative log-likelihood by

$$-\log p_\theta(\boldsymbol{x}_1) \le \underbrace{\mathrm{KL}\big(q(\boldsymbol{x}_{t_0} \mid \boldsymbol{x}_1) \,\|\, p(\boldsymbol{x}_{t_0})\big)}_{\text{prior}} + \sum_{i=1}^{N-1} \mathbb{E}_q \underbrace{\mathrm{KL}\big(q(\boldsymbol{x}_{t_i} \mid \boldsymbol{x}_{t_{i-1}}, \boldsymbol{x}_1) \,\|\, p_\theta(\boldsymbol{x}_{t_i} \mid \boldsymbol{x}_{t_{i-1}})\big)}_{\text{denoising step } i} + \underbrace{\mathbb{E}_q\big[-\log p_\theta(\boldsymbol{x}_1 \mid \boldsymbol{x}_{t_{N-1}})\big]}_{\text{reconstruction}}. \tag{27}$$

The key fact is that the encoder posterior $q(\boldsymbol{x}_t \mid \boldsymbol{x}_s, \boldsymbol{x}_1)$ for $s < t$, the distribution of a less noisy state given a noisier one and the data, is Gaussian with a mean that is linear in $\boldsymbol{x}_1$. If the decoder uses the same Gaussian with $\boldsymbol{x}_1$ replaced by a prediction $\hat{\boldsymbol{x}}_1(\boldsymbol{x}_s, s)$, each denoising term is a squared error,

$$\mathrm{KL}\big(q(\boldsymbol{x}_t \mid \boldsymbol{x}_s, \boldsymbol{x}_1) \,\|\, p_\theta(\boldsymbol{x}_t \mid \boldsymbol{x}_s)\big) = \tfrac{1}{2}\big(\mathrm{SNR}(t) - \mathrm{SNR}(s)\big)\,\big\|\boldsymbol{x}_1 - \hat{\boldsymbol{x}}_1(\boldsymbol{x}_s, s)\big\|^2, \qquad \mathrm{SNR}(t) = e^{\lambda_t} = \frac{\alpha_t^2}{\sigma_t^2}. \tag{28}$$

The derivation is in Appendix A.7. Training DDPM by maximum likelihood is therefore data prediction, weighted by how much signal-to-noise ratio each step adds.

The continuous-time limit and its invariance. As the grid is refined, the sum in Eq. (27) becomes an integral, $\sum_i \frac{1}{2}(\mathrm{SNR}(t_i) - \mathrm{SNR}(t_{i-1}))\|\cdot\|^2 \to \frac{1}{2}\int_0^1 \frac{\mathrm{d}}{\mathrm{d}t} e^{\lambda_t} \|\cdot\|^2 \,\mathrm{d}t$. Changing variables from t to λ and using $e^\lambda \|\boldsymbol{x}_1 - \hat{\boldsymbol{x}}_1\|^2 = \|\boldsymbol{x}_0 - \hat{\boldsymbol{x}}_0\|^2$ from Eq. (14) gives the variational diffusion model (VDM) objective [162],

$$\mathcal{L}_\infty(\boldsymbol{x}_1) = \tfrac{1}{2}\int_{\lambda_{\min}}^{\lambda_{\max}} \mathbb{E}_{\boldsymbol{x}_0}\big\|\boldsymbol{x}_0 - \hat{\boldsymbol{x}}_0(\boldsymbol{x}_\lambda, \lambda)\big\|^2 \,\mathrm{d}\lambda, \tag{29}$$

where $\boldsymbol{x}_\lambda$ denotes the noisy sample at log-SNR λ. The ELBO is *noise prediction with uniform weight in log-SNR*. Two consequences follow. First, the schedule enters only through the endpoints $\lambda_{\min}$ and $\lambda_{\max}$. Between them, any monotone schedule gives the same objective, because the rescaled input $\boldsymbol{x}_t/\alpha_t = \boldsymbol{x}_1 + e^{-\lambda_t/2}\boldsymbol{x}_0$ depends on t only through λ_t (this is the reparametrization argument of Section 2.3). The schedule still matters for the variance of the Monte Carlo estimate of the integral, which is why VDM learns it to reduce variance. Second, the same weighting appears from the SDE side: the likelihood weighting $g^2 = \sigma^2\dot{\lambda}$ of Song et al. [323] turns the score-matching loss into a bound on the KL divergence, and with $\hat{\boldsymbol{x}}_0 = -\sigma\boldsymbol{s}$ it reduces to Eq. (29).

Weighting is the real design choice. Every loss in common use can be written as a weighted version of Eq. (29),

$$\mathcal{L}_w = \tfrac{1}{2}\int w(\lambda)\,\mathbb{E}\|\boldsymbol{x}_0 - \hat{\boldsymbol{x}}_0(\boldsymbol{x}_\lambda, \lambda)\|^2\,\mathrm{d}\lambda, \tag{30}$$

and different methods differ in w alone [161]. The weight collects two choices: the density $\pi(t)$ from which training times are drawn, and the prediction target, which multiplies the loss by the factors in Table 3. As an example, take flow matching with the linear path $\alpha_t = t$, $\sigma_t = 1-t$. Then $\boldsymbol{u} = \boldsymbol{x}_1 - \boldsymbol{x}_0$, and a short calculation gives $\|\hat{\boldsymbol{u}} - \boldsymbol{u}\|^2 = \|\hat{\boldsymbol{x}}_0 - \boldsymbol{x}_0\|^2/t^2$ and $\mathrm{d}\lambda/\mathrm{d}t = 2/(t(1-t))$. Substituting into $\mathbb{E}_{t\sim\pi}\|\hat{\boldsymbol{u}} - \boldsymbol{u}\|^2$ yields

$$w(\lambda) = \pi(t)\,\frac{1-t}{t} = \pi(t)\,e^{-\lambda/2}, \qquad t = t(\lambda). \tag{31}$$

With uniform π, rectified flow puts weight $e^{-\lambda/2}$ on each log-SNR, which grows towards the noisy end. The logit-normal time sampling of Stable Diffusion 3 [84] multiplies this by a bump centred on intermediate noise levels, and the log-normal noise distribution and loss weight of EDM [152] have the same effect in other coordinates. Measured against the ELBO, all of these discount the least noisy levels, where the error concerns details that are hard to see, and spend capacity on the intermediate levels that decide global structure.

Kingma and Gao [161] give the weighting a likelihood interpretation. If w is monotonically decreasing in λ, then up to a constant $\mathcal{L}_w$ equals an average over noise levels λ', drawn with density proportional to $-w'(\lambda')$, of the ELBO of the data perturbed with Gaussian noise at level λ'. A monotone weighting is therefore maximum likelihood with Gaussian data augmentation. The uniform rectified flow weight in Eq. (31) is monotone, while bump-shaped weights such as the logit-normal one are not, and they fall outside this interpretation.

Exact likelihoods through the probability-flow ODE. The ELBO is only a bound, but the deterministic sampler also has an exact likelihood. Along a trajectory of the ODE $\dot{\boldsymbol{x}}_t = \boldsymbol{u}_t(\boldsymbol{x}_t)$, the continuity equation (Eq. (22)) implies

$$\frac{\mathrm{d}}{\mathrm{d}t}\log p_t(\boldsymbol{x}_t) = \partial_t \log p_t + \boldsymbol{u}_t \cdot \nabla \log p_t = -\nabla\cdot \boldsymbol{u}_t(\boldsymbol{x}_t), \tag{32}$$

because the continuity equation can be expanded as $\partial_t p_t = -p_t\nabla\cdot\boldsymbol{u}_t - \boldsymbol{u}_t\cdot\nabla p_t$. Integrating from $t=0$ to $t=1$ gives $\log p_1(\boldsymbol{x}_1) = \log p_0(\boldsymbol{x}_0) - \int_0^1 \nabla\cdot\boldsymbol{u}_t(\boldsymbol{x}_t)\,\mathrm{d}t$, where $\boldsymbol{x}_0$ is obtained by integrating the ODE backwards from $\boldsymbol{x}_1$. This is the instantaneous change of variables of continuous normalizing flows [55]. The divergence is estimated with Hutchinson's trace estimator $\nabla\cdot\boldsymbol{u} = \mathbb{E}_{\boldsymbol{\epsilon}}[\boldsymbol{\epsilon}^\top(\partial\boldsymbol{u}/\partial\boldsymbol{x})\boldsymbol{\epsilon}]$, which needs one vector–Jacobian product per sample [106]. Song et al. [322] use this to report bits per dimension for score-based models, and Huang et al. [130] relate the ODE likelihood to the variational bound of the SDE. The ODE likelihood belongs to the deterministic sampler and is not in general the likelihood of the reverse SDE.

Changing the target instead of the weight. All targets so far describe the instantaneous field at a single time, so a sampler needs many small steps to follow it. A different family of objectives learns the result of following the field over an interval. The object is the *flow map* $X_{s\to t}(\boldsymbol{x})$, which moves a point from time s to time t along the probability-flow ODE. Consistency models learn the map to the data end, $X_{s\to 1}$ [319, 325]. Shortcut models condition the network on the step size [93]. MeanFlow learns the average velocity over an interval, using an identity between the average and the instantaneous velocity that follows from differentiating the definition of the average [102]. Inductive moment matching [459] and flow map matching [31, 32] give further objectives. All of them can be trained from scratch without a teacher, because the instantaneous velocity is available through the conditional flow matching target. They are the subject of Section 5.

2.6 Couplings, optimal transport, Schrödinger bridges and control

So far noise and data were paired independently, and the only stochasticity in sampling came from the free noise level in Proposition 2.3. This section relaxes both choices. Changing the pairing leads to optimal transport. Changing the reference process leads to Schrödinger bridges. Changing the target distribution at the end of the path leads to stochastic optimal control, which is the mathematical core of reward fine-tuning in Section 4 and of guided sampling in Section 6.

Couplings are a free choice. Let $\pi(\boldsymbol{x}_0, \boldsymbol{x}_1)$ be any joint distribution with marginals $\mathcal{N}(0, I)$ and p_{data}, and set $\boldsymbol{x}_t = \alpha_t \boldsymbol{x}_1 + \sigma_t \boldsymbol{x}_0$ with $(\boldsymbol{x}_0, \boldsymbol{x}_1) \sim \pi$. The proof of Proposition 2.2 never used independence, so the conditional expectation $\boldsymbol{u}_t(\boldsymbol{x}) = \mathbb{E}_\pi[\dot\alpha_t \boldsymbol{x}_1 + \dot\sigma_t \boldsymbol{x}_0 \mid \boldsymbol{x}_t = \boldsymbol{x}]$ still transports $\mathcal{N}(0, I)$ to p_{data}, and flow matching still learns it. What breaks is the link to the score: the conditional law of $\boldsymbol{x}_t$ given $\boldsymbol{x}_1$ is no longer Gaussian, so Eqs. (11) and (25) do not hold. The coupling is chosen to make the field simpler. Minibatch optimal transport pairs the noise and data samples of each batch by solving a small assignment problem with cost $\|\boldsymbol{x}_1 - \boldsymbol{x}_0\|^2$ [274, 345]. As the batch grows the coupling approaches the optimal one, the conditional paths cross less, the regression target has lower variance, and the learned ODE has straighter trajectories that tolerate fewer integration steps.

The dynamic view of optimal transport. Why do optimal couplings give straight, non-crossing paths? Benamou and Brenier [17] showed that the squared Wasserstein distance is the least kinetic energy needed to move one density into the other,

$$W_2^2(\mu_0, \mu_1) = \min_{(\rho_t, \boldsymbol{u}_t)} \int_0^1 \int \|\boldsymbol{u}_t(\boldsymbol{x})\|^2 \rho_t(\boldsymbol{x})\, \mathrm{d}\boldsymbol{x}\, \mathrm{d}t \quad \text{s.t.} \quad \partial_t \rho_t + \nabla \cdot (\rho_t \boldsymbol{u}_t) = 0,\ \rho_0 = \mu_0,\ \rho_1 = \mu_1. \tag{33}$$

The constraint is the continuity equation Eq. (22), so this is a flow matching problem with an extra criterion that selects one velocity field among the many that generate a path. The minimizer moves every particle along a straight line at constant speed from $\boldsymbol{x}_0$ to its optimal partner $T(\boldsymbol{x}_0)$, and these lines do not cross at intermediate times. Flow matching with the linear path and the optimal coupling reproduces exactly this displacement interpolation, and its conditional and marginal velocities coincide. Its time-t marginals interpolate in the Wasserstein sense, which is also the geometry of the JKO scheme in Section 2.1.

Rectification never increases transport cost. With the independent coupling, the marginal ODE defines a new coupling $(\boldsymbol{X}_0, \boldsymbol{X}_1)$ by integrating from noise to data. Rectified flow [214] retrains

on this coupling. For every convex cost c the new coupling is no worse than the old one, and the proof is two applications of Jensen's inequality:

$$\mathbb{E}\, c(\boldsymbol{X}_1 - \boldsymbol{X}_0) = \mathbb{E}\, c\Big(\int_0^1 \boldsymbol{u}_t(\boldsymbol{X}_t)\,\mathrm{d}t\Big) \le \int_0^1 \mathbb{E}\, c(\boldsymbol{u}_t(\boldsymbol{X}_t))\,\mathrm{d}t \le \int_0^1 \mathbb{E}\, c(\boldsymbol{x}_1 - \boldsymbol{x}_0)\,\mathrm{d}t = \mathbb{E}\, c(\boldsymbol{x}_1 - \boldsymbol{x}_0). \quad (34)$$

The second inequality uses $\boldsymbol{u}_t(\boldsymbol{x}) = \mathbb{E}[\boldsymbol{x}_1 - \boldsymbol{x}_0 \mid \boldsymbol{x}_t = \boldsymbol{x}]$ for the linear path and the fact that $\boldsymbol{X}_t$ and $\boldsymbol{x}_t$ have the same law. Repeating the procedure straightens the paths further, which is the basis of reflow in Section 5.

Schrödinger bridges. Optimal transport is deterministic. Its stochastic counterpart replaces kinetic energy by relative entropy with respect to a reference process. Let Q be the law of a Brownian motion $\mathrm{d}\boldsymbol{x}_t = \varepsilon\,\mathrm{d}\boldsymbol{W}_t$ started from $\mathcal{N}(0, I)$. The Schrödinger bridge problem is

$$\min_P \ \mathrm{KL}(P \,\|\, Q) \quad \text{over path measures } P \text{ with } P_0 = \mathcal{N}(0, I),\ P_1 = p_{\mathrm{data}}. \quad (35)$$

The KL divergence between path measures splits into a part for the endpoints and a part for the paths given their endpoints, $\mathrm{KL}(P\|Q) = \mathrm{KL}(P_{01}\|Q_{01}) + \mathbb{E}_{P_{01}}\mathrm{KL}(P^{\boldsymbol{x}_0\boldsymbol{x}_1}\|Q^{\boldsymbol{x}_0\boldsymbol{x}_1})$. The second term vanishes when the bridges of P are the Brownian bridges of Q, $\boldsymbol{x}_t = (1-t)\boldsymbol{x}_0 + t\boldsymbol{x}_1 + \varepsilon\sqrt{t(1-t)}\,\boldsymbol{z}$. The first term reduces, after expanding $\log Q_{01}$ and dropping terms fixed by the marginals, to

$$\min_\pi \ \mathbb{E}_\pi\|\boldsymbol{x}_1 - \boldsymbol{x}_0\|^2 - 2\varepsilon^2 H(\pi), \quad (36)$$

which is entropy-regularized optimal transport [177]. The Schrödinger bridge is therefore an interpolant with an entropic OT coupling and Brownian-bridge noise. As $\varepsilon \to 0$ it recovers the Benamou–Brenier solution. Diffusion Schrödinger bridges solve Eq. (35) by iterative proportional fitting, alternately matching each marginal with learned forward and backward SDEs [33]. Diffusion Schrödinger bridge matching replaces this with iterative Markovian fitting, built from bridge-matching regressions of the kind used in flow matching [309]. Forward–backward SDE theory gives a likelihood objective for the same problem [57].

Changing the target: stochastic optimal control. Suppose a pretrained model defines a base process $\mathrm{d}\boldsymbol{x}_t = \boldsymbol{b}_t(\boldsymbol{x}_t)\,\mathrm{d}t + \varepsilon_t\,\mathrm{d}\boldsymbol{W}_t$ from $\mathcal{N}(0, I)$ to $p_1^{\mathrm{base}} \approx p_{\mathrm{data}}$, for instance one member of the family in Proposition 2.3. We want samples that score highly under a reward r without straying far from the base model. Add a control $\boldsymbol{c}$ to the drift, $\mathrm{d}\boldsymbol{x}_t = (\boldsymbol{b}_t + \varepsilon_t\boldsymbol{c}_t)\,\mathrm{d}t + \varepsilon_t\,\mathrm{d}\boldsymbol{W}_t$, and call the path measures $P^{\boldsymbol{c}}$ and P^{base}. The Girsanov theorem turns their KL divergence into a quadratic control cost,

$$\mathrm{KL}(P^{\boldsymbol{c}} \,\|\, P^{\mathrm{base}}) = \mathbb{E}_{P^{\boldsymbol{c}}} \int_0^1 \tfrac{1}{2}\|\boldsymbol{c}_t(\boldsymbol{x}_t)\|^2\,\mathrm{d}t. \quad (37)$$

The KL-regularized objective $\max_{\boldsymbol{c}} \mathbb{E}_{P^{\boldsymbol{c}}}[r(\boldsymbol{x}_1)] - \beta\,\mathrm{KL}(P^{\boldsymbol{c}}\|P^{\mathrm{base}})$ is therefore a stochastic optimal control problem. It can be rewritten as $\min_{\boldsymbol{c}} \mathrm{KL}(P^{\boldsymbol{c}}\|P^\star) + \mathrm{const}$, where

$$\frac{\mathrm{d}P^\star}{\mathrm{d}P^{\mathrm{base}}}(\boldsymbol{x}_{[0,1]}) = \frac{e^{r(\boldsymbol{x}_1)/\beta}}{Z}. \quad (38)$$

The optimal path measure is the base path measure reweighted by the exponentiated terminal reward. Conditioning a diffusion on such a reweighting is Doob's h-transform, and the optimal control is

$$\boldsymbol{c}_t^\star(\boldsymbol{x}) = \varepsilon_t \nabla \log h_t(\boldsymbol{x}), \qquad h_t(\boldsymbol{x}) = \mathbb{E}_{P^{\mathrm{base}}}\big[e^{r(\boldsymbol{x}_1)/\beta} \,\big|\, \boldsymbol{x}_t = \boldsymbol{x}\big]. \quad (39)$$

The function h_t solves the backward Kolmogorov equation of the base process, and $\beta \log h_t$ is the value function, which solves the Hamilton–Jacobi–Bellman equation. Three consequences organize the rest of the survey.

The fine-tuning target is an energy-based model. The terminal marginal of $P^\star$ is $p^\star(\boldsymbol{x}) \propto p_1^{\text{base}}(\boldsymbol{x})\, e^{r(\boldsymbol{x})/\beta}$, an EBM whose energy is the base energy minus the scaled reward. There is one subtlety. The control cannot change the initial noise, so the controlled process reaches $p^\star$ only if h_0 is constant, which means $\boldsymbol{x}_0$ and $\boldsymbol{x}_1$ must be independent under the base process. The memoryless noise level $\varepsilon_t^2 = \sigma_t^2 \dot{\lambda}_t$ of Section 2.4 has exactly this property, and that is why Adjoint Matching fine-tunes with it [79]. Section 4 develops this view [78, 80, 193, 352].

Guidance estimates h_t at sampling time. Without fine-tuning, one can approximate h_t while sampling. Replacing the expectation in Eq. (39) by its value at the Tweedie estimate gives $h_t(\boldsymbol{x}) \approx e^{r(\hat{\boldsymbol{x}}_1(\boldsymbol{x},t))/\beta}$ and the drift correction $\varepsilon_t^2 \nabla_{\boldsymbol{x}} r(\hat{\boldsymbol{x}}_1)/\beta$, which is the reward analogue of diffusion posterior sampling [66]. Estimating h_t with weighted particles instead gives Feynman–Kac reweighting and sequential Monte Carlo [313, 314, 388]. Both are treated in Section 6.

Sampling from unnormalized densities. If the base process is a simple diffusion with a known terminal law and the reward is chosen so that $p^\star \propto e^{-E}$ for a given energy E, the same control problem trains a sampler for the Boltzmann density e^{-E} without any data [21, 350, 438]. The energy-based starting point of Section 2.1 thus reappears at the end of the chapter, now as the target of a controlled diffusion rather than the object of learning.

2.7 Why diffusion models generalize

Every objective in this chapter is a regression whose minimizer is known. This raises a puzzle. Training uses a finite set $\{\boldsymbol{y}_1, \dots, \boldsymbol{y}_n\}$, so the loss is minimized not by the score of p_{data} but by the score of the empirical distribution $\hat{p} = \frac{1}{n}\sum_i \delta_{\boldsymbol{y}_i}$. That score can be written down, and it does not generalize.

The empirical optimum memorizes. Noising $\hat{p}$ gives a Gaussian mixture with one component per training point, $\hat{p}_t = \frac{1}{n}\sum_i \mathcal{N}(\alpha_t \boldsymbol{y}_i, \sigma_t^2 I)$. Its denoiser follows from Lemma 2.1 and the closed forms in Appendix A.8:

$$\hat{\boldsymbol{x}}_1(\boldsymbol{x}, t) = \sum_{i=1}^{n} w_i(\boldsymbol{x}, t)\, \boldsymbol{y}_i, \qquad w_i(\boldsymbol{x}, t) = \operatorname{softmax}_i\Big(- \frac{\|\boldsymbol{x} - \alpha_t \boldsymbol{y}_i\|^2}{2\sigma_t^2} \Big). \tag{40}$$

At high noise the weights are nearly uniform and the denoiser returns the mean of the training set. As $\sigma_t \to 0$ the softmax concentrates on the nearest training point, and the probability-flow ODE with this exact field carries almost every initial noise to one of the $\boldsymbol{y}_i$. A model that minimized the training loss perfectly would therefore only reproduce its training data. Since trained models do produce new samples, generalization must come from what the network fails to fit: its limited capacity, its architecture and the implicit regularization of training. This is the generative counterpart of the question why overparameterized classifiers generalize.

Evidence that networks learn a density. Kadkhodaie et al. [149] trained two networks on disjoint subsets of the same image dataset. With small subsets each network reproduced its own training images. With large enough subsets the two networks produced nearly the same image from the same initial noise, so they had learned nearly the same score, and that score could not be the empirical one of either subset. Analysing the trained denoisers, they found shrinkage in bases adapted to the image, with oscillating harmonic structure along contours, which they call

geometry-adaptive harmonic representations. Kamb and Ganguli [150] explain much of the behaviour of convolutional models from two constraints alone, locality and translation equivariance. The optimal score under these constraints assembles each output from local patches of different training images, which produces new images without any learning beyond the constraints. Memorization is nevertheless real at scale: training images can be extracted from large text-to-image models [40, 317], and it becomes more likely with smaller datasets and with conditioning, even on uninformative labels [107].

Theory: when does memorization set in? Biroli et al. [26] study the exact empirical score in the limit of large dimension and many samples. Sampling passes through two transitions. At the *speciation* time the trajectory commits to a class, and this time can be read off the spectrum of the data covariance. At the later *collapse* time it is captured by a single training point, and this time is set by an excess entropy of the data. How the collapse time depends on d and n quantifies the curse of dimensionality for an exactly trained model. Several 2026 results sharpen this picture.

- For linear generative models, Maillard and Goldt [237] compute the transition from memorization to generalization exactly. Models trained on disjoint data converge to each other once n grows linearly with d, but recovering the principal latent factors of the data is a separate, sharp transition. Agreement between two models, the criterion of Kadkhodaie et al. [149], therefore measures only one aspect of generalization.
- Merger and Goldt [245] treat memorization as a property of individual training points. Through the connection to kernel density estimation, a point is memorized when its neighbourhood is sparsely covered by the rest of the data, so one model can memorize isolated samples and generalize in dense regions at the same time.
- Hunt et al. [134] study analytically tractable models in which each pixel sees restricted information about the noisy input. Such a model memorizes when the mutual information between its observation and the training set exceeds $\log n$ and generalizes otherwise. Spatially local versions track trained networks early in training.
- Latourelle-Vigeant et al. [169] derive exact training trajectories for denoising score matching with kernels in the regime $n \asymp d$. They find three phases dominated by a generalizing spectral estimator, an interpolating estimator and a memorizing empirical Bayes estimator.
- For conditional models with random features, Kim and Lee [158] show that more informative conditions lead to memorization at smaller model widths. They also find a "malign generalization" regime, in which wider models predict the condition-dependent mean better while reducing the diversity within each condition.

Consequences. The empirical-optimum view makes three practical points. Memorization is local, so audits should look at rare and duplicated training examples rather than averages. Anything that sharpens the effective target, such as strong conditioning, guidance and long fine-tuning on small datasets, moves a model towards the regime of Eq. (40). This links the present section to the overfitting and reward-hacking failures of Section 4. Finally, the closed form Eq. (40) is cheap to evaluate in two dimensions. A planned companion animation compares it with a smoothed score on the same points and shows memorization appearing as the smoothing shrinks.

2.8 Representations and architectures

The mathematics of the previous sections holds in any Euclidean space. In practice two engineering choices decide what a model can learn at a given cost: the space in which the diffusion runs, and the network that parametrizes the field. This section treats both, and shows that several recent design rules have a simple reading in terms of the denoiser $\hat{\boldsymbol{x}}_1$.

Latent diffusion. A 1024×1024 RGB image has about three million coordinates, most of which encode details that a squared-error loss weights heavily but humans barely see. Latent diffusion [287] first trains an autoencoder $(\mathcal{E}, \mathcal{D})$ with a reconstruction, perceptual and adversarial loss, and then runs diffusion on $\boldsymbol{z} = \mathcal{E}(\boldsymbol{x})$. The original autoencoder downsamples by $f = 8$ in each spatial direction and keeps four channels, which reduces the dimension by a factor of 48. Deep compression autoencoders reach $f = 32$ or $f = 64$ with more channels [52], and video autoencoders compress space and time jointly by factors of several hundred [113]. The diffusion model then sees a sequence of

$$N = \frac{H\,W\,(T/f_T)}{f^2 p^2} \tag{41}$$

tokens for an image or video of height H, width W and T frames, temporal compression f_T and patch size p. Attention costs $O(N^2)$, so the compression factor sets the price of every training step. Figure 5 traces these factors for a small example.

What makes a latent space easy to model? Better reconstruction does not imply better generation. Adding channels improves reconstruction, but past a point the generative model gets worse, because the extra dimensions carry high-frequency content that is hard to predict from noise. Yue et al. [424] construct controlled tokenizer variants and find that generation quality follows three properties of the latent manifold, namely coherent spatial structure, local continuity and global semantic organization, more closely than it follows reconstruction fidelity. Their prior-aligned autoencoder shapes the latent space towards these properties directly. Unified Latents [121] take the likelihood view of Section 2.5 instead. They regularize the encoder with a diffusion prior and tie the encoder's output noise to the prior's minimum noise level, which turns the objective into an upper bound on the bitrate of the latent.

Semantic latents. A more radical option is to skip autoencoder training and diffuse in the feature space of a pretrained representation model. Representation autoencoders (RAE) keep a self-supervised encoder such as DINOv2 frozen and train only a decoder [452]. The latents are high-dimensional, so the network must be at least as wide as the token dimension and the noise schedule must be shifted with the dimension, a consequence of the fact that the same σ destroys less information in more coordinates. At text-to-image scale with a SigLIP-2 encoder, the dimension-dependent schedule remains necessary while the other architectural changes stop mattering [348]. Singh et al. [312] improve the encoder side by summing the last few layers of the representation model. UniSpace [404] argues that the final tokens of a semantic ViT lose detail because of the patch embedding rather than the transformer blocks, and reparametrizes the patches so that one representation serves understanding, generation and editing.

Representation alignment through Tweedie. REPA [420] adds an auxiliary loss to diffusion training. A small projection g_ϕ of an intermediate hidden state $\boldsymbol{h}_\theta(\boldsymbol{x}_t, t)$ of the denoiser is aligned with the features $\boldsymbol{f}(\boldsymbol{x}_1)$ that a frozen encoder computes on the *clean* image. With a squared error

in place of the cosine similarity used in practice, the loss is

$$\mathcal{L}_{\text{REPA}} = \mathbb{E}_{t,\boldsymbol{x}_1,\boldsymbol{x}_t} \big\| g_\phi(\boldsymbol{h}_\theta(\boldsymbol{x}_t, t)) - \boldsymbol{f}(\boldsymbol{x}_1) \big\|^2. \tag{42}$$

This is a regression of a function of $\boldsymbol{x}_1$ on the noisy input, so by Lemma 2.1 its minimizer is

$$g_\phi \circ \boldsymbol{h}_\theta(\boldsymbol{x}, t) = \mathbb{E}\big[\boldsymbol{f}(\boldsymbol{x}_1) \,\big|\, \boldsymbol{x}_t = \boldsymbol{x}\big], \tag{43}$$

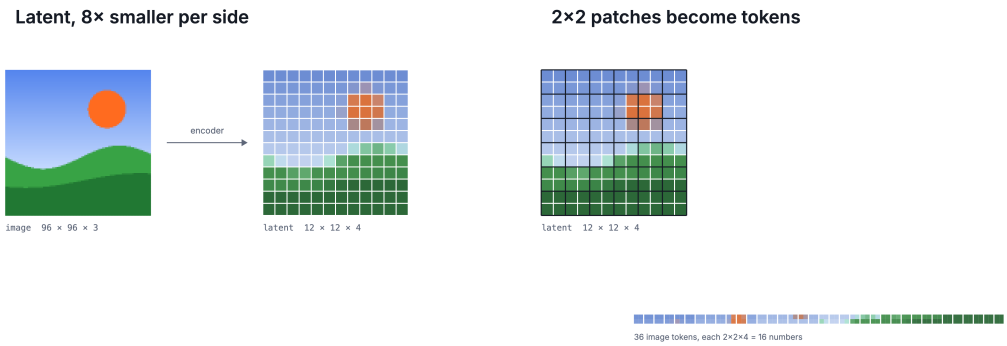
which is the Tweedie estimate of the semantic features. For a linear encoder it would be exactly $\boldsymbol{f}(\hat{\boldsymbol{x}}_1(\boldsymbol{x}, t))$, and the alignment would ask an early layer to have already computed a linear image of the denoiser output. For a nonlinear encoder it asks for the posterior mean in feature space, which is a denoising problem in its own right and one that is easier than predicting pixels because the encoder has discarded nuisance detail. REPA therefore supplies an intermediate target on the path to the final prediction, and the reported speed-ups in training are large. In the same spirit, Singh et al. [312] observe that in the latent space of a representation autoencoder REPA amounts to data prediction, so the two methods address the same quantity from the network side and from the latent side and can be combined. Training the autoencoder end-to-end with the diffusion loss alone collapses the latent, since a constant latent is trivial to denoise, but end-to-end training with the alignment loss works [176].

Back to pixels. The prediction target matters more in high dimensions. Li and He [187] argue that clean images lie near a low-dimensional manifold while noise and velocity do not, so a network with limited width can predict $\boldsymbol{x}_1$ for a large patch but cannot predict $\boldsymbol{x}_0$ or $\boldsymbol{u}$. With data prediction and large patches, a plain transformer on raw pixels becomes competitive with latent models. PixelDiT [422] and PixelGen [235] continue this line. In the language of Table 3, the choice of target leaves the minimizer unchanged but changes which function the network has to represent.

Backbones. The first diffusion models used the U-Net [288], a convolutional encoder–decoder with skip connections at every resolution and attention at the coarse ones. EDM2 shows how far careful normalization of a U-Net can go, by keeping the magnitudes of activations and weights constant throughout training [153]. The diffusion transformer (DiT) [267] replaces it with a standard transformer [354] on the patch tokens of Eq. (41). The time and class enter through adaptive layer normalization, whose scale and shift are regressed from the conditioning and initialized so that each block starts as the identity. DiT quality improves steadily with compute, and SiT [231] showed that the same backbone works with flow matching and the interpolants of Section 2.4. For text conditioning, the MMDiT of Stable Diffusion 3 [84] keeps separate weights for text and image tokens but concatenates both streams in one attention operation, so that every image token can read every text token and vice versa. Single-stream designs share all weights between modalities after a few modality-specific layers, which is more parameter-efficient at a fixed size [425]. Large text-to-image systems built on these backbones include Qwen-Image [384] and ERNIE-Image [207].

2.9 Discrete and language diffusion

Text, code, molecules given as graphs and the tokens of vector-quantized images take values in a finite set. Gaussian noise and scores are not available there, but the structure of the previous sections carries over almost line by line once the SDE is replaced by a continuous-time Markov chain. This section develops the correspondence and then turns to the diffusion language models that have become serious competitors to autoregressive models.



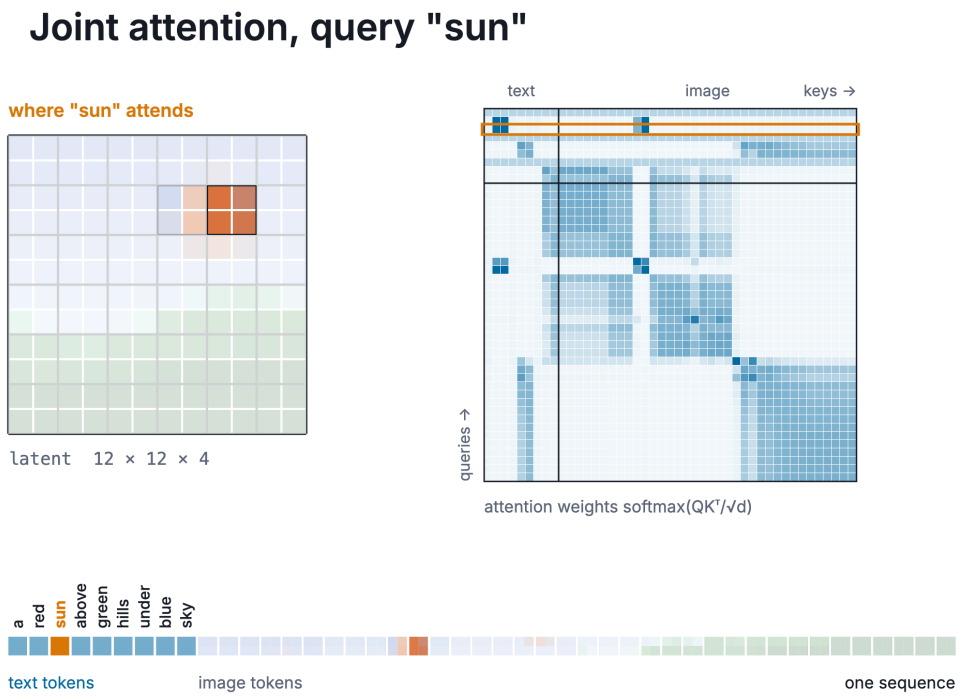


Figure 5: **From pixels to tokens** (Animation A3). An autoencoder with 8× downsampling maps a 96×96 image to a $12 \times 12 \times 4$ latent (top left), and 2×2 patches turn it into 36 tokens (top right). With the same factors a 1024×1024 image gives 4,096 tokens. In an MMDiT text and image tokens share one attention sequence, and one row of the attention matrix shows where a word reads from the image (bottom). The encoder and the attention weights in this figure are hand-built for illustration.

Continuous-time Markov chains. Let $\mathcal{X}$ be a finite state space. A Markov chain on $\mathcal{X}$ is described by a rate matrix $R_t(x, y) \geq 0$ for $y \neq x$, meaning that the probability of a jump from x to y in a short interval is $R_t(x, y)\,\mathrm{d}t$. Its marginals obey the Kolmogorov forward equation

$$\partial_t p_t(y) = \sum_{x \neq y} \big(p_t(x) R_t(x, y) - p_t(y) R_t(y, x)\big), \tag{44}$$

which plays the role of the Fokker–Planck equation Eq. (17): probability flows into y from every other state and out of y to every other state. D3PM [13] introduced discrete corruption processes in discrete time, including a uniform kernel that replaces a token by a random one and an absorbing kernel that replaces it by a special mask symbol `[M]`. Campbell et al. [38] formulated the continuous-time version.

Time reversal and the concrete score. As in Section 2.3, the corruption runs from data at $t = 1$ to noise at $t = 0$. Write $Q_t(y, x)$ for its rate of jumping from y to x as t decreases. The generative chain, which runs forward in t, has the rates

$$R_t(x, y) = Q_t(y, x)\,\frac{p_t(y)}{p_t(x)}, \qquad y \neq x. \tag{45}$$

This follows from the requirement that the generative chain jumps from x to y exactly as often as the corruption, followed backwards in time, jumps from y to x, that is $p_t(x)R_t(x, y) = p_t(y)Q_t(y, x)$, and it is the discrete analogue of the reverse-time SDE Eq. (19). The unknown is the ratio $p_t(y)/p_t(x)$ for neighbouring states, called the *concrete score*. On a lattice with $y = x + \boldsymbol{\delta}$ it is approximately $1 + \boldsymbol{\delta} \cdot \nabla \log p_t(x)$, so it carries exactly the information of the score. SEDD [218] learns it with a network $s_\theta(x, t)_y$ trained by the *denoising score entropy*

$$\mathbb{E}_{\boldsymbol{x}_1,\, x \sim p_t(\cdot \mid \boldsymbol{x}_1)} \sum_{y \neq x} w_t(x, y) \Big[s_\theta(x, t)_y - r_y \log s_\theta(x, t)_y \Big], \qquad r_y = \frac{p_t(y \mid \boldsymbol{x}_1)}{p_t(x \mid \boldsymbol{x}_1)}, \tag{46}$$

up to terms that do not depend on θ, with positive weights w_t given by the corruption rates. The function $s - r \log s$ is minimized at $s = r$, and averaging r_y over the posterior of $\boldsymbol{x}_1$ given x gives $\sum_{\boldsymbol{x}_1} p(\boldsymbol{x}_1 \mid x)\, p_t(y \mid \boldsymbol{x}_1)/p_t(x \mid \boldsymbol{x}_1) = p_t(y)/p_t(x)$. The minimizer is therefore the marginal concrete score, by the same argument as Lemma 2.1 with the squared error replaced by a Bregman divergence. Denoising score matching and score entropy are two instances of one principle: regress a tractable conditional quantity and obtain the intractable marginal one.

Masked diffusion and its ELBO. The absorbing process is the one that works best for language, and its theory simplifies drastically. Each of the L tokens is independently kept with probability α_t and replaced by `[M]` otherwise, with $\alpha_0 = 0$ and $\alpha_1 = 1$. The generative chain unmasks a masked token in $[t, t + \mathrm{d}t]$ with probability $\dot{\alpha}_t\,\mathrm{d}t/(1 - \alpha_t)$ and draws its value from a predicted distribution $p_\theta(x^i \mid \boldsymbol{x}_t)$. Unmasked tokens never change. MDLM [295] and MD4 [308] showed that the continuous-time ELBO is then a weighted masked cross-entropy,

$$-\log p_\theta(\boldsymbol{x}_1) \leq \int_0^1 \frac{\dot{\alpha}_t}{1 - \alpha_t}\, \mathbb{E}_{\boldsymbol{x}_t} \Big[\sum_{i:\, x_t^i = \texttt{[M]}} -\log p_\theta(x_1^i \mid \boldsymbol{x}_t) \Big]\, \mathrm{d}t. \tag{47}$$

This is the discrete counterpart of Eq. (29), and it has the same two properties. Substituting the masking fraction $m = 1 - \alpha_t$ turns the weight into $\mathrm{d}m/m$, so the bound does not depend on the

schedule between its endpoints. And since the posterior of a masked token given the visible ones does not depend on t, the optimal predictor is time-independent; the time input can be dropped [261, 453]. A masked diffusion model is thus a BERT-style masked language model trained with a specific distribution over masking ratios, and it is equivalent to an order-agnostic autoregressive model that predicts tokens in a random order [126]. Zheng et al. [453] also point out that sampling categorical distributions in low floating-point precision silently lowers the temperature, which had flattered earlier perplexity comparisons.

Discrete flow matching. Discrete flow matching [98] starts from the flow-matching side instead. It fixes a conditional path, for example the per-token mixture $p_t(x^i \mid \boldsymbol{x}_1) = \kappa_t\, \delta_{x_1^i} + (1-\kappa_t)\, p_0(x^i)$, writes down conditional rates that generate it, and marginalizes them with the posterior $p(\boldsymbol{x}_1 \mid \boldsymbol{x}_t)$, exactly as Proposition 2.2 marginalizes conditional velocities. With p_0 a point mass on `[M]` this reproduces masked diffusion. With p_0 uniform it gives uniform-state models, which can revise tokens they have already placed and which have a precise relation to Gaussian diffusion through the argmax map [296]. Pham et al. [270] prove non-asymptotic error bounds for time-discretized discrete flow matching on $\mathbb{Z}_m^d$ under minimal assumptions. Remasking samplers give masked models a way to revise their choices [360].

Parallel decoding and its price. The appeal of masked diffusion for language is parallel generation: a sampler can reveal many tokens per network evaluation. It reveals them independently, however. If tokens i and j are unmasked in the same step, they are drawn from $p_\theta(x^i \mid \boldsymbol{x}_t)\, p_\theta(x^j \mid \boldsymbol{x}_t)$ rather than from the joint conditional, and the error is the mutual information between them given the visible context. Decoding many coupled tokens at once produces incoherent combinations. Confidence-based samplers, inherited from MaskGIT [45], reveal only tokens whose predicted probability exceeds a threshold. This keeps the output coherent but biases the samples towards high-probability completions, much as guidance does in Section 2.11. Figure 6 shows both effects on a toy language whose conditionals are exact.

Diffusion language models in 2026. LLaDA showed that an 8B masked diffusion model trained from scratch can match autoregressive models of similar size [252], and iLLaDA keeps the masked objective through pre-training on 12T tokens and instruction tuning, with large gains in reasoning and code [253]. Other models start from autoregressive checkpoints and adapt them: Dream [412], LLaDA2.0 at 100B parameters [24], Seed Diffusion for code [327], the commercial Mercury models [138], and DiffusionGemma, which refines blocks of 256 tokens and is fine-tuned from a mixture-of-experts Gemma model with less than a tenth of the original training tokens [77]. Several lines of work explain where the approach helps. Under a fixed amount of unique data, diffusion models keep improving over many epochs after autoregressive models saturate [250]. Looping the early and middle layers gives masked models extra depth without extra parameters and matches baselines with up to 3.3 times fewer training FLOPs [175]. Mechanistically, masked models learn an induction circuit that copies from matching context in either direction [41]. There are also limits. Arbitrary-order generation lets a model postpone the uncertain tokens that matter for exploration in reasoning, which can collapse the diversity of solutions [251]. And parallel generation does not by itself make inference fast, since end-to-end latency depends on caching, block structure and hardware as much as on the number of steps [112].



Figure 6: **Parallel unmasking** (Animation A4). A toy language of 64 eight-token sentences with three coupled token pairs, so every conditional is exact (top left). Revealing four tokens per step samples coupled tokens independently and produces sentences such as "san kong" or "owl barks" (top right). Keeping only predictions with probability above $\tau = 0.35$ stays coherent in about three steps, at the cost of concentrating on the most probable sentences (bottom left). Share of valid sentences against the mean number of decoding steps, each point averaging 3000 runs (bottom right).

2.10 Autoregressive and hybrid visual generation

Diffusion generates all coordinates jointly and refines them over noise levels. Autoregressive models generate a sequence of tokens one after another, each exactly conditioned on its predecessors. The two used to be separate families, but much recent work places them on a single axis: how the joint distribution is factorized, and how each factor is parametrized.

One axis, several factorizations. An autoregressive model writes $p(\boldsymbol{z}) = \prod_{k=1}^{N} p(z_k \mid z_{<k})$ over discrete tokens and trains each factor by cross-entropy, which gives an exact likelihood and lets one model handle text and images alike. Masked diffusion (Section 2.9) is the same product over a random order with several factors evaluated per step. Next-scale prediction in VAR [344] factorizes over resolutions instead, $p(\boldsymbol{z}) = \prod_s p(\boldsymbol{r}_s \mid \boldsymbol{r}_{<s})$, where $\boldsymbol{r}_s$ is the token map at scale s of a multi-scale quantizer and all tokens of one scale are predicted in parallel. Continuous diffusion is the limit in which a single "token" is the whole image and the sequence runs over noise levels. These choices trade sequential steps against the dependence that each step must capture in parallel, which is the price discussed with Fig. 6.

Diffusion as a parametrization of one factor. Quantizing images into discrete tokens loses information. MAR [188] keeps continuous tokens and models each factor $p(\boldsymbol{z}_k \mid \boldsymbol{c}_k)$, where $\boldsymbol{c}_k$ is the output of a transformer over the tokens generated so far, with a small diffusion model:

$$\mathcal{L}_{\text{MAR}} = \mathbb{E}_{k,t,\boldsymbol{\epsilon}} \big\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\alpha_t \boldsymbol{z}_k + \sigma_t \boldsymbol{\epsilon},\, t,\, \boldsymbol{c}_k) \big\|^2. \tag{48}$$

The loss is the denoising loss of Section 2.2 for a single token, conditioned on its context, and gradients flow through $\boldsymbol{c}_k$ into the transformer. The theory of the previous sections applies unchanged to each factor. NextStep-1 scales this design to text-to-image generation with a flow-matching head [249]. Transfusion [458] goes the other way and puts a full diffusion model inside a language model. One transformer is trained with next-token prediction on text and with a diffusion loss on image patches, with causal attention across the sequence and bidirectional attention within each image.

Unified multimodal models. These designs have become the basis of models that both understand and generate images. Show-o2 [393] combines autoregressive text with flow matching on images in one model. Emu3.5 [72] is trained with next-token prediction on interleaved video and text and accelerates image generation by adapting the model to parallel discrete diffusion decoding. Manzano [191] uses a hybrid tokenizer with continuous features for understanding and discrete tokens for generation, and HunyuanImage 3.0 [341] generates images inside a large mixture-of-experts language model. Discrete diffusion language models are extended to images in the same way [139, 394]. Two 2026 designs mark the ends of the spectrum. UniAR [268] argues that a single shared discrete tokenizer is what makes understanding and generation reinforce each other, and models both autoregressively. UniDDT [367] instead decouples them, with a language model reading noisy visual features and a separate diffusion transformer producing the image. Which split of labour between the language model and the diffusion component scales best is open. Post-training of these models, including reinforcement learning on discrete visual tokens [103], is discussed in Section 4.

2.11 Conditioning and guidance

Conditional generation needs no new theory: a model of $p_t(\boldsymbol{x} \mid c)$ for a label, a prompt or an image is trained with the same losses, with c as an extra input. In practice, however, conditional models sampled at face value produce images that follow the prompt only loosely. Guidance is the family of sampling-time modifications that strengthen the condition. It is used by almost every deployed text-to-image and text-to-video model, and its mathematics is less innocent than its implementation.

Bayes' rule on scores. Since $p_t(\boldsymbol{x} \mid c) = p_t(\boldsymbol{x})\, p_t(c \mid \boldsymbol{x})/p(c)$ and $p(c)$ does not depend on $\boldsymbol{x}$,

$$\nabla \log p_t(\boldsymbol{x} \mid c) = \nabla \log p_t(\boldsymbol{x}) + \nabla \log p_t(c \mid \boldsymbol{x}), \qquad p_t(c \mid \boldsymbol{x}) = \mathbb{E}\big[p(c \mid \boldsymbol{x}_1) \,\big|\, \boldsymbol{x}_t = \boldsymbol{x}\big]. \tag{49}$$

The second identity holds because c and $\boldsymbol{x}_t$ are independent given $\boldsymbol{x}_1$. Classifier guidance [75] trains a classifier on noisy inputs to estimate $p_t(c \mid \boldsymbol{x})$ and adds $w\nabla \log p_t(c \mid \boldsymbol{x})$ to an unconditional score, with a scale $w > 1$ that over-weights the classifier.

Classifier-free guidance. Classifier-free guidance (CFG) [122] avoids the classifier. One network is trained both conditionally and unconditionally, by replacing c with a null token for a fraction of the training examples, and the two predictions are extrapolated:

$$\boldsymbol{s}_t^w(\boldsymbol{x} \mid c) = (1-w)\, \boldsymbol{s}_t(\boldsymbol{x}) + w\, \boldsymbol{s}_t(\boldsymbol{x} \mid c) = \boldsymbol{s}_t(\boldsymbol{x} \mid c) + (w-1)\big(\boldsymbol{s}_t(\boldsymbol{x} \mid c) - \boldsymbol{s}_t(\boldsymbol{x})\big). \tag{50}$$

The weights sum to one, and the velocity is an affine function of the score with coefficients that do not depend on c, $\boldsymbol{u} = (\dot\alpha_t/\alpha_t)\boldsymbol{x} + \frac{1}{2}\sigma_t^2 \dot\lambda_t \boldsymbol{s}$ by Eq. (25). The same extrapolation therefore applies to velocities, noise predictions and data predictions. By Eq. (49), the guided score is

$$\boldsymbol{s}_t^w = \nabla \log \pi_t^w, \qquad \pi_t^w(\boldsymbol{x}) \propto p_t(\boldsymbol{x})^{1-w} p_t(\boldsymbol{x} \mid c)^w \propto p_t(\boldsymbol{x})\, p_t(c \mid \boldsymbol{x})^w, \tag{51}$$

so CFG behaves like classifier guidance with an implicit classifier. At $t = 1$ the density $\pi_1^w \propto p_1(\boldsymbol{x})\, p(c \mid \boldsymbol{x})^w$ is a sharpened conditional, which puts more mass where the condition is unambiguous.

CFG does not sample its own target. It is tempting to conclude that CFG samples π_1^w. It does not, and Section 2.6 explains why. Sampling π_1^w means tilting the terminal distribution by the reward $r(\boldsymbol{x}) = w \log p(c \mid \boldsymbol{x})$ with $\beta = 1$. The time-t marginal of the tilted path measure Eq. (38) is $p_t(\boldsymbol{x})\, h_t(\boldsymbol{x})$, and comparing h_t with the factor used by CFG gives

$$h_t(\boldsymbol{x}) = \mathbb{E}\big[p(c \mid \boldsymbol{x}_1)^w \,\big|\, \boldsymbol{x}_t = \boldsymbol{x}\big] \;\geq\; \big(\mathbb{E}\big[p(c \mid \boldsymbol{x}_1) \,\big|\, \boldsymbol{x}_t = \boldsymbol{x}\big]\big)^w = p_t(c \mid \boldsymbol{x})^w \quad \text{for } w \geq 1, \tag{52}$$

by Jensen's inequality. The two agree only where the posterior of $\boldsymbol{x}_1$ is concentrated, that is at low noise. At high noise CFG uses the power of an average where the correct tilt uses the average of a power, and it misjudges which regions of $\boldsymbol{x}_t$ lead to a confident class. There is a second, independent problem: the densities π_t^w are not the marginals of any forward noising process, because powers do not commute with Gaussian convolution. The probability-flow ODE driven by $\boldsymbol{s}^w$ therefore does not keep its samples distributed as π_t^w, even if every score is exact. Bradley and Nakkiran [34] make this precise for the stochastic sampler: in the continuous limit, CFG is equivalent to a predictor–corrector scheme that alternates denoising steps with Langevin steps towards π_t^w, which is a different distribution from π_1^w. Chidambaram et al. [61] analyse guidance on mixtures where every quantity is exact and show that large guidance pushes samples towards the extremes of the

support of the conditional rather than sampling a sharpened density. Figure 7 shows this on an exact Gaussian mixture: at high w the samples leave the modes that the two classes share. Methods that sample the tilted distribution correctly reweight particles with Feynman–Kac weights [314] and are treated in Section 6.

Practical variants. Many modifications of Eq. (50) address its side effects, which are lower diversity, oversaturated colours and samples that leave the data manifold.

- *Where to guide.* Guidance at high noise mostly removes diversity, because it decides the coarse layout using the unreliable factor in Eq. (52), while guidance at very low noise has little effect. Restricting guidance to an interval of intermediate noise levels improves both quality and diversity [168].
- *What to guide against.* Autoguidance [154] replaces the unconditional model by a weaker version of the conditional one, smaller or trained for less time, and extrapolates $\boldsymbol{s}_{\text{weak}} + w(\boldsymbol{s} - \boldsymbol{s}_{\text{weak}})$. The weak model makes the same kind of errors as the good one, only larger, so the extrapolation moves away from those errors without changing the condition and without the diversity loss of CFG. Figure 7 compares the two.
- *How to combine.* CFG++ [67] keeps guided samples near the data manifold by renoising with the unconditional noise prediction and using guidance scales in $[0, 1]$. Adaptive projected guidance [294] splits the guidance term into components parallel and orthogonal to the conditional prediction and down-weights the parallel one, which causes oversaturation. CFG-Ctrl [361] reads Eq. (50) as proportional feedback control on the conditional–unconditional discrepancy and replaces it with a sliding-mode controller.
- *Removing the second pass.* Guidance doubles the cost of sampling. Guidance-free training [49] trains a single model whose output is the guided prediction, by expressing the conditional model as a combination of the sampling model and the unconditional one, and samples with one evaluation per step.

A careful 2026 comparison of eight variants on two rectified-flow transformers, with uncertainty estimates, found that none consistently beats plain CFG once the guidance scale is tuned for each method [304]. Guidance also complicates distillation (Section 5). When a student is trained to match the guided teacher, errors in the conditional and unconditional branches can cancel in the combination, so the objective does not determine the branches individually. This is harmless when both branches share the same negative condition, but it fails when the teacher's negative branch uses information that the student does not have [178].

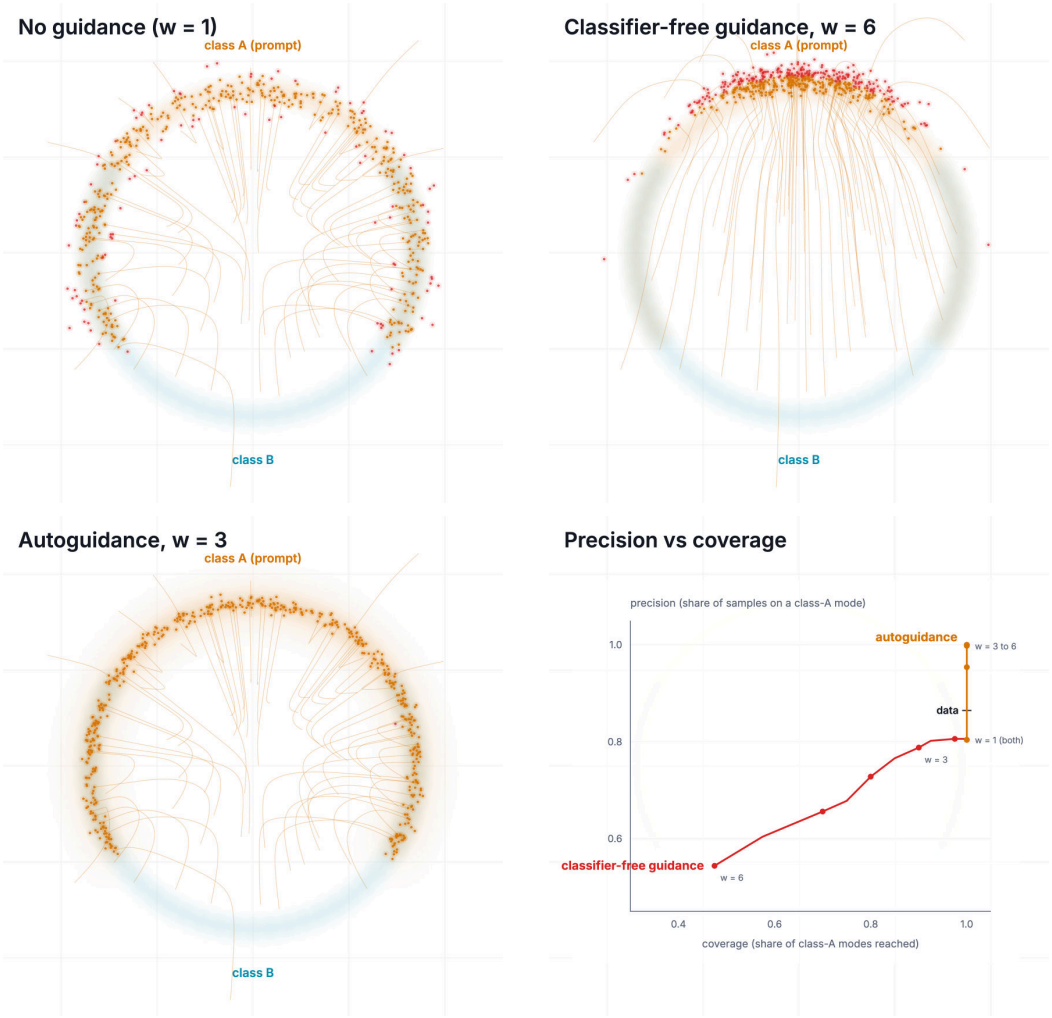


Figure 7: **Classifier-free guidance versus autoguidance** (Animation A5). Two classes share a ring of Gaussian modes and overlap at its two ends; all velocity fields are exact Gaussian-mixture fields. The conditional model for class A is slightly too wide, as a trained network would be. Top left: without guidance, some samples land between modes or on class-B modes (red). Top right: classifier-free guidance at $w = 6$ extrapolates away from the unconditional model, which pushes samples toward the centre of class A and drops its modes near the overlap. Bottom left: autoguidance extrapolates away from a weaker, wider version of the same conditional model; at $w = 3$ the samples are sharper while every class-A mode is still reached. Bottom right: precision and coverage over $w \in [1, 6]$, from 500 samples per setting.

3 Fine-Tuning and Adaptation

A pretrained model defines a velocity field $\boldsymbol{u}_\theta$ that transports noise to an approximation of the pretraining distribution. Fine-tuning moves this field towards a new distribution p^{new}: a subject, a style, images that respect a sketch or a depth map, edited versions of an input, or videos that start from a given frame. In all cases covered here the objective is unchanged. It is the conditional flow matching or denoising loss of Section 2, evaluated on new data and often with new conditioning inputs. The questions are which parameters to change, how to inject the new condition, and how to keep what the model already knows. Section 4 treats the different situation in which no target samples exist and only a reward is available.

Three ideas recur throughout the chapter. First, most adaptations change the model in a low-dimensional way, so a small number of new parameters suffices. Second, every adapter is initialized so that the adapted model starts exactly at the pretrained one, which makes the first gradient steps a small perturbation of a working sampler. Third, fine-tuning on few examples moves the model towards the empirical optimum of Section 2.7, so overfitting and memorization are the main failure modes, and much of the method design is regularization against them.

3.1 Parameter-efficient fine-tuning

Low-rank adaptation. LoRA [128] freezes each pretrained weight matrix $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and learns an additive update of rank r,

$$W = W_0 + \frac{\alpha}{r} BA, \qquad B \in \mathbb{R}^{d_{\text{out}} \times r},\ A \in \mathbb{R}^{r \times d_{\text{in}}}, \qquad B \leftarrow 0 \text{ at initialization.} \tag{53}$$

The number of trainable parameters per matrix drops from $d_{\text{out}} d_{\text{in}}$ to $r(d_{\text{out}} + d_{\text{in}})$, which for a 3072×3072 projection of a large DiT and $r = 16$ is a factor of 96. Because $B = 0$, the adapted field equals the pretrained field at the start of training. After training the update can be merged into W_0, so inference costs nothing extra, or kept separate and shared as a file of a few megabytes. In diffusion transformers LoRA is usually applied to the attention projections and often to the feed-forward layers, and the scale α can be changed at sampling time to interpolate between the base and the adapted model.

Why a low rank is enough. Pretrained language models can be fine-tuned within random subspaces of surprisingly low dimension, which suggests that adaptation has a low intrinsic dimension [2]. The low rank is a property of the reachable solutions rather than of the unconstrained optimum, however. Figure 8 shows this on a flow-matching toy. A full update of one weight matrix that moves a single mode is spread over many singular directions, with only 29% of its energy in the top one, yet a rank-one update produces the same samples. Many weight updates realize the same change of the field, and the low-rank family contains one of them. The constraint also acts as a regularizer. In language models LoRA learns less of the new domain than full fine-tuning but also forgets less of the old one [23], which is often the preferred trade-off for personalization.

Variants and merging. DoRA [212] separates each column of W into a magnitude and a direction, $W = \boldsymbol{m} \odot (W_0 + BA)/\|W_0 + BA\|_c$ with column norms $\|\cdot\|_c$, and trains the magnitude $\boldsymbol{m}$ directly while LoRA updates the direction. T-LoRA [315] makes the rank depend on the noise level. It observes that single-image customization overfits mainly at high noise, where the update decides layout and composition, and restricts the update there. Adapters trained separately, for a subject and for a style, should ideally combine by adding their updates. They often interfere when

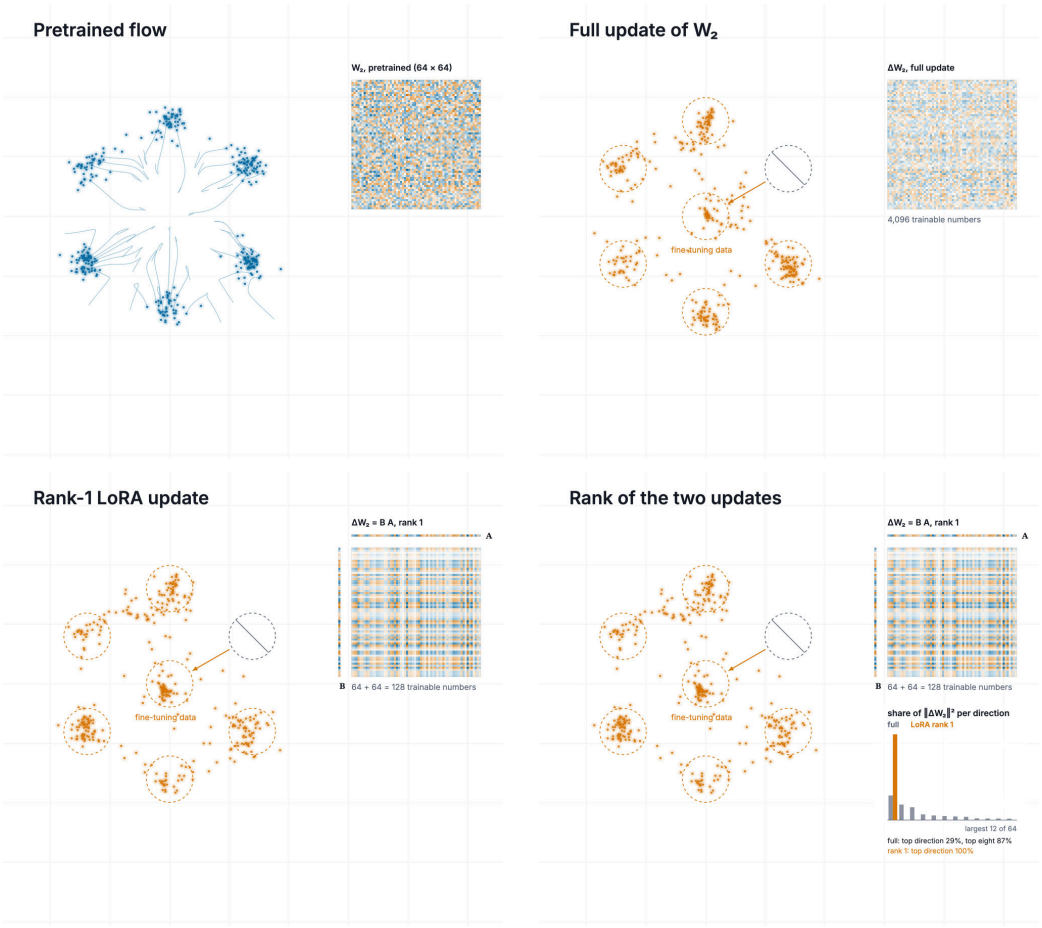


Figure 8: **LoRA in weight space** (Animation A6). A flow-matching MLP with a 64×64 hidden weight matrix W_2 is pretrained on six Gaussians on a ring (top left) and fine-tuned on data in which one mode has moved to the origin. Updating all 4,096 entries of W_2 moves that mode and keeps the other five (top right). A rank-1 update $W_2 + BA$ [128] with 128 trainable numbers gives the same samples (bottom left). The unconstrained update is not low rank: its top singular direction holds 29% of $\|\Delta W_2\|^2$ (bottom right). All networks are trained, and samples use the same noise for every model.

they write to the same directions. ZipLoRA [305] learns per-column merge coefficients that keep the two updates nearly orthogonal. Hypernetworks go one step further and predict adapter weights directly. Prompt2Effect [409] generates LoRA weights for a visual effect in an image-to-video model from a prompt in a single forward pass, conditioned on the frozen base weights.

3.2 Personalization

Personalization teaches a model a new concept, typically a subject or a style, from a handful of images, so that it can be placed in new contexts by prompting.

Optimization-based methods. Textual inversion [94] keeps the model frozen and optimizes only the embedding $\boldsymbol{v}$ of a new token S^*,

$$\boldsymbol{v}^\star = \arg\min_{\boldsymbol{v}} \; \mathbb{E}_{t,\boldsymbol{x}_1,\boldsymbol{x}_0} \big\| \hat{\boldsymbol{x}}_0(\boldsymbol{x}_t, t, c(\boldsymbol{v})) - \boldsymbol{x}_0 \big\|^2, \tag{54}$$

where $c(\boldsymbol{v})$ is a prompt such as "a photo of S^*". The model can only express what its conditioning space already reaches, which limits fidelity for unusual subjects. DreamBooth [290] fine-tunes the whole network on the subject images with a prompt that combines a rare identifier and the class name, and adds a *prior-preservation* term: the same loss on images that the frozen base model generates for the class prompt alone,

$$\mathcal{L} = \mathbb{E}\big\|\hat{\boldsymbol{x}}_0(\boldsymbol{x}_t, t, c_{\text{subj}}) - \boldsymbol{x}_0\big\|^2 + \lambda\, \mathbb{E}_{\boldsymbol{x}'_1 \sim p^{\text{base}}(\cdot|c_{\text{cls}})}\big\|\hat{\boldsymbol{x}}_0(\boldsymbol{x}'_t, t, c_{\text{cls}}) - \boldsymbol{x}'_0\big\|^2. \tag{55}$$

The second term is a self-distillation that keeps the field for the class prompt near the base field. It plays the role of the KL penalty to the base model in Section 2.6, enforced through samples instead of a path-space divergence. Without it the model drifts towards the few training images for every member of the class, which is the empirical-denoiser regime of Eq. (40). Custom Diffusion [166] restricts the update to the key and value projections of the cross-attention, the only weights that map text into the image stream, and trains several concepts jointly. In practice most optimization-based personalization today trains a LoRA with the DreamBooth objective.

Encoder-based methods. Per-subject optimization takes minutes and stores one adapter per subject. Encoder-based methods train once, on many subjects, a module that turns reference images into conditioning, and then personalize in a single forward pass. IP-Adapter [411] adds a separate cross-attention for image features beside the text cross-attention,

$$\boldsymbol{z}' = \text{Attn}(Q, K_{\text{txt}}, V_{\text{txt}}) + \lambda\, \text{Attn}(Q, K_{\text{img}}, V_{\text{img}}), \tag{56}$$

where only the new key and value projections are trained and λ sets the strength of the reference at sampling time. For faces, PhotoMaker [194] merges features of several reference photos into a stacked identity embedding, and InstantID [364] combines face-recognition features with a landmark-conditioned control network. In DiTs with joint attention a simpler route exists: append the tokens of the reference images to the sequence and let attention find the correspondences. DreamO [248] trains one such model for identity, subject, style and try-on. These generalist models need large datasets of reference images paired with the same subject in a new scene, which are synthesized by multi-stage pipelines. CRAFT [265] avoids composed targets altogether and fine-tunes only attention under a constrained reward, connecting personalization to the reward methods of Section 4.

3.3 Spatial and structural control

Text is a poor channel for geometry. Control methods condition generation on spatially aligned signals such as edges, depth, segmentation, human pose or a reference layout.

ControlNet and zero initialization. ControlNet [435] copies the encoder of a pretrained U-Net into a trainable branch, feeds it the control image, and adds its outputs back into the frozen network through convolutions whose weights are initialized to zero. With $\mathcal{F}$ the frozen block, $\mathcal{F}_c$ its trainable copy and $\mathcal{Z}_1, \mathcal{Z}_2$ the zero convolutions,

$$\boldsymbol{y} = \mathcal{F}(\boldsymbol{z}) + \mathcal{Z}_2(\mathcal{F}_c(\boldsymbol{z} + \mathcal{Z}_1(\boldsymbol{c}))). \tag{57}$$

At initialization $\boldsymbol{y} = \mathcal{F}(\boldsymbol{z})$, so training starts at the pretrained model, and the copied encoder starts from features that already understand images rather than from scratch. This is the same principle as $B = 0$ in Eq. (53) and the zero-initialized adaptive normalization of DiT in Section 2.8. T2I-Adapter [247] replaces the copied encoder by a small network that produces multi-scale features, which is cheaper and easy to combine with other adapters.

Control in diffusion transformers. DiTs have no encoder to copy, and their sequence structure offers a different mechanism. OminiControl [333] encodes the condition image with the model's own autoencoder, appends its tokens to the sequence and trains only a LoRA, adding about 0.1% parameters. Sharing position embeddings between condition and image tokens aligns them spatially for edge or depth maps, and shifting them supports conditions that are not aligned, such as a subject reference. EasyControl [441] restricts attention so that condition tokens do not attend to the image, which lets their keys and values be cached across steps. In-context LoRA [132] shows that a pretrained text-to-image DiT can already generate several related images as panels of one large image, and that a small LoRA on a few dozen examples is enough to turn this into tasks such as storyboards or visual identities. The common message is that joint attention turns conditioning into sequence concatenation, so most control tasks reduce to a short LoRA fine-tune on paired data.

3.4 Instruction-based editing

An editing model maps a source image and an instruction to an edited image. As a fine-tuning problem it differs from personalization in two ways: the condition includes a full image that the output must largely preserve, and training needs triplets of source, instruction and result, which do not occur naturally.

InstructPix2Pix and two-condition guidance. The training triplets of InstructPix2Pix [35] were generated with a language model, which wrote instructions and edited captions, and a text-to-image model, which rendered both captions with attention control so that the two images differ only in the edit. The edit model receives the source image c_I by concatenating its latent to the input channels. At sampling time the two conditions get separate guidance scales,

$$\hat{\boldsymbol{s}} = \boldsymbol{s}(\boldsymbol{x}) + w_I\big(\boldsymbol{s}(\boldsymbol{x} \mid c_I) - \boldsymbol{s}(\boldsymbol{x})\big) + w_T\big(\boldsymbol{s}(\boldsymbol{x} \mid c_I, c_T) - \boldsymbol{s}(\boldsymbol{x} \mid c_I)\big). \tag{58}$$

By the argument of Eq. (51), this follows the score of $p_t(\boldsymbol{x})\, p_t(c_I \mid \boldsymbol{x})^{w_I}\, p_t(c_T \mid c_I, \boldsymbol{x})^{w_T}$, one factor for faithfulness to the source and one for the instruction. The trade-off between preserving the image and following the instruction is therefore set by two exponents, with the caveats of Section 2.11.

In-context editing at scale. Current editors replace channel concatenation by sequence concatenation. FLUX.1 Kontext [29] appends the tokens of the context image to the image tokens of a flow-matching DiT, distinguished by a position offset, and trains generation and editing in one model. Qwen-Image [384] feeds the source image both through a vision-language model, for semantics, and through the autoencoder, for appearance. Step1X-Edit [213] also pairs a multimodal language model with a DiT. Seedream 4.0 [339] unifies generation and multi-image editing, and further technical reports follow the same pattern [332, 444]. Two trends stand out. Training data has moved from synthetic pairs produced by attention control to large curated mixtures that are filtered by vision-language models. And supervised fine-tuning is increasingly followed by reinforcement learning with a multimodal model as judge [195], the subject of Section 4.

3.5 Video adaptation

Video models are adapted in the same ways as image models, with time as an additional axis along which conditions can be specified.

Image-to-video and temporal modules. Image-to-video generation conditions a text-to-video model on a first frame. Stable Video Diffusion [30] concatenates the noise-augmented latent of the frame to the input channels of every frame, and current open models such as Wan [340] combine a channel-wise frame condition with a mask that marks which frames are given. When no video base model is available, AnimateDiff [110] inserts temporal attention layers into a frozen personalized image model and trains only these, so that every image LoRA inherits motion. Its motion LoRAs, trained on a few clips each, add camera movements such as zooms and pans.

Camera and motion control. Precise camera control needs a geometric condition. CameraCtrl [116] represents the camera of each frame by its Plücker embedding: for each pixel, the ray direction $\boldsymbol{d}$ and the moment $\boldsymbol{o} \times \boldsymbol{d}$ for the camera centre $\boldsymbol{o}$, a six-channel map that a small encoder injects into the temporal layers. MotionCtrl [373] conditions on camera poses and on object trajectories separately. Wan-Move [64] specifies motion by dense point trajectories and propagates the first-frame features along them in latent space, without additional modules. Audio is another driving signal, as in audio-driven character animation [95]. Editing can also be transferred from images: an image editor changes a few keyframes and a video model propagates the edits across time [359].

3.6 Risks of cheap adaptation

The low cost of adaptation has made adapters a medium of exchange, with public hubs hosting very large numbers of LoRAs, and this raises its own problems. Fine-tuning on a few images is exactly the regime in which models memorize (Section 2.7), so adapters trained on private photos can leak them. Differential privacy protects training data at a large cost in quality, and Miao et al. [246] propose adapting under a weaker guarantee that bounds reconstruction directly, with less loss in utility. Adapters can also hide their purpose. Qiu et al. [280] search for the conditioning input that best exposes what an adapter has learned, to support moderation of shared adapters, and Wei et al. [377] construct pairs of adapters that each look benign but redirect generation when loaded together. Since merging adapters is routine, the safety of a model is a property of the combination that users actually load and not of each adapter alone.

4 Post-Training: Preference and Reward Alignment

Pre-training teaches a model to reproduce its data distribution, including the parts of it that nobody wants: blurry photographs, misspelled text, images that ignore half of the prompt. Fine-tuning (Section 3) changes the data. Post-training changes the objective. A reward $r(\boldsymbol{x}, c)$ scores how good a sample $\boldsymbol{x}$ is for a condition c, and the model is trained to produce samples with high reward while staying close to a reference model p_{ref}, usually the pre-trained one. Almost every method in this chapter optimizes, exactly or approximately, the KL-regularized objective

$$\mathcal{J}(\theta) = \mathbb{E}_c\Big[\mathbb{E}_{\boldsymbol{x}\sim p_\theta(\cdot|c)} r(\boldsymbol{x}, c) - \beta\, \mathrm{KL}\big(p_\theta(\cdot \mid c) \,\|\, p_{\mathrm{ref}}(\cdot \mid c)\big)\Big]. \tag{59}$$

For each c, the bracket equals $-\beta \mathrm{KL}(p_\theta \| p^\star) + \beta \log Z$ with

$$p^\star(\boldsymbol{x} \mid c) = \frac{1}{Z}\, p_{\mathrm{ref}}(\boldsymbol{x} \mid c)\, e^{r(\boldsymbol{x},c)/\beta}, \tag{60}$$

which can be checked by expanding $\log p^\star$. The maximizer is therefore the reference model tilted by the exponentiated reward, the same energy-based target that appeared as the terminal marginal

of the optimal path measure Eq. (38) in Section 2.6. The temperature β sets how far the tilt may move: as $\beta \to \infty$ nothing changes, and as $\beta \to 0$ all mass goes to the reward maximizers within the support of p_{ref}.

The difficulty is that a diffusion or flow model does not give $\log p_\theta(\boldsymbol{x})$ cheaply, and Eq. (59) needs its gradient. The methods differ in how they get around this. Policy-gradient methods (Section 4.2) treat sampling as a sequence of Gaussian decisions, each with a tractable likelihood. Reward backpropagation (Section 4.3) differentiates the reward through the sampler. Preference optimization (Section 4.4) eliminates the reward and bounds the likelihood ratio with denoising losses. The control view (Section 4.5) solves for the optimal drift directly and leads to regression objectives that look like pre-training. All of them first need a reward, and Section 4.1 begins there. Section 4.6 discusses what goes wrong when the reward is optimized too well, and Section 4.7 turns to discrete diffusion language models.

4.1 Reward models

A reward for visual generation can be a program, such as an object detector that checks counts and positions for GenEval prompts [104] or an OCR system that checks rendered text, or it can be a learned model of human judgment. Programmatic rewards are reliable on their narrow task. Learned rewards cover aesthetics and overall prompt adherence, and they are where most of the difficulty lies.

From comparisons to scores. Humans compare more reliably than they score. The standard model of pairwise comparisons is the Bradley–Terry model, which assumes a latent score r such that

$$P(\boldsymbol{x}^w \succ \boldsymbol{x}^l \mid c) = \sigma\big(r(\boldsymbol{x}^w, c) - r(\boldsymbol{x}^l, c)\big), \qquad \mathcal{L}_{\mathrm{BT}}(\phi) = -\mathbb{E} \log \sigma\big(r_\phi(\boldsymbol{x}^w, c) - r_\phi(\boldsymbol{x}^l, c)\big), \tag{61}$$

where $\boldsymbol{x}^w$ is the preferred sample, $\boldsymbol{x}^l$ the rejected one and σ the logistic function. Only differences of r are identified, so a reward model is defined up to a constant per prompt, a fact that the group normalization of Section 4.2 exploits. The first generation of reward models fine-tuned CLIP-like encoders with this loss on large collections of comparisons between generated images: PickScore on the Pick-a-Pic dataset collected from users of a web application [163], ImageReward on expert annotations [396], and HPS v2 on its Human Preference Dataset [389]. HPSv3 replaces the CLIP backbone by a vision-language model and trains on 1.17M comparisons with an uncertainty-aware ranking loss [234].

Vision-language models as judges. A pretrained vision-language model already understands composition, text and spatial relations, which a CLIP score captures poorly. Recent reward models therefore fine-tune such models to judge. UnifiedReward trains one model to assess both understanding and generation, for images and videos, with pairwise ranking and pointwise scoring [369]. VisionReward decomposes a judgment into many fine-grained questions and combines the answers linearly, which makes the score interpretable [397]. RewardDance argues that a Bradley–Terry head is mismatched with a model trained by next-token prediction and instead reads the reward from the probability of an answer token, which lets reward models scale with the size of the underlying VLM [387]. RationalRewards has the judge write a multi-dimensional critique before scoring, recovers such rationales from ordinary preference data, and uses the critiques both as training rewards and for prompt refinement at test time [362]. Production systems combine several such judges. The post-training pipeline of Qwen-Image-2.0 uses VLM-based pointwise reward models

with chain-of-thought reasoning for alignment, aesthetics and portrait fidelity, and separate ones for instruction following and identity preservation in editing [401].

Video. Video rewards must judge motion and temporal consistency as well as single frames. VideoReward is a multi-dimensional reward model trained on pairwise annotations of modern video generators, and its authors derive several alignment algorithms for flow models from the KL-regularized objective Eq. (59) [209]. VisionReward and UnifiedReward also cover video. Physical plausibility is a further target. PhysMaster learns a representation of physical cues from the input image of an image-to-video model and trains it with reinforcement learning from human feedback [144].

Whatever its form, a learned reward is accurate only near the distribution it was trained on. Post-training pushes the generator towards regions where the reward is high, which may be regions the reward model has never seen. This is the root of reward hacking (Section 4.6).

4.2 Policy gradients on the denoising chain

Sampling as a Markov decision process. A stochastic sampler with steps $0 = t_0 < t_1 < \cdots < t_K = 1$ produces a chain $\boldsymbol{x}_{t_0}, \ldots, \boldsymbol{x}_{t_K}$ in which every transition is Gaussian. For the DDPM sampler, or for the Euler–Maruyama discretization of any SDE in Proposition 2.3, the transition is

$$\pi_\theta(\boldsymbol{x}_{k+1} \mid \boldsymbol{x}_k, c) = \mathcal{N}\big(\boldsymbol{x}_k + \Delta_k\, \boldsymbol{b}_\theta(\boldsymbol{x}_k, t_k, c),\ \varepsilon_{t_k}^2 \Delta_k\, I\big), \qquad \Delta_k = t_{k+1} - t_k, \tag{62}$$

with drift $\boldsymbol{b}_\theta$ and noise level ε_t. Read the state as $(\boldsymbol{x}_k, t_k, c)$, the action as the next sample $\boldsymbol{x}_{k+1}$, the policy as π_θ, and the reward as zero except at the last step, where it is $r(\boldsymbol{x}_{t_K}, c)$. This denoising MDP was introduced by DDPO [27] and DPOK [89]. Its value is that the likelihood of a whole trajectory is a product of Gaussians that can be evaluated exactly, although the marginal likelihood of the final sample cannot. The score-function (REINFORCE) identity then gives an unbiased gradient of the expected reward,

$$\nabla_\theta \mathbb{E}[r(\boldsymbol{x}_{t_K}, c)] = \mathbb{E}\Big[r(\boldsymbol{x}_{t_K}, c) \sum_{k=0}^{K-1} \nabla_\theta \log \pi_\theta(\boldsymbol{x}_{k+1} \mid \boldsymbol{x}_k, c)\Big], \tag{63}$$

which requires only reward values, not reward gradients. Any black-box reward, including a VLM judge or a human, can be used.

DDPO and DPOK. DDPO uses Eq. (63) with the reward replaced by an advantage, either directly or with the importance-sampled and clipped surrogate of PPO [302], which reuses each batch of trajectories for several gradient steps while keeping the policy ratio $\pi_\theta / \pi_{\theta_{\text{old}}}$ near one. It improved compressibility, aesthetic scores and prompt alignment measured by a VLM. DPOK adds the KL term of Eq. (59). The KL divergence between the path measures of two chains factorizes over steps, and for Gaussian steps with a shared variance each term is a squared distance between means,

$$\mathrm{KL}(\pi_\theta(\cdot \mid \boldsymbol{x}_k) \,\|\, \pi_{\text{ref}}(\cdot \mid \boldsymbol{x}_k)) = \frac{\Delta_k}{2\varepsilon_{t_k}^2} \big\| \boldsymbol{b}_\theta(\boldsymbol{x}_k, t_k) - \boldsymbol{b}_{\text{ref}}(\boldsymbol{x}_k, t_k) \big\|^2. \tag{64}$$

Summed over steps, this is the Euler discretization of the Girsanov cost Eq. (37) with control $\boldsymbol{c} = (\boldsymbol{b}_\theta - \boldsymbol{b}_{\text{ref}})/\varepsilon$. The path KL bounds the KL between the terminal marginals by the data-processing inequality, so penalizing it is a conservative version of Eq. (59). Section 4.5 shows that this bound is not tight in general, and that the gap biases the solution unless the noise schedule is chosen carefully.

Flow models and GRPO. Flow-matching models are usually sampled with an ODE, whose transitions are deterministic and have no density. Flow-GRPO [208] restores the MDP by sampling the same model with an SDE from Proposition 2.3 during training. The score in the drift is not a separate network: by the velocity–score identity Eq. (25), $\nabla \log p_t = 2(\boldsymbol{u}_t - (\dot\alpha_t/\alpha_t)\boldsymbol{x})/(\sigma_t^2 \dot\lambda_t)$, so the drift $\boldsymbol{b}_\theta = \boldsymbol{u}_\theta + \frac{1}{2}\varepsilon_t^2 \nabla \log p_t$ is an affine function of the learned velocity. The SDE has the same marginals as the ODE, so the trained model can still be sampled deterministically. Flow-GRPO optimizes the policy with group relative policy optimization (GRPO) [306], which was developed for language models and avoids a learned value function. For each prompt c it samples a group of G trajectories, normalizes their rewards within the group, and maximizes a clipped surrogate,

$$A_i = \frac{r_i - \operatorname{mean}(r_{1:G})}{\operatorname{std}(r_{1:G})}, \qquad \mathcal{J}_{\mathrm{GRPO}} = \mathbb{E}\Big[\frac{1}{GK}\sum_{i,k} \min\left(\rho_{i,k}A_i,\ \operatorname{clip}(\rho_{i,k}, 1-\delta, 1+\delta)A_i\right)\Big] - \beta\,\mathrm{KL}, \quad (65)$$

with per-step ratios $\rho_{i,k} = \pi_\theta(\boldsymbol{x}^i_{k+1} \mid \boldsymbol{x}^i_k)/\pi_{\theta_{\mathrm{old}}}(\boldsymbol{x}^i_{k+1} \mid \boldsymbol{x}^i_k)$ and the KL term Eq. (64). The group mean is a baseline for Eq. (63) that needs no critic, and it removes the per-prompt offset that a Bradley–Terry reward does not identify. Flow-GRPO also trains with far fewer denoising steps than it uses at inference, which reduces the cost of a rollout, and reports large gains on GenEval, text rendering and human-preference rewards. DanceGRPO [403] applies the same reformulation to both diffusion and flow models and to image and video generation with several reward models. Figure 9 shows a group, its advantages and the resulting update on a policy for which every quantity is exact, and it shows what the KL term buys: without it, the policy collapses onto a single mode.

Credit assignment and cost. A rollout is expensive, and Eq. (65) assigns the same terminal advantage to every step of a trajectory, although early steps decide the layout and late steps only details. Several variants spend the stochasticity and the credit more carefully. MixGRPO [183] samples with the SDE and optimizes only inside a sliding window of steps and uses the ODE elsewhere. TempFlow-GRPO [120] branches trajectories at chosen timesteps so that the effect of a single step on the reward can be isolated, and weights steps according to their noise level. BranchGRPO [192] organizes rollouts as a tree with shared prefixes and prunes low-value branches, and G^{2}RPO [462] evaluates several sampling directions per step. DenseGRPO [73] turns the sparse reward into a dense one by scoring the Tweedie estimate $\hat{\boldsymbol{x}}_1$ after each step and using the gain in reward as that step's reward. Pave-GRPO [201] keeps few-step rollouts but decomposes each coarse transition into finer sub-trajectories through the average velocity, so that more timesteps receive feedback at no extra sampling cost.

Stability of the ratio. The clipping in Eq. (65) is meant to keep updates small, but in flow models it behaves differently from language models. GRPO-Guard [363] observes that the distribution of the ratios has a mean below one and a variance that varies strongly across timesteps, so positive-advantage samples rarely reach the clipping boundary and over-confident updates go unchecked while the proxy reward keeps rising. λ-Controlled GRPO [370] traces these symptoms to a single per-step quantity that is fixed by the Gaussian kernel Eq. (62) and budgets the update across steps accordingly. Flow-DPPO [272] replaces ratio clipping by a constraint on the exact KL divergence between the old and new Gaussian step policies, which Eq. (64) makes available in closed form. Pref-GRPO [368] notes that the normalization in Eq. (65) amplifies tiny reward differences within a group into large advantages, and replaces pointwise rewards by win rates from a pairwise preference model.



Figure 9: **Group-relative policy gradients for a flow** (Animation A8). The policy is an exact Gaussian mixture on a spiral whose 110 component logits are trainable, sampled with the reverse SDE so that every Euler–Maruyama step has a Gaussian likelihood, as in Flow-GRPO [208]. For this policy the gradient of each step's log-likelihood with respect to the logits is available in closed form. Top left: a group of 16 rollouts for one prompt, coloured by the group-normalized advantage $A_i = (r_i - \bar{r})/\mathrm{std}(r)$ of their final reward (green above the group mean, red below). Top right: the logit update produced by this group. Rings mark components whose weight rises (green) or falls (red). Bottom left: after 80 updates with a KL penalty to the reference model ($\beta = 0.3$), the policy concentrates under the reward but keeps several modes. Bottom right: expected reward and effective number of modes during training. Without the KL term the reward reaches 0.97 while the policy collapses to about one mode.

Policy gradients without the reverse chain. The MDP formulation ties training to one sampler and evaluates likelihoods of individual noisy steps. A second family keeps the pre-training loss and reweights it. Flow Policy Optimization [240] uses the conditional flow-matching loss in place of the log-likelihood inside a PPO-style ratio. Advantage Weighted Matching [402] shows that DDPO is an implicit form of score matching with noisy targets and proposes to weight the flow-matching loss of each generated sample by its advantage instead, which lowers the variance. DiffusionNFT [454] contrasts positive and negative generations on the forward process, which removes the dependence on the solver and on the likelihood of the reverse steps, and DiffusionART [357] identifies and removes a force in such objectives that only partially cancels between positive and negative samples. DiPOD [147] shows that optimizing a variational surrogate can let the ELBO drift away from the true log-likelihood, so that the surrogate gradient no longer points uphill, and adds an on-policy ELBO regularizer. Section 4.5 explains why reweighted regression is the natural form of the problem.

4.3 Differentiable rewards

When the reward is a differentiable network, as all the learned rewards of Section 4.1 are, a much lower-variance gradient is available. Sample with a fixed initial noise $\boldsymbol{x}_0$ and fixed noise increments, so that the final sample $\boldsymbol{x}_1 = \Phi_\theta(\boldsymbol{x}_0, \boldsymbol{\xi})$ is a differentiable function of the parameters. The pathwise gradient is

$$\nabla_\theta\, r(\boldsymbol{x}_1) = \sum_{k=0}^{K-1} \Big(\frac{\partial \boldsymbol{x}_1}{\partial \boldsymbol{x}_{k+1}}\Big)^\top \nabla_{\boldsymbol{x}_1} r(\boldsymbol{x}_1) \cdot \frac{\partial \boldsymbol{x}_{k+1}}{\partial \theta}\Big|_{\boldsymbol{x}_k}, \qquad \frac{\partial \boldsymbol{x}_1}{\partial \boldsymbol{x}_{k+1}} = \prod_{j=k+1}^{K-1} \frac{\partial \boldsymbol{x}_{j+1}}{\partial \boldsymbol{x}_j}, \tag{66}$$

backpropagation through the whole sampler. Compared with Eq. (63), it uses the direction in which the reward increases rather than only its value, and a few hundred prompts are often enough to change a model substantially.

DRaFT, AlignProp and ReFL. The cost is memory, since every step's activations are needed for the backward pass, and the products of Jacobians in Eq. (66) can explode or vanish over many steps. DRaFT [68] trains LoRA weights with gradient checkpointing and observes that truncating the backward pass to the last K steps (DRaFT-K) is not only cheaper but often better. AlignProp [276] similarly backpropagates through a randomly truncated part of the chain. ReFL [396] goes furthest. It stops the sampler at a random late step t, forms the one-step Tweedie prediction $\hat{\boldsymbol{x}}_1(\boldsymbol{x}_t, t)$, and backpropagates the reward of that prediction through a single network call. These truncations bias the gradient: they ignore how the parameters affect the earlier, layout-defining steps. FlowBP [366] treats the short surrogate path used in the backward pass as a design choice in itself, for flow-matching models at modern scale.

Strengths and limits. Pathwise methods are the most sample-efficient, but they need a differentiable reward and therefore cannot use programs, humans or non-differentiable VLM judgments directly. They also optimize hard. Without an explicit KL term, which DRaFT and AlignProp do not use, the reward keeps improving until the samples exploit the reward model, and early stopping or scaling down the LoRA weights become the de facto regularizers (Section 4.6).

4.4 Preference optimization

Reward models are an intermediate step. If the data are pairwise preferences, one can ask whether the generator can be trained on them directly.

DPO. Direct preference optimization [282] starts from the optimum Eq. (60). Solving it for the reward gives $r(\boldsymbol{x}, c) = \beta \log \frac{p^\star(\boldsymbol{x}|c)}{p_{\text{ref}}(\boldsymbol{x}|c)} + \beta \log Z(c)$. Substituting this into the Bradley–Terry likelihood Eq. (61), the partition function cancels in the difference, and parametrizing $p^\star$ by the model gives

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E} \log \sigma\Big(\beta \log \frac{p_\theta(\boldsymbol{x}^w \mid c)}{p_{\text{ref}}(\boldsymbol{x}^w \mid c)} - \beta \log \frac{p_\theta(\boldsymbol{x}^l \mid c)}{p_{\text{ref}}(\boldsymbol{x}^l \mid c)}\Big). \tag{67}$$

The reward model is implicit in the policy, and training is a classification problem on fixed data, with no sampling.

Diffusion-DPO. For a diffusion model the log-likelihoods in Eq. (67) are intractable. Diffusion-DPO [356] replaces the terminal densities by the densities of whole chains, for which the KL-regularized objective is again of the form Eq. (59), and the log-ratio of chain densities is a sum of per-step log-ratios. Two approximations make this tractable. Jensen's inequality moves the expectation over chains outside the convex function $-\log \sigma$, which gives an upper bound on the loss. And the intractable reverse chain given $\boldsymbol{x}_1$ is replaced by the forward noising process $q(\boldsymbol{x}_t \mid \boldsymbol{x}_1)$, which can be sampled in closed form. Each per-step log-ratio then becomes a difference of the Gaussian KL terms of the ELBO Eq. (28), and these are weighted squared errors of the noise prediction. With noise $\boldsymbol{x}_0$, $\boldsymbol{x}_t = \alpha_t \boldsymbol{x}_1 + \sigma_t \boldsymbol{x}_0$ and the noise prediction $\hat{\boldsymbol{x}}_0$ of Table 3, define the improvement of the model over the reference on a sample,

$$\Delta_\theta(\boldsymbol{x}_1, t) = \big\|\hat{\boldsymbol{x}}_0^\theta(\boldsymbol{x}_t, t, c) - \boldsymbol{x}_0\big\|^2 - \big\|\hat{\boldsymbol{x}}_0^{\text{ref}}(\boldsymbol{x}_t, t, c) - \boldsymbol{x}_0\big\|^2. \tag{68}$$

The resulting loss is

$$\mathcal{L}_{\text{Diff-DPO}}(\theta) = -\mathbb{E}_{t, \boldsymbol{x}_0^w, \boldsymbol{x}_0^l} \log \sigma\Big(- \beta\, w(t) \big(\Delta_\theta(\boldsymbol{x}_1^w, t) - \Delta_\theta(\boldsymbol{x}_1^l, t)\big)\Big), \tag{69}$$

where $w(t)$ collects the ELBO weight and the number of steps. The model is rewarded for denoising the preferred image better, relative to the reference, than the rejected one. If it simply denoised both better, the two terms would cancel. Trained on Pick-a-Pic comparisons, Diffusion-DPO improved the visual appeal and prompt alignment of SDXL in human evaluations. VideoReward derives the analogous Flow-DPO for rectified-flow video models [209].

Variants. Preferences on final images say little about which denoising step was responsible. SPO [196] constructs preferences at each step: from a common $\boldsymbol{x}_t$ it samples several candidates for the next step, ranks them with a step-aware preference model, and trains on the best and worst. Diffusion-KTO [185] needs only binary per-image feedback, such as likes and dislikes, by maximizing a utility of each generation rather than a likelihood of pairs. An earlier method of Lee et al. [173] trained a reward model on human labels and fine-tuned the generator by reward-weighted likelihood, a precursor of the regression objectives in Section 4.5. Offline preference optimization has a basic limitation: it learns from samples of an older model, and its improvements saturate once the policy moves away from the data. The online methods of Section 4.2 generate their own training data.

4.5 A stochastic optimal control view

The methods so far approximate Eq. (59) in different ways. Control theory solves it. Represent the fine-tuned model as a base SDE with an added control, $\mathrm{d}\boldsymbol{x}_t = (\boldsymbol{b}_t + \varepsilon_t \boldsymbol{c}_t)\,\mathrm{d}t + \varepsilon_t\,\mathrm{d}\boldsymbol{W}_t$, as in Section 2.6. By Eqs. (37) to (39), the KL-regularized objective on path measures is maximized by the control $\boldsymbol{c}_t^\star = \varepsilon_t \nabla \log h_t$, with $h_t(\boldsymbol{x}) = \mathbb{E}_{P^{\text{base}}}[e^{r(\boldsymbol{x}_1)/\beta} \mid \boldsymbol{x}_t = \boldsymbol{x}]$ [79, 352].

The initial value function bias. Path-level regularization, as in DPOK, Flow-GRPO and Eq. (64), does not in general produce the terminal target Eq. (60). Under the optimal path measure the pair $(\boldsymbol{x}_0, \boldsymbol{x}_1)$ is distributed as $P_{01}^{\text{base}}(\boldsymbol{x}_0, \boldsymbol{x}_1)\, e^{r(\boldsymbol{x}_1)/\beta}/Z$. The controlled process, however, starts from the unchanged noise distribution p_0, so its terminal marginal is

$$p_1^{c^\star}(\boldsymbol{x}_1) = \int p_0(\boldsymbol{x}_0)\, P^\star(\boldsymbol{x}_1 \mid \boldsymbol{x}_0)\, \mathrm{d}\boldsymbol{x}_0 = p_1^{\text{base}}(\boldsymbol{x}_1)\, e^{r(\boldsymbol{x}_1)/\beta} \int \frac{P^{\text{base}}(\boldsymbol{x}_0 \mid \boldsymbol{x}_1)}{h_0(\boldsymbol{x}_0)}\, \mathrm{d}\boldsymbol{x}_0, \tag{70}$$

using $P^\star(\boldsymbol{x}_1 \mid \boldsymbol{x}_0) = P^{\text{base}}(\boldsymbol{x}_1 \mid \boldsymbol{x}_0)\, e^{r(\boldsymbol{x}_1)/\beta}/h_0(\boldsymbol{x}_0)$ and Bayes' rule. This equals $p_1^{\text{base}} e^{r/\beta}/Z$ for every reward exactly when h_0 is constant, that is when $\boldsymbol{x}_1$ is independent of $\boldsymbol{x}_0$ under the base process. For the probability-flow ODE, or for DDIM-like samplers with little noise, $\boldsymbol{x}_1$ is nearly a deterministic function of $\boldsymbol{x}_0$. Then $h_0(\boldsymbol{x}_0) \approx e^{r(\boldsymbol{x}_1(\boldsymbol{x}_0))/\beta}$, the integral nearly cancels the tilt, and the optimal control mostly reshuffles which noise leads to which image rather than changing the distribution of images. Domingo-Enrich et al. [79] call this the initial value function bias.

The memoryless schedule. The noise level $\varepsilon_t^2 = \sigma_t^2 \dot{\lambda}_t$ of Proposition 2.3, which reproduces the reverse-time SDE, makes $\boldsymbol{x}_1$ independent of $\boldsymbol{x}_0$ (Section 2.4), so it removes the bias. For a flow model the corresponding base drift follows from Eq. (25): since $\frac{1}{2}\sigma_t^2 \dot{\lambda}_t \nabla \log p_t = \boldsymbol{u}_t - (\dot{\alpha}_t/\alpha_t)\boldsymbol{x}$,

$$\boldsymbol{b}_t(\boldsymbol{x}) = \boldsymbol{u}_t(\boldsymbol{x}) + \tfrac{1}{2}\sigma_t^2 \dot{\lambda}_t \nabla \log p_t(\boldsymbol{x}) = 2\boldsymbol{u}_t(\boldsymbol{x}) - \frac{\dot{\alpha}_t}{\alpha_t}\, \boldsymbol{x}. \tag{71}$$

The tilt changes only the clean endpoint and leaves the noising law $\boldsymbol{x}_t \mid \boldsymbol{x}_1$ unchanged, so the fine-tuned process is again a diffusion model, now of $p^\star$, and Eq. (71) holds for it too. The fine-tuned velocity is $\boldsymbol{u}^\star = \boldsymbol{u} + \frac{1}{2}\varepsilon_t \boldsymbol{c}^\star$, and it can be sampled with any ODE or SDE solver. The memoryless schedule is needed only during training.

Adjoint Matching. It remains to estimate $\nabla \log h_t$. Differentiating h_t under the expectation, and using that $P^\star$ is P^{base} reweighted by $e^{r/\beta}/Z$, gives

$$\nabla \log h_t(\boldsymbol{x}) = \mathbb{E}_{P^\star}\big[\tilde{\boldsymbol{a}}_t \,\big|\, \boldsymbol{x}_t = \boldsymbol{x}\big], \qquad \tilde{\boldsymbol{a}}_t = J_{t\to 1}^\top \frac{\nabla r(\boldsymbol{x}_1)}{\beta}, \tag{72}$$

where $J_{t\to 1} = \partial \boldsymbol{x}_1/\partial \boldsymbol{x}_t$ is the Jacobian of the base dynamics along the path. The vector $\tilde{\boldsymbol{a}}_t$, the *lean adjoint*, solves the linear ODE $\dot{\tilde{\boldsymbol{a}}}_t = -(\nabla_{\boldsymbol{x}} \boldsymbol{b}_t(\boldsymbol{x}_t))^\top \tilde{\boldsymbol{a}}_t$ backwards from $\tilde{\boldsymbol{a}}_1 = \nabla r(\boldsymbol{x}_1)/\beta$, at the cost of one vector–Jacobian product per step. By Lemma 2.1, the conditional expectation in Eq. (72) is the minimizer of a regression, which gives the Adjoint Matching loss [79]

$$\mathcal{L}_{\text{AM}}(\theta) = \mathbb{E} \int_0^1 \big\| \boldsymbol{c}_\theta(\boldsymbol{x}_t, t) - \varepsilon_t\, \tilde{\boldsymbol{a}}_t \big\|^2\, \mathrm{d}t, \tag{73}$$

with trajectories drawn from the current model without gradients and the adjoint computed along them. Its fixed point is $\boldsymbol{c}^\star$. Unlike reward backpropagation, the loss needs no backward pass through the network across steps, only through the base drift, and unlike policy gradients it uses the reward gradient. Domingo-Enrich and Han [78] derive Adjoint Matching from the stochastic maximum principle, which extends it to controls that also change the diffusion coefficient and to convex running costs, and Thilges et al. [343] use it to train diffusion policies for maximum-entropy reinforcement learning.

Regression on the forward process. Because the tilted model is a diffusion model of $p^\star$ with the original noising law, its velocity is the marginal velocity of Proposition 2.2 with $\boldsymbol{x}_1 \sim p^\star$. Flow matching on samples from $p^\star$ would therefore train it directly. Such samples are not available, but samples from the base model are, and importance weighting gives

$$\mathbb{E}_{\boldsymbol{x}_1 \sim p^\star} \, \ell_{\mathrm{FM}}(\theta; \boldsymbol{x}_1) = \frac{1}{Z} \, \mathbb{E}_{\boldsymbol{x}_1 \sim p^{\mathrm{base}}} \big[e^{r(\boldsymbol{x}_1)/\beta} \, \ell_{\mathrm{FM}}(\theta; \boldsymbol{x}_1) \big], \tag{74}$$

where $\ell_{\mathrm{FM}}(\theta; \boldsymbol{x}_1)$ is the flow-matching loss Eq. (24) for a fixed clean sample. This is reward-weighted regression. With one tilt the weights have high variance. Repeating small tilts on samples from the current model composes them, $p_{k+1} \propto p_k e^{r/\beta_k}$, and normalizing the weights within a group of samples for the same prompt gives the advantage weights of Eq. (65). This explains why the reweighted pre-training losses at the end of Section 4.2 work. Several 2026 papers make the connection precise. Reinforce Adjoint Matching [20] combines the adjoint-matching optimality condition with a REINFORCE identity into a regression that draws a clean sample from the current model, scores it, noises it as in pre-training and regresses onto a reward-corrected target, with no SDE rollouts, adjoint sweeps or reward gradients. It reaches the peak reward of Flow-GRPO in up to 50 times fewer training steps on Stable Diffusion 3.5. Reward-based velocity matching [63] weights the velocity-matching loss by the reward and adds an anchor term, recovers the update of Reinforce Adjoint Matching and contains DiffusionNFT as a special case.

Unifying frameworks. Reward score matching [171] writes many existing methods as score matching against a value-guided target and finds that they differ mainly in how they estimate the value guidance and how strongly they update at each timestep. Domingo-Enrich et al. [80] relate adjoint-based methods, score-matching methods and non-equilibrium thermodynamics in one framework and show by bias–variance decompositions that Adjoint Matching has finite gradient variance while target and conditional score matching do not. Newton Matching [193] treats fine-tuning towards $p^{\mathrm{base}} e^{\tau r}$ as optimization over the manifold of densities and shows that the Newton direction for the reverse KL divergence is the negative Fisher–Rao gradient, which yields regression losses with locally quadratic convergence and recovers several existing methods as special cases. The picture that emerges is that post-training of diffusion and flow models, like pre-training, is best done by regression onto a target defined by the optimal control.

4.6 Reward hacking and regularization

The objective Eq. (59) assumes that the reward is right. A learned reward is a proxy, and maximizing a proxy beyond a point decreases the quantity it was meant to measure. Figure 10 shows the three regimes on an exact mixture with a linear reward. With a large β the tilt Eq. (60) moves mass towards high reward and stays on the data. With a smaller β it concentrates on the few highest-reward modes, which is mode collapse: diversity is lost even though every sample is good. With almost no regularization the samples leave the data, where the proxy is still high but the samples are no longer images of anything. The proxy reward rises monotonically with the KL divergence from the base model, while the true reward first rises and then falls.

Why it happens. The exact tilt Eq. (60) cannot leave the support of p_{ref}, since it is a reweighting. Two things break this guarantee in practice. First, regularization is often weak or absent. DRaFT and AlignProp use none, GRPO is often run with $\beta = 0$, and a path-level KL with few steps is a loose surrogate. Second, the support of a learned density is effectively everything, and a reward model extrapolates freely outside its training distribution, so regions of tiny base probability but

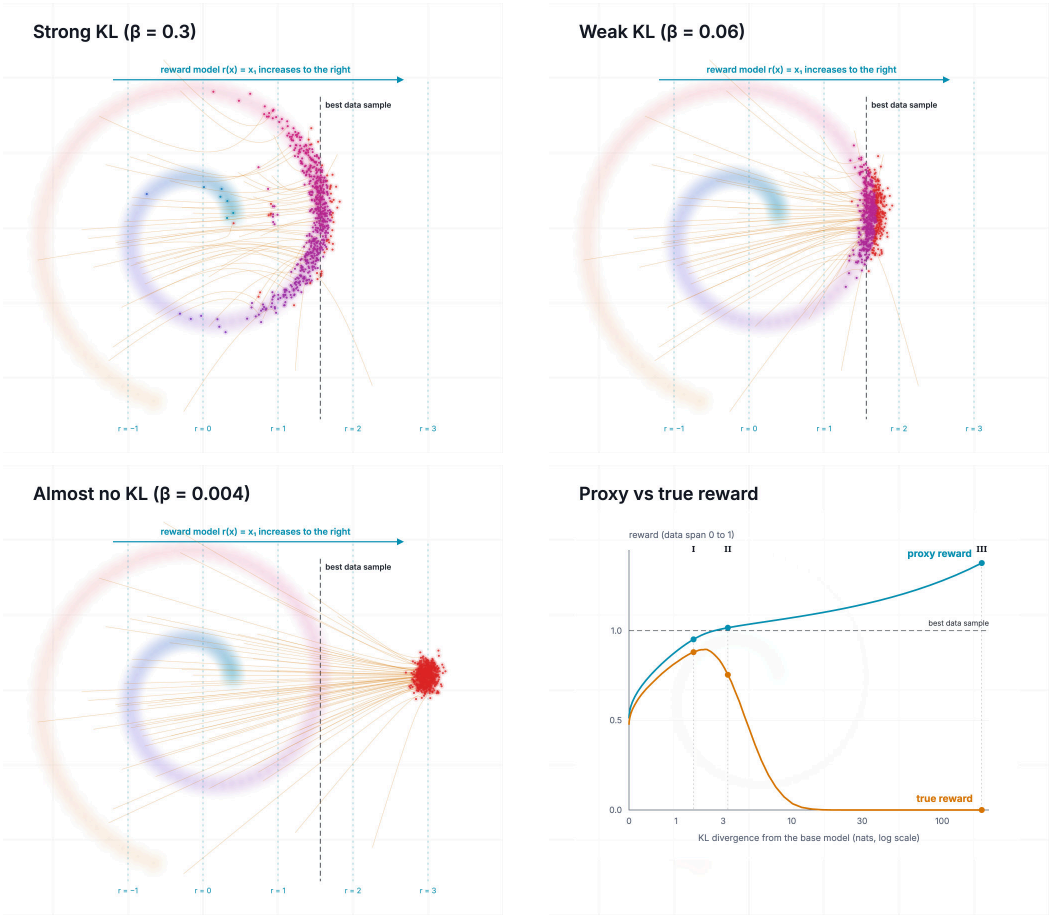


Figure 10: **Reward tilting and reward hacking** (Animation A7). The base model p is an exact Gaussian mixture on a spiral, and the reward model is the linear function $r(x) = x_1$. The optimum of $\max_q \mathbb{E}_q[r] - \beta\,\mathrm{KL}(q \,\|\, p)$ is $q^*(x) \propto p(x)\exp(r(x)/\beta)$, which for this model is again a Gaussian mixture: each component is reweighted by $\exp(\mu_{k,1}/\beta)$ and shifted by σ^2/β along the reward direction. Top left: with a strong KL term, mass moves toward high reward and stays on the data. Top right: a weaker KL term collapses the samples onto the few highest-reward modes. Bottom left: with almost no KL term, the samples leave the data (red) and receive higher proxy reward than any real sample. Bottom right: proxy reward and a toy true reward, which counts only samples within 2σ of a data mode, against the KL divergence from p. The true reward peaks at a moderate KL and then falls to zero.

large proxy reward exist and are found by the optimizer. Hong et al. [125] analyse text-to-image RL with aesthetic, preference and prompt-consistency rewards and find a common failure, images full of artifacts that the rewards score highly, which reward ensembles only partly prevent. They add a small learned artifact reward as a regularizer. Zhang et al. [443] trace over-optimization in part to a mismatch between the per-step structure of denoising and how methods assign reward across steps.

Remedies. The KL term is the principled remedy, and the choice of β trades reward against fidelity along the curve of Fig. 10. In practice, early stopping, LoRA scaling and evaluation on held-out rewards play the same role. GARDO [117] regularizes selectively: it penalizes only samples whose reward is uncertain, periodically moves the reference model towards the current policy so that the penalty does not hold back exploration, and amplifies rewards for samples that are both good and diverse. GRPO-Guard [363] and Pref-GRPO [368] address the over-optimization that comes from the GRPO update itself (Section 4.2). Multi-reward training has its own failure, a seesaw in which improving one reward degrades another. Flow-OPD [91] trains a separate expert for each reward with GRPO and then distils the experts into a single model with on-policy distillation. Qwen-Image-2.0-RL combines GRPO with on-policy distillation and a hybrid use of classifier-free guidance to preserve pre-trained capabilities [401]. Finally, the reward can be optimized at sampling time instead of in the weights (Section 6), which keeps the base model intact and makes the trade-off adjustable per prompt [303].

4.7 Reinforcement learning for discrete diffusion

Reinforcement learning with verifiable rewards, such as the correctness of a math answer or the result of unit tests, has driven much of the recent progress in reasoning for autoregressive language models, where GRPO [306] needs only the exact log-probability of each generated token. A masked diffusion language model (Section 2.9) has no such factorization. The probability of a completion depends on the order in which tokens were revealed, and the ELBO Eq. (47) bounds it only from one side.

d1 [448] adapts pretrained masked models to reasoning with masked supervised fine-tuning followed by diffu-GRPO, a policy-gradient method that estimates per-token log-probabilities from a single forward pass. DiRL [468] builds an efficient training and inference loop for blockwise models and proposes an unbiased GRPO implementation for them. Later methods use more of the structure of masked generation. Mask-aware policy gradients [281] model each step as two decisions, which tokens to place and which positions to remask, and show that the policy gradient splits into a token term and a masking term. CAPR [156] uses the denoising trace of a block-wise sampler to redistribute the final reward across blocks through a learned value head. CanvasAnneal [260] addresses exploration by placing teacher-written reasoning into the initial canvas and removing it gradually during training. The ELBO-drift analysis of DiPOD (Section 4.2) applies to these models as well [147]. The flexibility of arbitrary-order decoding also has a cost here, since a model can postpone the uncertain tokens on which exploration depends [251]. For visual tokens, X-Omni shows that reinforcement learning substantially improves a discrete autoregressive image generator [103], and reasoning-style reinforcement of image generation with chain-of-thought is studied by Guo et al. [111] and Jiang et al. [146].

5 Distillation and Few-Step Generation

A trained diffusion or flow model defines its samples through a differential equation, and a sampler pays for every step with one evaluation of a large network. The cost of a sample is the number of function evaluations (NFE) times the cost of one evaluation. Better solvers and schedules reduce the first factor without training [221, 291], but they cannot go much below ten steps, because the probability-flow trajectories are curved (Section 2.4). This section is about methods that train a new network, the student, to generate in one to four steps. The last subsection turns to the second factor and makes each evaluation cheaper.

It helps to organize the methods by what the student is asked to match. Table 4 gives the taxonomy that structures the section. Trajectory methods match the teacher's ODE solution sample by sample. Distribution methods match only the distribution of the student's outputs, through the scores of the noised distributions. Adversarial methods match distributions through a learned discriminator. Flow-map methods learn the solution operator of the ODE between any two times, which contains consistency models and several recent objectives as special cases, and which can be trained with or without a teacher. The distinction between matching samples and matching distributions recurs throughout: the first is a regression whose minimizer averages over what the student cannot resolve, and the second is a divergence whose minimizer can be sharp but may drop modes.

Table 4: A taxonomy of few-step methods by what the student matches.

Family	Object matched	Character of the loss	Examples
Trajectory	ODE endpoint of each noise sample	per-sample regression	progressive distillation, reflow, InstaFlow, consistency models, LCM
Distribution	noised marginals $q_{\theta,t}$ and p_t	reverse KL or Fisher divergence, mode-seeking	SDS, VSD, DMD, DMD2, SiD
Adversarial	samples against data or teacher samples	learned divergence	ADD, LADD, adversarial flow models
Flow map	solution operator $X_{s,t}$ for all $s < t$	regression with a self-consistency target	CTM, sCM, rCM, shortcut models, MeanFlow, AYF, TVM

5.1 Trajectory distillation

Regressing on the ODE endpoint. The deterministic sampler defines a map from noise to data: integrating the probability-flow ODE $\dot{\boldsymbol{x}}_t = \boldsymbol{u}_t(\boldsymbol{x}_t)$ from $\boldsymbol{x}_0 \sim \mathcal{N}(0, I)$ gives $\boldsymbol{x}_1 = X_{0,1}(\boldsymbol{x}_0)$. The most direct distillation trains a one-step student G_θ by regression on this map,

$$\mathcal{L}_{\mathrm{KD}}(\theta) = \mathbb{E}_{\boldsymbol{x}_0} \big\| G_\theta(\boldsymbol{x}_0) - X_{0,1}(\boldsymbol{x}_0) \big\|^2. \tag{75}$$

Luhman and Luhman [227] showed that this works on small images. It needs a dataset of teacher trajectories, each costing a full solve, and a network that can represent a map whose Jacobian is large and irregular wherever nearby noise samples end on different modes.

Progressive distillation. Salimans and Ho [297] make the regression easier by halving the number of steps in each round. A student initialized from the teacher learns to reproduce two deterministic DDIM steps of the teacher with one step of its own. Write a DDIM step from t to t' in terms of a data prediction $\hat{\boldsymbol{x}}$: the implied noise is $\hat{\boldsymbol{x}}_0 = (\boldsymbol{x}_t - \alpha_t \hat{\boldsymbol{x}})/\sigma_t$, and the step returns

$\boldsymbol{x}_{t'} = \alpha_{t'}\hat{\boldsymbol{x}} + \sigma_{t'}\hat{\boldsymbol{x}}_0$. If the teacher's two steps take $\boldsymbol{x}_t$ to $\boldsymbol{x}_{t''}$, the data prediction that makes a single student step from t to t'' land at the same point solves this linear equation,

$$\tilde{\boldsymbol{x}}_1 = \frac{\boldsymbol{x}_{t''} - (\sigma_{t''}/\sigma_t)\,\boldsymbol{x}_t}{\alpha_{t''} - (\sigma_{t''}/\sigma_t)\,\alpha_t}, \tag{76}$$

and the student regresses its data prediction on $\tilde{\boldsymbol{x}}_1$. After $\log_2 N$ rounds an N-step sampler becomes a one-step generator. The same paper introduced the velocity parametrization, because a noise prediction is useless at the pure-noise end where $\sigma_t = 1$ and every step must say something about the data. Guided teachers add one more stage that distills the two branches of classifier-free guidance into a single network conditioned on the guidance scale [244].

Reflow: change the coupling, then distill. The difficulty of Eq. (75) comes from curvature. One Euler step from $\boldsymbol{x}_0$ makes the error

$$X_{0,1}(\boldsymbol{x}_0) - (\boldsymbol{x}_0 + \boldsymbol{u}_0(\boldsymbol{x}_0)) = \int_0^1 (\boldsymbol{u}_t(\boldsymbol{X}_t) - \boldsymbol{u}_0(\boldsymbol{x}_0))\,\mathrm{d}t, \tag{77}$$

which vanishes only if the velocity is constant along the trajectory. For the linear path with the independent coupling the velocity at $t = 0$ is particularly uninformative. There $\boldsymbol{x}_t = \boldsymbol{x}_0$, so $\boldsymbol{u}_0(\boldsymbol{x}) = \mathbb{E}[\boldsymbol{x}_1 - \boldsymbol{x}_0 \mid \boldsymbol{x}_0 = \boldsymbol{x}] = \mathbb{E}[\boldsymbol{x}_1] - \boldsymbol{x}$, and one Euler step sends every sample to the data mean. Figure 11 shows this collapse on an exact mixture. Rectified flow [214] replaces the independent coupling by the coupling of the ODE itself, $(\boldsymbol{x}_0, X_{0,1}(\boldsymbol{x}_0))$, and retrains flow matching on it. By Eq. (34) the transport cost does not increase, and Liu et al. [214] also bound a straightness measure $\int_0^1 \mathbb{E}\|\boldsymbol{X}_1 - \boldsymbol{X}_0 - \boldsymbol{u}_t(\boldsymbol{X}_t)\|^2 \mathrm{d}t$, which is the expected squared size of the integrand in Eq. (77), and show that its smallest value over K rounds of reflow decreases like $1/K$. After one or two rounds a single Euler step is already close to the ODE solution, and a final round of Eq. (75) removes the rest. InstaFlow [215] applied this pipeline to Stable Diffusion and found that the reflow stage is what makes the subsequent one-step distillation succeed. Reflow and minibatch optimal transport (Section 2.6) pursue the same goal from opposite ends: optimal transport chooses a straighter coupling before training, and reflow extracts one from a trained model.

Consistency models. A consistency model [325] learns the map from every point of a trajectory to its endpoint, $f_\theta(\boldsymbol{x}_t, t) \approx X_{t,1}(\boldsymbol{x}_t)$. The boundary condition $f_\theta(\boldsymbol{x}, 1) = \boldsymbol{x}$ is built into the parametrization $f_\theta = c_{\text{skip}}(t)\,\boldsymbol{x} + c_{\text{out}}(t)\,F_\theta(\boldsymbol{x}, t)$ with $c_{\text{skip}}(1) = 1$ and $c_{\text{out}}(1) = 0$. The training signal is that the endpoint is the same for any two points of one trajectory. With a grid $t_0 < t_1 < \cdots < t_N = 1$, consistency distillation minimizes

$$\mathcal{L}_{\text{CD}}(\theta) = \mathbb{E}_{n, \boldsymbol{x}_{t_n}}\, d(f_\theta(\boldsymbol{x}_{t_n}, t_n),\, f_{\theta^-}(\hat{\boldsymbol{x}}_{t_{n+1}}, t_{n+1})), \tag{78}$$

where $\hat{\boldsymbol{x}}_{t_{n+1}}$ is one teacher ODE step from $\boldsymbol{x}_{t_n}$, d is a distance and θ^- is a stop-gradient copy of the weights. The target at t_{n+1} is closer to the data, where the boundary condition anchors it, and the information propagates backwards along the trajectory over training. Consistency training removes the teacher by replacing the ODE step with the conditional sample $\boldsymbol{x}_{t_{n+1}} = \alpha_{t_{n+1}}\boldsymbol{x}_1 + \sigma_{t_{n+1}}\boldsymbol{x}_0$ built from the same pair $(\boldsymbol{x}_0, \boldsymbol{x}_1)$ as $\boldsymbol{x}_{t_n}$. This works because the conditional velocity is an unbiased estimate of the marginal one (Proposition 2.2), so the two targets agree to first order in the step size. Song and Dhariwal [319] removed the exponential moving average from θ^-, which they showed biases consistency training, and replaced learned perceptual distances with a pseudo-Huber loss. Latent consistency models [228] distill Stable Diffusion in latent space by applying Eq. (78) to the ODE of the guided velocity, which folds classifier-free guidance into the student and gives

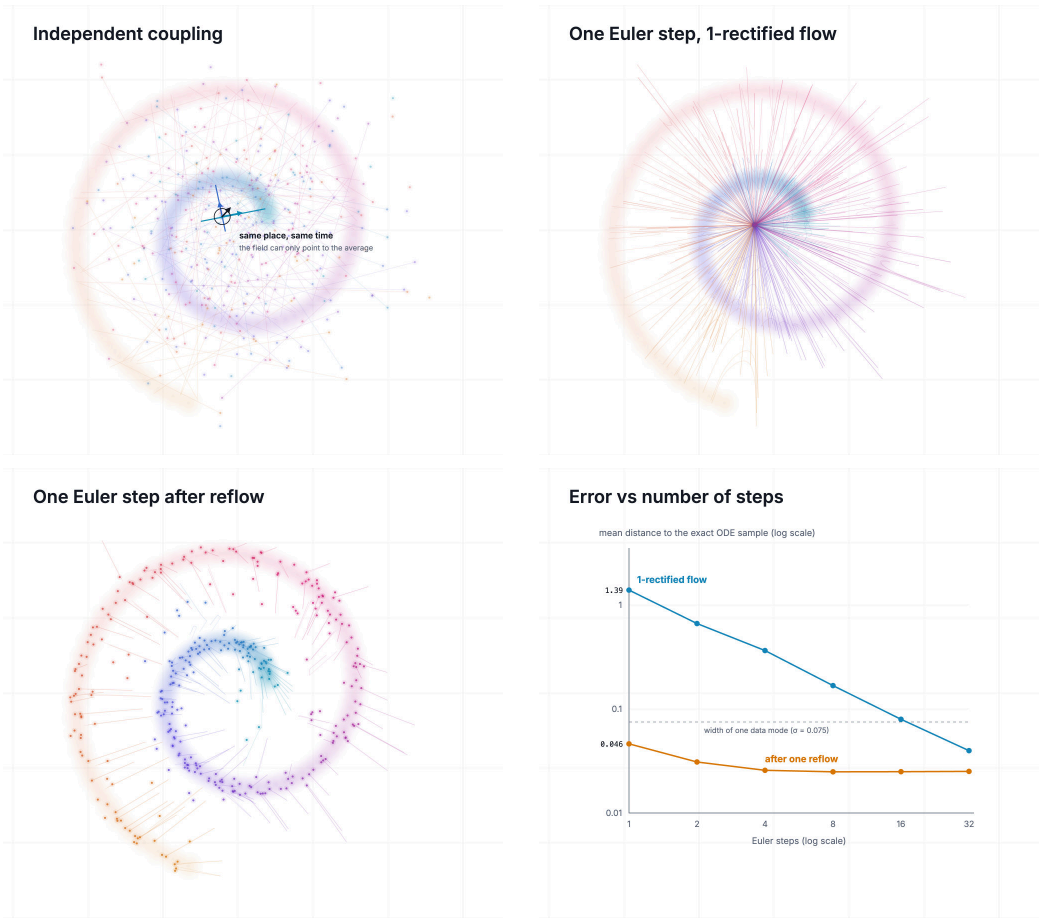


Figure 11: **Reflow straightens paths** (Animation A10). Independent noise-data pairs give straight training lines that cross, and where two cross at the same time the regressed velocity is their average (top left). The resulting 1-rectified flow is exact for this Gaussian mixture, but its velocity at $t = 0$ points every sample to the data mean, so one Euler step collapses all samples to one point (top right). After pairing each noise sample with its own ODE endpoint and refitting the field by kernel regression, one Euler step lands on the data (bottom left). Mean distance to the exact ODE sample against the number of Euler steps (bottom right).

two- to four-step samplers. Multi-step sampling with a consistency model alternates a jump to the data with renoising to a smaller noise level. The renoising discards the deterministic link to the trajectory, which is why quality stops improving, and can degrade, as steps are added [293]. Section 5.4 resolves this by learning jumps between arbitrary times.

5.2 Distribution matching

From samples to distributions. Trajectory methods ask the student to reproduce one particular map from noise to data. Any map that pushes $\mathcal{N}(0, I)$ to p_{data} would do, and asking for less makes the student's task easier. Let G_θ be a one-step generator, q_θ the law of $G_\theta(\boldsymbol{z})$ with $\boldsymbol{z} \sim \mathcal{N}(0, I)$, and $q_{\theta,t}$ and p_t the laws obtained by noising the generator's samples and the data with the same forward process. Distribution matching minimizes a divergence between $q_{\theta,t}$ and p_t, averaged over t. The noise matters: q_θ and p_{data} may have disjoint supports, which makes the KL divergence between them infinite, while their noised versions have full support and smooth scores at every $t < 1$.

Proposition 5.1 (Gradient of the noised reverse KL)**.** *Let $\boldsymbol{x}_t = \alpha_t G_\theta(\boldsymbol{z}) + \sigma_t \boldsymbol{\epsilon}$ with $\boldsymbol{z}, \boldsymbol{\epsilon} \sim \mathcal{N}(0, I)$, and let $\boldsymbol{s}_{p,t} = \nabla \log p_t$ and $\boldsymbol{s}_{q,t} = \nabla \log q_{\theta,t}$. Then*

$$\nabla_\theta \mathrm{KL}(q_{\theta,t} \,\|\, p_t) = \mathbb{E}_{\boldsymbol{z},\boldsymbol{\epsilon}}\Big[\alpha_t \left(\boldsymbol{s}_{q,t}(\boldsymbol{x}_t) - \boldsymbol{s}_{p,t}(\boldsymbol{x}_t)\right)^\top \frac{\partial G_\theta(\boldsymbol{z})}{\partial \theta}\Big]. \tag{79}$$

Derivation sketch. Write $\mathrm{KL}(q_{\theta,t} \| p_t) = \mathbb{E}_{\boldsymbol{z},\boldsymbol{\epsilon}}[\log q_{\theta,t}(\boldsymbol{x}_t) - \log p_t(\boldsymbol{x}_t)]$ and differentiate under the expectation. The parameter enters $\log q_{\theta,t}(\boldsymbol{x}_t)$ twice, through the point $\boldsymbol{x}_t$ and through the density itself. The first gives $\boldsymbol{s}_{q,t}(\boldsymbol{x}_t)^\top \partial \boldsymbol{x}_t / \partial \theta$. The second gives $\mathbb{E}_{q_{\theta,t}}[\nabla_\theta \log q_{\theta,t}] = \nabla_\theta \int q_{\theta,t} = 0$. The term $\log p_t(\boldsymbol{x}_t)$ depends on θ only through $\boldsymbol{x}_t$, and $\partial \boldsymbol{x}_t / \partial \theta = \alpha_t \, \partial G_\theta / \partial \theta$.

The update moves each generated sample along the difference of two scores, towards high density under the data and away from regions where the generator already puts its mass. The first score comes from the teacher, through Eq. (25). The second is the score of the student's own noised distribution, which changes as the student trains. Distribution matching distillation (DMD) [416] estimates it with a second diffusion model, the fake score network, trained online by denoising score matching on generator samples. With data predictions, Tweedie's formula Eq. (12) turns the score difference into a difference of denoisers, $\boldsymbol{s}_{q,t} - \boldsymbol{s}_{p,t} = (\alpha_t / \sigma_t^2)(\hat{\boldsymbol{x}}_q - \hat{\boldsymbol{x}}_p)$, so the student is pushed towards what the teacher would denoise its noised sample to and away from what the fake model would. Figure 12 shows the two parts on an exact mixture. The real score alone is a mode-seeking force and drives all samples to the nearest modes. The fake score repels samples from each other and spreads them over the whole support.

Score distillation as a degenerate case. Score distillation sampling (SDS) [275] optimized 3D scenes against a pretrained image model before DMD and uses the update $\mathbb{E}[w_t(\hat{\boldsymbol{\epsilon}}_p(\boldsymbol{x}_t) - \boldsymbol{\epsilon}) \, \partial \boldsymbol{x} / \partial \theta]$, where $\boldsymbol{x}$ is a rendered image and $\hat{\boldsymbol{\epsilon}}_p$ is the teacher's noise prediction. Since $\hat{\boldsymbol{\epsilon}}_p = -\sigma_t \boldsymbol{s}_{p,t}$, the first term is the real score. The second is also a score: if q were a point mass at the current image $\boldsymbol{x}$, its noised score at $\boldsymbol{x}_t$ would be $-(\boldsymbol{x}_t - \alpha_t \boldsymbol{x})/\sigma_t^2 = -\boldsymbol{\epsilon}/\sigma_t$. SDS is therefore Eq. (79) with the fake distribution collapsed to a single point, which explains its oversaturated, low-diversity results: nothing in the update spreads the samples. Variational score distillation (VSD) [372] treats the 3D parameters as a distribution represented by particles and estimates the fake score with a low-rank adaptation of the pretrained model trained on the particles' renders, which is the update of DMD applied to text-to-3D.



Figure 12: **Distribution matching with real and fake scores** (Animation A11). The real score of the noised data and the fake score of the noised generator samples, both drawn at noise level 0.2; their difference moves each sample (top left). Following the real score alone sends every sample to the nearest mode, and only 42 of the 110 modes are reached (top right). Moving the samples by the score difference at random noise levels, with the fake score refit after every step, spreads them over the whole spiral (bottom left). Maximum mean discrepancy to held-out data against the number of updates (bottom right). The real score is exact; the fake score is a Gaussian kernel density over 500 free particles that stand in for the generator.

Making it stable. The reverse KL is mode-seeking, and the fake score lags behind the generator. DMD therefore added a regression loss Eq. (75) on a precomputed set of teacher noise–image pairs. DMD2 [417] removed it, which removes the dataset cost and the tie to the teacher's paths, and restored stability with a two time-scale update in which the fake score network takes several steps per generator step. It also added a GAN loss on features of the fake score network and extended the method to several steps by training on the student's own intermediate samples, which avoids a mismatch between training and inference inputs. Score identity distillation (SiD) [461] replaces the KL divergence by a Fisher divergence between the two scores along the generator's diffusion and uses score identities to rewrite it in a form whose gradient can be estimated with the fake score network. It is data-free and reduces FID quickly during distillation.

What the objective really does with guidance. Text-to-image teachers are sampled with classifier-free guidance, and DMD uses the guided teacher as the real score. Liu et al. [204] decompose this objective and find that in text-to-image distillation the dominant term is not distribution matching at all but a term they call CFG augmentation, the guidance direction applied to the student's samples, while the distribution matching term acts as a regularizer that keeps training stable. Decoupling the two, for example by giving them different noise schedules, improves the result, and the method was used for the 8-step Z-Image model [425]. The guided teacher score is not the score of any noised distribution (Section 2.11), so it is not surprising that a "distribution matching" objective with a guided teacher does something else. Li et al. [178] analyse a related failure when the teacher's negative branch uses information the student does not see. Phased DMD [88] splits the noise range into subintervals with separate experts and derives the score matching objective within each subinterval, which restores diversity and motion in multi-step distillation of 20B-scale image and video models.

On-policy distillation. DMD trains the student on its own samples and uses the teacher only to score them. This is the structure of on-policy distillation in language models, and it extends the teacher's role from a sampler to a critic. Flow-OPD [91] uses it to merge several reward-specialized teachers, each trained with the policy gradient methods of Section 4.2, into one student. Reward Forcing [226] weights the distribution matching loss by a reward, which tilts the student towards high-reward samples in the sense of Eq. (60). The line between distillation and post-training is thin here: both optimize a divergence to a target distribution on samples from the current model.

5.3 Adversarial distillation

The discriminator as a density ratio. A discriminator trained to separate data from generator samples with the logistic loss converges to $D^* = p/(p+q)$, so its logit is $\log(p/q)$ and its input gradient is $\nabla \log p - \nabla \log q$. Distribution matching computes the same vector field from two diffusion models at every noise level, and Eq. (79) is the generator update of a GAN whose discriminator logit at noise level t is $\log(p_t/q_{\theta,t})$. Adversarial distillation estimates this ratio with a classifier instead. The advantage is that the classifier sees clean or lightly noised samples, where the score networks are least informative, and it sharpens fine detail. The cost is the familiar instability of GAN training.

ADD and LADD. Adversarial diffusion distillation (ADD) [299] trains a one- to four-step student from SDXL with two losses. A score distillation loss noises the student's output, lets the teacher denoise it and penalizes the distance between the two. An adversarial loss uses a discriminator built on frozen DINOv2 features in pixel space. ADD gave the first single-step, real-time

samples from a foundation model. Its discriminator requires decoding to pixels and fixes the feature space to that of a model trained on natural images. Latent adversarial diffusion distillation (LADD) [300] builds the discriminator from the features of a pretrained latent diffusion model and applies it to noised latents, which avoids decoding, supports high resolutions and lets the noise level select which features the discriminator emphasizes. LADD distilled the 8B Stable Diffusion 3 into SD3-Turbo. SANA-Sprint [50] combines a LADD term with the continuous-time consistency loss of Section 5.4. The pattern is common: a teacher-anchored loss that preserves diversity, plus an adversarial term that restores detail at one step.

Adversarial training without a teacher. Two recent methods train few-step generators adversarially without distillation. Adversarial flow models [199] train a generator with an adversarial objective but encourage it to learn a deterministic noise-to-data transport, which stabilizes training compared with a GAN that may learn any map. Their one-step models reach FID 2.38 on ImageNet 256×256. TwinFlow [59] trains the model in a self-adversarial way that needs neither a fixed pretrained teacher nor a separate discriminator network, and turns the 20B Qwen-Image into a one-step generator.

5.4 Flow maps: a unifying lens

The flow map and its three characterizations. Let $X_{s,t}$ be the solution operator of the probability-flow ODE from time s to time t: $X_{s,t}(\boldsymbol{x}_s) = \boldsymbol{x}_t$ along every trajectory. It is the object $X_{s\to t}$ introduced in Section 2.5. Three properties determine it, and each gives a family of losses [31, 32]:

$$\text{Lagrangian:} \quad \partial_t X_{s,t}(\boldsymbol{x}) = \boldsymbol{u}_t(X_{s,t}(\boldsymbol{x})), \qquad X_{s,s}(\boldsymbol{x}) = \boldsymbol{x}, \tag{80}$$

$$\text{Eulerian:} \quad \partial_s X_{s,t}(\boldsymbol{x}) + \big(\partial_{\boldsymbol{x}} X_{s,t}(\boldsymbol{x})\big)\, \boldsymbol{u}_s(\boldsymbol{x}) = 0, \tag{81}$$

$$\text{semigroup:} \quad X_{r,t} \circ X_{s,r} = X_{s,t} \quad \text{for } s \le r \le t. \tag{82}$$

The Lagrangian equation is the ODE itself, read at the end point. The Eulerian equation says that $X_{s,t}(\boldsymbol{x}_s)$ does not change as the start point slides along the trajectory, $\frac{\mathrm{d}}{\mathrm{d}s} X_{s,t}(\boldsymbol{x}_s) = 0$, and the chain rule turns this into a transport equation. The semigroup property says that two jumps compose into one (Fig. 13). Taking $t \to s$ in any of them recovers the velocity, $\partial_t X_{s,t}|_{t=s} = \boldsymbol{u}_s$, so a flow map contains a flow model. Consistency models are the Eulerian characterization with the end point fixed at $t = 1$, and Eq. (78) is a finite-difference version of it. Consistency trajectory models [157] were the first to learn the full two-time map, with an adversarial loss added. Boffi et al. [32] show that every distillation scheme for these characterizations can be turned into a teacher-free training scheme by self-distillation, because the velocity in Eqs. (80) and (81) can be replaced by the conditional flow matching target or by the model's own diagonal $t = s$. They find that Lagrangian losses, which need neither spatial derivatives nor bootstrapping from small steps, train most stably.

MeanFlow: the average velocity. Parametrize the map by its chord, $X_{s,t}(\boldsymbol{x}) = \boldsymbol{x} + (t - s)\, \bar{\boldsymbol{u}}(\boldsymbol{x}, s, t)$, where the average velocity over the interval is

$$\bar{\boldsymbol{u}}(\boldsymbol{x}_s, s, t) = \frac{1}{t - s} \int_s^t \boldsymbol{u}_r(\boldsymbol{x}_r)\, \mathrm{d}r. \tag{83}$$

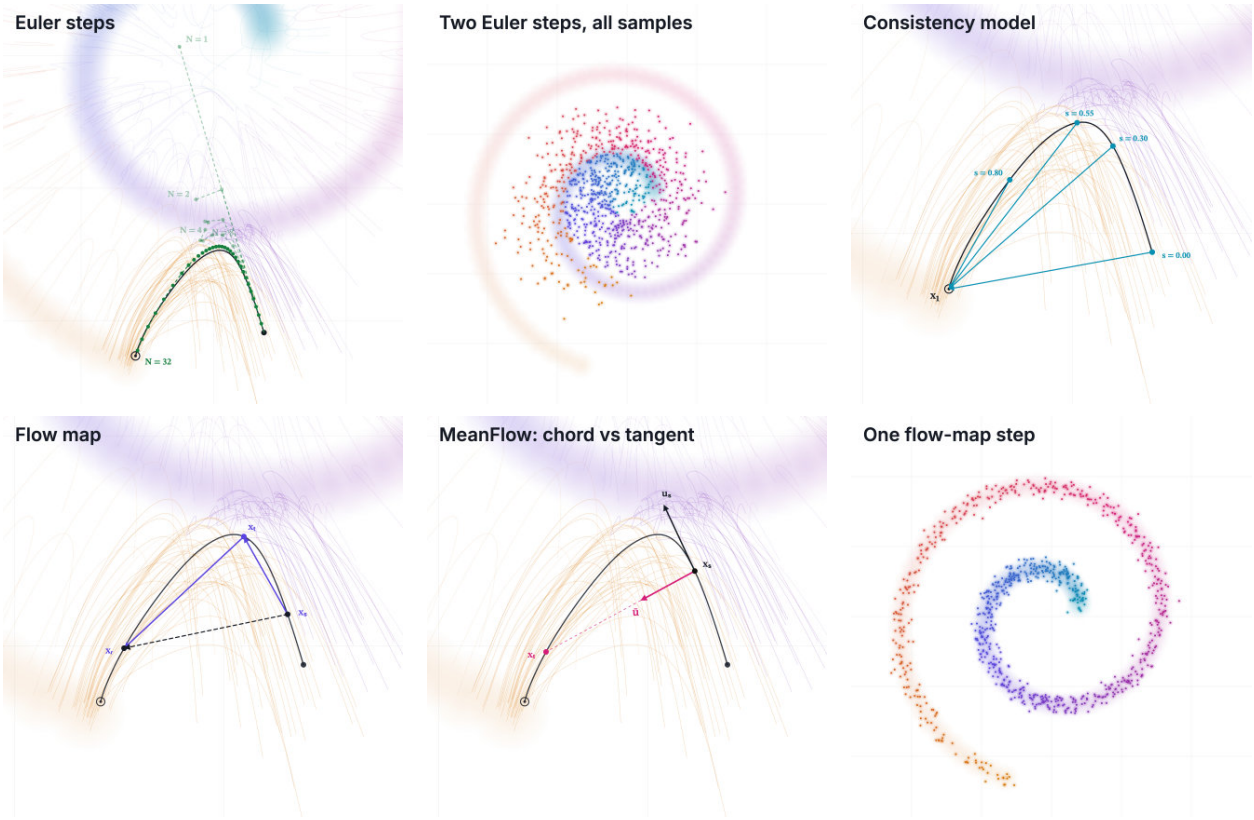


Figure 13: **The flow-map zoo on one exact teacher trajectory** (Animation A9). Top left: Euler sampling with $N = 1, 2, 4, 8, 32$ steps. One step lands on the data mean, and 8 steps reach a sharp sample on the wrong arm, because this trajectory turns outward only late. Top middle: with two steps the whole distribution is blurred between the arms. Top right: a consistency model maps every point x_s of the trajectory to its endpoint x_1. Bottom left: a flow map $X_{s,t}$ jumps between any two times, and two short jumps compose into a long one, $X_{t,r} \circ X_{s,t} = X_{s,r}$. Bottom middle: MeanFlow learns the average velocity $\bar{u} = (x_t - x_s)/(t - s)$, the slope of the chord, which tends to the instantaneous velocity u_s as $t \to s$. Bottom right: one step of the exact flow map $X_{0,1}$ recovers the sharp target. The teacher is the probability-flow ODE of a 110-component Gaussian mixture, solved in closed form, and every map is drawn as its ideal target.

Multiply by $t-s$ and differentiate with respect to s along the trajectory, keeping t fixed. The left side gives $-\bar{\boldsymbol{u}}+(t-s)\frac{\mathrm{d}}{\mathrm{d}s}\bar{\boldsymbol{u}}$ and the right side gives $-\boldsymbol{u}_s(\boldsymbol{x}_s)$, so

$$\bar{\boldsymbol{u}}(\boldsymbol{x}_s,s,t)=\boldsymbol{u}_s(\boldsymbol{x}_s)+(t-s)\Big(\partial_s\bar{\boldsymbol{u}}+\big(\partial_{\boldsymbol{x}}\bar{\boldsymbol{u}}\big)\,\boldsymbol{u}_s\Big)(\boldsymbol{x}_s,s,t). \tag{84}$$

This is the Eulerian equation Eq. (81) rewritten for the average velocity. MeanFlow [102] uses the right side as a regression target, with a stop-gradient on the network and with the conditional velocity $\boldsymbol{u}^{\mathrm{c}}=\dot{\alpha}_s\boldsymbol{x}_1+\dot{\sigma}_s\boldsymbol{x}_0$ in place of $\boldsymbol{u}_s$:

$$\mathcal{L}_{\mathrm{MF}}(\theta)=\mathbb{E}\Big\|\bar{\boldsymbol{u}}_\theta(\boldsymbol{x}_s,s,t)-\mathrm{sg}\Big[\boldsymbol{u}^{\mathrm{c}}+(t-s)\big(\partial_s\bar{\boldsymbol{u}}_\theta+\big(\partial_{\boldsymbol{x}}\bar{\boldsymbol{u}}_\theta\big)\,\boldsymbol{u}^{\mathrm{c}}\big)\Big]\Big\|^2. \tag{85}$$

The total derivative is one Jacobian–vector product of the network with tangent $(\boldsymbol{u}^{\mathrm{c}},1,0)$ in the inputs $(\boldsymbol{x},s,t)$. The substitution of $\boldsymbol{u}^{\mathrm{c}}$ for $\boldsymbol{u}_s$ is what makes training possible without a teacher, and it is legitimate for a specific reason: for fixed network weights the target is affine in the velocity. Its conditional expectation given $\boldsymbol{x}_s$ is therefore the target with the marginal velocity, and by Lemma 2.1 the regression has the same minimizer as if the marginal velocity were known. The same argument does not apply to targets that depend nonlinearly on the velocity. MeanFlow reached FID 3.43 with one step on ImageNet 256×256, trained from scratch. Pixel MeanFlow [225] predicts images on the data manifold while keeping the loss in velocity space, and generates in one step without an autoencoder.

Variants that remove the derivative. The Jacobian–vector product is costly at scale and interacts badly with fused attention kernels. Shortcut models [93] condition the network on the step size d and enforce the semigroup property for two equal halves, $\bar{\boldsymbol{u}}(\boldsymbol{x},s,2d)=\frac{1}{2}\big(\bar{\boldsymbol{u}}(\boldsymbol{x},s,d)+\bar{\boldsymbol{u}}(\boldsymbol{x}',s+d,d)\big)$ with $\boldsymbol{x}'=\boldsymbol{x}+d\,\bar{\boldsymbol{u}}(\boldsymbol{x},s,d)$, while the smallest step size is trained with flow matching. SplitMeanFlow [109] enforces the general interval splitting $(t-s)\bar{\boldsymbol{u}}(s,t)=(r-s)\bar{\boldsymbol{u}}(s,r)+(t-r)\bar{\boldsymbol{u}}(r,t)$, which is Eq. (82) for chords, and recovers Eq. (84) as $r\to s$. AlphaFlow [430] shows that the MeanFlow loss splits into a trajectory flow matching term and a trajectory consistency term whose gradients conflict, and anneals from the first to the full objective. Inductive moment matching [459] matches distributions rather than points across pairs of times with a moment-matching loss, which gives distribution-level convergence guarantees.

Continuous-time consistency at scale. As the grid in Eq. (78) is refined, the consistency loss becomes a continuous-time objective whose gradient involves $\frac{\mathrm{d}}{\mathrm{d}t}f_{\theta^-}(\boldsymbol{x}_t,t)=\partial_t f_{\theta^-}+(\partial_{\boldsymbol{x}}f_{\theta^-})\,\boldsymbol{u}_t$, the same Jacobian–vector product. Continuous-time training had been unstable. sCM [220] traced the instability to the time derivative and fixed it with a trigonometric parametrization, $\alpha_t=\sin(\frac{\pi}{2}t)$ and $\sigma_t=\cos(\frac{\pi}{2}t)$ in our convention, together with normalization of this tangent and adaptive weighting, which allowed a 1.5B model to reach FID 1.88 with two steps on ImageNet 512×512. rCM [455] wrote a FlashAttention-2 kernel for the Jacobian–vector product and scaled sCM to 14B text-to-video models. It found that sCM alone loses fine detail and attributes this to the mode-covering character of its forward-divergence objective. Adding a DMD loss as a regularizer, which is mode-seeking, gives quality comparable to DMD2 with better diversity in one to four steps. The pairing of a forward and a reverse divergence is a useful lens on the whole field: trajectory and flow-map regressions cover the modes but blur, distribution matching is sharp but drops modes, and the strongest recipes combine one of each.

Flow maps that keep improving with more steps. Consistency models degrade as steps are added, while a flow map sampled on a finer grid converges to the ODE solution. Align Your

Flow [293] shows this difference analytically and empirically, introduces two continuous-time flow-map objectives that generalize consistency and flow matching, and distills with autoguidance and a short adversarial fine-tuning. Terminal velocity matching [460] regularizes the map at its terminal time instead of its initial time, $\partial_t X_{s,t}$ against $\boldsymbol{u}_t(X_{s,t})$ as in Eq. (80), and proves an upper bound on the 2-Wasserstein distance to the data when the network is Lipschitz. Decoupled MeanFlow [174] converts a pretrained flow model into a flow map without architectural changes by conditioning only the final blocks on the target time, and finds that training a flow model first and converting it is cheaper than training a map from scratch. CMT [129] inserts a mid-training stage that regresses on solver trajectories before flow-map training. For distillation of large models, He et al. [119] warm up MeanFlow with a discrete version of its target before switching to the differential one, add a trajectory distribution alignment loss against the mean-seeking bias of few-step regression, and distill text-to-image models up to 80B parameters.

Where the training points come from. A flow map distilled on teacher trajectories that start from data may never visit the regions that the teacher actually reaches from noise. Tong et al. [347] sample only from the prior, which the teacher follows by construction, and correct the student's own compounding errors along the way. Their one-step student of an ImageNet teacher reaches FID 1.45 at 256×256. π-Flow [48] changes the output instead: the student predicts a network-free policy that produces velocities at several substeps, and it is trained by imitation, matching the policy's velocity to the teacher's along the policy's own trajectory with a flow matching loss. On 12B and 20B text-to-image models at four steps it keeps teacher-level quality with clearly higher diversity than DMD. SGFlow [105] learns the map and its implied velocity jointly by following non-gradient dynamics whose stationary point is the true flow map, which avoids model inverses and backpropagation through iterated model calls and comes with a proof that the stopgrad dynamics have the desired fixed point.

5.5 Causal and real-time video

Why autoregression, and what goes wrong. Video diffusion transformers denoise all frames jointly with bidirectional attention, so the first frame is ready only when the last one is. Interactive applications such as game engines and world models (Section 7) need frames one chunk at a time, conditioned on what has been shown and on new actions. An autoregressive video diffusion model factorizes the video as $p(\boldsymbol{x}^{1:K}) = \prod_k p(\boldsymbol{x}^k \mid \boldsymbol{x}^{<k})$, with causal attention and a key–value cache over past frames, and models each factor with a few-step diffusion model. The chain rule for the KL divergence shows where the difficulty lies:

$$\mathrm{KL}(q(\boldsymbol{x}^{1:K}) \,\|\, p(\boldsymbol{x}^{1:K})) = \sum_{k=1}^{K} \mathbb{E}_{\boldsymbol{x}^{<k} \sim q} \, \mathrm{KL}(q(\cdot \mid \boldsymbol{x}^{<k}) \,\|\, p(\cdot \mid \boldsymbol{x}^{<k})). \tag{86}$$

Teacher forcing trains each factor on real contexts $\boldsymbol{x}^{<k} \sim p$, while the model is deployed on its own contexts $\boldsymbol{x}^{<k} \sim q$. A model can be accurate on every real context and still drift once its own errors enter the context, because nothing in training tells it how to behave on contexts that real videos never produce. Figure 14 shows two models with identical teacher-forced losses and very different rollouts. Diffusion forcing [46] trains with independent noise levels per frame, which exposes the model to corrupted contexts but not to its own errors.

From bidirectional teachers to causal students. CausVid [418] adapts a pretrained bidirectional video model to causal attention and distills it from 50 to 4 steps with DMD. The bidirectional

teacher supervises the causal student, and the student is initialized by regression on the teacher's ODE trajectories. Causal Forcing [466] points out that this initialization is flawed: regression of a causal student on a bidirectional teacher's ODE endpoints requires that each noisy frame determine its clean frame given only the past, which fails when the teacher used the future, and the regression then returns a conditional expectation, the same averaging that collapses one-step samples in Fig. 11. Initializing from an autoregressive teacher removes the problem. Causal Forcing++ [446] replaces the stored ODE trajectories with causal consistency distillation, which learns the same conditional flow map from single online teacher steps, and reaches frame-wise generation with one or two steps.

Self Forcing. Self Forcing [133] closes the gap in Eq. (86) directly. During training it rolls the model out autoregressively with its key–value cache, exactly as at inference, and applies a distribution matching loss to the whole generated video. The expectation over contexts is then taken under q, as in Eq. (86), and the loss sees and penalizes drift. Training on full rollouts is affordable because the student is already few-step and gradients are truncated stochastically through the denoising chain. Causal-rCM [456] reads the two training modes through the divergence lens of Section 5.4: teacher forcing with a consistency loss is offline and forward-divergence, self forcing with DMD is on-policy and reverse-divergence, and the first is the best initialization for the second.

Long horizons. A student distilled from a teacher that generates five-second clips must extrapolate beyond the horizon it was trained on. Self-Forcing++ [71] extends generation towards minutes without long-video teachers, by using the short-horizon teacher to correct segments of the student's own long rollouts. Rolling Forcing [211] denoises a window of frames with increasing noise levels jointly, which relaxes strict causality between neighbouring frames, and keeps the cache of the first frames as an attention sink for global consistency. LongLive [408] supports prompt changes during a stream. Reward Forcing [226] replaces the static sink with a moving average of evicted tokens, which reduces copying of the first frame, and reports 23.1 frames per second on one H100. AnyFlow [108] distills flow-map transitions between arbitrary times instead of jumps to the data end, so that quality keeps improving with more steps, and uses a backward simulation through flow-map jumps to train on the student's own trajectories. Ultra Flash [224] cascades a streaming super-resolution stage after a low-resolution generator and reaches about 30 frames per second at 1K resolution. These systems are the basis of the interactive world models of Section 7.

5.6 Systems-level acceleration

Distillation reduces the number of evaluations. The methods in this subsection reduce the cost of each one, are mostly training-free and compose with distillation.

Fewer tokens. The cost of a transformer evaluation grows with the number of tokens (Eq. (41)), and for video the quadratic attention term dominates. Deep compression autoencoders [52] raise the spatial compression factor to 32 or more and reduce the token count before any other optimization. Multi-resolution sampling generates the early, coarse part of the trajectory at low resolution and switches to full resolution only for the final steps [457].

Caching. Consecutive denoising steps see similar inputs and produce similar features. DeepCache [233] reuses the high-level features of a U-Net across adjacent steps and recomputes only the shallow branch. Pyramid attention broadcast [450] observes that the change in attention outputs

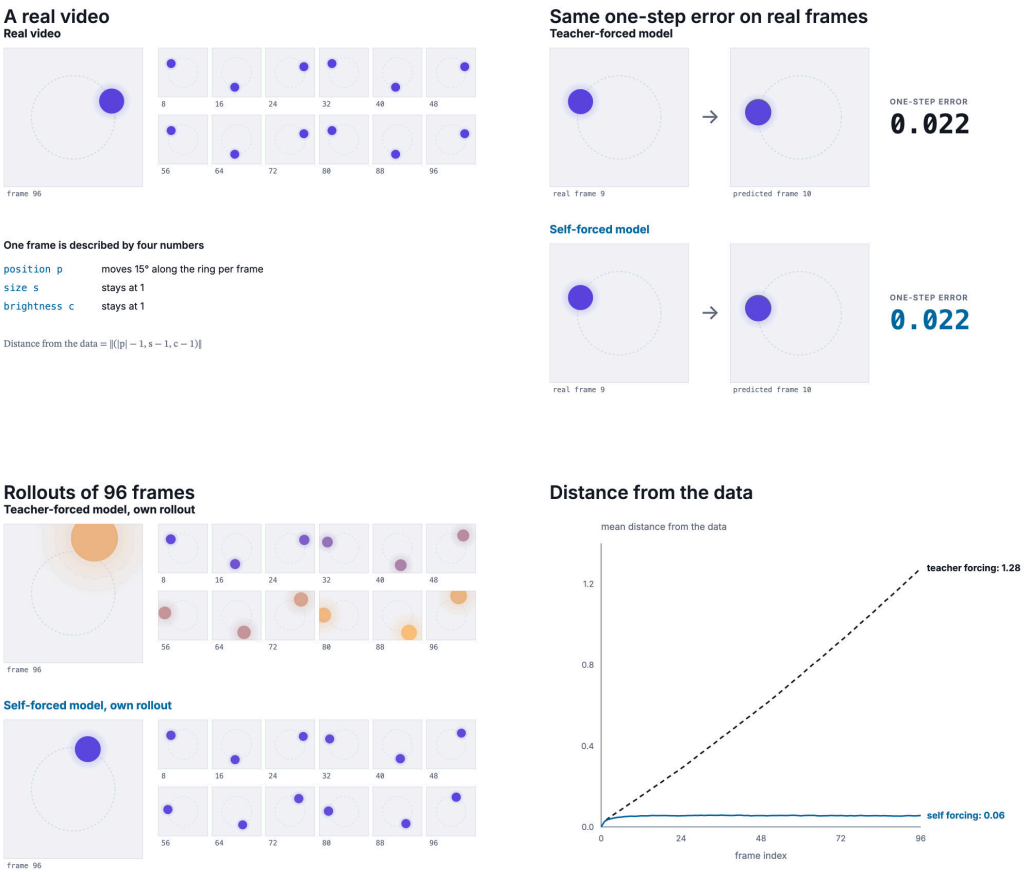


Figure 14: **Exposure bias in autoregressive video generation** (Animation A12). Each frame of a toy video is a ball on a ring with a position, a size and a brightness. Top left: in real videos the ball moves 15° per frame at constant size and brightness. Top right: two hand-specified per-frame models make the same small error on every real context frame, so their teacher-forced losses are identical. The second model also pulls drifted frames back toward the nearest real frame, a term that vanishes on real contexts. Bottom left: conditioned on their own outputs with shared noise, the first model drifts outward, grows and saturates, while the second stays near the data. Bottom right: the mean distance from the data over 400 rollouts grows steadily under teacher forcing and levels off for the self-forced model. Self Forcing [133] obtains such a correction by training on the model's own rollouts with a video-level distribution-matching loss.

across steps is smallest in the middle of sampling, reuses attention outputs over several steps with a different interval for each attention type, and reports real-time generation of 720p video. TeaCache [205] decides when to reuse by estimating the change in the output from the change in the timestep-modulated input, which is cheap to compute, so that the cache is refreshed only when the output is expected to change.

Sparse and linear attention. Attention maps in video transformers are dominated by a small fraction of entries. Sparse VideoGen [390] classifies heads online as spatial or temporal and computes only the corresponding blocks, with speedups above 2×. SLA [432] splits the attention weights into a few large, high-rank entries, computed exactly, and many small entries of low rank, computed with linear attention, and with brief fine-tuning removes 95% of the attention computation. Training-free methods continue to improve the trade-off between quality and speed: HyperVAttention [170] clusters tokens within local spatio-temporal windows, and Sol-Attn [179] selects key blocks during the online softmax pass and approximates the discarded ones. Autoregressive video adds a growing key–value cache. HeadCast [307] classifies heads once into sink, dummy, spatial and global types and gives each type its own cache, which reaches 1.95× at 1080p, and SparSTAR [172] recomputes block-sparse attention masks at each scale of next-scale video models.

Quantization. Q-Diffusion [190] showed that post-training quantization needs calibration data drawn across time steps, because activation statistics change along the trajectory. SVDQuant [184] quantizes weights and activations to 4 bits by moving activation outliers into the weights and absorbing them with a high-precision low-rank branch. SageAttention [431] quantizes the attention computation itself to 8 bits.

Putting it together. The factors multiply. TurboDiffusion [433] combines rCM step distillation, SageAttention, SLA and 8-bit weights and activations, and reports end-to-end speedups of 100 to 200 times on 14B video models on a single consumer GPU. Discrete diffusion language models have their own set of techniques, built around key–value caching for block diffusion and parallel decoding [25, 327, 385], surveyed by Gwak et al. [112]. Speed also changes what is possible after training: a few-step model makes reward fine-tuning (Section 4) and inference-time search (Section 6) cheaper per sample, and flow maps make look-ahead estimates of the final sample available at every step [292].

6 Inference-Time Scaling

Language models improve when they are allowed to spend more computation at inference, by sampling several answers and checking them or by reasoning before answering. Diffusion and flow models have always had a compute dial at inference, the number of solver steps, but turning it up stops helping after a few dozen steps [232]. The reason is simple. A more accurate solver samples the model's own distribution p_1^θ more faithfully, and once the discretization error is below the model error there is nothing left to gain. Further compute must change *what* is sampled, not how accurately.

This section treats the methods that do so without changing the weights. Almost all of them target the same distribution as the fine-tuning methods of Section 4, the reward-tilted model

$$p^\star(\boldsymbol{x}) = \frac{1}{Z}\, p_1^\theta(\boldsymbol{x})\, e^{r(\boldsymbol{x})/\beta}, \tag{87}$$

which is Eq. (60) with the pretrained model as reference. Fine-tuning amortizes the tilt into new weights. Inference-time methods pay for it per sample, with extra network evaluations and extra reward evaluations, and in exchange they need no training, work with rewards that are not differentiable, and can be switched per prompt. The control view of Section 2.6 organizes them. The tilted path measure Eq. (38) has marginals $p_t\, h_t$ with $h_t(\boldsymbol{x}) = \mathbb{E}[e^{r(\boldsymbol{x}_1)/\beta} \mid \boldsymbol{x}_t = \boldsymbol{x}]$, and every method below is a way to approximate h_t or to sample from the tilt without it: by drawing several samples and keeping the best (Section 6.2), by reweighting and resampling particles on the way (Section 6.3), by following the gradient of an estimate of $\log h_t$ (Section 6.4), or by revising a sample in a loop with a verifier or a critique. Inverse problems are the special case in which the reward is a log-likelihood (Section 6.5), and discrete diffusion language models bring their own tools (Section 6.6). We first treat the baseline, the solver itself (Section 6.1). Table 5 summarizes the methods by what they require.

Table 5: Inference-time methods by what they require from the reward and what they sample. Cost is in network evaluations for N samples or particles and K solver steps.

Method	Reward access	Cost	Target and failure mode
Best-of-N	final value	NK	Tilt with KL at most $\log N$; needs $N \sim 1/q$ to reach a reward of base probability q.
Noise search	final value	NK	Maximizer of $r \circ F$ near the noise prior; can overfit the verifier.
SMC, FK steering	intermediate values	NK	Consistent for $p^\star$ as $N \to \infty$; weight degeneracy with poor potentials.
Gradient guidance	gradient	$\approx 2K$	Biased approximation of $p^\star$; adversarial gradients.
Inverse solvers	likelihood gradient	$\approx 2K$	Approximate posterior; biased when $p(\boldsymbol{x}_1 \mid \boldsymbol{x}_t)$ is multimodal.
Reflection loops	critique or score	several samples	No explicit target; bounded by the verifier and the editor.

6.1 Solvers and step schedules

Sampling a flow or diffusion model means integrating the probability-flow ODE Eq. (20) or one of the SDEs of Proposition 2.3 from $t = 0$ to $t = 1$. The solver decides how close the result is to the model's distribution for a given number of function evaluations (NFE). Three ideas account for most of the progress.

Exponential integrators. The ODE has a linear part that can be integrated exactly. Write the velocity through the data prediction $\hat{\boldsymbol{x}}_1(\boldsymbol{x}, t)$. Since $\boldsymbol{u} = \dot\alpha_t \hat{\boldsymbol{x}}_1 + \dot\sigma_t \hat{\boldsymbol{x}}_0$ and $\hat{\boldsymbol{x}}_0 = (\boldsymbol{x} - \alpha_t \hat{\boldsymbol{x}}_1)/\sigma_t$, and since $\dot\lambda_t/2 = \dot\alpha_t/\alpha_t - \dot\sigma_t/\sigma_t$ for $\lambda = \log(\alpha^2/\sigma^2)$,

$$\frac{\mathrm{d}\boldsymbol{x}}{\mathrm{d}t} = \frac{\dot\sigma_t}{\sigma_t}\,\boldsymbol{x} + \frac{\alpha_t \dot\lambda_t}{2}\,\hat{\boldsymbol{x}}_1(\boldsymbol{x}, t). \tag{88}$$

Dividing by σ_t removes the linear term. With $\rho_t = \alpha_t/\sigma_t = e^{\lambda_t/2}$, which satisfies $\dot\rho_t = \rho_t \dot\lambda_t/2$, the equation becomes $\mathrm{d}(\boldsymbol{x}/\sigma_t) = \hat{\boldsymbol{x}}_1\, \mathrm{d}\rho_t$, so between two times $t < s$

$$\frac{\boldsymbol{x}_s}{\sigma_s} = \frac{\boldsymbol{x}_t}{\sigma_t} + \int_{\rho_t}^{\rho_s} \hat{\boldsymbol{x}}_1(\boldsymbol{x}_\tau, \tau)\, \mathrm{d}\rho_\tau. \tag{89}$$

This is exact. Only the integral of the network output needs approximating. Freezing $\hat{\boldsymbol{x}}_1$ at its value at the start of the step gives $\boldsymbol{x}_s = \sigma_s \hat{\boldsymbol{x}}_0 + \alpha_s \hat{\boldsymbol{x}}_1$, which is DDIM [318]: the step re-noises the current guess of the data with the current guess of the noise. Expanding $\hat{\boldsymbol{x}}_1$ to higher order in λ gives DPM-Solver [221] and DEIS [439], which use the noise prediction instead, and DPM-Solver++ [222], which uses the data prediction as above. The data prediction is bounded, so it stays stable under the large guidance scales of Eq. (50), where noise-prediction solvers of high order become unstable, and it can be clipped or thresholded. The multistep variants estimate the derivatives of $\hat{\boldsymbol{x}}_1$ from the outputs of previous steps, so a k-th order method still costs one evaluation per step and has global error $O(h^k)$ in the largest step h in λ. UniPC [449] adds a corrector that reuses the evaluation at the new point, which the next step needs anyway, and so raises the order by one at no extra cost. These solvers produce usable images in 10 to 20 steps.

Step schedules. The same solver can give very different results on different time grids, because the error per step depends on how curved the trajectory is at that time. EDM [152] parametrizes time by the noise level σ (with $\alpha = 1$) and spaces the steps as $\sigma_i = (\sigma_{\max}^{1/\rho} + \frac{i}{K-1}(\sigma_{\min}^{1/\rho} - \sigma_{\max}^{1/\rho}))^\rho$ with $\rho = 7$, which concentrates steps at low noise where fine detail is resolved. Align Your Steps [291] optimizes the schedule for a given model, solver and step count by minimizing a bound on the KL divergence between the continuous and the discretized process. Guidance, too, need not be applied at every step: restricting CFG to an intermediate interval of noise levels improves both sample quality and diversity [168]. The interpretation of CFG as a predictor-corrector scheme [34], and analyses of what guidance does in simple settings [61], treat the guidance rule as part of the sampler, and several training-free variants modify the guided direction to improve quality or reduce artefacts [47, 62].

Stochasticity as error correction. Deterministic solvers accumulate error, and nothing pulls a trajectory back once it has drifted off the data manifold. Stochastic samplers can. Adding noise and running the reverse SDE contracts the distance between distributions over time, which underlies polynomial convergence guarantees for SDE samplers that need only an accurate score in L^2 [56]. EDM's stochastic sampler adds a small amount of noise before each step and then takes a deterministic step. Restart sampling [400] alternates deterministic integration with jumps back to a higher noise level by the forward process, so that the contraction of the noise removes accumulated error while most of the path is integrated with the accurate ODE. Restart sampling is an early example of spending extra steps on something other than accuracy, and the same move, renoising and regenerating, reappears below in verifier-guided backtracking and in iterative refinement.

Why step scaling saturates. Let p_1^θ be the distribution of the exact ODE solution and $p_1^{\theta,K}$ that of a K-step solver. The error of the final sample splits into a model error $D(p_{\text{data}}, p_1^\theta)$, fixed by training, and a discretization error that vanishes as $O(K^{-k})$. Past the point where the second is smaller than the first, more steps do not help, and in practice the plateau arrives after a few dozen steps [232]. Distillation (Section 5) goes further in the same direction, trading the remaining steps for a new network. The rest of the section spends compute on a different axis.

6.2 Search over noise

A deterministic sampler is a map F from the initial noise $\boldsymbol{x}_0 \sim \mathcal{N}(0, I)$ to the sample $\boldsymbol{x}_1 = F(\boldsymbol{x}_0)$. Different noises give different images, and some score much higher under a reward r than others. The simplest form of inference-time scaling is to search this space.

Best-of-N. Draw N samples and keep the one with the highest reward. If the reward of a base sample has a continuous distribution function $\Phi(u) = \Pr[r(\boldsymbol{x}) \le u]$, the best of N has distribution function Φ^N, and the selected sample has density

$$p_N(\boldsymbol{x}) = N\,\Phi\big(r(\boldsymbol{x})\big)^{N-1}\,p_1^\theta(\boldsymbol{x}). \tag{90}$$

Best-of-N is thus itself a reward tilt, with the monotone weight $N\Phi(r)^{N-1}$ in place of $e^{r/\beta}$. Its divergence from the base model is

$$\mathrm{KL}(p_N \,\|\, p_1^\theta) = \log N - \frac{N-1}{N} \le \log N, \tag{91}$$

since $\mathrm{KL} = \log N + (N-1)\mathbb{E}_{p_N}[\log \Phi(r)]$ and $\Phi(r)$ of the selected sample is the maximum of N uniform variables, which has $\mathbb{E}[\log U_{\max}] = -1/N$. For a reward that is Gaussian with standard deviation s under the base model, the expected gain is $\mathbb{E}[\max_i r_i] - \mathbb{E}[r] \approx s\sqrt{2\log N}$, and by Eq. (91) this is about $s\sqrt{2\,\mathrm{KL}}$. The exponential tilt of the same Gaussian shifts its mean by s^2/β at a divergence $s^2/(2\beta^2)$, which is also a gain of $s\sqrt{2\,\mathrm{KL}}$. Per unit of divergence, best-of-N is therefore close to the optimal KL-regularized policy. Its weakness is cost: the divergence grows like $\log N$, so moving a given distance from the base model costs an exponential number of samples. A reward that the base model reaches with probability q requires $N \sim 1/q$. In Fig. 15 the reward region is reached by fewer than 1 in 400 base samples, and best-of-64 does not reach it.

Search algorithms and verifiers. Ma et al. [232] framed inference-time scaling for diffusion models as a search problem with two axes: the verifier that scores candidates and the algorithm that proposes them. Random search is best-of-N. Zero-order search starts from a pivot noise, samples candidates in a neighbourhood of it, moves the pivot to the best candidate and repeats. Search over paths applies the same idea to the intermediate states of the sampling trajectory, renoising them and continuing from several branches. Verifiers range from aesthetic and CLIP scores to preference models such as ImageReward [396] and HPS [234, 389] and to vision-language models used as judges [369]. The verifier matters as much as the algorithm. No single verifier suits all tasks, and a search that pushes one score far can exploit its blind spots, which is reward hacking at inference time (Section 4.6).

Noise space as an optimization domain. Search over $\boldsymbol{x}_0$ is black-box optimization of $r \circ F$ on $\mathbb{R}^d$, with two constraints. The candidates must remain plausible under the prior, since in high dimension almost all of the Gaussian mass lies near the sphere of radius $\sqrt{d}$ and noises far from it lead to unnatural samples. And F is expensive, so the search must make every evaluation count. Trust-region noise search (TRS) [303] adapts trust-region Bayesian optimization to this setting. It treats both the generator and the reward as black boxes and optimizes only the source noise. After a random warm-up, it maintains a few boxes in noise space centred on the best noises found so far, proposes candidates inside them, and enlarges a box after repeated successes and shrinks it after repeated failures. The boxes trade global exploration against local exploitation, and the method needs little tuning across text-to-image, molecule and protein design, where it improves on methods that optimize the source noise and on methods that optimize the entire sequence of sampling noises. In Fig. 15 it reaches a much higher reward than best-of-N with the same 64 evaluations, although it lags behind SMC when the budget is too small for more than one batch, because it sees only final rewards.

Why noise search is the right object for deterministic samplers. Section 4.5 showed that changing the drift of a deterministic sampler cannot tilt its output distribution, because the initial noise already determines the sample. Changing the noise distribution can. If $\boldsymbol{x}_1 = F(\boldsymbol{x}_0)$ with F invertible, then the tilted noise distribution

$$q^\star(\boldsymbol{x}_0) \propto \mathcal{N}(\boldsymbol{x}_0; 0, I)\, e^{r(F(\boldsymbol{x}_0))/\beta} \tag{92}$$

pushes forward exactly to $p^\star$ in Eq. (87), and $\mathrm{KL}(q \| \mathcal{N}(0, I)) = \mathrm{KL}(F_\# q \,\|\, p_1^\theta)$ for any q. For non-invertible generators, such as distilled one-step models, the data-processing inequality makes the noise-space divergence an upper bound on the output-space divergence. Noise Hypernetworks [86] use this to amortize test-time noise optimization. They train a small network that modulates the input noise of a distilled generator, with a tractable noise-space objective that rewards high r while keeping the modulated noise distribution close to the prior. The cost of search is paid once during training, and sampling again takes one step.

6.3 Sequential Monte Carlo and twisted sampling

Best-of-N wastes most of its compute on trajectories that were hopeless early on. Sequential Monte Carlo (SMC) reallocates compute during sampling: it runs N trajectories, called particles, in parallel, weights them by how promising they look, and periodically replaces the unpromising ones by copies of the promising ones.

The Feynman–Kac construction. Discretize the sampler into Markov kernels $p^\theta(\boldsymbol{x}_{k+1} \mid \boldsymbol{x}_k)$ on a grid $0 = t_0 < \cdots < t_K = 1$. The target is the tilted path distribution $p^\theta(\boldsymbol{x}_{0:K})\, e^{r(\boldsymbol{x}_K)/\beta}/Z$, the discrete form of Eq. (38). Choose positive *potentials* $G_k(\boldsymbol{x}_k)$ with G_0 constant and $G_K = e^{r/\beta}$, and define intermediate targets $\pi_k(\boldsymbol{x}_{0:k}) \propto p^\theta(\boldsymbol{x}_{0:k})\, G_k(\boldsymbol{x}_k)$. If particles are moved by a proposal kernel $q(\boldsymbol{x}_{k+1} \mid \boldsymbol{x}_k)$, the incremental importance weight that turns π_k into π_{k+1} is

$$w_{k+1} = \frac{p^\theta(\boldsymbol{x}_{k+1} \mid \boldsymbol{x}_k)}{q(\boldsymbol{x}_{k+1} \mid \boldsymbol{x}_k)} \cdot \frac{G_{k+1}(\boldsymbol{x}_{k+1})}{G_k(\boldsymbol{x}_k)}. \tag{93}$$

With the base kernel as proposal the first factor is one. The product of the second factor over k telescopes to $e^{r(\boldsymbol{x}_K)/\beta}/G_0$, so the final weighted particles target $p^\star$ for *any* choice of intermediate potentials, and the estimate is consistent as $N \to \infty$. The potentials affect only the variance. Whenever the effective sample size $\mathrm{ESS} = (\sum_i w^i)^2 / \sum_i (w^i)^2$ falls below a threshold such as $N/2$, the particles are resampled in proportion to their weights and the weights are reset. The ideal potential is $G_k = h_{t_k}$, the quantity of Eq. (39). If in addition the proposal is the h-transformed kernel $q \propto p^\theta(\boldsymbol{x}_{k+1} \mid \boldsymbol{x}_k)\, h_{t_{k+1}}(\boldsymbol{x}_{k+1})$, then every weight equals one and SMC reduces to exact sampling from the optimally controlled process. Every practical method is an approximation of h_t that is used either as a potential, as a proposal, or both, with the weights correcting the error.

Look-ahead potentials. The common approximation evaluates the reward at the Tweedie estimate, $G_k(\boldsymbol{x}) = \exp\left(r(\hat{\boldsymbol{x}}_1(\boldsymbol{x}, t_k))/\beta\right)$. Feynman–Kac steering [313] uses such intermediate rewards and studies several potentials built from them, including the difference between consecutive intermediate rewards, which telescopes as above, and the running maximum. With a human preference reward, it lets a smaller text-to-image model with a few particles outperform larger ones, and it also steers text diffusion models. Figure 15 shows the mechanism: by the middle of sampling, the resampling has concentrated all particles in the part of the space that leads to the reward

bump, which best-of-N with the same number of trajectories rarely visits. Soft value-based decoding (SVDD) [189] is a related greedy scheme: at each step it draws several candidates from the pretrained kernel and keeps one according to an estimate of the soft value $\beta \log h_t$, which needs no reward gradient and applies to continuous and discrete diffusion alike.

Twisted proposals. The Twisted Diffusion Sampler (TDS) [388] also changes the proposal. It uses the approximation $\tilde{h}_t(\boldsymbol{x}) = p(y \mid \hat{\boldsymbol{x}}_1(\boldsymbol{x}, t))$ of the conditional likelihood both as a potential and, through its gradient, as a guided proposal, in the manner of Sections 6.4 and 6.5. The weights Eq. (93) correct the bias of the heuristic, so TDS is asymptotically exact, and in conditional image generation it improves on the heuristic with as few as two particles.

Feynman–Kac correctors and the drift–weight trade-off. The continuous-time version of Eq. (93) shows that the drift and the weights are interchangeable. Suppose particles follow $\mathrm{d}\boldsymbol{x}_t = \boldsymbol{b}_t(\boldsymbol{x}_t)\,\mathrm{d}t + \varepsilon_t\,\mathrm{d}\boldsymbol{W}_t$ for some drift $\boldsymbol{b}_t$ and carry log-weights that evolve as $\mathrm{d}\log w_t = g_t(\boldsymbol{x}_t)\,\mathrm{d}t$. The weighted particle density then obeys the Fokker–Planck equation with an added source term $(g_t - \bar{g}_t)\,\pi_t$, where $\bar{g}_t = \mathbb{E}_{\pi_t} g_t$. For it to follow a prescribed path of densities π_t, one needs

$$g_t = \partial_t \log \pi_t + \nabla \cdot \boldsymbol{b}_t + \boldsymbol{b}_t \cdot \nabla \log \pi_t - \tfrac{1}{2}\varepsilon_t^2 \big(\Delta \log \pi_t + \|\nabla \log \pi_t\|^2\big) \tag{94}$$

up to a constant, which self-normalization removes. This follows by dividing the target equation $\partial_t \pi = -\nabla \cdot (\pi \boldsymbol{b}) + \tfrac{1}{2}\varepsilon^2 \Delta \pi + (g - \bar{g})\pi$ by π and using $\Delta\pi/\pi = \Delta \log \pi + \|\nabla \log \pi\|^2$. Any drift can be used, and the weights account for whatever the drift gets wrong. Feynman–Kac correctors (FKC) [314] derive such weights for paths built from pretrained score models, namely annealed densities p_t^{γ}, geometric averages and products of several models, and simulate them with SMC. This samples, for example, the product of experts that classifier-free guidance only approximates (Eq. (52)). DriftLite [284] uses the freedom in Eq. (94) in the other direction: it chooses the drift to minimize the variance or the energy of the weights, which reduces weight degeneracy at little extra cost, and it improves on both pure guidance and plain SMC on Gaussian mixtures, particle systems and protein–ligand co-folding.

Advanced. Two lines of work remove the remaining approximations. URGE [358] weights whole trajectories by a Girsanov change of measure between the guided and the base path measures, which requires neither scores nor Hessians nor the PDE terms of Eq. (94), and shows that the path-wise weight and the particle-wise weights produce the same terminal law. The Energy-based Feynman–Kac Corrector [379] addresses the fact that all of the above assume the pretrained model is exact. More particles reduce Monte Carlo error but not the mismatch between the model and the intended path. Given a reference energy, it derives Feynman–Kac dynamics that track a prescribed path even with an imperfect model and corrects the endpoint mismatch on the fly. In the other direction, Andersson and Zhao [10] use a diffusion model inside SMC, as a differentiable resampling step for particle filters.

What the look-ahead gets wrong. The approximation $h_t(\boldsymbol{x}) \approx e^{r(\hat{\boldsymbol{x}}_1)/\beta}$ makes two errors. By Jensen's inequality, $\mathbb{E}[e^{r(\boldsymbol{x}_1)/\beta} \mid \boldsymbol{x}_t] \geq e^{\mathbb{E}[r(\boldsymbol{x}_1)|\boldsymbol{x}_t]/\beta}$, so it ignores the spread of rewards among the possible outcomes. More seriously, it replaces $\mathbb{E}[r(\boldsymbol{x}_1) \mid \boldsymbol{x}_t]$ by $r(\mathbb{E}[\boldsymbol{x}_1 \mid \boldsymbol{x}_t])$. At high noise the posterior of $\boldsymbol{x}_1$ is spread over several modes, and the posterior mean lies between them, where the reward was never trained and may be meaningless: a blurry average image is not what an aesthetic model was built to score. Flow Map Trajectory Tilting [292] replaces the Tweedie estimate by the flow map $X_{t\to 1}(\boldsymbol{x})$ of Section 5.4, which jumps to an actual sample on the data manifold. Exploiting the relation between the flow map and the velocity field, it proves better reward ascent

than gradient methods that use the denoiser, and it can be used either for exact sampling with importance weights or as a search for local maximizers of the tilted distribution. CS-VGB [143] accepts imperfect intermediate verifiers and repairs the target instead. It repeatedly denoises and renoises between checkpoints, accepting or backtracking according to the verifier. With correctly paired denoising and renoising transitions, the stationary distribution at the clean checkpoint is the tilted target even when the intermediate verifier is wrong, and the number of updates grows only linearly with the number of checkpoints.

Applications. SMC is most useful when the reward is cheap relative to the generator and not differentiable. In protein design, Feynman–Kac steering of RFdiffusion with potentials derived from ProteinMPNN and structural relaxation improves predicted interface energetics and binder designability [115], and Proteina-Complexa couples a flow-based binder generator with inference-time optimization, unifying generative and hallucination-based design [76]. The unified views of fine-tuning and sampling in Domingo-Enrich et al. [80] and Li et al. [193] treat inference-time sampling of $p^\star$ and fine-tuning toward it as the same problem, which also covers sampling from unnormalized densities [438].

6.4 Verifiers, reward guidance and reflection

When the reward is differentiable, its gradient can steer each trajectory directly, without particles.

Gradient guidance. By Eq. (39), the optimal drift correction is $\varepsilon_t^2 \nabla \log h_t$. Replacing h_t by the look-ahead $e^{r(\hat{\boldsymbol{x}}_1)/\beta}$ gives the correction $\varepsilon_t^2 \nabla_{\boldsymbol{x}} r(\hat{\boldsymbol{x}}_1(\boldsymbol{x}, t))/\beta$ already noted in Section 2.6. Its gradient passes through the denoiser. From Tweedie's formula Eq. (12), $\partial \hat{\boldsymbol{x}}_1 / \partial \boldsymbol{x} = (I + \sigma_t^2 \nabla^2 \log p_t)/\alpha_t$, and the second-order Tweedie identity $\mathrm{Cov}[\boldsymbol{x}_1 \mid \boldsymbol{x}_t] = (\sigma_t^2/\alpha_t^2)(I + \sigma_t^2 \nabla^2 \log p_t)$ turns this into

$$\nabla_{\boldsymbol{x}}\, r(\hat{\boldsymbol{x}}_1(\boldsymbol{x}, t)) = \frac{\alpha_t}{\sigma_t^2}\, \mathrm{Cov}[\boldsymbol{x}_1 \mid \boldsymbol{x}_t = \boldsymbol{x}]\, \nabla r(\hat{\boldsymbol{x}}_1). \tag{95}$$

The reward gradient is filtered through the posterior covariance, so the guidance moves the sample only in directions in which the outcome is still uncertain. The exact correction Eq. (72) instead averages $J_{t\to 1}^\top \nabla r(\boldsymbol{x}_1)$ over the tilted outcomes, with the Jacobian of the full flow rather than of the one-step denoiser. Gradient guidance is cheap, about one backward pass through the denoiser per step, but it is biased, and with an adversarially brittle reward model it follows gradients that increase the score without improving the image. Universal guidance [16] makes this practical for arbitrary off-the-shelf networks, such as segmentation, face recognition and detection models, by combining the forward guidance above with a backward step that optimizes $\hat{\boldsymbol{x}}_1$ directly and with repeated renoising at each step. Guided Action Flow [406] applies the same principle to robotics: it learns a small action-value critic for a frozen flow-matching vision-language-action policy and steers the action flow with the critic's gradient, raising the success rate of real-robot manipulation from 60.0% to 82.5%.

Verifiers. All of the methods above are only as good as the verifier. For images, the common verifiers are preference models trained on human comparisons (Section 4.1). Vision-language models are increasingly used instead, either as judges that answer whether an image matches a prompt, or as unified reward models for understanding and generation [369]. RationalRewards [362] trains a reward model to write a multi-dimensional critique before it scores. The critique serves twice: as a fine-grained reward for reinforcement learning, and at test time in a generate, critique and refine loop that turns the critique into a revised prompt and improves outputs without updating

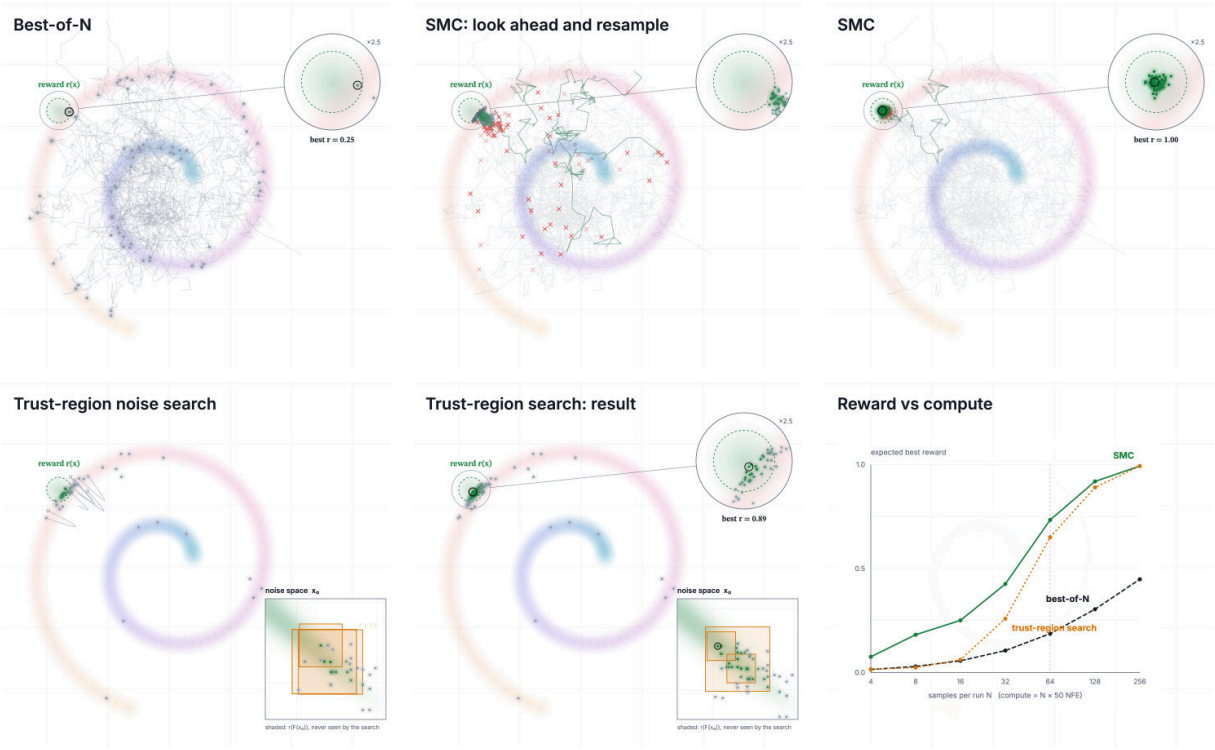


Figure 15: **Best-of-N, sequential Monte Carlo and trust-region noise search at equal compute** (Animation A13). The base model is the exact SDE of a Gaussian mixture, and the reward r is a narrow bump outside the outer arm that fewer than 1 in 400 base samples reach with $r > 0.5$. Top left: best-of-N with $N = 64$ independent trajectories returns its highest-reward sample ($r = 0.25$ in this run); the inset magnifies the reward region. Top middle: SMC weights particles by the change in reward of the look-ahead estimate $\hat{x}_1 = \mathbb{E}[x_1 \mid x_t]$ and resamples when the effective sample size falls below $N/2$; red crosses mark dropped particles and green lines the surviving ancestry. Top right: with the same 64 particles and 50 steps, all final SMC samples land on the bump. Bottom left and middle: trust-region noise search [303] treats the sampler as a black box, maps each initial noise x_0 to a sample $F(x_0)$ with the deterministic ODE, and observes only $r(F(x_0))$. After a random warm-up on 20% of the budget it proposes candidates inside $k = 3$ boxes (inset, noise space) centred on the best noises found so far, grows or shrinks each box after repeated successes or failures, and re-centres the boxes after every batch. With 64 candidates, the compute of 64 particles, it reaches $r = 0.89$ in this run. The shading in the inset is the true $r(F(x_0))$, which the search never sees. Bottom right: expected best reward against N. Best-of-N is computed in closed form from exact samples, and SMC and trust-region search are averaged over 160 runs per N. Both fail at small N when nothing comes near the bump. Trust-region search lags SMC there because it only sees final rewards, and it catches up once the budget allows a few batches.

any parameters. For an 8B model, the authors report preference accuracy competitive with much larger proprietary judges.

Reflection and reasoning before generating. A critique can also be fed back to the generator. Reflect-DiT [186] equips a diffusion transformer with in-context reflection: it conditions on previously generated images together with textual feedback that describes what to change, so each new sample is a revision rather than an independent draw. In autoregressive image generation, Guo et al. [111] study chain-of-thought style verification, preference optimization and their combination, with a reward model that assesses partial generations. GoT [90] lets a multimodal language model write an explicit reasoning chain about semantic and spatial layout before a diffusion model draws, and T2I-R1 [146] trains a semantic-level plan and token-level generation jointly with reinforcement learning. These methods move compute from denoising to planning and checking. Video generation has been proposed as a reasoning medium in its own right, with frames playing the role of intermediate thoughts [346]. Not every loop needs an external verifier: iterative partial refinement [151] renoises a subset of regions of an already generated sample and regenerates them conditioned on the rest, which improves global consistency for sequential diffusion models without any reward.

Video. Inference-time search is expensive for video, where one candidate can take minutes. Stream-T1 [349] exploits the structure of streaming generation, in which chunks are produced in a few steps each (Section 5.5). It initializes the noise of each new chunk from the noise of previous chunks that scored well and prunes candidates by reward at the chunk level, which turns a global search into a sequence of short local ones.

6.5 Inverse problems

An inverse problem asks for $\boldsymbol{x}$ given a measurement $y = \mathcal{A}(\boldsymbol{x}) + \boldsymbol{n}$, with a known forward operator $\mathcal{A}$ and noise $\boldsymbol{n} \sim \mathcal{N}(0, \sigma_y^2 I)$. Examples are inpainting, super-resolution, deblurring, computed tomography and accelerated MRI. With a diffusion model as the prior, the posterior is

$$p(\boldsymbol{x} \mid y) \propto p_1^\theta(\boldsymbol{x}) \, \exp\Big(- \frac{\|y - \mathcal{A}(\boldsymbol{x})\|^2}{2\sigma_y^2} \Big), \tag{96}$$

which is Eq. (87) with the log-likelihood as the reward and $\beta = 1$. Every method of Section 6.3 therefore applies, and TDS was evaluated on exactly such problems. The literature on inverse problems developed largely in parallel, around the gradient approximation.

Diffusion posterior sampling. By Eq. (49), the posterior score is $\nabla \log p_t(\boldsymbol{x}) + \nabla \log p_t(y \mid \boldsymbol{x})$ with $p_t(y \mid \boldsymbol{x}) = \int p(y \mid \boldsymbol{x}_1)\, p(\boldsymbol{x}_1 \mid \boldsymbol{x}_t = \boldsymbol{x}) \, \mathrm{d}\boldsymbol{x}_1$. The second term requires an integral over all clean images compatible with $\boldsymbol{x}_t$. Diffusion posterior sampling (DPS) [66] replaces $p(\boldsymbol{x}_1 \mid \boldsymbol{x}_t)$ by a point mass at its mean,

$$p_t(y \mid \boldsymbol{x}) \approx p(y \mid \hat{\boldsymbol{x}}_1(\boldsymbol{x}, t)), \qquad \nabla \log p_t(y \mid \boldsymbol{x}) \approx -\frac{1}{\sigma_y^2} \Big(\frac{\partial \hat{\boldsymbol{x}}_1}{\partial \boldsymbol{x}} \Big)^\top \nabla \mathcal{A}(\hat{\boldsymbol{x}}_1)^\top (\mathcal{A}(\hat{\boldsymbol{x}}_1) - y). \tag{97}$$

This is gradient guidance with the measurement log-likelihood as reward, and Eq. (95) applies unchanged. In practice the step is rescaled by a factor $\zeta / \|y - \mathcal{A}(\hat{\boldsymbol{x}}_1)\|$, because the approximation is poor at high noise and the unscaled step is unstable. DPS handles nonlinear operators and noisy measurements and needs one backward pass through the denoiser per step.

The bias of the point estimate. The approximation is exact only if $p(\boldsymbol{x}_1 \mid \boldsymbol{x}_t)$ is a point mass, which is true only at $t = 1$. When the posterior over clean images is multimodal, the likelihood at its mean is not the mean of the likelihood, and DPS steers toward images that agree with the measurement at the mean rather than toward the right mixture of modes. Figure 16 makes this visible in two dimensions, where the exact posterior is available in closed form: DPS satisfies the measurement but puts 42% of its samples on an arm that holds 9% of the posterior mass, and no step size brings the total variation distance below about 0.35. A Gaussian approximation $p(\boldsymbol{x}_1 \mid \boldsymbol{x}_t) \approx \mathcal{N}(\hat{\boldsymbol{x}}_1, r_t^2 I)$ reduces this bias for linear $\mathcal{A}$, since then $p_t(y \mid \boldsymbol{x}) \approx \mathcal{N}(y; \mathcal{A}\hat{\boldsymbol{x}}_1, \sigma_y^2 I + r_t^2 \mathcal{A}\mathcal{A}^\top)$ in closed form.

Successors. Denoising diffusion restoration models (DDRM) [155] handle linear problems in the spectral basis of the singular value decomposition of $\mathcal{A}$, where measured and unmeasured coordinates decouple, and sample the posterior efficiently without problem-specific training. REDdiff [239] takes a variational view. It fits a simple distribution to the posterior by minimizing a KL divergence, which leads to a measurement loss plus a regularizer built from denoising at many noise levels, in the spirit of regularization by denoising. DAPS [428] decouples consecutive steps. At each noise level it samples a clean estimate from an approximate posterior given $\boldsymbol{x}_t$ and y, and then renoises that estimate independently to the next level. The decoupling lets later steps correct large errors from earlier ones, which matters for strongly nonlinear problems such as phase retrieval. For latent diffusion models, the measurement acts on the decoded image, and PSLD [289] adds a term that keeps the latent consistent under encoding and decoding, with provable recovery in a linear model. In medical imaging, score-based priors were among the first diffusion models used for CT and MRI reconstruction [65, 324]. There, because the measurement process is known and the posterior is the object of interest, calibrated uncertainty matters more than for creative generation, and asymptotically exact methods such as those of Section 6.3 are especially attractive.

6.6 Test-time compute for diffusion language models

Masked diffusion language models (Section 2.9) decode by unmasking tokens in parallel over a series of steps. Their inference-time scaling has a different shape from that of continuous models, for two reasons. First, a standard masked sampler never revisits a decision: by the reverse rates Eq. (45) of the absorbing process, an unmasked token stays fixed, so an early error cannot be repaired. Second, deployed samplers rarely follow the reverse process. They unmask the tokens on which the model is most confident, a greedy rule that improves quality but makes generation nearly deterministic and collapses the diversity on which search depends.

Remasking. ReMDM [360] derives a sampler from a discrete diffusion model with a custom backward process in which already decoded tokens can be remasked. It applies to pretrained masked models without retraining, and it gives masked diffusion a genuine compute dial: as the number of steps grows, the quality of generated text approaches that of autoregressive models, and with a small budget it degrades more gracefully. LLaDA2.1 adds token editing to a large diffusion language model, so that decoded tokens can also be revised [25].

Particles and verification. The SMC machinery of Section 6.3 transfers directly. Self-rewarding SMC [229] runs several decoding trajectories in parallel and uses the model's own trajectory-level confidence as the potential, so that resampling steers toward globally confident completions instead of locally greedy ones, without an external reward. Prism [15] organizes search hierarchically: it

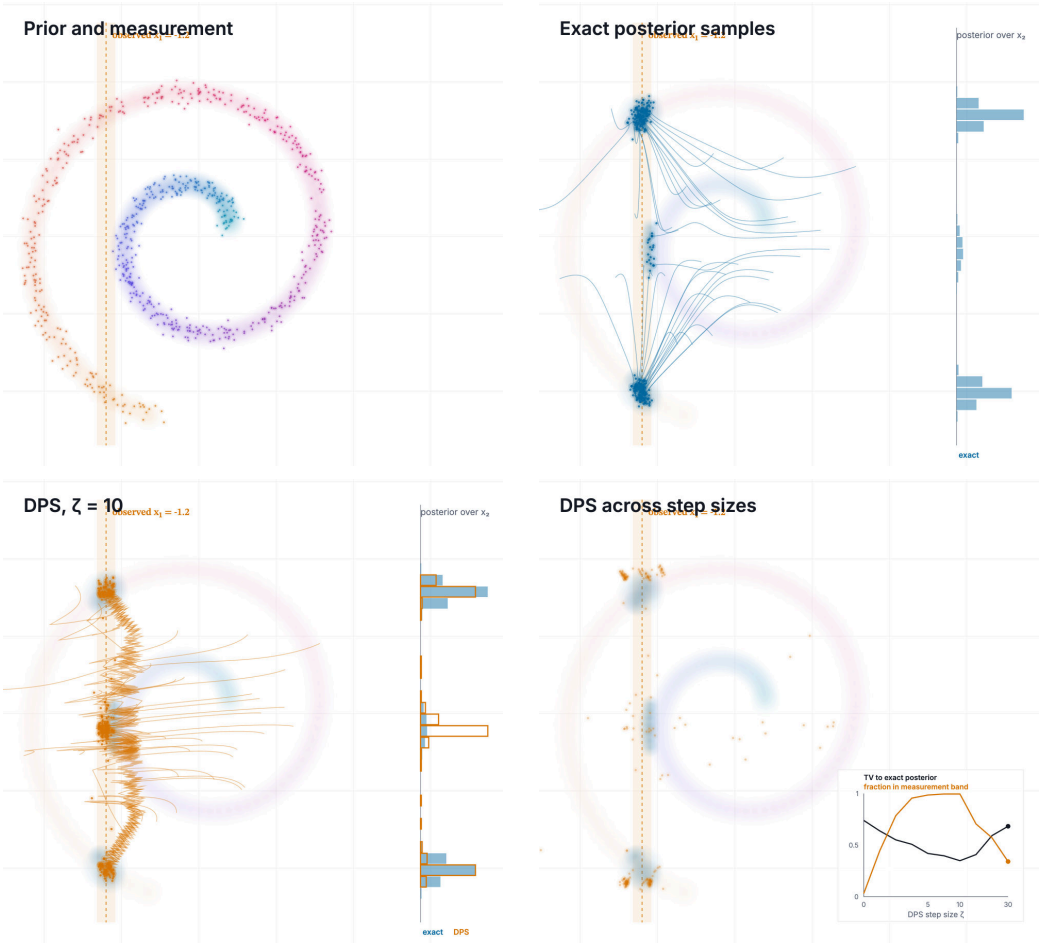


Figure 16: **Posterior sampling with an exact reference** (Animation A14). The prior is the spiral mixture and only the horizontal coordinate is observed, $y = x_1 + n$ with $\sigma_y = 0.06$ (top left). For a Gaussian mixture prior and a linear Gaussian measurement the posterior is again a mixture with a closed-form velocity, so its samples are exact (top right). It puts 9% of its mass on the middle arm. DPS [66], run with the exact prior denoiser, satisfies the measurement but places 42% of its samples on that arm (bottom left). Sweeping the step size ζ trades measurement consistency against stability, and the total variation distance to the exact posterior never falls below about 0.35 (bottom right).

prunes trajectories and reallocates compute within an early-to-mid denoising window, branches locally by remasking low-confidence parts while keeping high-confidence tokens, and replaces external verifiers by self-evaluation prompts on intermediate completions, with gains on mathematical reasoning and code benchmarks across several diffusion language models. For sequence-level rewards, nested SMC [44] addresses the overoptimism of best-of-N and the weight degeneracy of bootstrap SMC, and identifies and corrects errors in earlier formulations of Feynman–Kac steering that bias the final estimates.

Order and drafting. The freedom to decode in any order looks like a larger search space, but Ni et al. [251] find that for reasoning it can hurt: models use the flexibility to postpone uncertain tokens, which are exactly the decisions where exploration matters, and solution coverage collapses. A fixed left-to-right order with standard GRPO elicits reasoning more effectively. TiDAR [210] combines the two regimes within one forward pass: it drafts tokens in parallel by diffusion and accepts them autoregressively, using structured attention masks so that drafting and verification share the computation. The draft-and-verify structure is a form of test-time compute that buys speed rather than quality. Which of these mechanisms, remasking, particles, self-verification or ordering, gives the best return per unit of compute is not yet settled, and the question is studied mostly on mathematics and code, where answers can be checked.

Summary. Inference-time scaling turns a sampler into a search procedure, and the tilted target Eq. (87) gives the different procedures a common yardstick. Best-of-N is near optimal per unit of divergence but exponentially expensive. SMC is consistent and allocates compute adaptively, but it depends on potentials that approximate h_t. Gradient guidance is cheap and biased. Noise search is the natural object for deterministic samplers, and it can be amortized into a hypernetwork, which brings the methods back to fine-tuning. Across all of them the limiting factor is increasingly the verifier rather than the search, which is why reward models that reason and critique (Section 4.1) appear both here and in post-training.

7 Applications

The previous sections treated diffusion and flow models as a single method. This section looks at what the method is used for. The core recipe is the same in every domain: choose a representation of the data, choose an interpolation path from noise to data, and train a network to predict the velocity or the clean sample (Section 2). The domains differ in the representation and in what is conditioned on. Images and videos live in the latent space of an autoencoder. Robot actions are short trajectories in joint space. Protein backbones are sets of rigid frames, crystals live on a torus and weather states on a sphere. Each of these choices changes the interpolation path and the symmetries the network has to respect, and the subsections below make this concrete. We concentrate on work from 2024 to 2026. Earlier application surveys cover the first wave [39, 405].

7.1 Image generation and editing

Text-to-image generation is the application in which most of the methods of this survey were first developed and tested, and it remains the main commercial use of diffusion and flow models.

The frontier landscape. By 2024 the field had converged on one architecture. A convolutional autoencoder compresses the image by a factor of 8 or 16 per side [287], the latent is cut into

patches, and a diffusion transformer with joint attention over text and image tokens predicts the velocity of a rectified flow [84, 267]. Stable Diffusion 3 established this recipe at scale and showed that biasing the training times toward the middle of the path, where the image content is decided, improves samples (Section 2.5). Most open and closed models released since then follow it, and Table 6 lists representative ones. Imagen 3 is a latent diffusion model described mainly through its evaluations [137]. FLUX.1 Kontext uses the same flow-matching backbone for generation and editing [29]. Seedream 3.0 and 4.0 are bilingual Chinese and English models with an emphasis on typography and, in the later version, on unified generation and multi-image editing [96, 339]. Qwen-Image [384] and Qwen-Image-2.0 [444] condition a multimodal diffusion transformer on a vision-language model instead of a pure text encoder, and the latter is followed by a dedicated reinforcement-learning stage [401]. LongCat-Image [243] and Kandinsky 5.0 [11] are recent open families with detailed reports on data curation, and UltraFlux [413] trains a FLUX-based transformer natively at 4K resolution, which requires changes to the positional encoding, the autoencoder and the loss weighting at the same time.

Table 6: Representative text-to-image and editing models. "Open" means that weights of at least one released variant are public.

Model	Weights	Design points
SD3 [84]	open	Rectified flow, MM-DiT with joint text and image attention, logit-normal time sampling.
Imagen 3 [137]	closed	Latent diffusion; report focuses on quality and responsibility evaluation.
FLUX.1 Kontext [29]	open	Flow matching; editing by appending context-image tokens to the sequence.
Seedream 3.0/4.0 [96, 339]	closed	Bilingual, typography, high resolution; 4.0 unifies generation and multi-image editing.
Qwen-Image [384, 444]	open	Vision-language condition encoder, curriculum from no text to paragraph-level text.
HunyuanImage 3.0 [341]	open	Native multimodal autoregressive MoE, about 80B total and 13B active parameters.
LongCat-Image [243]	open	Bilingual text rendering, reward models in an RL stage.
OmniGen2 [386]	open	Separate text and image decoding paths on top of a multimodal language model.

Native multimodal generators. A second family puts image generation inside a language model. Transfusion trains one transformer with a next-token loss on text and a diffusion loss on image patches [458], MAR replaces the categorical output head of an autoregressive image model by a small diffusion head [188], and Show-o2 attaches a flow-matching head to a language model [393]. These are the hybrids of Section 2.10. Recent systems scale the idea. HunyuanImage 3.0 is an autoregressive mixture-of-experts model that unifies understanding and generation [341], Emu3.5 is pretrained on more than ten trillion interleaved vision-language tokens and converts its token-by-token decoding into parallel discrete diffusion for speed [72], and OmniGen2, TUNA and UniDDT study how the visual representation should be shared between understanding and generation [216, 367, 386]. The question of which representation the generator should operate in connects to the representation-alignment results of Section 2.8. Tokenizers trained for semantics rather than pixel accuracy scale better as latent spaces for generation [410], and a recent survey organizes this interplay between representation learning and generation [399]. One consequence of the discrete

side is relevant for Section 6: beam search over the tokens of an autoregressive image model is an effective inference-time search, while the same budget spent on noise search for a continuous diffusion model gives smaller gains [285].

Text rendering and compositionality. Two failure modes of early models are now the main axes of competition. The first is text rendering. A glyph occupies a few latent pixels, and the text encoder sees subword tokens, not letters, so the model must learn the spelling of every word from captioned images. Reports address this with data rather than architecture. Qwen-Image follows a curriculum that starts from images without text and moves to paragraph-level text [384], Seedream 3.0 lists fine-grained typography as a main target [96], and HunyuanVideo 1.5 adds a glyph-aware text encoding [383]. The second is compositionality: binding attributes to the right object, placing objects in the right relation and generating the right number of them. Guidance (Section 2.11) increases prompt adherence on average but does not by itself solve binding, which is why most of the reward models of Section 4.1 include compositional terms.

Editing. Instruction-based editing is treated as a fine-tuning problem in Section 3.4. The application side adds two observations. First, data is the bottleneck. Editing triplets of a source image, an instruction and an edited image are expensive to collect, and pipelines now mine them automatically with generators and vision-language judges [167]. Second, the editor no longer needs to be large: a pipeline that lets a 2B-parameter vision-language model guide a compact diffusion backbone reaches the quality of much larger open editors [6]. Benchmarks have moved from "was the instruction followed" to "is the result physically consistent". PICABench checks, for example, whether removing an object also removes its shadow and reflection, and finds that current editors often do not [279].

Evaluation. Distribution metrics such as FID compare feature statistics of generated and real images and do not see the prompt. Prompt-conditioned benchmarks fill this gap. GenEval generates images for templated prompts and checks object presence, count, color and position with a detector [104]. T2I-CompBench++ contains 8,000 compositional prompts in four groups (attribute binding, object relationships, numeracy and complex compositions) and uses specialized metrics per group [131]. Arenas collect pairwise human votes. GenAI-Arena [145] ranks image and video models from such votes, and the ranking model is the Bradley-Terry model of Eq. (61) with one scalar per model instead of one per image,

$$\Pr(\text{model } A \text{ preferred over } B) = \sigma(s_A - s_B), \tag{98}$$

with the scores s fitted by maximum likelihood on all votes. The same model that trains reward models in Section 4 thus also produces the public leaderboards. Arena scores are robust to benchmark overfitting but depend on the prompts users choose to type, and they mix aesthetic preference with correctness. Detector-based benchmarks are the opposite. A model can saturate them, as frontier models increasingly do on GenEval, while still failing on prompts outside the templates. A complete evaluation therefore reports both kinds.

Few-step deployment. Production systems rarely run the 50 solver steps of the base model. Distilled students (Section 5) and few-step flow maps are standard, and recent work extends one-step MeanFlow generators from class labels to text prompts, which turns out to require a text encoder whose representation is discriminative enough for a single step to resolve the prompt [445].

7.2 Video

Video generation is the application with the largest compute budgets and the fastest change. The early video diffusion model of Ho et al. used a factorized space-time U-Net and trained jointly on images and videos [124]. Latent video models followed [30], and the current generation uses the recipe of the image models with three changes: a spatiotemporal autoencoder that also compresses time, a transformer that attends over all space-time patches, and a much larger investment in data curation.

Model landscape. Table 7 lists representative models. Movie Gen is a 30B-parameter transformer with a context of 73K video tokens, which corresponds to 16 seconds at 16 frames per second, and generates synchronized audio with a separate model [273]. Open models have closed much of the gap to closed ones. HunyuanVideo [165] and Wan [340] were released with weights, and HunyuanVideo 1.5 reaches comparable quality with 8.3B parameters [383]. LongCat-Video pretrains on video continuation to obtain long-video capability [242], Waver unifies text-to-video, image-to-video and text-to-image in one model [440], and Kandinsky 5.0 includes video models from 2B to 19B parameters [11]. On the closed side, Seedance 1.0 supports multi-shot generation [97] and Kling-Omni unifies generation, editing and reasoning over multimodal inputs [164]. Sora 2 and Veo 3 are documented through product releases rather than technical reports. Efficiency is a separate axis. SANA-Video uses linear attention and a constant-memory cache to generate minute-long video on a single consumer GPU [51], and LTX-Video was designed for real-time generation [113]. The acceleration methods of Section 5.6 and the distillation methods of Section 5, including transition-matching distillation of video models [254], are what make interactive use possible.

Table 7: Representative video generation models. Parameter counts are those stated in the reports.

Model	Weights	Design points
Movie Gen [273]	closed	30B transformer, 73K-token context, separate audio model.
HunyuanVideo [165, 383]	open	1.5: 8.3B parameters, sliding-tile attention, glyph-aware text encoding.
Wan [340]	open	14B, custom spatiotemporal VAE, released with smaller variants.
Seedance [97, 337, 338]	closed	Multi-shot (1.0); joint audio-video (1.5); four input modalities (2.0).
Kling-Omni [164]	closed	Unified generation, editing and reasoning over text, image and video inputs.
LongCat-Video [242]	open	13.6B, pretrained on video continuation for long videos.
LTX-2 [114]	open	14B video and 5B audio streams joined by cross-attention.
SANA-Video [51]	open	Linear attention, block-wise autoregression with constant-memory cache.

Joint audio and video. The newest systems generate the soundtrack together with the frames. Two designs exist. A cascade first generates video and then audio conditioned on it, as Movie Gen does. A joint model denoises both at once. Write the state as $\boldsymbol{x} = (\boldsymbol{x}^{\mathrm{v}}, \boldsymbol{x}^{\mathrm{a}})$ with separate

interpolation times t_{v} and t_{a}. A network that predicts the pair of velocities

$$(\boldsymbol{u}_\theta^{\mathrm{v}}, \boldsymbol{u}_\theta^{\mathrm{a}})(\boldsymbol{x}_{t_{\mathrm{v}}}^{\mathrm{v}}, \boldsymbol{x}_{t_{\mathrm{a}}}^{\mathrm{a}}, t_{\mathrm{v}}, t_{\mathrm{a}}, c) \tag{99}$$

covers joint generation with $t_{\mathrm{v}} = t_{\mathrm{a}}$, video-to-audio with $t_{\mathrm{v}} = 1$ (clean video given) and audio-to-video with $t_{\mathrm{a}} = 1$. This is the same idea as per-frame noise levels in Diffusion Forcing [46], applied to modalities instead of frames, and the flow-matching loss Eq. (24) applies to each block unchanged. Ovi uses two identical transformer towers with blockwise cross-modal fusion [219], LTX-2 an asymmetric pair of streams with shared timestep conditioning [114], and MOVA and Seedance 1.5 are further open and closed examples [311, 337]. Seedance 2.0 accepts text, image, audio and video as references [338]. Audio-driven models such as Wan-S2V and HuMo condition on a given speech track to animate a person with synchronized lips and body motion [54, 95].

Long and causal video. A bidirectional model generates a fixed-length clip in one pass. Longer or streaming generation requires an autoregressive factorization over chunks, which Section 5.5 develops together with its failure mode, the gap between training on real context and generating from self-generated context (Fig. 14). On the application side, FramePack compresses older frames more strongly so that thousands of frames fit in a fixed context and adds sampling orders that reduce drift [436], and Helios reports a 14B model that runs at 19.5 frames per second on a single H100 and generates minute-scale video [423]. Real-time models are also used for interaction with a person rather than a game, as in streaming avatars and live video editing [434].

Camera and motion control. Control signals enter as extra conditions (Section 3.5). Camera trajectories are encoded per frame [116, 373], object motion as dense point tracks in latent space [64], and motion learned from example clips through adapters [447]. Video-As-Prompt treats a reference video as an in-context prompt for a frozen video transformer and thereby unifies many semantic controls in one model [22].

Video models as general vision models. Large video models perform tasks they were not trained for. Prompted with an image and an instruction, Veo 3 segments objects, detects edges, edits images and solves simple visual puzzles [380], and video generation has been proposed as a reasoning medium in which the model "thinks" by drawing [346]. Physical plausibility is weaker than visual quality, and reinforcement learning with physics-based rewards is one response [144]. These observations motivate the use of video models as world models in the next subsection.

7.3 World models and interactive simulation

A world model predicts how an environment evolves in response to actions. With frames $\boldsymbol{x}^k$ and actions a^k it is a conditional distribution over futures,

$$p(\boldsymbol{x}^{1:K} \mid \boldsymbol{x}^0, a^{1:K}) = \prod_{k=1}^{K} p(\boldsymbol{x}^k \mid \boldsymbol{x}^{<k}, a^k), \tag{100}$$

and a diffusion world model represents each factor by a conditional flow or diffusion model, with velocity $\boldsymbol{u}_\theta(\boldsymbol{x}_t^k, t; \boldsymbol{x}^{<k}, a^k)$. The factorization is the same as for causal video in Section 5.5. The difference is that the conditioning sequence is chosen online by a user or an agent and is not known in advance, so the model must generate in real time, one chunk after the other, and must remain consistent with what it has shown before.

From games to open worlds. DIAMOND trains a reinforcement-learning agent entirely inside a diffusion world model of Atari games and argues that the visual detail preserved by diffusion, which discrete-latent world models lose, matters for the agent [8]. GameNGen runs the game DOOM as a diffusion model at 20 frames per second on one TPU, and human raters are only slightly better than chance at telling short clips of the simulation from the real game [353]. Genie learns a latent action space from unlabeled internet videos and generates playable environments from a single image with an autoregressive token model rather than a diffusion model [36]. UniSim learns an interactive simulator of real-world scenes from a mixture of datasets with different kinds of actions [407]. The 2025 and 2026 systems scale this to open domains with keyboard and mouse control. Matrix-Game 2.0 generates on the fly with few-step autoregressive diffusion [118], Hunyuan-GameCraft maps keyboard and mouse inputs to a shared camera representation [182], and Yume-1.5, LingBot-World, AlayaWorld and DreamForge-World target text control, sub-second latency, playable long horizons and consumer GPUs, respectively [14, 238, 286, 335]. Magpie takes a different route: a conventional engine executes the game rules and keeps the state, and the generative model only renders it, which makes the outcome of an interaction reproducible [427].

Compounding error. Figure 17 shows the central difficulty with a toy game in which the next-frame model is specified by hand. The factorization Eq. (100) lets us bound the error of a rollout by the errors of single steps.

Proposition 7.1 (Rollout error from one-step error)**.** *Let p and q be two distributions over $\boldsymbol{x}^{1:K}$ with the same initial frame and actions, and let $\varepsilon_k = \mathbb{E}_{\boldsymbol{x}^{<k}\sim p}\,\mathrm{TV}(p(\cdot \mid \boldsymbol{x}^{<k}), q(\cdot \mid \boldsymbol{x}^{<k}))$ be the one-step error of q on contexts drawn from p. Then*

$$\mathrm{TV}\big(p(\boldsymbol{x}^{1:K}), q(\boldsymbol{x}^{1:K})\big) \le \sum_{k=1}^{K} \varepsilon_k. \tag{101}$$

Derivation sketch. Define hybrids h_j that sample the first j frames from p and the rest from q, so $h_K = p$ and $h_0 = q$. The distributions h_j and h_{j-1} agree on $\boldsymbol{x}^{<j}$, which is drawn from p, and on how frames after j are generated given their past. They differ only in the kernel for frame j. Since the total variation between two joint laws with the same first marginal and the same later kernels is the expected total variation of the differing kernel, $\mathrm{TV}(h_j, h_{j-1}) = \varepsilon_j$, and the triangle inequality over $j = 1, \dots, K$ gives Eq. (101).

The bound is linear in K, which looks benign, but it has two catches. It becomes vacuous once $\sum_k \varepsilon_k \ge 1$, which for a one-step error of a few percent happens within tens of frames. And ε_k is measured on real contexts, which is exactly what teacher-forced training controls. The bound says nothing about which of the many models with the same ε_k behaves well once the rollout has left the support of real frames, and in that regime errors compound in a different sense. If the state is perturbed by e_{k-1} and the learned dynamics is L-Lipschitz with one-step error ε, then $e_k \le L\, e_{k-1} + \varepsilon$, so

$$e_K \le \varepsilon\, \frac{L^K - 1}{L - 1}, \tag{102}$$

which grows geometrically when $L > 1$. The two models of Fig. 17 have the same teacher-forced loss and differ only in L off the data: the self-forced variant contracts its own errors back toward valid states ($L < 1$ near the data) and keeps the game state over 64 actions, while the other one smears and drifts. GameNGen counters drift by adding noise to the context frames during training [353], the Diffusion Forcing family trains with independent noise levels per frame [46], and Self Forcing trains on self-generated context directly [133].

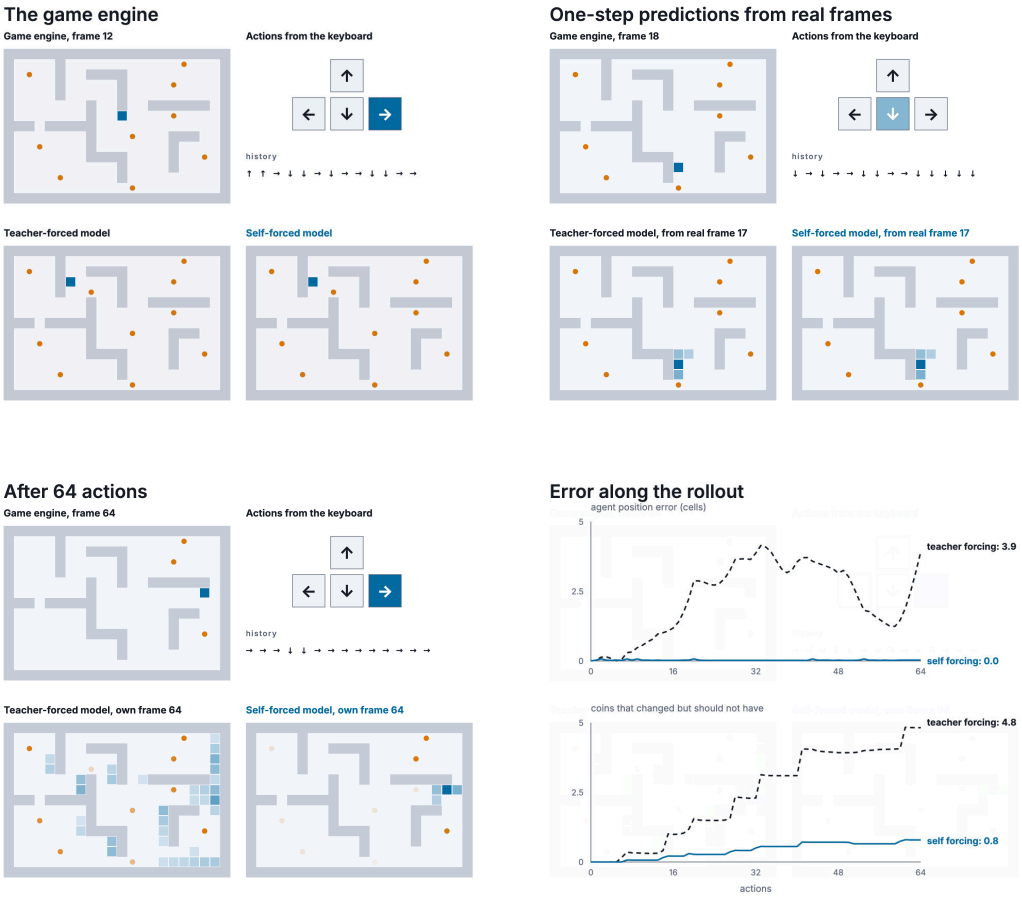


Figure 17: **Compounding errors in an action-conditioned world model** (Animation A15). Top left: a grid game in which each key press moves the agent one cell, walls block it and coins vanish when collected. Top right: a hand-specified next-frame model, conditioned on the frame and the key, moves most of the agent's intensity one cell and a little of it too far, too short or sideways. A second variant first sharpens its context, which leaves real frames unchanged, so both have the same one-step loss. Bottom left: after 64 actions generated on its own frames, the first model has smeared the agent, moved it away from the engine's position and partly erased coins it never reached, while the self-forced variant keeps the game state. Bottom right: position error and coin error along the rollout. The same train-test gap appears in autoregressive video (Figure 14).

Memory and consistency. A world model should show the same room when the user turns around. Short context windows forget it, and long ones are too slow for real time. Recent systems therefore separate a short window that drives control from a sparse long-term memory. WorldPlay rebuilds the context from geometrically important past frames and adds a training scheme against forgetting [331], Matrix-Game 3.0 adds a memory for 720p real-time generation [375], ReWorld confines most attention heads to recent frames and retrieves older frames from a bank indexed by camera pose [58], WorldCrafter learns an implicit 3D-aware memory queried by the target view [421], and Wonder conditions on a dense coordinate field rendered along the camera path [395]. SANA-WM uses hybrid linear attention to make minute-long generation affordable [465]. Benchmarks for these properties are recent. WorldRoamBench measures action following, drift, physical plausibility and memory over long horizons [398], and WBench evaluates multi-turn interactions in five dimensions [419].

World foundation models for physical AI. For robotics and driving, world models are trained on curated real video. The Cosmos platform provides pretrained world models, tokenizers and post-training recipes [256]. Cosmos-Predict2.5 is a flow-based model trained on 200M clips and refined by reinforcement learning, released at 2B and 14B [257], and Cosmos 3 extends the family to an omnimodal model that also generates actions [258]. PAN combines a language-model backbone for long-horizon latent dynamics with a video diffusion decoder [263], and Emu3.5 describes a native multimodal model trained on video frames and transcripts as a world learner [72]. Their use for robot learning is the topic of Section 7.5.

7.4 3D, audio and motion

3D from 2D priors. Large 3D datasets are much smaller than image datasets, so the first text-to-3D methods borrowed a 2D model. DreamFusion optimizes the parameters ψ of a 3D representation whose renderings $\boldsymbol{x} = g(\psi)$ should look like samples of a text-to-image model [275]. In our notation, with $\boldsymbol{x}_t = \alpha_t \boldsymbol{x} + \sigma_t \boldsymbol{x}_0$ and the model's noise prediction $\hat{\boldsymbol{x}}_0(\boldsymbol{x}_t, t, c)$, score distillation sampling uses the gradient

$$\nabla_\psi \mathcal{L}_{\text{SDS}} = \mathbb{E}_{t,\boldsymbol{x}_0}\Big[w(t)(\hat{\boldsymbol{x}}_0(\boldsymbol{x}_t, t, c) - \boldsymbol{x}_0)\,\frac{\partial g(\psi)}{\partial \psi}\Big], \tag{103}$$

which is the gradient of the denoising loss with the Jacobian of the network left out. Since $\hat{\boldsymbol{x}}_0 - \boldsymbol{x}_0$ is proportional to $-\nabla \log p_t$ by Tweedie's formula Eq. (12), the update moves each rendering toward higher density under the noised 2D model, which is the same mode-seeking reverse-KL objective as distribution matching distillation (Section 5.2). Its known symptoms, oversaturated and overly smooth objects, are the mode collapse of the reverse KL, and ProlificDreamer replaces the single 3D estimate by a distribution that is optimized with a variational score distillation [372].

Native 3D generation. The current generation trains directly on 3D assets. TRELLIS introduces a structured latent, a sparse voxel grid with features from a vision foundation model, that decodes to radiance fields, Gaussians or meshes, and trains rectified-flow transformers with up to 2B parameters on it [391]. Its successor uses a native sparse voxel structure that represents arbitrary topology and physically based materials [392]. Hunyuan3D 2.0 separates a flow-based shape model from a texture model [451], PartCrafter denoises several part meshes jointly from one image [200], and SAM 3D reconstructs geometry, texture and layout of objects in cluttered natural images from data annotated with humans and models in the loop [298]. For scenes, HunyuanWorld 1.0 and HY-World 2.0 generate navigable 3D worlds from text or images, the latter through panorama

generation, trajectory planning and expansion into a Gaussian-splatting representation [135, 336]. Explicit 3D scenes are the alternative to the video world models of Section 7.3: they are consistent by construction but less dynamic.

Audio and speech. AudioLDM brought latent diffusion to text-to-audio generation with a latent space aligned to text through contrastive language-audio pretraining [206]. Stable Audio Open is an open-weights text-to-audio model trained on Creative Commons data that generates stereo audio at 44.1 kHz [85]. For speech, VibeVoice uses next-token diffusion: an autoregressive language model emits one continuous latent per step, and a small diffusion head samples it, the same hybrid as MAR (Section 2.10), which lets it synthesize up to 90 minutes of multi-speaker speech [269]. Joint audio-video models were covered in Section 7.2.

Human motion. The Motion Diffusion Model generates sequences of human poses with a transformer and classifier-free guidance [342]. It predicts the clean sample rather than the noise, which allows geometric losses on joint positions and foot contact to be applied directly to the prediction, an early example of the target choice discussed in Section 2.5.

7.5 Robotics

Why generative policies. A policy maps an observation o (camera images, proprioception, an instruction) to actions. Demonstrations are multimodal: a person may pass an obstacle on the left or on the right. A regression policy trained with the squared error learns the conditional mean $\mathbb{E}[A \mid o]$, which for two equally likely modes is the average of left and right, a path through the obstacle. This is the same quantity as the denoiser at pure noise in Eq. (12), and a diffusion policy replaces it with the full conditional distribution. Diffusion Policy introduced this for visuomotor control and reported an average improvement of 46.9% over prior methods on 12 manipulation tasks [60]. An earlier line, Diffuser, generated whole state-action trajectories and planned by guided sampling and inpainting [141], the robotic counterpart of the reward guidance in Section 6.4.

Action chunks and flow matching. Current policies generate a chunk of H future actions $A = (a_k, \dots, a_{k+H-1})$ at once. We write the flow time as $\tau \in [0, 1]$ to distinguish it from control time. With noise $\varepsilon \sim \mathcal{N}(0, I)$ and the interpolant $A^\tau = \tau A + (1 - \tau)\varepsilon$, the policy is trained with the conditional flow-matching loss

$$\mathcal{L}(\theta) = \mathbb{E}_{(o,A),\varepsilon,\tau} \big\| \boldsymbol{u}_\theta(A^\tau, \tau; o) - (A - \varepsilon) \big\|^2, \tag{104}$$

which is Eq. (24) with the observation as the condition. At test time the robot integrates the ODE from noise in a few steps, executes the first $h \le H$ actions and replans. Chunking reduces the number of decisions per second, smooths the motion and captures temporal correlation within the chunk. π_0 builds such an action expert on a pretrained vision-language model and trains it with flow matching on data from several robots [28], and $\pi_{0.5}$ co-trains on heterogeneous data, including web data and high-level semantic prediction, to generalize to new homes [271]. These vision-language-action models (VLAs) are now the main use of flow matching in robotics.

Shaping the flow for actions. Because actions are low-dimensional and latency matters, the design choices of Sections 2 and 5 are revisited. Action trajectories concentrate energy at low frequencies, and FreqFM generates them in DCT coordinates with a frequency-matched source distribution [255]. The isotropic Gaussian source is replaced by a library of learned priors in

LAFM [236]. On the solver side, more integration steps do not necessarily raise task success, and Coda spends part of the budget on a learned endpoint correction instead [334], while Kinematic MeanFlow adapts MeanFlow (Section 5.4) to one-step action generation after finding that the plain version collapses [87].

Reinforcement learning for flow policies. Imitation alone does not exceed the demonstrator. Policy-gradient methods need log-likelihoods, which a deterministic flow does not provide, and the solutions are those of Section 4.2. π_{RL} either learns a noise network that turns sampling into a Markov decision process with tractable likelihoods or converts the ODE into an SDE [53]. SimpleVLA-RL scales reinforcement learning for VLAs [180], and ForesightFlow augments each action chunk with a predicted success potential so that the same flow can score its own candidates for best-of-K selection [241], the robotic analogue of Section 6.2.

Video models in the loop. World models from Section 7.3 enter robotics in three ways. As simulators, they evaluate policies: a Veo-based simulator has been used to evaluate Gemini Robotics policies in and out of distribution [101], and VLA-RFT fine-tunes a policy with rewards computed on rollouts of a learned simulator [181]. As backbones, they become policies: Cosmos Policy fine-tunes a pretrained video model to output actions in a single post-training stage without new modules [159], and mimic-video and Genie Envisioner decode actions from the latent dynamics of a video model with a flow-matching head [198, 262]. The argument for this design is that video pretraining teaches physical dynamics that static image-text pretraining does not.

7.6 Science: proteins, molecules, materials, medicine and weather

Scientific data come with symmetries and constraints that images lack. Rotating a molecule does not change it, crystals are periodic and weather states live on a sphere. The common thread of this subsection is how such structure enters the path and the network.

Symmetry. Let a group G act on the data by orthogonal maps, such as rotations of all atoms. A generative model should give the same probability to $\boldsymbol{x}$ and $g\boldsymbol{x}$. The following proposition, used by equivariant diffusion [127], says how to obtain this with a flow.

Proposition 7.2 (Invariant marginals from an equivariant field)**.** *If the source p_0 is G-invariant and the velocity is G-equivariant, $\boldsymbol{u}(g\boldsymbol{x}, t) = g\,\boldsymbol{u}(\boldsymbol{x}, t)$ for all $g \in G$, then every marginal p_t of the flow is G-invariant. Conversely, if the coupling of $(\boldsymbol{x}_0, \boldsymbol{x}_1)$ is G-invariant and the conditional velocity is G-equivariant, the marginal velocity Eq. (23) is G-equivariant.*

Derivation sketch. Let ϕ_t be the flow map. The curve $g\phi_t(\boldsymbol{x})$ starts at $g\boldsymbol{x}$ and satisfies $\frac{\mathrm{d}}{\mathrm{d}t} g\phi_t(\boldsymbol{x}) = g\,\boldsymbol{u}(\phi_t(\boldsymbol{x}), t) = \boldsymbol{u}(g\phi_t(\boldsymbol{x}), t)$, so by uniqueness of ODE solutions $\phi_t(g\boldsymbol{x}) = g\phi_t(\boldsymbol{x})$. For a set B, $p_t(gB) = p_0(\phi_t^{-1}(gB)) = p_0(g\phi_t^{-1}(B)) = p_0(\phi_t^{-1}(B)) = p_t(B)$. For the converse, the marginal velocity is a conditional expectation over pairs $(\boldsymbol{x}_0, \boldsymbol{x}_1)$ whose joint law is invariant, and applying g to both endpoints maps the conditional path through $\boldsymbol{x}$ to the one through $g\boldsymbol{x}$.

Two details follow from the proposition. Translations have no invariant probability distribution on $\mathbb{R}^{3N}$, so the source is a Gaussian restricted to the subspace with zero center of mass, and the positions are centered before training. And the network must be equivariant, either by construction, as in E(3)-equivariant graph networks [127], or approximately, by training a general transformer on randomly rotated data. Recent protein models increasingly take the second route [1, 371].

Protein backbones on SE(3). A protein backbone is a chain of residues, and the N, Cα and C atoms of each residue form a nearly rigid triangle. Each residue can therefore be described by a frame $T_i = (R_i, \boldsymbol{x}_i) \in \mathrm{SE}(3)$, a rotation $R_i \in \mathrm{SO}(3)$ and the position $\boldsymbol{x}_i$ of its Cα atom, and the backbone of N residues is a point of $\mathrm{SE}(3)^N$ (Fig. 18). FrameDiff developed diffusion on this space, with Brownian motion on the rotations, and generated designable monomers of up to 500 residues [415]. FrameFlow replaces diffusion by flow matching on the same space, which needs fewer sampling steps [414]. The flow-matching path is the geodesic interpolant of each factor. For translations, with $\boldsymbol{x}_i^0$ drawn from a centered Gaussian,

$$\boldsymbol{x}_i^t = t\,\boldsymbol{x}_i^1 + (1-t)\,\boldsymbol{x}_i^0, \qquad \dot{\boldsymbol{x}}_i^t = \boldsymbol{x}_i^1 - \boldsymbol{x}_i^0. \tag{105}$$

For rotations, with R_i^0 drawn uniformly from SO(3), the geodesic from R_i^0 to R_i^1 is

$$R_i^t = R_i^0 \exp(t\,\Omega_i), \qquad \Omega_i = \log\big((R_i^0)^\top R_i^1\big) \in \mathfrak{so}(3), \tag{106}$$

where exp and log are the matrix exponential and logarithm and Ω_i is a skew-symmetric matrix whose axis and angle are those of the relative rotation. Differentiating gives $\dot{R}_i^t = R_i^t\,\Omega_i$, and since the remaining rotation from R_i^t to R_i^1 is $\exp((1-t)\Omega_i)$, the target can be written without R_i^0 as

$$\dot{R}_i^t = \frac{R_i^t\,\log\big((R_i^t)^\top R_i^1\big)}{1-t}, \tag{107}$$

the rotational analogue of the velocity $(\boldsymbol{x}^1 - \boldsymbol{x}^t)/(1-t)$ of a straight line. The network predicts the clean frames $\hat{T}_i^1$ and the loss compares translations in Euclidean norm and rotations through the norm of the skew matrix, which is the geodesic distance on SO(3). The uniform distribution on SO(3) is invariant under all rotations and the translation source is centered, so Proposition 7.2 applies with an SE(3)-equivariant network.

Protein design. RFdiffusion fine-tuned the RoseTTAFold structure-prediction network as a denoiser of residue frames and designed binders, symmetric assemblies and scaffolds for functional motifs, many of which were validated experimentally [376]. Its successors scaffold enzyme active sites at the level of atoms [3], design all-atom interactions with small molecules and nucleic acids [37] and design antibodies with atomic accuracy [18]. Chroma made conditioning on symmetry, shape, secondary structure and natural-language annotations programmable through composable conditioners [140]. Proteina scales a flow-based backbone generator with a scalable transformer, hierarchical fold-class labels and millions of synthetic structures [100], and La-Proteina generates all atoms jointly with the sequence by modeling the backbone explicitly and the side chains through per-residue latent variables of fixed size [99]. Joint sequence-structure models without an autoencoder stage [223], compact all-atom generators for enzyme design [381] and all-to-all interaction design across molecule types [437] extend the range. Inference-time methods from Section 6 apply directly: Feynman-Kac steering enforces design constraints with particle filtering [115], and binder design benefits from scaling test-time compute [76].

Structure prediction as generation. AlphaFold 3 predicts the structure of complexes of proteins, nucleic acids, small molecules and ions with a diffusion module that operates directly on atom coordinates [1]. Generating the structure instead of regressing it returns a distribution over conformations and handles the ambiguity of flexible regions. Open reproductions and extensions followed. Boltz-1 reaches comparable accuracy with open weights [382], Boltz-2 adds binding-affinity prediction [266], BoltzGen turns the model into a binder generator [328], and Chai-1 and

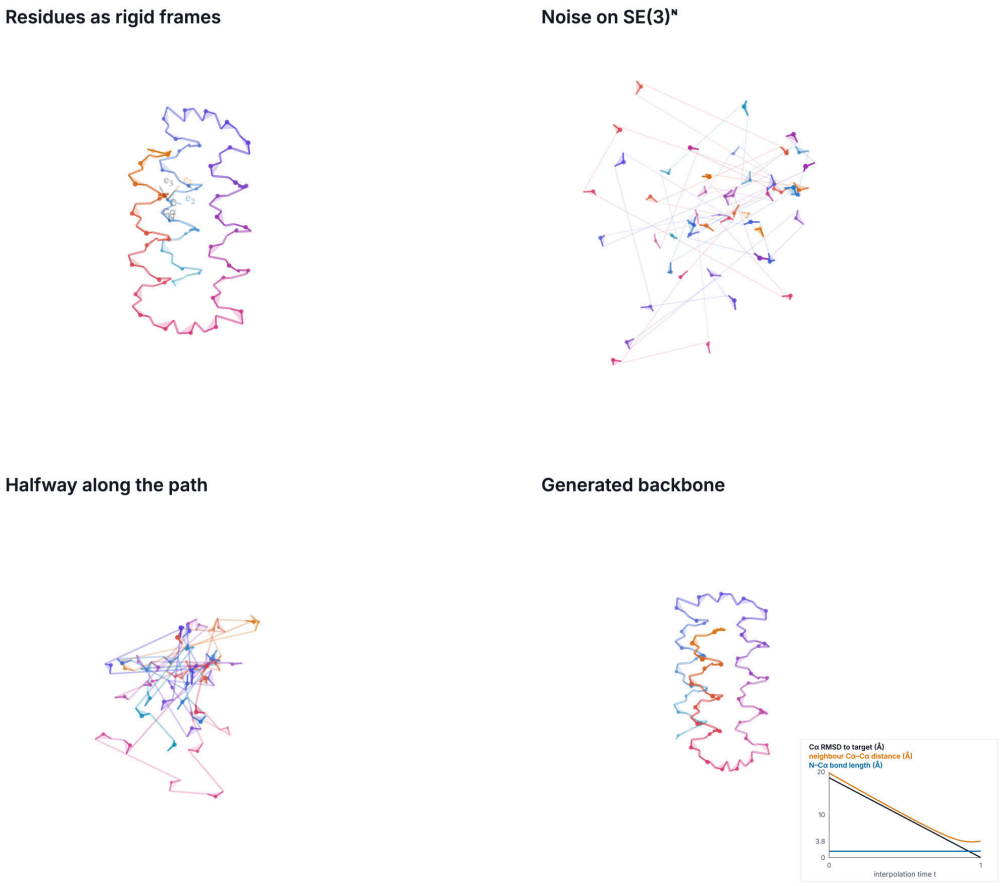


Figure 18: **Backbone generation on SE(3)** (Animation A16). Each residue is a rigid frame (R_i, x_i) with its N, Cα and C atoms at fixed ideal coordinates (top left). Noise draws translations from a centered Gaussian and rotations uniformly from SO(3) (top right). The conditional flow-matching path moves translations linearly and rotations along SO(3) geodesics toward an idealized three-helix bundle (bottom row). Bond lengths inside each residue are exact throughout, so only the arrangement of the frames has to be generated.

Chai-2 cover structure prediction and zero-shot antibody design [42, 43]. SimpleFold shows that a flow-matching model with only general-purpose transformer blocks is competitive in folding [371], and all-atom flow maps distilled from cofolding models need only a few steps, which makes search over many samples affordable [301].

Molecules. Small molecules combine continuous coordinates with categorical atom types. The E(3)-equivariant diffusion model EDM diffuses both jointly with an equivariant network [127]. Equivariant flow matching regularizes the coordinate path with an equivariant optimal transport, which straightens it (Section 2.6), and treats atom types with a separate path [326]. FlowMol extends flow matching to the mixed continuous and categorical state, connecting to the discrete flows of Section 2.9 [81]. DiffDock frames molecular docking as generation on the manifold of ligand poses, a product of translations, rotations and torsion angles, and diffuses on each factor [69].

Materials. A crystal is a periodic arrangement of atoms in a unit cell, described by atom types, fractional coordinates in $[0, 1)^3$ and a lattice matrix. Fractional coordinates live on a torus, so the noise is a wrapped Gaussian and the interpolation takes the shortest path modulo one. MatterGen diffuses all three components jointly and is fine-tuned with adapter modules to steer generation toward target chemistry, symmetry and properties [426]. Recent work lets the number of atoms change during generation [259], predicts organic crystal structures with unit-cell flow matching [160, 217], learns all-atom flow maps for fast candidate generation [83] and applies GRPO-style post-training (Section 4.2) to the joint coordinate and lattice state [330].

Cells. At the level of cells, Lingshu-Cell models single-cell transcriptomes with masked discrete diffusion and simulates responses to perturbations [429], an application of the discrete models of Section 2.9 outside language.

Medicine. Medical imaging is mainly an inverse problem. Accelerated MRI measures a subsampled Fourier transform and CT a set of projections, so the measurement is $\boldsymbol{y} = \mathcal{A}\boldsymbol{x} + \eta$ with a known linear operator $\mathcal{A}$, and the posterior samplers of Section 6.5 apply with a prior trained on fully sampled images [65, 324]. A prior trained once can then be used with any sampling pattern, which a supervised reconstruction network cannot. Two further uses are synthetic data for rare conditions and privacy-preserving data sharing. Both depend on whether the model memorizes its training images, the question studied in Section 2.7, which matters more for patient data than for any other domain in this section.

Weather. A weather forecast is a distribution, and ensembles of forecasts quantify its spread. GenCast generates an ensemble of 15-day global forecasts with a conditional diffusion model that steps autoregressively from the recent atmospheric states and reports greater skill than the operational ensemble of the European Centre for Medium-Range Weather Forecasts [278]. The factorization is Eq. (100) without actions, and the compounding-error analysis of Section 7.3 applies. Ensembles are scored by the continuous ranked probability score of the forecast distribution F for an observation y,

$$\mathrm{CRPS}(F, y) = \mathbb{E}_{X\sim F}|X - y| - \tfrac{1}{2}\,\mathbb{E}_{X,X'\sim F}|X - X'|, \tag{108}$$

which is a proper score: its expectation over y is minimized by the true distribution. The first term rewards accuracy and the second rewards spread, and both can be estimated from ensemble members. Because it needs only samples, the CRPS can also serve as a training loss. FGN trains an ensemble of perturbed models directly on it and reports better ensemble skill than the previous state of the art [7], and Swift fine-tunes a single-step consistency model autoregressively with it, which keeps 6-hourly forecasts stable for up to 75 days [329]. The trend repeats the one in images and video: multi-step diffusion first establishes quality, and distillation or direct distributional objectives then make it fast enough for the application.

8 Open Problems and Outlook

The previous sections followed a model through its lifecycle: pretraining with a denoising or velocity objective (Section 2), adaptation to new tasks and data (Section 3), alignment with rewards and preferences (Section 4), compression into few-step samplers (Section 5) and search at inference time (Section 6). Each stage was presented with its own objective and its own guarantees. This section collects the questions that cut across the stages and that we consider open. Some are conceptual, such as whether diffusion and autoregression are distinct model classes at all. Others are practical,

such as how to divide a fixed compute budget between training, post-training and sampling. We state each problem, summarize what is known, and where possible give the small calculation that makes the trade-off precise.

8.1 Autoregression and diffusion as one family

A spectrum rather than a dichotomy. Sections 2.9 and 2.10 described models that combine the two factorizations. Block diffusion [12] makes the connection explicit. A sequence is split into blocks $\boldsymbol{x} = (\boldsymbol{x}^1, \ldots, \boldsymbol{x}^B)$ and the likelihood factorizes autoregressively over blocks,

$$\log p_\theta(\boldsymbol{x}) = \sum_{b=1}^{B} \log p_\theta(\boldsymbol{x}^b \mid \boldsymbol{x}^{<b}), \tag{109}$$

while each conditional is a discrete diffusion model over the tokens of one block, trained with the masked diffusion bound Eq. (47). A block length of one token recovers an autoregressive model, and a single block covering the whole sequence recovers a masked diffusion model. Intermediate block sizes keep the parallel decoding of diffusion inside a block and allow key-value caching of the completed blocks, as in an autoregressive transformer, and they support generation of arbitrary length. The block size is then a hyperparameter that trades quality against parallelism, and the two classical model families are its endpoints.

When does diffusion win? The comparison between the endpoints depends on what is scarce. Prabhudesai et al. [277] trained both families under a fixed amount of unique data and many epochs. Autoregressive models were better when compute was the limiting resource. When data was limited and compute plentiful, masked diffusion models kept improving with repeated passes over the same data and eventually outperformed them. The authors derive a critical compute threshold at which the ordering flips, and they attribute the difference to the implicit augmentation of diffusion training: a masked model sees each sequence under many random orders and masking ratios, while an autoregressive model always sees the same left-to-right factorization. If high-quality text becomes the bottleneck for language model pretraining, this result suggests that the choice of factorization should be revisited for that regime, not only for speed.

Scaling diffusion language models. Diffusion language models have moved from small proofs of concept [295, 296] to models trained at the scale of their autoregressive counterparts [24, 25, 252, 412]. LLaDA-MoE [464] uses a mixture-of-experts backbone with 7B parameters in total, of which 1.4B are active per token, trained on about 20T tokens. It outperforms earlier dense diffusion language models with more active parameters, and its instruction-tuned version is reported to be comparable to an autoregressive model of similar active size. Open frameworks that standardize training, inference and evaluation, such as dLLM [463], make it easier to compare design choices that were previously entangled with different codebases. What is still missing is a clear account of the inference cost. A diffusion language model can decode many tokens per forward pass, but it cannot cache its context in the same way, and its quality depends on the unmasking schedule and on the test-time strategies of Section 6.6. A fair comparison has to fix the quality and report the total compute, not the number of forward passes.

Native multimodal models. For images and video the same question appears in a different form. Unified models generate text autoregressively and images by diffusion or flow matching inside one transformer [393, 458], or they predict continuous tokens autoregressively with a small

diffusion head per token [188]. Others treat image generation as next-token prediction over a very large multimodal corpus [72]. The comparison also depends on the inference budget. Riise et al. [285] report that visual autoregressive models benefit more from beam search than diffusion models do, and they attribute the advantage to the discrete token space, which allows early pruning of partial generations and reuse of computation. The open problem is to determine which properties are inherent to the factorization and which come from the training recipe, the tokenizer or the search algorithm. We expect the answer to be a model that can choose its decoding order and block size per task, with the classical families as special cases.

8.2 Evaluation that tracks human judgement

Distribution metrics. FID fits a Gaussian to Inception features of real and generated images and computes the Fréchet distance between the two Gaussians. Both assumptions are questionable for modern generators: the features were trained on ImageNet classification and do not represent the content of text-to-image prompts well, and the feature distribution is far from Gaussian. Jayasumana et al. [142] show that FID can disagree with human raters and can even worsen when image quality visibly improves. They propose CMMD, the maximum mean discrepancy between CLIP embeddings of real and generated images under a Gaussian kernel k,

$$\mathrm{MMD}^2(P, Q) = \mathbb{E}_{\boldsymbol{x},\boldsymbol{x}'\sim P}[k(\boldsymbol{x}, \boldsymbol{x}')] + \mathbb{E}_{\boldsymbol{y},\boldsymbol{y}'\sim Q}[k(\boldsymbol{y}, \boldsymbol{y}')] - 2\,\mathbb{E}_{\boldsymbol{x}\sim P,\,\boldsymbol{y}\sim Q}[k(\boldsymbol{x}, \boldsymbol{y})]. \tag{110}$$

The MMD makes no distributional assumption, and its unbiased estimator does not depend on the sample size in the way the FID estimator does. It is still a distance between feature distributions, so it inherits whatever the feature extractor fails to see.

Prompt-level benchmarks and arenas. Compositional benchmarks such as GenEval [104] check whether the prompt was followed, using detectors or vision-language models as automatic judges (Section 7.1). Their scores saturate as models improve, and at that point they mostly measure the errors of the judge. Human arenas [145] avoid fixed judges and rank models with the Bradley-Terry model of Eq. (98), but they reflect the prompts and preferences of the people who vote, and they cannot be used as a training signal without being contaminated by it. For video and world models (Sections 7.2 and 7.3) the gap is larger. A short clip can look plausible frame by frame and still violate object permanence or physics, and an interactive model can drift only after many steps of a rollout, as the bound Eq. (101) suggests. Benchmarks that test consistency over long horizons and under interventions are still scarce.

The judge is also the target. The same preference models serve as rewards in Section 4, as verifiers in Section 6 and as evaluation metrics [234, 369, 389, 396]. A model fine-tuned or searched against a reward and then evaluated with the same reward is graded by its own objective, and an improvement in the score is then evidence of optimization, not of quality. At a minimum, papers should report results on judges that were not used for training or search, together with a human study. More generally, evaluation needs held-out judges in the same sense in which learning needs held-out data.

8.3 Reward over-optimization

Why the proxy keeps improving while the target does not. Section 4.6 described reward hacking empirically. A short calculation shows why it is unavoidable without additional

structure. Write $\lambda = 1/\beta$ for the inverse regularization strength in the tilted target Eq. (60), $p_\lambda(\boldsymbol{x}) \propto p(\boldsymbol{x})\, e^{\lambda r(\boldsymbol{x})}$. For any function f,

$$\frac{\mathrm{d}}{\mathrm{d}\lambda}\mathbb{E}_{p_\lambda}[f] = \mathbb{E}_{p_\lambda}[f\, r] - \mathbb{E}_{p_\lambda}[f]\,\mathbb{E}_{p_\lambda}[r] = \mathrm{Cov}_{p_\lambda}(f, r), \tag{111}$$

which follows by differentiating the normalizing constant. With $f = r$ the derivative is $\mathrm{Var}_{p_\lambda}(r) \geq 0$, so the proxy reward increases monotonically as the regularization is weakened. With $f = r^\star$, the true but unobserved quality, the derivative is $\mathrm{Cov}_{p_\lambda}(r^\star, r)$. Under the base model the two rewards are positively correlated, which is why the proxy was chosen. But the tilt concentrates p_λ on the region where r is largest, and in that region the residual $r - r^\star$ is typically large, since the largest values of the proxy are selected precisely where its error is positive. Once the covariance under p_λ becomes negative, further optimization lowers the true quality while raising the measured reward. The KL term in Eq. (59) only controls how far p_λ moves from the base model. It does not know in which directions the reward is trustworthy.

What helps. The empirical remedies of Section 4.6 can be read through Eq. (111). Ensembles and uncertainty penalties shrink the reward where models disagree, which reduces the covariance between the proxy error and r [443]. Regularizers that act on the update rather than on the distribution, such as the importance-ratio corrections in GRPO-Guard [363], prevent the optimizer from moving fastest in the directions where the gradient estimate is least reliable. Adaptive regularization towards a moving reference with explicit diversity terms [117] keeps p_λ from collapsing onto a few modes. Hong et al. [125] study how reward hacking arises in reinforcement learning for text-to-image models. A different strategy avoids fine-tuning altogether. DOIT [467] realizes the Doob h-transform Eq. (39) as a correction to the drift that is computed by simulation at inference time. It requires no training and no differentiable reward, and it comes with a high-probability convergence guarantee to the high-reward target. Its experiments are on offline reinforcement learning benchmarks, so its behaviour on large image models remains to be tested. Because the base model is unchanged, the strength of the adaptation can be chosen per query and undone at no cost, which makes over-optimization easier to detect and to control. The open problem is a principled way to specify where a reward can be trusted, for example as a region of the data manifold, so that the tilt is applied only there.

8.4 Data provenance and memorization

Memorization is the optimum of the training objective. Section 2.7 showed that the minimizer of the denoising loss on a finite training set is the empirical denoiser Eq. (40), whose sampler reproduces the training images exactly. Generalization is therefore a property of the inductive biases and of the optimization, not of the objective. Extraction attacks find training images in the outputs of large text-to-image models [40, 317], and controlled studies relate memorization to dataset size, duplication and conditioning [107]. Several recent analyses explain why trained networks usually generalize. Denoisers trained on enough data adapt to a geometric basis of the images and converge to the same function for disjoint training sets [149]. Locality of the denoiser explains part of the gap between the empirical and the learned denoiser [245]. For a two-layer denoising autoencoder, Zhang et al. [442] prove that memorization corresponds to storing individual training samples in the weights, which produces localized, spiky hidden representations, whereas generalization corresponds to capturing local data statistics, which produces balanced representations. This suggests a practical diagnostic, since the representations of a trained network can be inspected without access to an attacker. Related work studies when memorization arises during training and

how it depends on the model scale [158, 169]. What is missing is a guarantee: a training procedure with a certified bound on how close a generated sample can come to any training sample, at a cost in quality that is acceptable for large models.

Watermarking as hypothesis testing. The opposite question is whether a given image was produced by a given model. Watermarks answer it with a statistical test. The Stable Signature [92] fine-tunes the latent decoder so that every generated image carries a fixed binary signature $m \in \{0,1\}^k$, which a pretrained extractor recovers. If the extractor returns k bits that agree with m in s positions, then under the null hypothesis that the image is not watermarked the matches are independent fair coins, and the false positive rate of the threshold $s \geq \tau$ is

$$\Pr(S \geq \tau) = \sum_{j=\tau}^{k} \binom{k}{j} 2^{-k}, \qquad S \sim \text{Binomial}(k, \tfrac{1}{2}). \tag{112}$$

For $k = 48$ bits a threshold of $\tau = 41$ gives a false positive rate below 10^{-6}. Tree-Ring watermarks [378] instead place a pattern in the Fourier transform of the initial noise $\boldsymbol{x}_0$ and detect it by inverting the deterministic sampler from the image back to the noise. Neither method requires changing the training data, and Tree-Ring does not change the model at all.

Interactions with the rest of the lifecycle. Watermarks interact with the methods of this survey in ways that are rarely tested. Tree-Ring relies on the sampler being invertible and on the initial noise being chosen by the provider. Noise search (Section 6.2) replaces the initial noise by an optimized one, and distillation (Section 5) replaces the sampler by a few-step generator that is not easily inverted. A decoder watermark survives both but is removed if the decoder is fine-tuned again or replaced. A provenance scheme for open-weight models has to survive the adaptation methods of Section 3, which can be applied by anyone. For generated training data, such as the synthetic editing triplets of Section 7.1, the question also applies within a single organization: if the outputs of one generation of models become the training data of the next, it should be possible to tell which images were generated and by which model.

8.5 Compute-optimal trade-offs across the lifecycle

Pretraining scaling laws. For autoregressive language models, scaling laws relate the loss to the model size, the data and the compute, and they determine how to allocate a fixed budget between parameters and tokens. For diffusion transformers, Liang et al. [197] fit a power law of the pretraining loss in compute over the range from 10^{17} to 6×10^{18} FLOPs and use it to predict the compute-optimal model size and the loss of larger models. Such laws are a starting point, but they cover only one stage, and the loss is not the quantity of interest. A model with a slightly worse loss can be a better starting point for post-training, distillation or search.

A budget across stages. Consider a model that will produce M samples over its deployment. Let C_{pre} and C_{post} be the pretraining and post-training compute, D the one-time cost of distillation, and c the compute per sample, which is the number of function evaluations K times the cost f of one evaluation times the number N of candidates drawn per output, for example in best-of-N. The total cost is

$$C_{\text{total}} = C_{\text{pre}} + C_{\text{post}} + D + M\,N\,K\,f. \tag{113}$$

Two elementary consequences follow. First, distillation from K to k steps pays for itself when $MN(K-k)f > D$, so it is almost always worthwhile for a widely deployed model and rarely

worthwhile for a research prototype. Second, the value of test-time search depends on how its gain scales with N. For best-of-N the expected reward gain grows like $s\sqrt{2\log N}$ for a reward with standard deviation s (Section 6.2), so each doubling of N buys less, and the marginal gain per unit of compute decays like $1/(N\sqrt{\log N})$. Fine-tuning towards the same tilted target has a fixed cost and no per-sample cost. Best-of-N and the tilted target achieve almost the same reward per unit of KL divergence, as shown after Eq. (91). Hence for a fixed target and a large M, post-training is the cheaper way to reach it, while search is the better choice when the reward changes per query, when it is not available at training time, or when the base model must stay unchanged.

What is not known. This accounting ignores interactions between the stages, which is where the open questions lie. Distillation changes the noise-to-sample map and can make search less effective, because a one-step generator has fewer intermediate states to steer. Post-training reduces the diversity on which best-of-N relies. A more capable verifier increases the value of search but also the risk of reward hacking at inference time. Ma et al. [232] showed that inference compute can substitute for model size in some regimes, and Prabhudesai et al. [277] showed that the best factorization depends on the ratio of data to compute. A joint scaling law for $(C_{\text{pre}}, C_{\text{post}}, D, N, K)$, measured on a quality metric that people agree with, does not exist yet and would be one of the most useful results for practitioners.

8.6 Beyond iterative sampling

Learning the map instead of the field. Most of the models in this survey learn an instantaneous quantity, a score or a velocity, and generate by integrating it. Section 5.4 described models that learn the finite-time map between two noise levels directly [93, 102]. Transition models [374] derive an exact continuous-time equation for the state transition over any finite interval and train a single network that can be sampled with any number of steps, from one to many, with quality that improves as steps are added. This removes the distinction between a base model and its distilled student, and it makes the number of steps a choice at inference time rather than at training time.

Moving the iteration into training. Drifting models [74] go one step further. They define a drifting field that moves generated samples and reaches equilibrium when the generated and data distributions match, and they use it as a training objective, so that the optimizer evolves the pushforward distribution of a one-step generator during training. Iterative refinement then happens over training steps instead of sampling steps, and inference takes a single network evaluation by design. It is not yet clear how such models interact with the rest of the lifecycle. Post-training methods of Section 4 that rely on the stochastic sampling path, inference-time methods of Section 6 that rely on intermediate states, and guidance methods of Section 2.11 that combine conditional and unconditional fields all assume an iterative sampler. If one-step models trained from scratch become competitive, these methods will need versions that act on the single map, as noise search already does.

Closing remarks. Diffusion and flow models began as a way to train generative models with a simple regression loss. Over a few years they acquired an entire lifecycle of adaptation, alignment, compression and search, and the same mathematical objects recur throughout: the tilted distribution, the Doob h-transform, the probability flow and Tweedie's formula. We expect the next advances to come less from new objectives for a single stage and more from understanding how the stages interact, and from evaluation that tells us reliably whether a change has improved the model for the people who use it.

A Derivations for the fundamentals

This appendix proves the identities used in Section 2. We assume throughout that densities are smooth and positive and that all boundary terms in integrations by parts vanish, which holds for example when the densities and the fields involved decay faster than any polynomial. Notation follows Section 2: $\boldsymbol{x}_t = \alpha_t \boldsymbol{x}_1 + \sigma_t \boldsymbol{x}_0$, with $\boldsymbol{x}_0 \sim \mathcal{N}(0, I)$ independent of $\boldsymbol{x}_1 \sim p_{\text{data}}$.

A.1 Implicit score matching

Expanding the square in Eq. (7),

$$J_{\text{ESM}}(\theta) = \tfrac{1}{2}\mathbb{E}_p\|\boldsymbol{s}_\theta\|^2 - \mathbb{E}_p[\boldsymbol{s}_\theta \cdot \nabla \log p] + \tfrac{1}{2}\mathbb{E}_p\|\nabla \log p\|^2.$$

The last term does not depend on θ. For the middle term, $p\,\nabla \log p = \nabla p$, so by the divergence theorem

$$\mathbb{E}_p[\boldsymbol{s}_\theta \cdot \nabla \log p] = \int \boldsymbol{s}_\theta \cdot \nabla p \,\mathrm{d}\boldsymbol{x} = -\int p\, \nabla \cdot \boldsymbol{s}_\theta \,\mathrm{d}\boldsymbol{x} = -\mathbb{E}_p[\nabla \cdot \boldsymbol{s}_\theta],$$

where the boundary term $\oint p\, \boldsymbol{s}_\theta \cdot \boldsymbol{n}$ vanishes by assumption. Substituting gives Eq. (8).

A.2 Regression onto a noisy target and denoising score matching

Proof of Lemma 2.1. Write $m(X) = \mathbb{E}[Y \mid X]$. Then $\mathbb{E}\|f(X) - Y\|^2 = \mathbb{E}\|f(X) - m(X)\|^2 + \mathbb{E}\|m(X) - Y\|^2 + 2\,\mathbb{E}[(f(X) - m(X)) \cdot (m(X) - Y)]$. Conditioning the last expectation on X and using $\mathbb{E}[m(X) - Y \mid X] = 0$ shows that it vanishes.

Marginal score as a conditional expectation. Differentiating $p_t(\boldsymbol{x}) = \int p_t(\boldsymbol{x} \mid \boldsymbol{x}_1) p_{\text{data}}(\boldsymbol{x}_1)\,\mathrm{d}\boldsymbol{x}_1$ under the integral and dividing by $p_t(\boldsymbol{x})$,

$$\nabla \log p_t(\boldsymbol{x}) = \int \frac{p_t(\boldsymbol{x} \mid \boldsymbol{x}_1) p_{\text{data}}(\boldsymbol{x}_1)}{p_t(\boldsymbol{x})} \nabla \log p_t(\boldsymbol{x} \mid \boldsymbol{x}_1)\,\mathrm{d}\boldsymbol{x}_1 = \mathbb{E}\big[\nabla \log p_t(\boldsymbol{x}_t \mid \boldsymbol{x}_1) \,\big|\, \boldsymbol{x}_t = \boldsymbol{x}\big],$$

because the fraction is the posterior density of $\boldsymbol{x}_1$ given $\boldsymbol{x}_t = \boldsymbol{x}$. With Eq. (9), this is Eq. (11). Applying Lemma 2.1 with $X = \boldsymbol{x}_t$ and $Y = \nabla \log p_t(\boldsymbol{x}_t \mid \boldsymbol{x}_1)$ shows that J_{DSM} and the explicit objective $\tfrac{1}{2}\mathbb{E}\|\boldsymbol{s}_\theta(\boldsymbol{x}_t, t) - \nabla \log p_t(\boldsymbol{x}_t)\|^2$ differ by the constant $\tfrac{1}{2}\mathbb{E}\|Y - \mathbb{E}[Y \mid X]\|^2$.

Tweedie's formula and the target weights. Taking the conditional expectation of Eq. (1) given $\boldsymbol{x}_t = \boldsymbol{x}$ yields $\boldsymbol{x} = \alpha_t \hat{\boldsymbol{x}}_1 + \sigma_t \hat{\boldsymbol{x}}_0$. Substituting $\hat{\boldsymbol{x}}_0 = -\sigma_t \nabla \log p_t(\boldsymbol{x})$ from Eq. (11) gives Eq. (12). The same identity holds for any pair of estimates that is consistent with the input, $\boldsymbol{x} = \alpha_t \hat{\boldsymbol{x}}_1 + \sigma_t \hat{\boldsymbol{x}}_0$, while the labels satisfy $\boldsymbol{x} = \alpha_t \boldsymbol{x}_1 + \sigma_t \boldsymbol{x}_0$. Subtracting gives $\alpha_t(\hat{\boldsymbol{x}}_1 - \boldsymbol{x}_1) = -\sigma_t(\hat{\boldsymbol{x}}_0 - \boldsymbol{x}_0)$, and therefore

$$\hat{\boldsymbol{u}}_t - (\dot{\alpha}_t \boldsymbol{x}_1 + \dot{\sigma}_t \boldsymbol{x}_0) = \dot{\alpha}_t(\hat{\boldsymbol{x}}_1 - \boldsymbol{x}_1) + \dot{\sigma}_t(\hat{\boldsymbol{x}}_0 - \boldsymbol{x}_0) = \Big(\dot{\alpha}_t - \dot{\sigma}_t \frac{\alpha_t}{\sigma_t}\Big)(\hat{\boldsymbol{x}}_1 - \boldsymbol{x}_1),$$

which gives Eq. (14).

A.3 The Fokker–Planck equation

Let $\mathrm{d}\boldsymbol{y} = \boldsymbol{b}(\boldsymbol{y}, \tau)\,\mathrm{d}\tau + g_\tau\,\mathrm{d}\boldsymbol{W}$ and let φ be smooth with compact support. Itô's formula gives

$$\mathrm{d}\varphi(\boldsymbol{y}_\tau) = \Big(\boldsymbol{b} \cdot \nabla\varphi + \tfrac{1}{2} g_\tau^2\, \Delta\varphi\Big)(\boldsymbol{y}_\tau)\,\mathrm{d}\tau + g_\tau \nabla\varphi(\boldsymbol{y}_\tau) \cdot \mathrm{d}\boldsymbol{W}_\tau.$$

The stochastic integral has mean zero, so

$$\frac{\mathrm{d}}{\mathrm{d}\tau}\int \varphi\, \rho_\tau \,\mathrm{d}\boldsymbol{y} = \int \Big(\boldsymbol{b} \cdot \nabla\varphi + \tfrac{1}{2} g_\tau^2 \Delta\varphi\Big)\rho_\tau \,\mathrm{d}\boldsymbol{y} = \int \varphi\Big(-\nabla \cdot (\boldsymbol{b}\rho_\tau) + \tfrac{1}{2} g_\tau^2 \Delta\rho_\tau\Big)\mathrm{d}\boldsymbol{y},$$

where we integrated by parts once in the first term and twice in the second. Since φ is arbitrary, Eq. (17) follows.

Coefficients of the linear SDE. For Eq. (15), the conditional mean $m_\tau = \mathbb{E}[\boldsymbol{y}_\tau \mid \boldsymbol{y}_0]$ satisfies $\dot m = fm$, and the conditional variance per coordinate v_τ satisfies $\dot v = 2fv + g^2$, again by Itô's formula. With $m_\tau = \alpha_{1-\tau}\boldsymbol{y}_0$ and $v_\tau = \sigma_{1-\tau}^2$, and $\frac{\mathrm{d}}{\mathrm{d}\tau} = -\frac{\mathrm{d}}{\mathrm{d}t}$, the first equation gives $f = -\dot\alpha_t/\alpha_t$ and the second gives $g^2 = -2\sigma_t\dot\sigma_t + 2(\dot\alpha_t/\alpha_t)\sigma_t^2$. Since $\dot\lambda_t = 2(\dot\alpha_t/\alpha_t - \dot\sigma_t/\sigma_t)$, this is Eq. (16).

A.4 The probability-flow ODE and the reverse-time SDE

Using $\nabla\rho = \rho\nabla\log\rho$, the diffusion term of Eq. (17) is $\frac12 g^2\Delta\rho = \nabla\cdot(\frac12 g^2\rho\nabla\log\rho)$, which gives Eq. (18). A continuity equation with velocity $\boldsymbol{v}_\tau$ is the Fokker–Planck equation of the ODE $\dot{\boldsymbol{y}} = \boldsymbol{v}_\tau(\boldsymbol{y})$ (the case $g = 0$ of Eq. (17)), which proves the claim about the probability-flow ODE.

For the reversal, set $p_t = \rho_{1-t}$, so that $\partial_t p_t = -\partial_\tau\rho_\tau$ and

$$\partial_t p_t = \nabla\cdot\big((\boldsymbol{b} - \tfrac12 g^2\nabla\log p_t)p_t\big) = -\nabla\cdot\big((-\boldsymbol{b} + g^2\nabla\log p_t)p_t\big) + \tfrac12 g^2\Delta p_t,$$

where the second form adds $-\nabla\cdot(\frac12 g^2 p_t\nabla\log p_t) + \frac12 g^2\Delta p_t = 0$. The right-hand side is the Fokker–Planck operator of Eq. (19), so that SDE, started from p_0, has marginals p_t. The pathwise statement, that the reversed process is a weak solution of Eq. (19), is the theorem of Anderson [9].

Our convention. With $\boldsymbol{b} = f\boldsymbol{y}$, $f = -\dot\alpha_t/\alpha_t$ and $g^2 = \sigma_t^2\dot\lambda_t$, the reverse SDE has drift $-\boldsymbol{b} + g^2\nabla\log p_t = (\dot\alpha_t/\alpha_t)\boldsymbol{x} + \sigma_t^2\dot\lambda_t\nabla\log p_t$, which is Eq. (21). The probability-flow ODE in τ has velocity $\boldsymbol{b} - \frac12 g^2\nabla\log\rho$, and reversing time flips its sign, which gives Eq. (20).

A.5 The marginal velocity

Let φ be smooth with compact support. Since $\boldsymbol{x}_t$ is differentiable in t for each fixed pair $(\boldsymbol{x}_0, \boldsymbol{x}_1)$, with derivative $\dot{\boldsymbol{x}}_t = \dot\alpha_t\boldsymbol{x}_1 + \dot\sigma_t\boldsymbol{x}_0$, dominated convergence gives

$$\frac{\mathrm{d}}{\mathrm{d}t}\mathbb{E}\varphi(\boldsymbol{x}_t) = \mathbb{E}\big[\nabla\varphi(\boldsymbol{x}_t)\cdot\dot{\boldsymbol{x}}_t\big] = \mathbb{E}\big[\nabla\varphi(\boldsymbol{x}_t)\cdot\mathbb{E}[\dot{\boldsymbol{x}}_t \mid \boldsymbol{x}_t]\big] = \int \nabla\varphi\cdot\boldsymbol{u}_t\, p_t\,\mathrm{d}\boldsymbol{x},$$

by the tower property and the definition Eq. (23). Integrating by parts, $\frac{\mathrm{d}}{\mathrm{d}t}\int\varphi\, p_t = -\int\varphi\,\nabla\cdot(p_t\boldsymbol{u}_t)$ for all φ, which is the weak form of Eq. (22). The same argument applies to any coupling of $\boldsymbol{x}_0$ and $\boldsymbol{x}_1$ and to any differentiable interpolation, which is the setting of stochastic interpolants [5]. Uniqueness of the ODE solution, and hence the statement that integrating $\boldsymbol{u}_t$ from p_0 reproduces p_t, requires $\boldsymbol{u}_t$ to be locally Lipschitz, which holds for Gaussian paths at all $t < 1$.

A.6 The velocity–score identity

Taking conditional expectations of $\dot{\boldsymbol{x}}_t = \dot\alpha_t\boldsymbol{x}_1 + \dot\sigma_t\boldsymbol{x}_0$ gives $\boldsymbol{u}_t = \dot\alpha_t\hat{\boldsymbol{x}}_1 + \dot\sigma_t\hat{\boldsymbol{x}}_0$. Substituting $\hat{\boldsymbol{x}}_0 = -\sigma_t\nabla\log p_t$ and Eq. (12),

$$\boldsymbol{u}_t(\boldsymbol{x}) = \frac{\dot\alpha_t}{\alpha_t}\big(\boldsymbol{x} + \sigma_t^2\nabla\log p_t(\boldsymbol{x})\big) - \dot\sigma_t\sigma_t\nabla\log p_t(\boldsymbol{x}) = \frac{\dot\alpha_t}{\alpha_t}\boldsymbol{x} + \sigma_t^2\Big(\frac{\dot\alpha_t}{\alpha_t} - \frac{\dot\sigma_t}{\sigma_t}\Big)\nabla\log p_t(\boldsymbol{x}),$$

and the bracket equals $\frac12\dot\lambda_t$. This is Eq. (25). Comparing with Eq. (20) shows that the marginal velocity of flow matching and the probability-flow velocity coincide. Proposition 2.3 follows from Eq. (17) as in the sketch in the main text.

A.7 The ELBO as a weighted denoising loss

The bound. Write $\boldsymbol{x}_{0:N-1}$ for the latents $\boldsymbol{x}_{t_0}, \dots, \boldsymbol{x}_{t_{N-1}}$ and $q(\boldsymbol{x}_{0:N-1} \mid \boldsymbol{x}_1)$ for the joint law of the corruption process. For any such q, Jensen's inequality gives $\log p_\theta(\boldsymbol{x}_1) \geq \mathbb{E}_q[\log p_\theta(\boldsymbol{x}_1, \boldsymbol{x}_{0:N-1}) - \log q(\boldsymbol{x}_{0:N-1} \mid \boldsymbol{x}_1)]$. The corruption process is Markov from data to noise, so by Bayes' rule its joint law can be written in the generative direction as $q(\boldsymbol{x}_{t_0} \mid \boldsymbol{x}_1) \prod_{i=1}^{N-1} q(\boldsymbol{x}_{t_i} \mid \boldsymbol{x}_{t_{i-1}}, \boldsymbol{x}_1)$. The generative model factorizes in the same order, $p(\boldsymbol{x}_{t_0}) \prod_{i=1}^{N-1} p_\theta(\boldsymbol{x}_{t_i} \mid \boldsymbol{x}_{t_{i-1}})\, p_\theta(\boldsymbol{x}_1 \mid \boldsymbol{x}_{t_{N-1}})$. Matching the factors term by term gives Eq. (27).

The posterior. Let $s < t$, so that $\boldsymbol{x}_s$ is the noisier state. The corruption step from t to s is $\boldsymbol{x}_s = \alpha_{s|t}\boldsymbol{x}_t + \sigma_{s|t}\boldsymbol{\epsilon}$ with $\alpha_{s|t} = \alpha_s/\alpha_t$ and $\sigma^2_{s|t} = \sigma_s^2 - \alpha^2_{s|t}\sigma_t^2$, which is non-negative because $\lambda_s \leq \lambda_t$. Bayes' rule combines this Gaussian likelihood with the prior $q(\boldsymbol{x}_t \mid \boldsymbol{x}_1) = \mathcal{N}(\alpha_t \boldsymbol{x}_1, \sigma_t^2 I)$. The precisions add to $\alpha^2_{s|t}/\sigma^2_{s|t} + 1/\sigma_t^2 = \sigma_s^2/(\sigma^2_{s|t}\sigma_t^2)$, so

$$q(\boldsymbol{x}_t \mid \boldsymbol{x}_s, \boldsymbol{x}_1) = \mathcal{N}(\boldsymbol{\mu}(\boldsymbol{x}_s, \boldsymbol{x}_1), \varsigma^2 I), \qquad \varsigma^2 = \frac{\sigma^2_{s|t}\sigma_t^2}{\sigma_s^2}, \qquad \boldsymbol{\mu} = \frac{\alpha_{s|t}\sigma_t^2}{\sigma_s^2}\,\boldsymbol{x}_s + \frac{\alpha_t \sigma^2_{s|t}}{\sigma_s^2}\,\boldsymbol{x}_1.$$

One step of the bound. Let the decoder be $p_\theta(\boldsymbol{x}_t \mid \boldsymbol{x}_s) = \mathcal{N}(\boldsymbol{\mu}(\boldsymbol{x}_s, \hat{\boldsymbol{x}}_1(\boldsymbol{x}_s, s)), \varsigma^2 I)$. Two Gaussians with equal covariance have $\mathrm{KL} = \|\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2\|^2/(2\varsigma^2)$, and only the $\boldsymbol{x}_1$ coefficient of $\boldsymbol{\mu}$ contributes:

$$\mathrm{KL} = \frac{1}{2\varsigma^2}\,\frac{\alpha_t^2 \sigma^4_{s|t}}{\sigma_s^4}\,\|\boldsymbol{x}_1 - \hat{\boldsymbol{x}}_1\|^2 = \frac{\alpha_t^2 \sigma^2_{s|t}}{2\sigma_s^2\sigma_t^2}\,\|\boldsymbol{x}_1 - \hat{\boldsymbol{x}}_1\|^2.$$

Substituting $\sigma^2_{s|t}$ gives $\alpha_t^2\sigma^2_{s|t}/(\sigma_s^2\sigma_t^2) = \alpha_t^2/\sigma_t^2 - \alpha_s^2/\sigma_s^2 = \mathrm{SNR}(t) - \mathrm{SNR}(s)$, which is Eq. (28).

Continuous time. As the mesh of the grid tends to zero,

$$\sum_i \tfrac{1}{2}\big(\mathrm{SNR}(t_i) - \mathrm{SNR}(t_{i-1})\big)\,\mathbb{E}\|\boldsymbol{x}_1 - \hat{\boldsymbol{x}}_1(\boldsymbol{x}_{t_{i-1}}, t_{i-1})\|^2 \;\longrightarrow\; \tfrac{1}{2}\int_0^1 \tfrac{\mathrm{d}}{\mathrm{d}t} e^{\lambda_t}\,\mathbb{E}\|\boldsymbol{x}_1 - \hat{\boldsymbol{x}}_1(\boldsymbol{x}_t, t)\|^2\,\mathrm{d}t.$$

With $\mathrm{d}e^\lambda = e^\lambda \mathrm{d}\lambda$ and $e^\lambda\|\boldsymbol{x}_1 - \hat{\boldsymbol{x}}_1\|^2 = \|\boldsymbol{x}_0 - \hat{\boldsymbol{x}}_0\|^2$ from Eq. (14), this is Eq. (29). The prior term tends to zero as $\alpha_{t_0} \to 0$ and $\sigma_{t_0} \to 1$, and the reconstruction term depends only on the least noisy level $\lambda_{\max}$.

Invariance to the schedule. Write $\boldsymbol{x}_t = \alpha_t(\boldsymbol{x}_1 + e^{-\lambda_t/2}\boldsymbol{x}_0)$ and parametrize the network through the rescaled input, $\hat{\boldsymbol{x}}_0(\boldsymbol{x}, t) = f_\theta(\boldsymbol{x}/\alpha_t, \lambda_t)$. The law of $\boldsymbol{x}_t/\alpha_t$ depends on t only through λ_t, so the integrand of Eq. (29) is a fixed function of λ for every schedule. Two schedules with the same $\lambda_{\min}$ and $\lambda_{\max}$ define the same objective.

Monotone weights are ELBOs of noisy data. Following Kingma and Gao [161], let $\ell(\lambda) = \frac{1}{2}\mathbb{E}\|\boldsymbol{x}_0 - \hat{\boldsymbol{x}}_0(\boldsymbol{x}_\lambda, \lambda)\|^2$. Treat the noisy sample at level λ' as the data. Its ELBO involves only the noisier levels, so up to terms that do not depend on θ, its negative is $\int_{\lambda_{\min}}^{\lambda'} \ell(\lambda)\,\mathrm{d}\lambda$. Averaging over λ' with a density p and exchanging the integrals gives

$$\int p(\lambda') \int_{\lambda_{\min}}^{\lambda'} \ell(\lambda)\,\mathrm{d}\lambda\,\mathrm{d}\lambda' = \int \ell(\lambda)\, w(\lambda)\,\mathrm{d}\lambda, \qquad w(\lambda) = \int_\lambda^{\lambda_{\max}} p(\lambda')\,\mathrm{d}\lambda'.$$

This w is decreasing in λ. Conversely, any decreasing w with $w(\lambda_{\max}) = 0$ arises in this way with $p = -w'$, and a positive value of $w(\lambda_{\max})$ adds the ELBO of the clean data with weight $w(\lambda_{\max})$.

A.8 Closed-form fields for Gaussian mixtures

Let $p_{\mathrm{data}} = \sum_k \pi_k \mathcal{N}(\boldsymbol{\mu}_k, \Sigma_k)$. Since $\boldsymbol{x}_t$ is an affine function of independent Gaussians given the component k,

$$p_t(\boldsymbol{x}) = \sum_k \pi_k\, \mathcal{N}(\boldsymbol{x};\, \alpha_t\boldsymbol{\mu}_k,\, C_{k,t}), \qquad C_{k,t} = \alpha_t^2\Sigma_k + \sigma_t^2 I.$$

Let $w_k(\boldsymbol{x}) = \pi_k \mathcal{N}(\boldsymbol{x}; \alpha_t \boldsymbol{\mu}_k, C_{k,t})/p_t(\boldsymbol{x})$ be the posterior probability of component k. Then

$$\nabla \log p_t(\boldsymbol{x}) = \sum_k w_k(\boldsymbol{x})\, C_{k,t}^{-1}(\alpha_t \boldsymbol{\mu}_k - \boldsymbol{x}), \qquad \hat{\boldsymbol{x}}_1(\boldsymbol{x}) = \sum_k w_k(\boldsymbol{x}) \Big(\boldsymbol{\mu}_k + \alpha_t \Sigma_k C_{k,t}^{-1}(\boldsymbol{x} - \alpha_t \boldsymbol{\mu}_k) \Big),$$

where the second expression is the Gaussian conditional mean within each component, averaged with the posterior weights. The velocity follows from Eq. (25) or from $\boldsymbol{u}_t = \dot{\alpha}_t \hat{\boldsymbol{x}}_1 + \dot{\sigma}_t(\boldsymbol{x} - \alpha_t \hat{\boldsymbol{x}}_1)/\sigma_t$. For isotropic components $\Sigma_k = s_k^2 I$ all matrix inverses are scalar divisions, and this isotropic case is what `web/animations/lib/gmm.js` implements for the animations. The empirical distribution of a training set $\{\boldsymbol{x}^{(i)}\}_{i=1}^n$ is the limit $s_k \to 0$ with $\boldsymbol{\mu}_k = \boldsymbol{x}^{(i)}$ and $\pi_k = 1/n$, which gives the closed-form optimal score used in Section 2.7.

References

[1] Josh Abramson, Jonas Adler, Jack Dunger, et al. Accurate structure prediction of biomolecular interactions with AlphaFold 3. *Nature*, 630(8016):493–500, 2024. doi: 10.1038/s41586-024-07487-w. URL https://doi.org/10.1038/s41586-024-07487-w.

[2] Armen Aghajanyan, Luke Zettlemoyer, and Sonal Gupta. Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning, 2020. URL https://arxiv.org/abs/2012.13255.

[3] Woody Ahern, Jason Yim, Doug Tischer, et al. Atom-level enzyme active site scaffolding using RFdiffusion2. *Nature Methods*, 23(1):96–105, 2025. doi: 10.1038/s41592-025-02975-x. URL https://doi.org/10.1038/s41592-025-02975-x.

[4] Michael S. Albergo and Eric Vanden-Eijnden. Building Normalizing Flows with Stochastic Interpolants, 2022. URL https://arxiv.org/abs/2209.15571.

[5] Michael S. Albergo, Nicholas M. Boffi, and Eric Vanden-Eijnden. Stochastic Interpolants: A Unifying Framework for Flows and Diffusions, 2023. URL https://arxiv.org/abs/2303.08797.

[6] Grigorii Alekseenko, Aleksandr Gordeev, Irina Tolstykh, et al. VIBE: Visual Instruction Based Editor, 2026. URL https://arxiv.org/abs/2601.02242.

[7] Ferran Alet, Ilan Price, Andrew El-Kadi, et al. Skillful joint probabilistic weather forecasting from marginals, 2025. URL https://arxiv.org/abs/2506.10772.

[8] Eloi Alonso, Adam Jelley, Vincent Micheli, et al. Diffusion for World Modeling: Visual Details Matter in Atari, 2024. URL https://arxiv.org/abs/2405.12399.

[9] Brian D. O. Anderson. Reverse-time diffusion equation models. *Stochastic Processes and their Applications*, 12(3):313–326, 1982. doi: 10.1016/0304-4149(82)90051-5.

[10] Jennifer Rosina Andersson and Zheng Zhao. Diffusion differentiable resampling, 2025. URL https://arxiv.org/abs/2512.10401.

[11] Vladimir Arkhipkin, Vladimir Korviakov, Nikolai Gerasimenko, et al. Kandinsky 5.0: A Family of Foundation Models for Image and Video Generation, 2025. URL https://arxiv.org/abs/2511.14993.

[12] Marianne Arriola, Aaron Gokaslan, Justin T. Chiu, et al. Block Diffusion: Interpolating Between Autoregressive and Diffusion Language Models, 2025. URL https://arxiv.org/abs/2503.09573.

[13] Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured Denoising Diffusion Models in Discrete State-Spaces, 2021. URL https://arxiv.org/abs/2107.03006.

[14] Daniyel Ayupov and Artur Markov-Tsoy. DreamForge-World 0.1 Preview: A Low-Compute Real-Time Controllable World Model, 2026. URL https://arxiv.org/abs/2606.30292.

[15] Jinbin Bai, Yixuan Li, Yuchen Zhu, et al. Prism: Efficient Test-Time Scaling via Hierarchical Search and Self-Verification for Discrete Diffusion Language Models, 2026. URL https://arxiv.org/abs/2602.01842.

[16] Arpit Bansal, Hong-Min Chu, Avi Schwarzschild, et al. Universal Guidance for Diffusion Models, 2023. URL https://arxiv.org/abs/2302.07121.

[17] Jean-David Benamou and Yann Brenier. A computational fluid mechanics solution to the Monge-Kantorovich mass transfer problem. *Numerische Mathematik*, 84(3):375–393, 2000. doi: 10.1007/s002110050002.

[18] Nathaniel R. Bennett, Joseph L. Watson, Robert J. Ragotte, et al. Atomically accurate de novo design of antibodies with RFdiffusion. *Nature*, 649(8095):183–193, 2025. doi: 10.1038/s41586-025-09721-5. URL https://doi.org/10.1038/s41586-025-09721-5.

[19] Joe Benton, Valentin De Bortoli, Arnaud Doucet, and George Deligiannidis. Nearly d-Linear Convergence Bounds for Diffusion Models via Stochastic Localization, 2023. URL https://arxiv.org/abs/2308.03686.

[20] Andreas Bergmeister, Stefanie Jegelka, Nikolas Nüsken, Carles Domingo-Enrich, and Jakiw Pidstrigach. Reinforce Adjoint Matching: Scaling RL Post-Training of Diffusion and Flow-Matching Models, 2026. URL https://arxiv.org/abs/2605.10759.

[21] Julius Berner, Lorenz Richter, and Karen Ullrich. An optimal control perspective on diffusion-based generative modeling, 2022. URL https://arxiv.org/abs/2211.01364.

[22] Yuxuan Bian, Xin Chen, Zenan Li, et al. Video-As-Prompt: Unified Semantic Control for Video Generation, 2025. URL https://arxiv.org/abs/2510.20888.

[23] Dan Biderman, Jacob Portes, Jose Javier Gonzalez Ortiz, et al. LoRA Learns Less and Forgets Less, 2024. URL https://arxiv.org/abs/2405.09673.

[24] Tiwei Bie, Maosong Cao, Kun Chen, et al. LLaDA2.0: Scaling Up Diffusion Language Models to 100B, 2025. URL https://arxiv.org/abs/2512.15745.

[25] Tiwei Bie, Maosong Cao, Xiang Cao, et al. LLaDA2.1: Speeding Up Text Diffusion via Token Editing, 2026. URL https://arxiv.org/abs/2602.08676.

[26] Giulio Biroli, Tony Bonnaire, Valentin de Bortoli, and Marc Mézard. Dynamical Regimes of Diffusion Models, 2024. URL https://arxiv.org/abs/2402.18491.

[27] Kevin Black, Michael Janner, Yilun Du, Ilya Kostrikov, and Sergey Levine. Training Diffusion Models with Reinforcement Learning, 2023. URL https://arxiv.org/abs/2305.13301.

[28] Kevin Black, Noah Brown, Danny Driess, et al. π_0: A Vision-Language-Action Flow Model for General Robot Control, 2024. URL https://arxiv.org/abs/2410.24164.

[29] Black Forest Labs, Stephen Batifol, Andreas Blattmann, et al. FLUX.1 Kontext: Flow Matching for In-Context Image Generation and Editing in Latent Space, 2025. URL https://arxiv.org/abs/2506.15742.

[30] Andreas Blattmann, Tim Dockhorn, Sumith Kulal, et al. Stable Video Diffusion: Scaling Latent Video Diffusion Models to Large Datasets, 2023. URL https://arxiv.org/abs/2311.15127.

[31] Nicholas M. Boffi, Michael S. Albergo, and Eric Vanden-Eijnden. Flow map matching with stochastic interpolants: A mathematical framework for consistency models, 2024. URL https://arxiv.org/abs/2406.07507.

[32] Nicholas M. Boffi, Michael S. Albergo, and Eric Vanden-Eijnden. How to build a consistency model: Learning flow maps via self-distillation, 2025. URL https://arxiv.org/abs/2505.18825.

[33] Valentin De Bortoli, James Thornton, Jeremy Heng, and Arnaud Doucet. Diffusion Schrödinger Bridge with Applications to Score-Based Generative Modeling, 2021. URL https://arxiv.org/abs/2106.01357.

[34] Arwen Bradley and Preetum Nakkiran. Classifier-Free Guidance is a Predictor-Corrector, 2024. URL https://arxiv.org/abs/2408.09000.

[35] Tim Brooks, Aleksander Holynski, and Alexei A. Efros. InstructPix2Pix: Learning to Follow Image Editing Instructions, 2022. URL https://arxiv.org/abs/2211.09800.

[36] Jake Bruce, Michael Dennis, Ashley Edwards, et al. Genie: Generative Interactive Environments, 2024. URL https://arxiv.org/abs/2402.15391.

[37] Jasper Butcher, Rohith Krishna, Raktim Mitra, et al. De novo Design of All-atom Biomolecular Interactions with RFdiffusion3. bioRxiv preprint, 2025. URL https://doi.org/10.1101/2025.09.18.676967.

[38] Andrew Campbell, Joe Benton, Valentin De Bortoli, et al. A Continuous Time Framework for Discrete Denoising Models, 2022. URL https://arxiv.org/abs/2205.14987.

[39] Hanqun Cao, Cheng Tan, Zhangyang Gao, et al. A Survey on Generative Diffusion Model, 2022. URL https://arxiv.org/abs/2209.02646.

[40] Nicholas Carlini, Jamie Hayes, Milad Nasr, et al. Extracting Training Data from Diffusion Models, 2023. URL https://arxiv.org/abs/2301.13188.

[41] Andy Catruna and Emilian Radoi. Induction in Both Directions: A Mechanistic Analysis of In-Context Learning in Masked Diffusion Language Models, 2026. URL https://arxiv.org/abs/2607.15893.

[42] Chai Discovery, Jacques Boitreaud, Jack Dent, et al. Chai-1: Decoding the molecular interactions of life. bioRxiv preprint, 2024. URL https://doi.org/10.1101/2024.10.10.615955.

[43] Chai Discovery Team, Jacques Boitreaud, Jack Dent, et al. Zero-shot antibody design in a 24-well plate. bioRxiv preprint, 2025. URL https://doi.org/10.1101/2025.07.05.663018.

[44] Lohithsai Yadala Chanchu, Hany Abdulsamad, and Christian A. Naesseth. Discrete Diffusion Inference-Time Control with Nested Sequential Monte Carlo, 2026. URL https://arxiv.org/abs/2608.20123.

[45] Huiwen Chang, Han Zhang, Lu Jiang, Ce Liu, and William T. Freeman. MaskGIT: Masked Generative Image Transformer, 2022. URL https://arxiv.org/abs/2202.04200.

[46] Boyuan Chen, Diego Marti Monso, Yilun Du, et al. Diffusion Forcing: Next-token Prediction Meets Full-Sequence Diffusion, 2024. URL https://arxiv.org/abs/2407.01392.

[47] Chubin Chen, Jiashu Zhu, Xiaokun Feng, et al. Stochastic Self-Guidance for Training-Free Enhancement of Diffusion Models, 2025. URL https://arxiv.org/abs/2508.12880.

[48] Hansheng Chen, Kai Zhang, Hao Tan, et al. pi-Flow: Policy-Based Few-Step Generation via Imitation Distillation, 2025. URL https://arxiv.org/abs/2510.14974.

[49] Huayu Chen, Kai Jiang, Kaiwen Zheng, et al. Visual Generation Without Guidance, 2025. URL https://arxiv.org/abs/2501.15420.

[50] Junsong Chen, Shuchen Xue, Yuyang Zhao, et al. SANA-Sprint: One-Step Diffusion with Continuous-Time Consistency Distillation, 2025. URL https://arxiv.org/abs/2503.09641.

[51] Junsong Chen, Yuyang Zhao, Jincheng Yu, et al. SANA-Video: Efficient Video Generation with Block Linear Diffusion Transformer, 2025. URL https://arxiv.org/abs/2509.24695.

[52] Junyu Chen, Han Cai, Junsong Chen, et al. Deep Compression Autoencoder for Efficient High-Resolution Diffusion Models, 2024. URL https://arxiv.org/abs/2410.10733.

[53] Kang Chen, Zhihao Liu, Tonghe Zhang, et al. π_{RL}: Online RL Fine-tuning for Flow-based Vision-Language-Action Models, 2025. URL https://arxiv.org/abs/2510.25889.

[54] Liyang Chen, Tianxiang Ma, Jiawei Liu, et al. HuMo: Human-Centric Video Generation via Collaborative Multi-Modal Conditioning, 2025. URL https://arxiv.org/abs/2509.08519.

[55] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural Ordinary Differential Equations, 2018. URL https://arxiv.org/abs/1806.07366.

[56] Sitan Chen, Sinho Chewi, Jerry Li, et al. Sampling is as easy as learning the score: theory for diffusion models with minimal data assumptions, 2022. URL https://arxiv.org/abs/2209.11215.

[57] Tianrong Chen, Guan-Horng Liu, and Evangelos A. Theodorou. Likelihood Training of Schrödinger Bridge using Forward-Backward SDEs Theory, 2021. URL https://arxiv.org/abs/2110.11291.

[58] Zhifei Chen, Luozhou Wang, Guibao Shen, et al. ReWorld: An Interactive World Model with Long-Horizon Memory, 2026. URL https://arxiv.org/abs/2608.23565.

[59] Zhenglin Cheng, Peng Sun, Jianguo Li, and Tao Lin. TwinFlow: Realizing One-step Generation on Large Models with Self-adversarial Flows, 2025. URL https://arxiv.org/abs/2512.05150.

[60] Cheng Chi, Zhenjia Xu, Siyuan Feng, et al. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion, 2023. URL https://arxiv.org/abs/2303.04137.

[61] Muthu Chidambaram, Khashayar Gatmiry, Sitan Chen, Holden Lee, and Jianfeng Lu. What does guidance do? A fine-grained analysis in a simple setting, 2024. URL https://arxiv.org/abs/2409.13074.

[62] Hyunmin Cho, Donghoon Ahn, Susung Hong, et al. TAG: Tangential Amplifying Guidance for Hallucination-Resistant Sampling, 2025. URL https://arxiv.org/abs/2510.04533.

[63] Jaemoo Choi, Wei Guo, Yuchen Zhu, et al. Scaling Reinforcement Learning for Diffusion Models via Velocity Matching, 2026. URL https://arxiv.org/abs/2608.23664.

[64] Ruihang Chu, Yefei He, Zhekai Chen, et al. Wan-Move: Motion-controllable Video Generation via Latent Trajectory Guidance, 2025. URL https://arxiv.org/abs/2512.08765.

[65] Hyungjin Chung and Jong Chul Ye. Score-based diffusion models for accelerated MRI, 2021. URL https://arxiv.org/abs/2110.05243.

[66] Hyungjin Chung, Jeongsol Kim, Michael T. Mccann, Marc L. Klasky, and Jong Chul Ye. Diffusion Posterior Sampling for General Noisy Inverse Problems, 2022. URL https://arxiv.org/abs/2209.14687. The Eleventh International Conference on Learning Representations (ICLR) 2023.

[67] Hyungjin Chung, Jeongsol Kim, Geon Yeong Park, Hyelin Nam, and Jong Chul Ye. CFG++: Manifold-constrained Classifier Free Guidance for Diffusion Models, 2024. URL https://arxiv.org/abs/2406.08070.

[68] Kevin Clark, Paul Vicol, Kevin Swersky, and David J Fleet. Directly Fine-Tuning Diffusion Models on Differentiable Rewards, 2023. URL https://arxiv.org/abs/2309.17400.

[69] Gabriele Corso, Hannes Stärk, Bowen Jing, Regina Barzilay, and Tommi Jaakkola. DiffDock: Diffusion Steps, Twists, and Turns for Molecular Docking, 2022. URL https://arxiv.org/abs/2210.01776.

[70] Florinel-Alin Croitoru, Vlad Hondru, Radu Tudor Ionescu, and Mubarak Shah. Diffusion Models in Vision: A Survey, 2022. URL https://arxiv.org/abs/2209.04747.

[71] Justin Cui, Jie Wu, Ming Li, et al. Self-Forcing++: Towards Minute-Scale High-Quality Video Generation, 2025. URL https://arxiv.org/abs/2510.02283.

[72] Yufeng Cui, Honghao Chen, Haoge Deng, et al. Emu3.5: Native Multimodal Models are World Learners, 2025. URL https://arxiv.org/abs/2510.26583.

[73] Haoyou Deng, Keyu Yan, Chaojie Mao, et al. DenseGRPO: From Sparse to Dense Reward for Flow Matching Model Alignment, 2026. URL https://arxiv.org/abs/2601.20218.

[74] Mingyang Deng, He Li, Tianhong Li, Yilun Du, and Kaiming He. Generative Modeling via Drifting, 2026. URL https://arxiv.org/abs/2602.04770.

[75] Prafulla Dhariwal and Alex Nichol. Diffusion Models Beat GANs on Image Synthesis, 2021. URL https://arxiv.org/abs/2105.05233.

[76] Kieran Didi, Zuobai Zhang, Guoqing Zhou, et al. Scaling Atomistic Protein Binder Design with Generative Pretraining and Test-Time Compute, 2026. URL https://arxiv.org/abs/2603.27950.

[77] DiffusionGemma Team, Adrien Ali Taïga, James Assiene, et al. DiffusionGemma Technical Report, 2026. URL https://arxiv.org/abs/2608.00146.

[78] Carles Domingo-Enrich and Jiequn Han. Adjoint Matching through the Lens of the Stochastic Maximum Principle in Optimal Control, 2026. URL https://arxiv.org/abs/2604.08580.

[79] Carles Domingo-Enrich, Michal Drozdzal, Brian Karrer, and Ricky T. Q. Chen. Adjoint Matching: Fine-tuning Flow and Diffusion Generative Models with Memoryless Stochastic Optimal Control, 2024. URL https://arxiv.org/abs/2409.08861.

[80] Carles Domingo-Enrich, Yuanqi Du, and Michael S. Albergo. A unified perspective on fine-tuning and sampling with diffusion and flow models, 2026. URL https://arxiv.org/abs/2605.00229.

[81] Ian Dunn and David Ryan Koes. Mixed Continuous and Categorical Flow Matching for 3D De Novo Molecule Generation, 2024. URL https://arxiv.org/abs/2404.19739.

[82] Bradley Efron. Tweedie's Formula and Selection Bias. *Journal of the American Statistical Association*, 106(496):1602–1614, 2011. doi: 10.1198/jasa.2011.tm11181.

[83] Thomas Egg, Harry Winston Sullivan, Ellad B. Tadmor, and Stefano Martiniani. OMatG-flash: An All-Atom Flow Map with Reinforce Adjoint Matching for Scalable Materials Discovery, 2026. URL https://arxiv.org/abs/2609.26402.

[84] Patrick Esser, Sumith Kulal, Andreas Blattmann, et al. Scaling Rectified Flow Transformers for High-Resolution Image Synthesis, 2024. URL https://arxiv.org/abs/2403.03206.

[85] Zach Evans, Julian D. Parker, CJ Carr, et al. Stable Audio Open, 2024. URL https://arxiv.org/abs/2407.14358.

[86] Luca Eyring, Shyamgopal Karthik, Alexey Dosovitskiy, Nataniel Ruiz, and Zeynep Akata. Noise Hypernetworks: Amortizing Test-Time Compute in Diffusion Models, 2025. URL https://arxiv.org/abs/2508.09968.

[87] Jiawei Fan, Sifeng Wang, Yuqing Hou, and Anbang Yao. Kinematic MeanFlow: One-Step Action Generation Policy for Robotic Foundation Models, 2026. URL https://arxiv.org/abs/2610.00864.

[88] Xiangyu Fan, Zesong Qiu, Zhuguanyu Wu, et al. Phased DMD: Few-step Distribution Matching Distillation via Score Matching within Subintervals, 2025. URL https://arxiv.org/abs/2510.27684.

[89] Ying Fan, Olivia Watkins, Yuqing Du, et al. DPOK: Reinforcement Learning for Fine-tuning Text-to-Image Diffusion Models, 2023. URL https://arxiv.org/abs/2305.16381.

[90] Rongyao Fang, Chengqi Duan, Kun Wang, et al. GoT: Unleashing Reasoning Capability of Multimodal Large Language Model for Visual Generation and Editing, 2025. URL https://arxiv.org/abs/2503.10639.

[91] Zhen Fang, Wenxuan Huang, Yu Zeng, et al. Flow-OPD: On-Policy Distillation for Flow Matching Models, 2026. URL https://arxiv.org/abs/2605.08063.

[92] Pierre Fernandez, Guillaume Couairon, Hervé Jégou, Matthijs Douze, and Teddy Furon. The Stable Signature: Rooting Watermarks in Latent Diffusion Models, 2023. URL https://arxiv.org/abs/2303.15435.

[93] Kevin Frans, Danijar Hafner, Sergey Levine, and Pieter Abbeel. One Step Diffusion via Shortcut Models, 2024. URL https://arxiv.org/abs/2410.12557.

[94] Rinon Gal, Yuval Alaluf, Yuval Atzmon, et al. An Image is Worth One Word: Personalizing Text-to-Image Generation using Textual Inversion, 2022. URL https://arxiv.org/abs/2208.01618.

[95] Xin Gao, Li Hu, Siqi Hu, et al. Wan-S2V: Audio-Driven Cinematic Video Generation, 2025. URL https://arxiv.org/abs/2508.18621.

[96] Yu Gao, Lixue Gong, Qiushan Guo, et al. Seedream 3.0 Technical Report, 2025. URL https://arxiv.org/abs/2504.11346.

[97] Yu Gao, Haoyuan Guo, Tuyen Hoang, et al. Seedance 1.0: Exploring the Boundaries of Video Generation Models, 2025. URL https://arxiv.org/abs/2506.09113.

[98] Itai Gat, Tal Remez, Neta Shaul, et al. Discrete Flow Matching, 2024. URL https://arxiv.org/abs/2407.15595.

[99] Tomas Geffner, Kieran Didi, Zhonglin Cao, et al. La-Proteina: Atomistic Protein Generation via Partially Latent Flow Matching, 2025. URL https://arxiv.org/abs/2507.09466.

[100] Tomas Geffner, Kieran Didi, Zuobai Zhang, et al. Proteina: Scaling Flow-based Protein Structure Generative Models, 2025. URL https://arxiv.org/abs/2503.00710.

[101] Gemini Robotics Team, Krzysztof Choromanski, Coline Devin, et al. Evaluating Gemini Robotics Policies in a Veo World Simulator, 2025. URL https://arxiv.org/abs/2512.10675.

[102] Zhengyang Geng, Mingyang Deng, Xingjian Bai, J. Zico Kolter, and Kaiming He. Mean Flows for One-step Generative Modeling, 2025. URL https://arxiv.org/abs/2505.13447.

[103] Zigang Geng, Yibing Wang, Yeyao Ma, et al. X-Omni: Reinforcement Learning Makes Discrete Autoregressive Image Generative Models Great Again, 2025. URL https://arxiv.org/abs/2507.22058.

[104] Dhruba Ghosh, Hanna Hajishirzi, and Ludwig Schmidt. GenEval: An Object-Focused Framework for Evaluating Text-to-Image Alignment, 2023. URL https://arxiv.org/abs/2310.11513.

[105] Mark Goldstein, Anshuk Uppal, Raghav Singhal, Aahlad Puli, and Rajesh Ranganath. Flow Map Learning via Nongradient Vector Flow, 2026. URL https://arxiv.org/abs/2607.26398.

[106] Will Grathwohl, Ricky T. Q. Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. FFJORD: Free-form Continuous Dynamics for Scalable Reversible Generative Models, 2018. URL https://arxiv.org/abs/1810.01367.

[107] Xiangming Gu, Chao Du, Tianyu Pang, et al. On Memorization in Diffusion Models, 2023. URL https://arxiv.org/abs/2310.02664.

[108] Yuchao Gu, Guian Fang, Yuxin Jiang, et al. AnyFlow: Any-Step Video Diffusion Model with On-Policy Flow Map Distillation, 2026. URL https://arxiv.org/abs/2605.13724.

[109] Yi Guo, Wei Wang, Zhihang Yuan, et al. SplitMeanFlow: Interval Splitting Consistency in Few-Step Generative Modeling, 2025. URL https://arxiv.org/abs/2507.16884.

[110] Yuwei Guo, Ceyuan Yang, Anyi Rao, et al. AnimateDiff: Animate Your Personalized Text-to-Image Diffusion Models without Specific Tuning, 2023. URL https://arxiv.org/abs/2307.04725.

[111] Ziyu Guo, Renrui Zhang, Chengzhuo Tong, et al. Can We Generate Images with CoT? Let's Verify and Reinforce Image Generation Step by Step, 2025. URL https://arxiv.org/abs/2501.13926.

[112] Daehoon Gwak, Minhyung Lee, Junwoo Park, and Jaegul Choo. Accelerating Masked Diffusion Large Language Models: A Survey of Efficient Inference Techniques, 2026. URL https://arxiv.org/abs/2607.12829.

[113] Yoav HaCohen, Nisan Chiprut, Benny Brazowski, et al. LTX-Video: Realtime Video Latent Diffusion, 2024. URL https://arxiv.org/abs/2501.00103.

[114] Yoav HaCohen, Benny Brazowski, Nisan Chiprut, et al. LTX-2: Efficient Joint Audio-Visual Foundation Model, 2026. URL https://arxiv.org/abs/2601.03233.

[115] Erik Hartman, Jonas Wallin, Johan Malmström, and Jimmy Olsson. Controllable protein design with particle-based Feynman-Kac steering, 2025. URL https://arxiv.org/abs/2511.09216.

[116] Hao He, Yinghao Xu, Yuwei Guo, et al. CameraCtrl: Enabling Camera Control for Text-to-Video Generation, 2024. URL https://arxiv.org/abs/2404.02101.

[117] Haoran He, Yuxiao Ye, Jie Liu, et al. GARDO: Reinforcing Diffusion Models without Reward Hacking, 2025. URL https://arxiv.org/abs/2512.24138.

[118] Xianglong He, Chunli Peng, Zexiang Liu, et al. Matrix-game 2.0: An open-source, real-time, and streaming interactive world model, 2025. URL https://arxiv.org/abs/2508.13009.

[119] Xiao He, Yang Li, Peizhen Zhang, et al. Stabilizing, Scaling & Enhancing MeanFlow for Large-scale Diffusion Distillation, 2026. URL https://arxiv.org/abs/2605.17834.

[120] Xiaoxuan He, Siming Fu, Yuke Zhao, et al. TempFlow-GRPO: When Timing Matters for GRPO in Flow Models, 2025. URL https://arxiv.org/abs/2508.04324.

[121] Jonathan Heek, Emiel Hoogeboom, Thomas Mensink, and Tim Salimans. Unified Latents (UL): How to train your latents, 2026. URL https://arxiv.org/abs/2602.17270.

[122] Jonathan Ho and Tim Salimans. Classifier-Free Diffusion Guidance, 2022. URL https://arxiv.org/abs/2207.12598.

[123] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 2020.

[124] Jonathan Ho, Tim Salimans, Alexey Gritsenko, et al. Video Diffusion Models, 2022. URL https://arxiv.org/abs/2204.03458.

[125] Yunqi Hong, Kuei-Chun Kao, Hengguang Zhou, and Cho-Jui Hsieh. Understanding Reward Hacking in Text-to-Image Reinforcement Learning, 2026. URL https://arxiv.org/abs/2601.03468.

[126] Emiel Hoogeboom, Alexey A. Gritsenko, Jasmijn Bastings, et al. Autoregressive Diffusion Models, 2021. URL https://arxiv.org/abs/2110.02037.

[127] Emiel Hoogeboom, Victor Garcia Satorras, Clément Vignac, and Max Welling. Equivariant Diffusion for Molecule Generation in 3D, 2022. URL https://arxiv.org/abs/2203.17003.

[128] Edward J. Hu, Yelong Shen, Phillip Wallis, et al. LoRA: Low-Rank Adaptation of Large Language Models, 2021. URL https://arxiv.org/abs/2106.09685.

[129] Zheyuan Hu, Chieh-Hsin Lai, Yuki Mitsufuji, and Stefano Ermon. CMT: Mid-Training for Efficient Learning of Consistency, Mean Flow, and Flow Map Models, 2025. URL https://arxiv.org/abs/2509.24526.

[130] Chin-Wei Huang, Jae Hyun Lim, and Aaron Courville. A Variational Perspective on Diffusion-Based Generative Models and Score Matching, 2021. URL https://arxiv.org/abs/2106.02808.

[131] Kaiyi Huang, Chengqi Duan, Kaiyue Sun, et al. T2I-CompBench++: An Enhanced and Comprehensive Benchmark for Compositional Text-to-image Generation, 2023. URL https://arxiv.org/abs/2307.06350.

[132] Lianghua Huang, Wei Wang, Zhi-Fan Wu, et al. In-Context LoRA for Diffusion Transformers, 2024. URL https://arxiv.org/abs/2410.23775.

[133] Xun Huang, Zhengqi Li, Guande He, Mingyuan Zhou, and Eli Shechtman. Self Forcing: Bridging the Train-Test Gap in Autoregressive Video Diffusion, 2025. URL https://arxiv.org/abs/2506.08009.

[134] Henry Hunt, Mason Kamb, and Surya Ganguli. An exact information theory of generalization phase transitions in Bayesian diffusion models, 2026. URL https://arxiv.org/abs/2607.08041.

[135] HunyuanWorld Team, Zhenwei Wang, Yuhao Liu, et al. HunyuanWorld 1.0: Generating Immersive, Explorable, and Interactive 3D Worlds from Words or Pixels, 2025. URL https://arxiv.org/abs/2507.21809.

[136] Aapo Hyvärinen. Estimation of Non-Normalized Statistical Models by Score Matching. *Journal of Machine Learning Research*, 6(24):695–709, 2005. URL https://jmlr.org/papers/v6/hyvarinen05a.html.

[137] Imagen-Team-Google, Jason Baldridge, Jakob Bauer, et al. Imagen 3, 2024. URL https://arxiv.org/abs/2408.07009.

[138] Inception Labs, Samar Khanna, Siddhant Kharbanda, et al. Mercury: Ultra-Fast Language Models Based on Diffusion, 2025. URL https://arxiv.org/abs/2506.17298.

[139] Inclusion AI, Tiwei Bie, Haoxing Chen, et al. LLaDA2.0-Uni: Unifying Multimodal Understanding and Generation with Diffusion Large Language Model, 2026. URL https://arxiv.org/abs/2604.20796.

[140] John B. Ingraham, Max Baranov, Zak Costello, et al. Illuminating protein space with a programmable generative model. *Nature*, 623(7989):1070–1078, 2023. doi: 10.1038/s41586-023-06728-8. URL https://doi.org/10.1038/s41586-023-06728-8.

[141] Michael Janner, Yilun Du, Joshua B. Tenenbaum, and Sergey Levine. Planning with Diffusion for Flexible Behavior Synthesis, 2022. URL https://arxiv.org/abs/2205.09991.

[142] Sadeep Jayasumana, Srikumar Ramalingam, Andreas Veit, et al. Rethinking FID: Towards a Better Evaluation Metric for Image Generation, 2023. URL https://arxiv.org/abs/2401.09603.

[143] Kijung Jeon and Molei Tao. Imperfect Verifiers, Exact Targets: Inference-Time Scaling for Continuous Generation, 2026. URL https://arxiv.org/abs/2610.15281.

[144] Sihui Ji, Xi Chen, Xin Tao, Pengfei Wan, and Hengshuang Zhao. PhysMaster: Mastering Physical Representation for Video Generation via Reinforcement Learning, 2025. URL https://arxiv.org/abs/2510.13809.

[145] Dongfu Jiang, Max Ku, Tianle Li, et al. GenAI Arena: An Open Evaluation Platform for Generative Models, 2024. URL https://arxiv.org/abs/2406.04485. NeurIPS 2024.

[146] Dongzhi Jiang, Ziyu Guo, Renrui Zhang, et al. T2I-R1: Reinforcing Image Generation with Collaborative Semantic-level and Token-level CoT, 2025. URL https://arxiv.org/abs/2505.00703.

[147] Haozhe Jiang, Haiwen Feng, Pieter Abbeel, et al. DiPOD: Diffusion Policy Optimization without Drifting Apart, 2026. URL https://arxiv.org/abs/2606.13795.

[148] Richard Jordan, David Kinderlehrer, and Felix Otto. The Variational Formulation of the Fokker–Planck Equation. *SIAM Journal on Mathematical Analysis*, 29(1):1–17, 1998. doi: 10.1137/S0036141096303359.

[149] Zahra Kadkhodaie, Florentin Guth, Eero P. Simoncelli, and Stéphane Mallat. Generalization in diffusion models arises from geometry-adaptive harmonic representations, 2023. URL https://arxiv.org/abs/2310.02557.

[150] Mason Kamb and Surya Ganguli. An analytic theory of creativity in convolutional diffusion models, 2024. URL https://arxiv.org/abs/2412.20292.

[151] Taegu Kang, Jaesik Yoon, and Sungjin Ahn. Inference-Time Scaling in Diffusion Models through Iterative Partial Refinement, 2026. URL https://arxiv.org/abs/2605.19317.

[152] Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the Design Space of Diffusion-Based Generative Models, 2022. URL https://arxiv.org/abs/2206.00364.

[153] Tero Karras, Miika Aittala, Jaakko Lehtinen, et al. Analyzing and Improving the Training Dynamics of Diffusion Models, 2023. URL https://arxiv.org/abs/2312.02696.

[154] Tero Karras, Miika Aittala, Tuomas Kynkäänniemi, et al. Guiding a Diffusion Model with a Bad Version of Itself, 2024. URL https://arxiv.org/abs/2406.02507.

[155] Bahjat Kawar, Michael Elad, Stefano Ermon, and Jiaming Song. Denoising Diffusion Restoration Models, 2022. URL https://arxiv.org/abs/2201.11793.

[156] Anant Khandelwal and Manish Gupta. Read the Trace, Steer the Path: Trajectory-Aware Reinforcement Learning for Diffusion Language Models, 2026. URL https://arxiv.org/abs/2606.04396.

[157] Dongjun Kim, Chieh-Hsin Lai, Wei-Hsiang Liao, et al. Consistency Trajectory Models: Learning Probability Flow ODE Trajectory of Diffusion, 2023. URL https://arxiv.org/abs/2310.02279.

[158] Gwangho Kim and Sungyoon Lee. Memorization and Malign Generalization in Conditional Diffusion Models with Random Features, 2026. URL https://arxiv.org/abs/2610.11288.

[159] Moo Jin Kim, Yihuai Gao, Tsung-Yi Lin, et al. Cosmos Policy: Fine-Tuning Video Models for Visuomotor Control and Planning, 2026. URL https://arxiv.org/abs/2601.16163.

[160] Nayoung Kim, Kiyoung Seong, and Sungsoo Ahn. Packora: Systematic Design for Generative Molecular Crystal Structure Prediction, 2026. URL https://arxiv.org/abs/2608.26962.

[161] Diederik P. Kingma and Ruiqi Gao. Understanding Diffusion Objectives as the ELBO with Simple Data Augmentation, 2023. URL https://arxiv.org/abs/2303.00848.

[162] Diederik P. Kingma, Tim Salimans, Ben Poole, and Jonathan Ho. Variational Diffusion Models, 2021. URL https://arxiv.org/abs/2107.00630.

[163] Yuval Kirstain, Adam Polyak, Uriel Singer, et al. Pick-a-Pic: An Open Dataset of User Preferences for Text-to-Image Generation, 2023. URL https://arxiv.org/abs/2305.01569.

[164] Kling Team, Jialu Chen, Yuanzheng Ci, et al. Kling-Omni Technical Report, 2025. URL https://arxiv.org/abs/2512.16776.

[165] Weijie Kong, Qi Tian, Zijian Zhang, et al. HunyuanVideo: A Systematic Framework For Large Video Generative Models, 2024. URL https://arxiv.org/abs/2412.03603.

[166] Nupur Kumari, Bingliang Zhang, Richard Zhang, Eli Shechtman, and Jun-Yan Zhu. Multi-Concept Customization of Text-to-Image Diffusion, 2022. URL https://arxiv.org/abs/2212.04488.

[167] Maksim Kuprashevich, Grigorii Alekseenko, Irina Tolstykh, et al. NoHumansRequired: Autonomous High-Quality Image Editing Triplet Mining, 2025. URL https://arxiv.org/abs/2507.14119.

[168] Tuomas Kynkäänniemi, Miika Aittala, Tero Karras, et al. Applying Guidance in a Limited Interval Improves Sample and Distribution Quality in Diffusion Models, 2024. URL https://arxiv.org/abs/2404.07724.

[169] Hugo Latourelle-Vigeant, Sinho Chewi, Aram-Alexandre Pooladian, John Sous, and Theodor Misiakiewicz. Generalization, memorization, and overfitting for diffusion models trained in the lazy high-dimensional regime, 2026. URL https://arxiv.org/abs/2608.23938.

[170] Dongyeun Lee, Amir Zandieh, Vahab Mirrokni, Junmo Kim, and Insu Han. HyperVAttention: Efficient Sparse Attention with Spatio-Temporal Clustering for Video Diffusion, 2026. URL https://arxiv.org/abs/2607.03012.

[171] Jeongjae Lee, Jinho Chang, Jeongsol Kim, and Jong Chul Ye. Reward Score Matching: Unifying Reward-based Fine-tuning for Flow and Diffusion Models, 2026. URL https://arxiv.org/abs/2604.17415.

[172] Jongbeom Lee, Hyunwoo Yu, Jincheol Yang, Jaemin Choi, and Suk-Ju Kang. SparSTAR: Sparse Attention for SpaceTime AutoRegressive Video Synthesis, 2026. URL https://arxiv.org/abs/2608.10519.

[173] Kimin Lee, Hao Liu, Moonkyung Ryu, et al. Aligning Text-to-Image Models using Human Feedback, 2023. URL https://arxiv.org/abs/2302.12192.

[174] Kyungmin Lee, Sihyun Yu, and Jinwoo Shin. Decoupled MeanFlow: Turning Flow Models into Flow Maps for Accelerated Sampling, 2025. URL https://arxiv.org/abs/2510.24474.

[175] Sanghyun Lee, Chunsan Hong, Seungryong Kim, et al. Looped Diffusion Language Models, 2026. URL https://arxiv.org/abs/2605.26106.

[176] Xingjian Leng, Jaskirat Singh, Yunzhong Hou, et al. REPA-E: Unlocking VAE for End-to-End Tuning with Latent Diffusion Transformers, 2025. URL https://arxiv.org/abs/2504.10483.

[177] Christian Léonard. A survey of the Schrödinger problem and some of its connections with optimal transport, 2013. URL https://arxiv.org/abs/1308.0215.

[178] Bingnan Li, Haozhe Wang, Haozhong Xiong, et al. Rethinking Classifier-Free Guidance in On-Policy Diffusion Distillation, 2026. URL https://arxiv.org/abs/2607.24731.

[179] Haopeng Li, Yitong Li, Junsong Chen, et al. Sol-Attn: Accelerating Video Generation Inference via On-the-Fly Attention Sparsification, 2026. URL https://arxiv.org/abs/2607.24027.

[180] Haozhan Li, Yuxin Zuo, Jiale Yu, et al. SimpleVLA-RL: Scaling VLA Training via Reinforcement Learning, 2025. URL https://arxiv.org/abs/2509.09674.

[181] Hengtao Li, Pengxiang Ding, Runze Suo, et al. VLA-RFT: Vision-Language-Action Reinforcement Fine-tuning with Verified Rewards in World Simulators, 2025. URL https://arxiv.org/abs/2510.00406.

[182] Jiaqi Li, Junshu Tang, Zhiyong Xu, et al. Hunyuan-GameCraft: High-dynamic Interactive Game Video Generation with Hybrid History Condition, 2025. URL https://arxiv.org/abs/2506.17201.

[183] Junzhe Li, Yutao Cui, Tao Huang, et al. MixGRPO: Unlocking Flow-based GRPO Efficiency with Mixed ODE-SDE, 2025. URL https://arxiv.org/abs/2507.21802.

[184] Muyang Li, Yujun Lin, Zhekai Zhang, et al. SVDQuant: Absorbing Outliers by Low-Rank Components for 4-Bit Diffusion Models, 2024. URL https://arxiv.org/abs/2411.05007.

[185] Shufan Li, Konstantinos Kallidromitis, Akash Gokul, Yusuke Kato, and Kazuki Kozuka. Aligning Diffusion Models by Optimizing Human Utility, 2024. URL https://arxiv.org/abs/2404.04465.

[186] Shufan Li, Konstantinos Kallidromitis, Akash Gokul, et al. Reflect-DiT: Inference-Time Scaling for Text-to-Image Diffusion Transformers via In-Context Reflection, 2025. URL https://arxiv.org/abs/2503.12271.

[187] Tianhong Li and Kaiming He. Back to Basics: Let Denoising Generative Models Denoise, 2025. URL https://arxiv.org/abs/2511.13720.

[188] Tianhong Li, Yonglong Tian, He Li, Mingyang Deng, and Kaiming He. Autoregressive Image Generation without Vector Quantization, 2024. URL https://arxiv.org/abs/2406.11838.

[189] Xiner Li, Yulai Zhao, Chenyu Wang, et al. Derivative-Free Guidance in Continuous and Discrete Diffusion Models with Soft Value-Based Decoding, 2024. URL https://arxiv.org/abs/2408.08252.

[190] Xiuyu Li, Yijiang Liu, Long Lian, et al. Q-Diffusion: Quantizing Diffusion Models, 2023. URL https://arxiv.org/abs/2302.04304.

[191] Yanghao Li, Rui Qian, Bowen Pan, et al. MANZANO: A Simple and Scalable Unified Multimodal Model with a Hybrid Vision Tokenizer, 2025. URL https://arxiv.org/abs/2509.16197.

[192] Yuming Li, Yikai Wang, Yuying Zhu, et al. BranchGRPO: Stable and Efficient GRPO with Structured Branching in Diffusion Models, 2025. URL https://arxiv.org/abs/2509.06040.

[193] Zeyang Li, Yunan Wang, Paolo Giaretta, and Navid Azizan. Newton Matching for Generative Modeling: A Unified Framework for Fine-Tuning and Sampling, 2026. URL https://arxiv.org/abs/2609.05727.

[194] Zhen Li, Mingdeng Cao, Xintao Wang, et al. PhotoMaker: Customizing Realistic Human Photos via Stacked ID Embedding, 2023. URL https://arxiv.org/abs/2312.04461.

[195] Zongjian Li, Zheyuan Liu, Qihui Zhang, et al. Uniworld-V2: Reinforce Image Editing with Diffusion Negative-aware Finetuning and MLLM Implicit Feedback, 2025. URL https://arxiv.org/abs/2510.16888.

[196] Zhanhao Liang, Yuhui Yuan, Shuyang Gu, et al. Aesthetic Post-Training Diffusion Models from Generic Preferences with Step-by-step Preference Optimization, 2024. URL https://arxiv.org/abs/2406.04314.

[197] Zhengyang Liang, Hao He, Ceyuan Yang, and Bo Dai. Scaling Laws For Diffusion Transformers, 2024. URL https://arxiv.org/abs/2410.08184.

[198] Yue Liao, Pengfei Zhou, Siyuan Huang, et al. Genie Envisioner: A Unified World Foundation Platform for Robotic Manipulation, 2025. URL https://arxiv.org/abs/2508.05635.

[199] Shanchuan Lin, Ceyuan Yang, Zhijie Lin, Hao Chen, and Haoqi Fan. Adversarial Flow Models, 2025. URL https://arxiv.org/abs/2511.22475.

[200] Yuchen Lin, Chenguo Lin, Panwang Pan, et al. PartCrafter: Structured 3D Mesh Generation via Compositional Latent Diffusion Transformers, 2025. URL https://arxiv.org/abs/2506.05573.

[201] Pengyang Ling, Jiazi Bu, Yujie Zhou, et al. Pave-GRPO: Beyond Instantaneous Guidance through Principled Average Velocity Decomposition, 2026. URL https://arxiv.org/abs/2606.01636.

[202] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.

[203] Yaron Lipman, Marton Havasi, Peter Holderrieth, et al. Flow Matching Guide and Code, 2024. URL https://arxiv.org/abs/2412.06264.

[204] Dongyang Liu, Peng Gao, David Liu, et al. Decoupled DMD: CFG Augmentation as the Spear, Distribution Matching as the Shield, 2025. URL https://arxiv.org/abs/2511.22677.

[205] Feng Liu, Shiwei Zhang, Xiaofeng Wang, et al. Timestep Embedding Tells: It's Time to Cache for Video Diffusion Model, 2024. URL https://arxiv.org/abs/2411.19108.

[206] Haohe Liu, Zehua Chen, Yi Yuan, et al. AudioLDM: Text-to-Audio Generation with Latent Diffusion Models, 2023. URL https://arxiv.org/abs/2301.12503.

[207] Jiaxiang Liu, Zhida Feng, Pengyu Zou, et al. ERNIE-Image Technical Report, 2026. URL https://arxiv.org/abs/2605.25347.

[208] Jie Liu, Gongye Liu, Jiajun Liang, et al. Flow-GRPO: Training Flow Matching Models via Online RL, 2025. URL https://arxiv.org/abs/2505.05470.

[209] Jie Liu, Gongye Liu, Jiajun Liang, et al. Improving Video Generation with Human Feedback, 2025. URL https://arxiv.org/abs/2501.13918.

[210] Jingyu Liu, Xin Dong, Zhifan Ye, et al. TiDAR: Think in Diffusion, Talk in Autoregression, 2025. URL https://arxiv.org/abs/2511.08923.

[211] Kunhao Liu, Wenbo Hu, Jiale Xu, Ying Shan, and Shijian Lu. Rolling Forcing: Autoregressive Long Video Diffusion in Real Time, 2025. URL https://arxiv.org/abs/2509.25161.

[212] Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, et al. DoRA: Weight-Decomposed Low-Rank Adaptation, 2024. URL https://arxiv.org/abs/2402.09353.

[213] Shiyu Liu, Yucheng Han, Peng Xing, et al. Step1X-Edit: A Practical Framework for General Image Editing, 2025. URL https://arxiv.org/abs/2504.17761.

[214] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow, 2022. URL https://arxiv.org/abs/2209.03003.

[215] Xingchao Liu, Xiwen Zhang, Jianzhu Ma, Jian Peng, and Qiang Liu. InstaFlow: One Step is Enough for High-Quality Diffusion-Based Text-to-Image Generation, 2023. URL https://arxiv.org/abs/2309.06380.

[216] Zhiheng Liu, Weiming Ren, Haozhe Liu, et al. TUNA: Taming Unified Visual Representations for Native Unified Multimodal Models, 2025. URL https://arxiv.org/abs/2512.02014.

[217] Alston Lo, Luka Mucko, Austin H. Cheng, et al. Fast Organic Crystal Structure Prediction with Unit Cell Flow Matching, 2026. URL https://arxiv.org/abs/2606.03199.

[218] Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete Diffusion Modeling by Estimating the Ratios of the Data Distribution, 2023. URL https://arxiv.org/abs/2310.16834.

[219] Chetwin Low, Weimin Wang, and Calder Katyal. Ovi: Twin Backbone Cross-Modal Fusion for Audio-Video Generation, 2025. URL https://arxiv.org/abs/2510.01284.

[220] Cheng Lu and Yang Song. Simplifying, Stabilizing and Scaling Continuous-Time Consistency Models, 2024. URL https://arxiv.org/abs/2410.11081.

[221] Cheng Lu, Yuhao Zhou, Fan Bao, et al. DPM-Solver: A Fast ODE Solver for Diffusion Probabilistic Model Sampling in Around 10 Steps, 2022. URL https://arxiv.org/abs/2206.00927.

[222] Cheng Lu, Yuhao Zhou, Fan Bao, et al. DPM-Solver++: Fast Solver for Guided Sampling of Diffusion Probabilistic Models, 2022. URL https://arxiv.org/abs/2211.01095.

[223] Jiarui Lu, Yuyang Wang, Yizhe Zhang, et al. SimpleDesign: A Joint Model for Protein Sequence and Structure Codesign, 2026. URL https://arxiv.org/abs/2609.03377.

[224] Xin Lu, Jie Huang, Zihao Fan, et al. Ultra Flash: Scaling Real-Time Streaming Video Generation to High Resolutions, 2026. URL https://arxiv.org/abs/2606.09150.

[225] Yiyang Lu, Susie Lu, Qiao Sun, et al. One-step Latent-free Image Generation with Pixel Mean Flows, 2026. URL https://arxiv.org/abs/2601.22158.

[226] Yunhong Lu, Yanhong Zeng, Haobo Li, et al. Reward Forcing: Efficient Streaming Video Generation with Rewarded Distribution Matching Distillation, 2025. URL https://arxiv.org/abs/2512.04678.

[227] Eric Luhman and Troy Luhman. Knowledge Distillation in Iterative Generative Models for Improved Sampling Speed, 2021. URL https://arxiv.org/abs/2101.02388.

[228] Simian Luo, Yiqin Tan, Longbo Huang, Jian Li, and Hang Zhao. Latent Consistency Models: Synthesizing High-Resolution Images with Few-Step Inference, 2023. URL https://arxiv.org/abs/2310.04378.

[229] Ziwei Luo, Ziqi Jin, Lei Wang, Lidong Bing, and Thomas B. Schön. Self-Rewarding Sequential Monte Carlo for Masked Diffusion Language Models, 2026. URL https://arxiv.org/abs/2602.01849.

[230] Chenrui Ma, Xi Xiao, Lin Zhao, et al. Drift Flow Matching, 2026. URL https://arxiv.org/abs/2605.17244.

[231] Nanye Ma, Mark Goldstein, Michael S. Albergo, et al. SiT: Exploring Flow and Diffusion-based Generative Models with Scalable Interpolant Transformers, 2024. URL https://arxiv.org/abs/2401.08740.

[232] Nanye Ma, Shangyuan Tong, Haolin Jia, et al. Inference-Time Scaling for Diffusion Models beyond Scaling Denoising Steps, 2025. URL https://arxiv.org/abs/2501.09732.

[233] Xinyin Ma, Gongfan Fang, and Xinchao Wang. DeepCache: Accelerating Diffusion Models for Free, 2023. URL https://arxiv.org/abs/2312.00858.

[234] Yuhang Ma, Yunhao Shui, Xiaoshi Wu, Keqiang Sun, and Hongsheng Li. HPSv3: Towards Wide-Spectrum Human Preference Score, 2025. URL https://arxiv.org/abs/2508.03789.

[235] Zehong Ma, Ruihan Xu, and Shiliang Zhang. PixelGen: Improving Pixel Diffusion with Perceptual Supervision, 2026. URL https://arxiv.org/abs/2602.02493.

[236] Bruno Machado, Alexandre Chapin, Emmanuel Dellandrea, and Liming Chen. Flowing With Purpose: Latent Action Guided Flow Matching Policies For Robotic Manipulation, 2026. URL https://arxiv.org/abs/2606.23420.

[237] Antoine Maillard and Sebastian Goldt. Memorisation, convergence and generalisation in generative models, 2026. URL https://arxiv.org/abs/2605.21402.

[238] Xiaofeng Mao, Zhen Li, Chuanhao Li, et al. Yume-1.5: A Text-Controlled Interactive World Generation Model, 2025. URL https://arxiv.org/abs/2512.22096.

[239] Morteza Mardani, Jiaming Song, Jan Kautz, and Arash Vahdat. A Variational Perspective on Solving Inverse Problems with Diffusion Models, 2023. URL https://arxiv.org/abs/2305.04391.

[240] David McAllister, Songwei Ge, Brent Yi, et al. Flow Matching Policy Gradients, 2025. URL https://arxiv.org/abs/2507.21053.

[241] Yunpeng Mei, Jiakai He, Hongjie Cao, et al. Potential-Guided Flow Matching for Vision-Language-Action Policy Improvement, 2026. URL https://arxiv.org/abs/2606.04968.

[242] Meituan LongCat Team, Xunliang Cai, Qilong Huang, et al. LongCat-Video Technical Report, 2025. URL https://arxiv.org/abs/2510.22200.

[243] Meituan LongCat Team, Hanghang Ma, Haoxian Tan, et al. LongCat-Image Technical Report, 2025. URL https://arxiv.org/abs/2512.07584.

[244] Chenlin Meng, Robin Rombach, Ruiqi Gao, et al. On Distillation of Guided Diffusion Models, 2022. URL https://arxiv.org/abs/2210.03142.

[245] Claudia Merger and Sebastian Goldt. Local Coverage Governs Memorization in Diffusion Models, 2026. URL https://arxiv.org/abs/2606.14390.

[246] Boming Miao, Tao Zhang, Netanel Raviv, et al. Protecting Sensitive Data in Image Synthesis via PAC-Private Adaptation for Diffusion Models, 2026. URL https://arxiv.org/abs/2610.04038.

[247] Chong Mou, Xintao Wang, Liangbin Xie, et al. T2I-Adapter: Learning Adapters to Dig out More Controllable Ability for Text-to-Image Diffusion Models, 2023. URL https://arxiv.org/abs/2302.08453.

[248] Chong Mou, Yanze Wu, Wenxu Wu, et al. DreamO: A Unified Framework for Image Customization, 2025. URL https://arxiv.org/abs/2504.16915.

[249] NextStep Team, Chunrui Han, Guopeng Li, et al. NextStep-1: Toward Autoregressive Image Generation with Continuous Tokens at Scale, 2025. URL https://arxiv.org/abs/2508.10711.

[250] Jinjie Ni, Qian Liu, Longxu Dou, et al. Diffusion Language Models are Super Data Learners, 2025. URL https://arxiv.org/abs/2511.03276.

[251] Zanlin Ni, Shenzhi Wang, Yang Yue, et al. The Flexibility Trap: Rethinking the Value of Arbitrary Order in Diffusion Language Models, 2026. URL https://arxiv.org/abs/2601.15165.

[252] Shen Nie, Fengqi Zhu, Zebin You, et al. Large Language Diffusion Models, 2025. URL https://arxiv.org/abs/2502.09992.

[253] Shen Nie, Qiyang Min, Shaoxuan Xu, et al. Improved Large Language Diffusion Models, 2026. URL https://arxiv.org/abs/2606.25331.

[254] Weili Nie, Julius Berner, Nanye Ma, et al. Transition Matching Distillation for Fast Video Generation, 2026. URL https://arxiv.org/abs/2601.09881. Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2026, pp. 4645-4655.

[255] Haochen Niu, Shengye Dong, Hao Liu, Peiwen Lin, and Wang Chuang. Frequency-Conditioned Flow Matching for Vision-Language-Action Models, 2026. URL https://arxiv.org/abs/2609.10405.

[256] NVIDIA, Niket Agarwal, Arslan Ali, et al. Cosmos World Foundation Model Platform for Physical AI, 2025. URL https://arxiv.org/abs/2501.03575.

[257] NVIDIA, Arslan Ali, Junjie Bai, et al. World Simulation with Video Foundation Models for Physical AI, 2025. URL https://arxiv.org/abs/2511.00062.

[258] NVIDIA, Aditi, Niket Agarwal, et al. Cosmos 3: Omnimodal World Models for Physical AI, 2026. URL https://arxiv.org/abs/2606.02800.

[259] Andrey Okhotin, Maksim Nakhodnov, Nikita Kazeev, et al. MiAD: Mirage Atom Diffusion for De Novo Crystal Generation, 2025. URL https://arxiv.org/abs/2511.14426.

[260] Blake Olson, Yuhang Song, Emmett McQuinn, and Yuan Shangguan. CanvasAnneal: Curriculum Reinforcement Learning for Diffusion Language Models, 2026. URL https://arxiv.org/abs/2609.13060.

[261] Jingyang Ou, Shen Nie, Kaiwen Xue, et al. Your Absorbing Discrete Diffusion Secretly Models the Conditional Distributions of Clean Data, 2024. URL https://arxiv.org/abs/2406.03736.

[262] Jonas Pai, Liam Achenbach, Victoriano Montesinos, et al. mimic-video: Video-Action Models for Generalizable Robot Control Beyond VLAs, 2025. URL https://arxiv.org/abs/2512.15692.

[263] PAN Team, Zihan Liu, Yi Gu, et al. PAN: A World Model for General, Actionable, and Long-Horizon World Simulation, 2025. URL https://arxiv.org/abs/2511.09057.

[264] Peter Pao-Huang, Xiaojie Qiu, and Stefano Ermon. Generative Modeling with Flux Matching, 2026. URL https://arxiv.org/abs/2605.07319.

[265] Jihun Park, Kyoungmin Lee, Jongmin Gim, et al. CRAFT: Constrained Reward via Attention Fine-Tuning for Subject Personalization without Composed Targets, 2026. URL https://arxiv.org/abs/2608.14403.

[266] Saro Passaro, Gabriele Corso, Jeremy Wohlwend, et al. Boltz-2: Towards Accurate and Efficient Binding Affinity Prediction. bioRxiv preprint, 2025. URL https://doi.org/10.1101/2025.06.14.659707.

[267] William Peebles and Saining Xie. Scalable Diffusion Models with Transformers, 2022. URL https://arxiv.org/abs/2212.09748.

[268] Wujian Peng, Lingchen Meng, Yuxuan Cai, et al. Unified Multimodal Autoregressive Modeling with Shared Context-Visual Tokenizer is Key to Unification, 2026. URL https://arxiv.org/abs/2606.18249.

[269] Zhiliang Peng, Jianwei Yu, Wenhui Wang, et al. VibeVoice Technical Report, 2025. URL https://arxiv.org/abs/2508.19205.

[270] Le-Tuyet-Nhi Pham, Giovanni Conforti, Zhenjie Ren, and Alain Durmus. Discrete Flow Matching: Convergence Guarantees Under Minimal Assumptions, 2026. URL https://arxiv.org/abs/2605.08882.

[271] Physical Intelligence, Kevin Black, Noah Brown, et al. $\pi_{0.5}$: a Vision-Language-Action Model with Open-World Generalization, 2025. URL https://arxiv.org/abs/2504.16054.

[272] Bowen Ping, Xiangxin Zhou, Penghui Qi, et al. Flow-DPPO: Divergence Proximal Policy Optimization for Flow Matching Models, 2026. URL https://arxiv.org/abs/2606.11025.

[273] Adam Polyak, Amit Zohar, Andrew Brown, et al. Movie Gen: A Cast of Media Foundation Models, 2024. URL https://arxiv.org/abs/2410.13720.

[274] Aram-Alexandre Pooladian, Heli Ben-Hamu, Carles Domingo-Enrich, et al. Multisample Flow Matching: Straightening Flows with Minibatch Couplings, 2023. URL https://arxiv.org/abs/2304.14772.

[275] Ben Poole, Ajay Jain, Jonathan T. Barron, and Ben Mildenhall. DreamFusion: Text-to-3D using 2D Diffusion, 2022. URL https://arxiv.org/abs/2209.14988.

[276] Mihir Prabhudesai, Anirudh Goyal, Deepak Pathak, and Katerina Fragkiadaki. Aligning Text-to-Image Diffusion Models with Reward Backpropagation, 2023. URL https://arxiv.org/abs/2310.03739.

[277] Mihir Prabhudesai, Mengning Wu, Amir Zadeh, Katerina Fragkiadaki, and Deepak Pathak. Diffusion Beats Autoregressive in Data-Constrained Settings, 2025. URL https://arxiv.org/abs/2507.15857.

[278] Ilan Price, Alvaro Sanchez-Gonzalez, Ferran Alet, et al. Probabilistic weather forecasting with machine learning. *Nature*, 637(8044):84–90, 2024. doi: 10.1038/s41586-024-08252-9. URL https://doi.org/10.1038/s41586-024-08252-9.

[279] Yuandong Pu, Le Zhuo, Songhao Han, et al. PICABench: How Far Are We from Physically Realistic Image Editing?, 2025. URL https://arxiv.org/abs/2510.17681.

[280] Yupeng Qiu, Han Fang, Dhananjai Sharma, and Ee-Chien Chang. Discovering Undisclosed Concepts in LoRA Adapters for Diffusion-Based AIGC Models, 2026. URL https://arxiv.org/abs/2610.18990.

[281] Haran Raajesh, Kulin Shah, Adam Klivans, and Philipp Krähenbühl. Mask-Aware Policy Gradients for Diffusion Language Models, 2026. URL https://arxiv.org/abs/2607.15200.

[282] Rafael Rafailov, Archit Sharma, Eric Mitchell, et al. Direct Preference Optimization: Your Language Model is Secretly a Reward Model, 2023. URL https://arxiv.org/abs/2305.18290.

[283] Aditya Ranganath and Mukesh Singhal. A Unified Measure-Theoretic View of Diffusion, Score-Based, and Flow Matching Generative Models, 2026. URL https://arxiv.org/abs/2605.06829.

[284] Yinuo Ren, Wenhao Gao, Lexing Ying, Grant M. Rotskoff, and Jiequn Han. DriftLite: Lightweight Drift Control for Inference-Time Scaling of Diffusion Models, 2025. URL https://arxiv.org/abs/2509.21655.

[285] Erik Riise, Mehmet Onurcan Kaya, and Dim P. Papadopoulos. Visual Autoregressive Models Beat Diffusion Models on Inference Time Scaling, 2025. URL https://arxiv.org/abs/2510.16751.

[286] Robbyant Team, Zelin Gao, Qiuyu Wang, et al. Advancing Open-source World Models, 2026. URL https://arxiv.org/abs/2601.20540.

[287] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-Resolution Image Synthesis with Latent Diffusion Models, 2021. URL https://arxiv.org/abs/2112.10752.

[288] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional Networks for Biomedical Image Segmentation, 2015. URL https://arxiv.org/abs/1505.04597.

[289] Litu Rout, Negin Raoof, Giannis Daras, et al. Solving Linear Inverse Problems Provably via Posterior Sampling with Latent Diffusion Models, 2023. URL https://arxiv.org/abs/2307.00619.

[290] Nataniel Ruiz, Yuanzhen Li, Varun Jampani, et al. DreamBooth: Fine Tuning Text-to-Image Diffusion Models for Subject-Driven Generation, 2022. URL https://arxiv.org/abs/2208.12242.

[291] Amirmojtaba Sabour, Sanja Fidler, and Karsten Kreis. Align Your Steps: Optimizing Sampling Schedules in Diffusion Models, 2024. URL https://arxiv.org/abs/2404.14507.

[292] Amirmojtaba Sabour, Michael S. Albergo, Carles Domingo-Enrich, et al. Test-time scaling of diffusions with flow maps, 2025. URL https://arxiv.org/abs/2511.22688.

[293] Amirmojtaba Sabour, Sanja Fidler, and Karsten Kreis. Align Your Flow: Scaling Continuous-Time Flow Map Distillation, 2025. URL https://arxiv.org/abs/2506.14603.

[294] Seyedmorteza Sadat, Otmar Hilliges, and Romann M. Weber. Eliminating Oversaturation and Artifacts of High Guidance Scales in Diffusion Models, 2024. URL https://arxiv.org/abs/2410.02416. The Thirteenth International Conference on Learning Representations (ICLR 2025).

[295] Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, et al. Simple and Effective Masked Diffusion Language Models, 2024. URL https://arxiv.org/abs/2406.07524.

[296] Subham Sekhar Sahoo, Justin Deschenaux, Aaron Gokaslan, et al. The Diffusion Duality, 2025. URL https://arxiv.org/abs/2506.10892.

[297] Tim Salimans and Jonathan Ho. Progressive Distillation for Fast Sampling of Diffusion Models, 2022. URL https://arxiv.org/abs/2202.00512.

[298] SAM 3D Team, Xingyu Chen, Fu-Jen Chu, et al. SAM 3D: 3Dfy Anything in Images, 2025. URL https://arxiv.org/abs/2511.16624.

[299] Axel Sauer, Dominik Lorenz, Andreas Blattmann, and Robin Rombach. Adversarial Diffusion Distillation, 2023. URL https://arxiv.org/abs/2311.17042.

[300] Axel Sauer, Frederic Boesel, Tim Dockhorn, et al. Fast High-Resolution Image Synthesis with Latent Adversarial Diffusion Distillation, 2024. URL https://arxiv.org/abs/2403.12015.

[301] Gianluca Scarpellini, Ron Shprints, Peter Holderrieth, et al. Few-step Cofolding with All-Atom Flow Maps, 2026. URL https://arxiv.org/abs/2606.08375.

[302] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal Policy Optimization Algorithms, 2017. URL https://arxiv.org/abs/1707.06347.

[303] Niklas Schweiger, Daniel Cremers, and Karnik Ram. Trust-Region Noise Search for Black-Box Alignment of Diffusion and Flow Models, 2026. URL https://arxiv.org/abs/2603.14504.

[304] Artem Sergievskii, Artyom Turevich, and Sergey Kastryulin. Revisiting Classifier-Free Guidance Methods in Latent Diffusion Models, 2026. URL https://arxiv.org/abs/2608.16786.

[305] Viraj Shah, Nataniel Ruiz, Forrester Cole, et al. ZipLoRA: Any Subject in Any Style by Effectively Merging LoRAs, 2023. URL https://arxiv.org/abs/2311.13600.

[306] Zhihong Shao, Peiyi Wang, Qihao Zhu, et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models, 2024. URL https://arxiv.org/abs/2402.03300.

[307] Jinliang Shen, Lianghao Su, Zheming Li, et al. HeadCast: Casting Attention Heads for Efficient Autoregressive Video Generation, 2026. URL https://arxiv.org/abs/2607.20125.

[308] Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis K. Titsias. Simplified and Generalized Masked Diffusion for Discrete Data, 2024. URL https://arxiv.org/abs/2406.04329.

[309] Yuyang Shi, Valentin De Bortoli, Andrew Campbell, and Arnaud Doucet. Diffusion Schrödinger Bridge Matching, 2023. URL https://arxiv.org/abs/2303.16852.

[310] Daniil Shlenskii, Alexander Varlamov, Nazar Buzun, and Alexander Korotin. Unlocking the Duality between Flow and Field Matching, 2026. URL https://arxiv.org/abs/2602.02261.

[311] SII-OpenMOSS Team, Donghua Yu, Mingshu Chen, et al. MOVA: Towards Scalable and Synchronized Video-Audio Generation, 2026. URL https://arxiv.org/abs/2602.08794.

[312] Jaskirat Singh, Boyang Zheng, Zongze Wu, et al. Improved Baselines with Representation Autoencoders, 2026. URL https://arxiv.org/abs/2605.18324.

[313] Raghav Singhal, Zachary Horvitz, Ryan Teehan, et al. A General Framework for Inference-time Scaling and Steering of Diffusion Models, 2025. URL https://arxiv.org/abs/2501.06848.

[314] Marta Skreta, Tara Akhound-Sadegh, Viktor Ohanesian, et al. Feynman-Kac Correctors in Diffusion: Annealing, Guidance, and Product of Experts, 2025. URL https://arxiv.org/abs/2503.02819.

[315] Vera Soboleva, Aibek Alanov, Andrey Kuznetsov, and Konstantin Sobolev. T-LoRA: Single Image Diffusion Model Customization Without Overfitting, 2025. URL https://arxiv.org/abs/2507.05964.

[316] Jascha Sohl-Dickstein, Eric A. Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep Unsupervised Learning using Nonequilibrium Thermodynamics, 2015. URL https://arxiv.org/abs/1503.03585.

[317] Gowthami Somepalli, Vasu Singla, Micah Goldblum, Jonas Geiping, and Tom Goldstein. Diffusion Art or Digital Forgery? Investigating Data Replication in Diffusion Models, 2022. URL https://arxiv.org/abs/2212.03860.

[318] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising Diffusion Implicit Models, 2020. URL https://arxiv.org/abs/2010.02502.

[319] Yang Song and Prafulla Dhariwal. Improved Techniques for Training Consistency Models, 2023. URL https://arxiv.org/abs/2310.14189.

[320] Yang Song and Stefano Ermon. Generative Modeling by Estimating Gradients of the Data Distribution, 2019. URL https://arxiv.org/abs/1907.05600.

[321] Yang Song and Diederik P. Kingma. How to Train Your Energy-Based Models, 2021. URL https://arxiv.org/abs/2101.03288.

[322] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, et al. Score-Based Generative Modeling through Stochastic Differential Equations, 2020. URL https://arxiv.org/abs/2011.13456.

[323] Yang Song, Conor Durkan, Iain Murray, and Stefano Ermon. Maximum Likelihood Training of Score-Based Diffusion Models, 2021. URL https://arxiv.org/abs/2101.09258.

[324] Yang Song, Liyue Shen, Lei Xing, and Stefano Ermon. Solving Inverse Problems in Medical Imaging with Score-Based Generative Models, 2021. URL https://arxiv.org/abs/2111.08005.

[325] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency Models, 2023. URL https://arxiv.org/abs/2303.01469.

[326] Yuxuan Song, Jingjing Gong, Minkai Xu, et al. Equivariant Flow Matching with Hybrid Probability Transport, 2023. URL https://arxiv.org/abs/2312.07168.

[327] Yuxuan Song, Zheng Zhang, Cheng Luo, et al. Seed Diffusion: A Large-Scale Diffusion Language Model with High-Speed Inference, 2025. URL https://arxiv.org/abs/2508.02193.

[328] Hannes Stark, Felix Faltings, MinGyu Choi, et al. BoltzGen: Toward Universal Binder Design. bioRxiv preprint, 2025. URL https://doi.org/10.1101/2025.11.20.689494.

[329] Jason Stock, Troy Arcomano, and Rao Kotamarthi. Swift: An Autoregressive Consistency Model for Efficient Weather Forecasting, 2025. URL https://arxiv.org/abs/2509.25631.

[330] Kaixiang Su, Hongfei Xue, and Qiang Zhu. CrystalGRPO: Target-Aligned and Coverage-Preserving Reinforcement Learning for Flow-Based Crystal Structure Prediction, 2026. URL https://arxiv.org/abs/2608.06582.

[331] Wenqiang Sun, Haiyu Zhang, Haoyuan Wang, et al. WorldPlay: Towards Long-Term Geometric Consistency for Real-Time Interactive World Modeling, 2025. URL https://arxiv.org/abs/2512.14614.

[332] Super Intelligence Team, Changhao Qiao, Chao Hui, et al. FireRed-Image-Edit-1.0 Technical Report, 2026. URL https://arxiv.org/abs/2602.13344.

[333] Zhenxiong Tan, Songhua Liu, Xingyi Yang, Qiaochu Xue, and Xinchao Wang. OminiControl: Minimal and Universal Control for Diffusion Transformer, 2024. URL https://arxiv.org/abs/2411.15098.

[334] Zhipeng Tang, Xinda Chen, Weining Rao, et al. Fewer Steps, Better Actions: Rethinking Flow-Matching Inference for VLA Policies, 2026. URL https://arxiv.org/abs/2609.21216.

[335] AlayaWorld Team, Kaipeng Zhang, Chuanhao Li, et al. AlayaWorld: Long-Horizon and Playable Video World Generation, 2026. URL https://arxiv.org/abs/2607.06291.

[336] Team HY-World, Chenjie Cao, Xuhui Zuo, et al. HY-World 2.0: A Multi-Modal World Model for Reconstructing, Generating, and Simulating 3D Worlds, 2026. URL https://arxiv.org/abs/2604.14268.

[337] Team Seedance, Heyi Chen, Siyan Chen, et al. Seedance 1.5 pro: A Native Audio-Visual Joint Generation Foundation Model, 2025. URL https://arxiv.org/abs/2512.13507.

[338] Team Seedance, De Chen, Liyang Chen, et al. Seedance 2.0: Advancing Video Generation for World Complexity, 2026. URL https://arxiv.org/abs/2604.14148.

[339] Team Seedream, Yunpeng Chen, Yu Gao, et al. Seedream 4.0: Toward Next-generation Multimodal Image Generation, 2025. URL https://arxiv.org/abs/2509.20427.

[340] Team Wan, Ang Wang, Baole Ai, et al. Wan: Open and Advanced Large-Scale Video Generative Models, 2025. URL https://arxiv.org/abs/2503.20314.

[341] Tencent Hunyuan Foundation Model Team. HunyuanImage 3.0 Technical Report, 2025. URL https://arxiv.org/abs/2509.23951.

[342] Guy Tevet, Sigal Raab, Brian Gordon, et al. Human Motion Diffusion Model, 2022. URL https://arxiv.org/abs/2209.14916.

[343] Serge Thilges, Onur Celik, Denis Blessing, Emiliyan Gospodinov, and Gerhard Neumann. Scalable Maximum Entropy Reinforcement Learning for Diffusion Policies via Adjoint Matching, 2026. URL https://arxiv.org/abs/2606.22630.

[344] Keyu Tian, Yi Jiang, Zehuan Yuan, Bingyue Peng, and Liwei Wang. Visual Autoregressive Modeling: Scalable Image Generation via Next-Scale Prediction, 2024. URL https://arxiv.org/abs/2404.02905.

[345] Alexander Tong, Kilian Fatras, Nikolay Malkin, et al. Improving and generalizing flow-based generative models with minibatch optimal transport, 2023. URL https://arxiv.org/abs/2302.00482.

[346] Jingqi Tong, Yurong Mou, Hangcheng Li, et al. Thinking with Video: Video Generation as a Promising Multimodal Reasoning Paradigm, 2025. URL https://arxiv.org/abs/2511.04570.

[347] Shangyuan Tong, Nanye Ma, Saining Xie, and Tommi Jaakkola. Flow Map Distillation Without Data, 2025. URL https://arxiv.org/abs/2511.19428.

[348] Shengbang Tong, Boyang Zheng, Ziteng Wang, et al. Scaling Text-to-Image Diffusion Transformers with Representation Autoencoders, 2026. URL https://arxiv.org/abs/2601.16208.

[349] Yijing Tu, Shaojin Wu, Mengqi Huang, et al. Stream-T1: Test-Time Scaling for Streaming Video Generation, 2026. URL https://arxiv.org/abs/2605.04461.

[350] Belinda Tzen and Maxim Raginsky. Theoretical guarantees for sampling and inference in generative models with latent diffusions, 2019. URL https://arxiv.org/abs/1903.01608.

[351] Masatoshi Uehara, Yulai Zhao, Tommaso Biancalani, and Sergey Levine. Understanding Reinforcement Learning-Based Fine-Tuning of Diffusion Models: A Tutorial and Review, 2024. URL https://arxiv.org/abs/2407.13734.

[352] Masatoshi Uehara, Yulai Zhao, Kevin Black, et al. Fine-Tuning of Continuous-Time Diffusion Models as Entropy-Regularized Control, 2024. URL https://arxiv.org/abs/2402.15194.

[353] Dani Valevski, Yaniv Leviathan, Moab Arar, and Shlomi Fruchter. Diffusion Models Are Real-Time Game Engines, 2024. URL https://arxiv.org/abs/2408.14837.

[354] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. Attention Is All You Need, 2017. URL https://arxiv.org/abs/1706.03762.

[355] Pascal Vincent. A Connection Between Score Matching and Denoising Autoencoders. *Neural Computation*, 23(7):1661–1674, 2011. doi: 10.1162/NECO_a_00142.

[356] Bram Wallace, Meihua Dang, Rafael Rafailov, et al. Diffusion Model Alignment Using Direct Preference Optimization, 2023. URL https://arxiv.org/abs/2311.12908.

[357] Zhenglin Wan, Jiayu Chen, Zhengfeiyang Zhang, et al. DiffusionART: Efficient, Stable and Scalable Reinforcement of Diffusion Models, 2026. URL https://arxiv.org/abs/2610.12572.

[358] Chenyang Wang, Weizhong Wang, Yinuo Ren, Jose Blanchet, and Yiping Lu. Simple Approximation and Derivative Free Inference-Time Scaling for Diffusion Models via Sequential Monte Carlo on Path Measures, 2026. URL https://arxiv.org/abs/2605.17850.

[359] Feng Wang, Zijie Li, Ceyuan Yang, Alan Yuille, and Peng Wang. Transforming Image Editors into Video Editors, 2026. URL https://arxiv.org/abs/2610.11037.

[360] Guanghan Wang, Yair Schiff, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Remasking Discrete Diffusion Models with Inference-Time Scaling, 2025. URL https://arxiv.org/abs/2503.00307.

[361] Hanyang Wang, Yiyang Liu, Jiawei Chi, et al. CFG-Ctrl: Control-Based Classifier-Free Diffusion Guidance, 2026. URL https://arxiv.org/abs/2603.03281.

[362] Haozhe Wang, Cong Wei, Weiming Ren, et al. RationalRewards: Reasoning Rewards Scale Visual Generation Both Training and Test Time, 2026. URL https://arxiv.org/abs/2604.11626.

[363] Jing Wang, Jiajun Liang, Jie Liu, et al. GRPO-Guard: Mitigating Implicit Over-Optimization in Flow Matching via Regulated Clipping, 2025. URL https://arxiv.org/abs/2510.22319.

[364] Qixun Wang, Xu Bai, Haofan Wang, et al. InstantID: Zero-shot Identity-Preserving Generation in Seconds, 2024. URL https://arxiv.org/abs/2401.07519.

[365] Runqian Wang and Yilun Du. Equilibrium Matching: Generative Modeling with Implicit Energy-Based Models, 2025. URL https://arxiv.org/abs/2510.02300.

[366] Ruoyu Wang, Boye Niu, Xiangxin Zhou, et al. Exploring the Design Space of Reward Backpropagation for Flow Matching, 2026. URL https://arxiv.org/abs/2606.11075.

[367] Shuai Wang, Liang Li, Yang Chen, et al. UniDDT: Unifying Multimodal Understanding and Generation with Decoupled Diffusion Transformer, 2026. URL https://arxiv.org/abs/2606.16255.

[368] Yibin Wang, Zhimin Li, Yuhang Zang, et al. Pref-GRPO: Pairwise Preference Reward-based GRPO for Stable Text-to-Image Reinforcement Learning, 2025. URL https://arxiv.org/abs/2508.20751.

[369] Yibin Wang, Yuhang Zang, Hao Li, Cheng Jin, and Jiaqi Wang. Unified Reward Model for Multimodal Understanding and Generation, 2025. URL https://arxiv.org/abs/2503.05236.

[370] Yufeng Wang, Parivesh Priye, Meeshawn Marathe, and Ramit Pahwa. λ-Controlled GRPO: Turning Flow-Matching Ratio Instability into a Budgeted Resource, 2026. URL https://arxiv.org/abs/2609.22041.

[371] Yuyang Wang, Jiarui Lu, Navdeep Jaitly, Josh Susskind, and Miguel Angel Bautista. SimpleFold: Folding Proteins is Simpler than You Think, 2025. URL https://arxiv.org/abs/2509.18480.

[372] Zhengyi Wang, Cheng Lu, Yikai Wang, et al. ProlificDreamer: High-Fidelity and Diverse Text-to-3D Generation with Variational Score Distillation, 2023. URL https://arxiv.org/abs/2305.16213.

[373] Zhouxia Wang, Ziyang Yuan, Xintao Wang, et al. MotionCtrl: A Unified and Flexible Motion Controller for Video Generation, 2023. URL https://arxiv.org/abs/2312.03641.

[374] Zidong Wang, Yiyuan Zhang, Xiaoyu Yue, et al. Transition Models: Rethinking the Generative Learning Objective, 2025. URL https://arxiv.org/abs/2509.04394.

[375] Zile Wang, Zexiang Liu, Jiaxing Li, et al. Matrix-Game 3.0: Real-Time and Streaming Interactive World Model with Long-Horizon Memory, 2026. URL https://arxiv.org/abs/2604.08995.

[376] Joseph L. Watson, David Juergens, Nathaniel R. Bennett, et al. De novo design of protein structure and function with RFdiffusion. *Nature*, 620(7976):1089–1100, 2023. doi: 10.1038/s41586-023-06415-8. URL https://doi.org/10.1038/s41586-023-06415-8.

[377] Jin Wei, Rundong Li, Ruihao Yang, et al. LoRango: It Takes Two LoRAs to Unlock Hidden Behaviors in Diffusion Models, 2026. URL https://arxiv.org/abs/2609.25884.

[378] Yuxin Wen, John Kirchenbauer, Jonas Geiping, and Tom Goldstein. Tree-Ring Watermarks: Fingerprints for Diffusion Images that are Invisible and Robust, 2023. URL https://arxiv.org/abs/2305.20030.

[379] Zuokai Wen, Louis Grenioux, Weinan E, and Jiequn Han. Error-Corrected Inference-Time Scaling for Imperfect Diffusion Models, 2026. URL https://arxiv.org/abs/2610.01933.

[380] Thaddäus Wiedemer, Yuxuan Li, Paul Vicol, et al. Video models are zero-shot learners and reasoners, 2025. URL https://arxiv.org/abs/2509.20328.

[381] Nicholas J. Williams, Ward Haddadin, Matteo P. Ferla, et al. Emyx: Fast and efficient all-atom protein generation, 2026. URL https://arxiv.org/abs/2606.19377.

[382] Jeremy Wohlwend, Gabriele Corso, Saro Passaro, et al. Boltz-1 Democratizing Biomolecular Interaction Modeling. bioRxiv preprint, 2024. URL https://doi.org/10.1101/2024.11.19.624167.

[383] Bing Wu, Chang Zou, Changlin Li, et al. HunyuanVideo 1.5 Technical Report, 2025. URL https://arxiv.org/abs/2511.18870.

[384] Chenfei Wu, Jiahao Li, Jingren Zhou, et al. Qwen-Image Technical Report, 2025. URL https://arxiv.org/abs/2508.02324.

[385] Chengyue Wu, Hao Zhang, Shuchen Xue, et al. Fast-dLLM v2: Efficient Block-Diffusion LLM, 2025. URL https://arxiv.org/abs/2509.26328.

[386] Chenyuan Wu, Pengfei Zheng, Ruiran Yan, et al. OmniGen2: Towards Instruction-Aligned Multimodal Generation, 2025. URL https://arxiv.org/abs/2506.18871.

[387] Jie Wu, Yu Gao, Zilyu Ye, et al. RewardDance: Reward Scaling in Visual Generation, 2025. URL https://arxiv.org/abs/2509.08826.

[388] Luhuan Wu, Brian L. Trippe, Christian A. Naesseth, David M. Blei, and John P. Cunningham. Practical and Asymptotically Exact Conditional Sampling in Diffusion Models, 2023. URL https://arxiv.org/abs/2306.17775. NeurIPS 2023.

[389] Xiaoshi Wu, Yiming Hao, Keqiang Sun, et al. Human Preference Score v2: A Solid Benchmark for Evaluating Human Preferences of Text-to-Image Synthesis, 2023. URL https://arxiv.org/abs/2306.09341.

[390] Haocheng Xi, Shuo Yang, Yilong Zhao, et al. Sparse VideoGen: Accelerating Video Diffusion Transformers with Spatial-Temporal Sparsity, 2025. URL https://arxiv.org/abs/2502.01776.

[391] Jianfeng Xiang, Zelong Lv, Sicheng Xu, et al. Structured 3D Latents for Scalable and Versatile 3D Generation, 2024. URL https://arxiv.org/abs/2412.01506.

[392] Jianfeng Xiang, Xiaoxue Chen, Sicheng Xu, et al. Native and Compact Structured Latents for 3D Generation, 2025. URL https://arxiv.org/abs/2512.14692.

[393] Jinheng Xie, Zhenheng Yang, and Mike Zheng Shou. Show-o2: Improved Native Unified Multimodal Models, 2025. URL https://arxiv.org/abs/2506.15564.

[394] Yi Xin, Qi Qin, Siqi Luo, et al. Lumina-DiMOO: An Omni Diffusion Large Language Model for Multi-Modal Generation and Understanding, 2025. URL https://arxiv.org/abs/2510.06308.

[395] Jiacong Xu, Hanwen Jiang, Zhixin Shu, et al. Wonder: Video World Model Done Better, 2026. URL https://arxiv.org/abs/2607.26037.

[396] Jiazheng Xu, Xiao Liu, Yuchen Wu, et al. ImageReward: Learning and Evaluating Human Preferences for Text-to-Image Generation, 2023. URL https://arxiv.org/abs/2304.05977.

[397] Jiazheng Xu, Yu Huang, Jiale Cheng, et al. VisionReward: Fine-Grained Multi-Dimensional Human Preference Learning for Image and Video Generation, 2024. URL https://arxiv.org/abs/2412.21059.

[398] Ting-Bing Xu, Jiacheng Sui, Zhe Gao, et al. WorldRoamBench: An Open-World Benchmark for Long-Horizon Stability of Interactive World Models, 2026. URL https://arxiv.org/abs/2606.31672.

[399] Yanchen Xu, Sida Huang, Zhenyu Gu, et al. Representation Learning in Diffusion and Flow-based Model: An Application Aspect, 2026. URL https://arxiv.org/abs/2608.24068.

[400] Yilun Xu, Mingyang Deng, Xiang Cheng, et al. Restart Sampling for Improving Generative Processes, 2023. URL https://arxiv.org/abs/2306.14878.

[401] Yixian Xu, Kaiyuan Gao, Yuxiang Chen, et al. Qwen-Image-2.0-RL Technical Report, 2026. URL https://arxiv.org/abs/2606.27608.

[402] Shuchen Xue, Chongjian Ge, Shilong Zhang, Yichen Li, and Zhi-Ming Ma. Advantage Weighted Matching: Aligning RL with Pretraining in Diffusion Models, 2025. URL https://arxiv.org/abs/2509.25050.

[403] Zeyue Xue, Jie Wu, Yu Gao, et al. DanceGRPO: Unleashing GRPO on Visual Generation, 2025. URL https://arxiv.org/abs/2505.07818.

[404] Jinbo Yan, Limeng Qiao, Jie Qin, et al. UniSpace: Unified Visual Representation and Scalable Multimodal Modeling, 2026. URL https://arxiv.org/abs/2608.08676.

[405] Ling Yang, Zhilong Zhang, Yang Song, et al. Diffusion Models: A Comprehensive Survey of Methods and Applications, 2022. URL https://arxiv.org/abs/2209.00796.

[406] Liuhaichen Yang, Zhuang Jiang, Chenchao Sheng, et al. Guided Action Flow: Value-Guided Sampling for Frozen Vision-Language-Action Policies, 2026. URL https://arxiv.org/abs/2607.02092.

[407] Sherry Yang, Yilun Du, Kamyar Ghasemipour, et al. Learning Interactive Real-World Simulators, 2023. URL https://arxiv.org/abs/2310.06114.

[408] Shuai Yang, Wei Huang, Ruihang Chu, et al. LongLive: Real-time Interactive Long Video Generation, 2025. URL https://arxiv.org/abs/2509.22622.

[409] Xiaomeng Yang, Yanyu Li, Gordon Guocheng Qian, et al. Prompt2Effect: Training-Free Image-to-Video Model Specialization via LoRA Generation, 2026. URL https://arxiv.org/abs/2606.13971.

[410] Jingfeng Yao, Yuda Song, Yucong Zhou, and Xinggang Wang. Towards Scalable Pre-training of Visual Tokenizers for Generation, 2025. URL https://arxiv.org/abs/2512.13687.

[411] Hu Ye, Jun Zhang, Sibo Liu, Xiao Han, and Wei Yang. IP-Adapter: Text Compatible Image Prompt Adapter for Text-to-Image Diffusion Models, 2023. URL https://arxiv.org/abs/2308.06721.

[412] Jiacheng Ye, Zhihui Xie, Lin Zheng, et al. Dream 7B: Diffusion Large Language Models, 2025. URL https://arxiv.org/abs/2508.15487.

[413] Tian Ye, Song Fei, and Lei Zhu. UltraFlux: Data-Model Co-Design for High-quality Native 4K Text-to-Image Generation across Diverse Aspect Ratios, 2025. URL https://arxiv.org/abs/2511.18050.

[414] Jason Yim, Andrew Campbell, Andrew Y. K. Foong, et al. Fast protein backbone generation with SE(3) flow matching, 2023. URL https://arxiv.org/abs/2310.05297.

[415] Jason Yim, Brian L. Trippe, Valentin De Bortoli, et al. SE(3) diffusion model with application to protein backbone generation, 2023. URL https://arxiv.org/abs/2302.02277.

[416] Tianwei Yin, Michaël Gharbi, Richard Zhang, et al. One-step Diffusion with Distribution Matching Distillation, 2023. URL https://arxiv.org/abs/2311.18828. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) 2024.

[417] Tianwei Yin, Michaël Gharbi, Taesung Park, et al. Improved Distribution Matching Distillation for Fast Image Synthesis, 2024. URL https://arxiv.org/abs/2405.14867.

[418] Tianwei Yin, Qiang Zhang, Richard Zhang, et al. From Slow Bidirectional to Fast Autoregressive Video Diffusion Models, 2024. URL https://arxiv.org/abs/2412.07772.

[419] Kaining Ying, Hengrui Hu, Siyu Ren, et al. WBench: A Comprehensive Multi-turn Benchmark for Interactive Video World Model Evaluation, 2026. URL https://arxiv.org/abs/2605.25874.

[420] Sihyun Yu, Sangkyung Kwak, Huiwon Jang, et al. Representation Alignment for Generation: Training Diffusion Transformers Is Easier Than You Think, 2024. URL https://arxiv.org/abs/2410.06940.

[421] Wangbo Yu, Kunhao Liu, Wenbo Hu, et al. WorldCrafter: Consistent Video World Model with Implicit 3D-aware Memory, 2026. URL https://arxiv.org/abs/2609.24984.

[422] Yongsheng Yu, Wei Xiong, Weili Nie, et al. PixelDiT: Pixel Diffusion Transformers for Image Generation, 2025. URL https://arxiv.org/abs/2511.20645.

[423] Shenghai Yuan, Yuanyang Yin, Zongjian Li, et al. Helios: Real Real-Time Long Video Generation Model, 2026. URL https://arxiv.org/abs/2603.04379.

[424] Zhengrong Yue, Taihang Hu, Mengting Chen, et al. What Matters for Diffusion-Friendly Latent Manifold? Prior-Aligned Autoencoders for Latent Diffusion, 2026. URL https://arxiv.org/abs/2605.07915.

[425] Z-Image Team, Huanqia Cai, Sihan Cao, et al. Z-Image: An Efficient Image Generation Foundation Model with Single-Stream Diffusion Transformer, 2025. URL https://arxiv.org/abs/2511.22699.

[426] Claudio Zeni, Robert Pinsler, Daniel Zügner, et al. A generative model for inorganic materials design. *Nature*, 639(8055):624–632, 2025. doi: 10.1038/s41586-025-08628-5. URL https://doi.org/10.1038/s41586-025-08628-5.

[427] Xiaoyu Zhan, Xinyu Wang, Xiaohong Zhang, et al. Magpie: Real-Time World Renderer for Interactive Games, 2026. URL https://arxiv.org/abs/2608.27168.

[428] Bingliang Zhang, Wenda Chu, Julius Berner, et al. Improving Diffusion Inverse Problem Solving with Decoupled Noise Annealing, 2024. URL https://arxiv.org/abs/2407.01521.

[429] Han Zhang, Guo-Hua Yuan, Chaohao Yuan, et al. Lingshu-Cell: A generative cellular world model for transcriptome modeling toward virtual cells, 2026. URL https://arxiv.org/abs/2603.25240.

[430] Huijie Zhang, Aliaksandr Siarohin, Willi Menapace, et al. AlphaFlow: Understanding and Improving MeanFlow Models, 2025. URL https://arxiv.org/abs/2510.20771.

[431] Jintao Zhang, Jia Wei, Haofeng Huang, et al. SageAttention: Accurate 8-Bit Attention for Plug-and-play Inference Acceleration, 2024. URL https://arxiv.org/abs/2410.02367. The Thirteenth International Conference on Learning Representations (ICLR 2025).

[432] Jintao Zhang, Haoxu Wang, Kai Jiang, et al. SLA: Beyond Sparsity in Diffusion Transformers via Fine-Tunable Sparse-Linear Attention, 2025. URL https://arxiv.org/abs/2509.24006.

[433] Jintao Zhang, Kaiwen Zheng, Kai Jiang, et al. TurboDiffusion: Accelerating Video Diffusion Models by 100-200 Times, 2025. URL https://arxiv.org/abs/2512.16093.

[434] Jintao Zhang, Kai Jiang, Jintao Chen, et al. Vidu S2: Real-Time Interactive, Editable, and Spatial Video Generation, 2026. URL https://arxiv.org/abs/2609.11638.

[435] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. Adding Conditional Control to Text-to-Image Diffusion Models, 2023. URL https://arxiv.org/abs/2302.05543.

[436] Lvmin Zhang, Shengqu Cai, Muyang Li, Gordon Wetzstein, and Maneesh Agrawala. Frame Context Packing and Drift Prevention in Next-Frame-Prediction Video Diffusion Models, 2025. URL https://arxiv.org/abs/2504.12626.

[437] Odin Zhang, Xujun Zhang, Haitao Lin, et al. ODesign: A World Model for Biomolecular Interaction Design, 2025. URL https://arxiv.org/abs/2510.22304.

[438] Qinsheng Zhang and Yongxin Chen. Path Integral Sampler: a stochastic control approach for sampling, 2021. URL https://arxiv.org/abs/2111.15141.

[439] Qinsheng Zhang and Yongxin Chen. Fast Sampling of Diffusion Models with Exponential Integrator, 2022. URL https://arxiv.org/abs/2204.13902.

[440] Yifu Zhang, Hao Yang, Yuqi Zhang, et al. Waver: Wave Your Way to Lifelike Video Generation, 2025. URL https://arxiv.org/abs/2508.15761.

[441] Yuxuan Zhang, Yirui Yuan, Yiren Song, Haofan Wang, and Jiaming Liu. EasyControl: Adding Efficient and Flexible Control for Diffusion Transformer, 2025. URL https://arxiv.org/abs/2503.07027.

[442] Zekai Zhang, Xiao Li, Xiang Li, et al. Generalization of Diffusion Models Arises with a Balanced Representation Space, 2025. URL https://arxiv.org/abs/2512.20963.

[443] Ziyi Zhang, Sen Zhang, Yibing Zhan, et al. Confronting Reward Overoptimization for Diffusion Models: A Perspective of Inductive and Primacy Biases, 2024. URL https://arxiv.org/abs/2402.08552. International Conference on Machine Learning, pp. 60396-60413, 2024.

[444] Bing Zhao, Chenfei Wu, Deqing Li, et al. Qwen-Image-2.0 Technical Report, 2026. URL https://arxiv.org/abs/2605.10730.

[445] Chenxi Zhao, Chen Zhu, Xiaokun Feng, et al. Extending One-Step Image Generation from Class Labels to Text via Discriminative Text Representation, 2026. URL https://arxiv.org/abs/2604.18168.

[446] Min Zhao, Hongzhou Zhu, Kaiwen Zheng, et al. Causal Forcing++: Scalable Few-Step Autoregressive Diffusion Distillation for Real-Time Interactive Video Generation, 2026. URL https://arxiv.org/abs/2605.15141.

[447] Rui Zhao, Yuchao Gu, Jay Zhangjie Wu, et al. MotionDirector: Motion Customization of Text-to-Video Diffusion Models, 2023. URL https://arxiv.org/abs/2310.08465.

[448] Siyan Zhao, Devaansh Gupta, Qinqing Zheng, and Aditya Grover. d1: Scaling Reasoning in Diffusion Large Language Models via Reinforcement Learning, 2025. URL https://arxiv.org/abs/2504.12216.

[449] Wenliang Zhao, Lujia Bai, Yongming Rao, Jie Zhou, and Jiwen Lu. UniPC: A Unified Predictor-Corrector Framework for Fast Sampling of Diffusion Models, 2023. URL https://arxiv.org/abs/2302.04867.

[450] Xuanlei Zhao, Xiaolong Jin, Kai Wang, and Yang You. Real-Time Video Generation with Pyramid Attention Broadcast, 2024. URL https://arxiv.org/abs/2408.12588.

[451] Zibo Zhao, Zeqiang Lai, Qingxiang Lin, et al. Hunyuan3D 2.0: Scaling Diffusion Models for High Resolution Textured 3D Assets Generation, 2025. URL https://arxiv.org/abs/2501.12202.

[452] Boyang Zheng, Nanye Ma, Shengbang Tong, and Saining Xie. Diffusion Transformers with Representation Autoencoders, 2025. URL https://arxiv.org/abs/2510.11690.

[453] Kaiwen Zheng, Yongxin Chen, Hanzi Mao, et al. Masked Diffusion Models are Secretly Time-Agnostic Masked Models and Exploit Inaccurate Categorical Sampling, 2024. URL https://arxiv.org/abs/2409.02908.

[454] Kaiwen Zheng, Huayu Chen, Haotian Ye, et al. DiffusionNFT: Online Diffusion Reinforcement with Forward Process, 2025. URL https://arxiv.org/abs/2509.16117.

[455] Kaiwen Zheng, Yuji Wang, Qianli Ma, et al. Large Scale Diffusion Distillation via Score-Regularized Continuous-Time Consistency, 2025. URL https://arxiv.org/abs/2510.08431.

[456] Kaiwen Zheng, Guande He, Min Zhao, et al. Causal-rCM: A Unified Teacher-Forcing and Self-Forcing Open Recipe for Autoregressive Diffusion Distillation in Streaming Video Generation and Interactive World Models, 2026. URL https://arxiv.org/abs/2606.25473.

[457] Xingyu Zheng, Xianglong Liu, Yifu Ding, et al. Multi-Resolution Flow Matching: Training-Free Diffusion Acceleration via Staged Sampling, 2026. URL https://arxiv.org/abs/2607.01642.

[458] Chunting Zhou, Lili Yu, Arun Babu, et al. Transfusion: Predict the Next Token and Diffuse Images with One Multi-Modal Model, 2024. URL https://arxiv.org/abs/2408.11039.

[459] Linqi Zhou, Stefano Ermon, and Jiaming Song. Inductive Moment Matching, 2025. URL https://arxiv.org/abs/2503.07565.

[460] Linqi Zhou, Mathias Parger, Ayaan Haque, and Jiaming Song. Terminal Velocity Matching, 2025. URL https://arxiv.org/abs/2511.19797.

[461] Mingyuan Zhou, Huangjie Zheng, Zhendong Wang, Mingzhang Yin, and Hai Huang. Score identity Distillation: Exponentially Fast Distillation of Pretrained Diffusion Models for One-Step Generation, 2024. URL https://arxiv.org/abs/2404.04057.

[462] Yujie Zhou, Pengyang Ling, Jiazi Bu, et al. Fine-Grained GRPO for Precise Preference Alignment in Flow Models, 2025. URL https://arxiv.org/abs/2510.01982.

[463] Zhanhui Zhou, Lingjie Chen, Hanghang Tong, and Dawn Song. dLLM: Simple Diffusion Language Modeling, 2026. URL https://arxiv.org/abs/2602.22661.

[464] Fengqi Zhu, Zebin You, Yipeng Xing, et al. LLaDA-MoE: A Sparse MoE Diffusion Language Model, 2025. URL https://arxiv.org/abs/2509.24389.

[465] Haoyi Zhu, Haozhe Liu, Yuyang Zhao, et al. SANA-WM: Efficient Minute-Scale World Modeling with Hybrid Linear Diffusion Transformer, 2026. URL https://arxiv.org/abs/2605.15178.

[466] Hongzhou Zhu, Min Zhao, Guande He, et al. Causal Forcing: Autoregressive Diffusion Distillation Done Right for High-Quality Real-Time Interactive Video Generation, 2026. URL https://arxiv.org/abs/2602.02214.

[467] Qijie Zhu, Zeqi Ye, Han Liu, Zhaoran Wang, and Minshuo Chen. Training-Free Adaptation of Diffusion Models via Doob's h-Transform, 2026. URL https://arxiv.org/abs/2602.16198.

[468] Ying Zhu, Jiaxin Wan, Xiaoran Liu, et al. DiRL: An Efficient Post-Training Framework for Diffusion Language Models, 2025. URL https://arxiv.org/abs/2512.22234.